



Synopsys PHY IP

MIPI M-PHY Type 1 G5 for Intel3

Databook

dwc_mipi_mphy_type1_g5_22_in3_ns – Product Code: H952-0

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.

www.synopsys.com

Contents

Revision History	9
Preface	13
Databook Organization	13
Web Resources	13
Reference Documentation	14
STAR on the Web (SotW)	14
Synopsys Statement on Inclusivity and Diversity	14
Customer Support	14
Chapter 1	
Product Overview	17
1.1 Quick Reference	18
1.2 General Product Description	19
1.3 Features	20
1.4 Applications	22
1.5 Standards Compliance	22
1.5.1 Configuration Settings	22
1.5.2 MIPI MPHY	22
1.6 Process Information	23
Chapter 2	
Deliverables	25
2.1 Introduction	26
2.2 Front-End Views	26
2.3 Back-End Views	27
2.4 Signal-Integrity Views	27
2.5 Test Views	27
Chapter 3	
Functional Description	29
3.1 PHY	30
3.2 PMA	31
3.2.1 Lane Receiver (RX)	33
3.2.2 Lane Transmitter (TX)	37

3.2.3 Support Block	40
3.3 Raw PCS	43

Chapter 4

PHY Usage and Configuration	47
4.1 PHY Power-Up	48
4.2 Control Register Interface	49
4.2.1 APB Interface	52
4.2.2 JTAG Interface	60
4.3 Firmware Clock Interface	62
4.4 PHY Initialization Sequence	64
4.4.1 Reference Clock Control Signals	67
4.4.2 Common Control Signals	67
4.4.3 MPLL Control Signals	68
4.4.4 TX Control Signals	68
4.4.5 RX Control Signals	68
4.5 ROM Interface	69
4.5.1 Unused ROM Locations	69
4.6 SRAM Interface	70
4.6.1 Bypassing SRAM Bootloading	70
4.7 Context Restore	74
4.7.1 Overview	74
4.7.2 Context Restore Architecture	74
4.7.3 Context Restore Memory Structure	76
4.7.4 Overriding Context Restore Values	78
4.8 MPLL Configuration Update	80
4.9 Lane Configuration Update	81
4.10 TX Datapath	84
4.10.1 TX Clock Alignment	84
4.10.2 TX Skew Requirements	84
4.10.3 TX Lane-to-Lane Deskewing Modes	86
4.11 RX Datapath	88
4.12 RX CDR PPM Detector	90
4.12.1 Configuring the RX CDR PPM Detector	90
4.13 RX Margining	91
4.13.1 RX Margin Stop	94
4.13.2 Margining Capability	94
4.14 Fast Context Switching	95
4.14.1 Usage Overview	95
4.14.2 Initial Calibration	95
4.14.3 Subsequent Rate Changes	98
4.14.4 Force/Skip Re-Calibration	103
4.15 TX and RX Power States	105
4.15.1 TX and RX Power State Description	105
4.15.2 Power State Transition Times for TX and RX	106
4.16 TX Equalization and Adaptation	109
4.16.1 TX Equalization Coefficient Mapping	109
4.16.2 TX Equalization, Initialization and Presets	109

4.16.3 TX Equalization and Adaptation	109
4.17 RX Equalization and Adaptation	110
4.18 Test Modes	112
4.18.1 Boundary Scan (ACJTAG)	112
4.18.2 Burn-In	115
4.18.3 IDDQ	115
4.18.4 Flyover	115
4.18.5 Reference Clock Over TX	116
4.18.6 Scan (At-Speed, Stuck-At)	116
4.18.7 Scan Dump	120
4.18.8 Diagnostics	121
4.18.9 Analog DC Test Capabilities	148
4.18.10 Digital Test Bus (DTB)	151
4.19 Power Gating	156
4.19.1 Isolation Between VP and VDD Domains	159
4.19.2 Register Preservation	160
4.20 Configuration for a Disabled PHY	165
4.21 Notes for Configuring the PHY	166
4.22 Configuring the PHY	167

Chapter 5

Verilog Model Description	169
5.1 Overview	170
5.2 Verilog Function	171
5.3 Serial Line Modeling	172
5.4 Verilog Model Files and Simulators	173
5.5 Fast Simulation Mode	174
5.5.1 Interrupt Disable Mode	180
5.6 Simulation Environment	181

Chapter 6

Physical-Level Implementation	183
6.1 PMA Assembly	184
6.1.1 Flip-Chip Pinout	184
6.1.2 PMA Size	184
6.2 PMA Placement	185
6.2.1 PMA Rotation and Mirroring	185
6.2.2 PMA Abutment	186
6.3 PMA Fill	188
6.4 Operating Conditions	188
6.4.1 Power Supply Limits	188
6.4.2 Temperature	188
6.4.3 Electromigration	188
6.4.4 Aging Specification	189
6.5 Power Supply Connectivity	190
6.5.1 Power and Ground Isolation	190
6.6 ESD	191

6.6.1 ESD Requirements	191
6.6.2 ESD Protection Strategy	191
6.6.3 ESD Back-to-Back Diode Connection to Core VSS	192
6.7 Reference Clock	194
6.8 Reference Resistor	194
6.8.1 Reference Resistor Contention Bus	195
6.9 ATB and DTB Signals	197
6.10 Raw PCS Synthesis	197
6.10.1 Synthesis Database	197
6.10.2 Raw PHY (PCS and PMA) Constraints	198

Chapter 7

Board- and Package-Level Implementation	207
7.1 Package Type	208
7.2 RX and TX Differential Signal Guidelines	208
7.2.1 Package Guidelines – RX and TX Maximum Allowed Package Loss	208
7.2.2 Signal Integrity Analysis	208
7.3 Supplies Requirements	209
7.3.1 Power Supplies Delivery to Pads	209
7.3.2 Package and Board Guidelines – Power Supply Sharing	213
7.3.3 Recommended Maximum Off Silicon Die Power Supply Inductance	213
7.3.4 Package Guidelines – Substrate Routing	214
7.4 Reference Resistor	216
7.4.1 Package and Board Requirements – Reference Resistor	216
7.4.2 Reference Resistor Implementation for a Single PMA	217
7.4.3 Sharing a Reference Resistor Between Multiple PMAs	217
7.4.4 Internal Reference Resistor ATE Calibration	218
7.5 Reference Clock	220
7.6 Board Requirements – External Components	220

Chapter 8

Reference Clock Implementation	221
8.1 Reference Clock Requirements	222
8.1.1 Reference Clock Specification	222
8.1.2 Reference Clock: Electrical Characteristics	224
8.1.3 Repeat Clock Specification	225
8.2 Reference Clock Implementation	226
8.2.1 External Off-Chip Clock Sources	226
8.2.2 On-Chip Clock Sources	227
8.2.3 Reference Clock Repeater	228
8.2.4 Reference Clock Distribution	229

Chapter 9

Electrical Specifications	231
9.1 Power Dissipation Overview	232
9.2 Power Dissipation	233

Appendix A
Support and Lane Configuration Settings.....235

Appendix B
TX Equalization Values.....237

Appendix C
Configuration and Compile Macro ifdef Settings239

Appendix D
Cell Mapping for Hand-Placed Cells.....241

Revision History

The following table lists the revision history of the PHY for each release. For a detailed description of PHY component updates, refer to the PHY release notes.

 Note	■ Links and references to section, table, figure and table numbers in this table are only assured to be valid for the version in which the change is made. ■ In some instances, documentation-only updates occur. The Synopsys IP product information (https://www.synopsys.com/designware-ip.html) has the latest information. ■ Documentation only updates are designated using the following numbering structure <phy_version>_d<x>.
---	---

PHY Version	Date	Description
4.00a_pre3	February 23, 2024	Updated: ■ “General Product Description” on page 19 ■ “Features” on page 20 ■ “RX Analog Front End” on page 33 ■ “TX Termination” on page 37 ■ “APB Interface” on page 52 ■ “Power State Transition Times for TX and RX” on page 106 ■ “Analog Test Bus Overview” on page 148 ■ “PMA Size” on page 184 ■ “Reference Resistor” on page 194 ■ “Reference Resistor Contention Bus” on page 195 ■ “Reference Resistor” on page 216 ■ “Sharing a Reference Resistor Between Multiple PMAs” on page 217 ■ “Internal Reference Resistor ATE Calibration” on page 218 ■ Table “Clock Distribution Impact” on page 229 ■ “TX Equalization Values” on page 237 ■ For “Signal Descriptions” and “Register Descriptions” updates refer to the <i>Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual</i> ■ Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions

PHY Version	Date	Description
4.00a_pre2	November 22, 2023	Added: ■ “RX CDR PPM Detector” on page 90 ■ “BCM Technology Mapping” on page 199 ■ Appendix D, “Cell Mapping for Hand-Placed Cells” Updated: ■ “Preface” on page 13 ■ “General Product Description” on page 19 ■ “Process Information” on page 23 ■ Figure “PMA Architecture (Single Lane)” on page 32 ■ “MPLL” on page 41 ■ “Raw PCS” on page 43 ■ “Control Register Interface” on page 49 ■ “APB Interface” on page 52 ■ “Firmware Clock Interface” on page 62 ■ “SRAM Interface” on page 70 ■ Table “Naming Convention Mapping” on page 78 ■ “RX Margining” on page 91 ■ “Fast Context Switching” on page 95 ■ “Power State Transition Times for TX and RX” on page 106 ■ “Boundary Scan (ACJTAG)” on page 112 ■ Note at “Burn-In” on page 115 ■ “Reference Clock Over TX” on page 116 ■ “Scan (At-Speed, Stuck-At)” on page 116 ■ “At-Speed Scan Clock Input Frequencies” on page 119 ■ “RX-to-TX Parallel Loopback” on page 122 ■ “Overview” on page 170 ■ “Fast Simulation Mode” on page 174 ■ “PMA Assembly” on page 184 ■ “PMA Size” on page 184 ■ “Power and Ground Isolation” on page 190 ■ “ESD Requirements” on page 191 ■ “Synthesis Database” on page 197 ■ “Hand-Placed Logic Constraints” on page 198 ■ “vph Supply Requirements” on page 211 ■ “Package Guidelines—Substrate Routing” on page 214 ■ Appendix B, “TX Equalization Values” ■ Appendix C, “Configuration and Compile Macro ifdef Settings” ■ For “Signal Descriptions” and “Register Descriptions” updates refer to the <i>DesignWare® Cores MIPI M-PHY Type 1 G5 for Intel3 Reference Manual</i>
4.00a_pre1	March 10, 2023	Initial preliminary release.

Preface

Databook Organization

This databook is organized as follows:

- [Chapter 1, “Product Overview”](#), provides an introduction to the Synopsys MIPI M-PHY Type 1 G5 and its features.
- [Chapter 2, “Deliverables”](#), describes the deliverables included in the PHY full-package release.
- [Chapter 3, “Functional Description”](#), provides a functional overview of the PHY and describes its test mode.
- [Chapter 4, “PHY Usage and Configuration”](#), explains the usage of PHY features and lists the PHY configuration settings for the protocols and data rates that are supported.
- [Chapter 5, “Verilog Model Description”](#), provides an overview on PHY Verilog model simulation.
- [Chapter 6, “Physical-Level Implementation”](#), describes the physical implementation of the PHY.
- [Chapter 7, “Board- and Package-Level Implementation”](#), describes requirements and guidelines for implementing the PHY at the board and package level.
- [Chapter 8, “Reference Clock Implementation”](#), describes the reference clock that the PHY supports.
- [Chapter 9, “Electrical Specifications”](#), provides PHY electrical specifications for the Intel3 technology.
- [Appendix A, “Support and Lane Configuration Settings”](#), provides support and lane configuration values for configuring the PHY for your application.
- [Appendix B, “TX Equalization Values”](#), provides TX equalization values per protocol for configuring the PHY for your application.
- [Appendix C, “Configuration and Compile Macro ifdef Settings”](#), provides Configuration and Compile Macro ifdef Settings.
- [Appendix D, “Cell Mapping for Hand-Placed Cells”](#), provides the cell mapping for hand-placed cells required for place and route.

Web Resources

- Synopsys IP product information: <https://www.synopsys.com/designware-ip.html>
- Your custom Synopsys Component IP page: <https://www.synopsys.com/dw/mydesignware.php>
- Documentation through SolvNetPlus: <https://solvnetplus.synopsys.com> (Synopsys password required)

- Synopsys Common Licensing (SCL): <https://www.synopsys.com/support/licensing-installation-computeplatforms/licensing.html>

Reference Documentation

- IEEE Standard 1149.1-2001, IEEE
- IEEE Standard 1149.6-2003, IEEE
- MIPI Alliance Specification for M-PHY, Version 5.0 Revision 05, 31 May 2021
- MIPI M-PHY Physical Layer Conformance Test Suite, Version 1.00 Revision Candidate 02, 21 March 2013
- MIPI Alliance Specification for Unified Protocol (UniProSM), Version 2.0 Revision 03, 12 August 2020
- JEDEC STANDARD Universal Flash Storage (UFS), JESD220-1, SEPTEMBER 2013

STAR on the Web (SotW)

You must review all STARs on the Web (SotWs) associated with your product. SotWs are considered a part of the Synopsys documentation suite, and show critical information related to your product. To review product SotWs, refer to the Synopsys IP product information:

<https://www.synopsys.com/designware-ip.html>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

Synopsys provides the following ways to contact Customer Support:

- *For fastest response*, enter a case through SolvNetPlus:
 - a. <https://solvnetplus.synopsys.com>



Note

SolvNetPlus does not support Internet Explorer.

- b. Click the **Cases** menu and then click **Create a New Case** (below is the list of cases).
- c. Complete the mandatory fields that are marked with an asterisk and click **Save**.
Make sure to include the following:
 - **Product L1:** DesignWare Cores

■ **Product L2:** MIPI M-PHY

- For more information about general usage information, refer to the following article in SolvNetPlus:
<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>
- Or, send an e-mail message to support_center@synopsys.com (your e-mail will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
 - Include the Product L1 and Product L2 names, process, and Version number in your e-mail so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

1

Product Overview

This chapter includes the following sections:

- [“Quick Reference”](#) on page 18
- [“General Product Description”](#) on page 19
- [“Features”](#) on page 20
- [“Applications”](#) on page 22
- [“Standards Compliance”](#) on page 22
- [“Process Information”](#) on page 23

1.1 Quick Reference

This section provides the most relevant details of the Synopsys MIPI M-PHY Type 1 G5 for Intel3, which are provided along with other details throughout this databook.

Features
Receiver: AFE containing CTLE and VGAs, and 5-tap DFE Transmitter: 3-tap FFE, low power hybrid (voltage and current mode) driver PLL: Two independent PLLs, based on one LC-tank and one RO-tank oscillators Lane configurations: x2

Junction Temperature	EM	Process Technology
-40 ~ 125°C	88KPOH@105°C	Intel3

Power Supplies (V)	Metal Stacks Supported
Core Voltage (vp, vpdig): - Typical (Nominal): 0.75 - Minimum: 0.75 - 7% - Maximum: 0.75 + 10% IO Voltage (vph): - Typical (Nominal): 1.2 - Minimum: 1.2 - 7% - Maximum: 1.2 + 10%	m17_6x_2ya_2yb_2yc_2yd_1ye_1ga_mim3x_1gb_bumpi

Minimum ESD Protection Levels	Test Modes Supported
CDM: 6A peak discharge current HBM: 2 kV	Boundary Scan (ACJTAG), Burn-In, IDDQ, Scan (At-Speed, Stuck-At), Diagnostics (serial and parallel loopback), and Normal Operation

Specifications	
MIPI M-PHY Type1 G5	IEEE Standard 1149.1-2001, IEEE IEEE Standard 1149.6-2003, IEEE MIPI Alliance Specification for M-PHY, Version 5.0 Revision 05, 31 May 2021

1.2 General Product Description

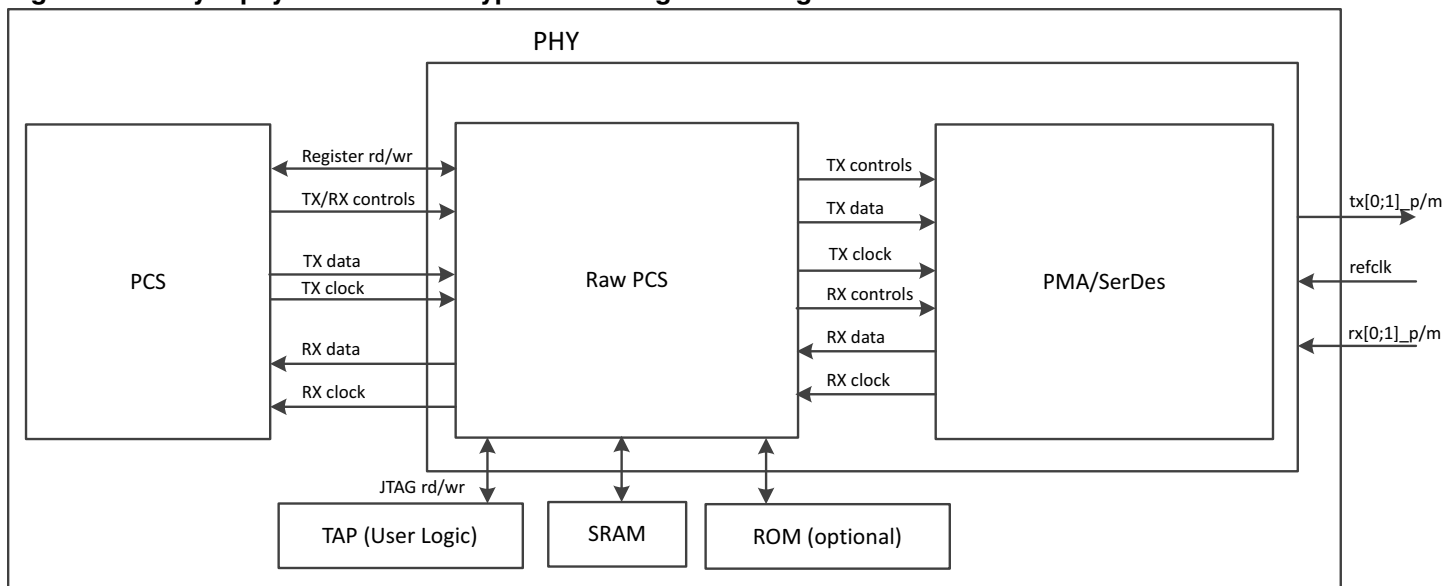
The Synopsys MIPI M-PHY Type 1 G5 (`dwc_mipi_mphy_type1_g5_phy_2tx_2rx_ns`) provides full-duplex transceivers. It consists of the following:

- A hard macro (physical medium attachment (PMA)/SerDes): `dwc_mipi_mphy_type1_g5_pma_2tx_2rx_ns`
- A soft RTL module (raw physical coding sublayer (PCS)) prefixed with: `dwc_mipi_mphy_type1_g5_pcs_2tx_2rx_ns`

Figure 1-1 shows the hierarchy of the PHY and its instantiation with a protocol-specific PCS connecting to a TX/RX transceiver pair to create a protocol-specific PHY.

The PHY architecture supports aggregation of multiple PMA to create higher-lane configurations.

Figure 1-1 Synopsys MIPI M-PHY Type 1 G5 Usage and Integration Model



Note

Mandatory SRAM size: 20K single-port SRAM, consisting of 16-bit locations.

The PHY supports a source-synchronous clocking interface on the transmit (TX) and receive (RX) datapaths. The TX parallel data (per lane) is synchronous to the TX clock input of the corresponding lane. Similarly, the RX parallel data is output-synchronized (per lane) to an RX clock that is recovered from the serial data stream that corresponds to that lane.

The PHY supports different bit-width options for the TX and RX datapath interfaces. Table 1-1 lists the general relation between baud rates and corresponding parallel data widths that are supported. For specific bit-widths supported for each protocol, see section “Power Gating” on page 156.

Table 1-1 Data Widths Supported

Protocol/Datarate	TX/RX Interface Width
M-PHY HS-G(1,2,3,4,5)	20-bit

1.3 Features

The PHY provides the following features:

- Highly customizable physical medium attachment (PMA) configuration (controlled by PCS):
 - M-PHY 5.0:
 - Supports 80-bit (up to HS-Gear 5) RMMI interface
- Single, dual and quad channel macros configurations
- Two independent PLLs, based on 1 LC-tank and 1 RO-tank oscillators
- Extensive PMA debug capability via read/write and read-only registers in PCS
- Support for data rates within the following ranges:
 - High-Speed: Gear 1, 2, 3, 4 and 5 (Rate A and B)
 - Low-Speed: PWM-Gear 1
- Register-based control of all PCS-to-PMA signals
- Adaptive and configurable RX continuous time linear equalizer (CTLE) and decision feedback equalizer (DFE)
- Adaptive and programmable TX equalization
- Boundary scan (ACJTAG), Burn-In, IDDQ, Scan (At-Speed, Stuck-At Fault) Diagnostics (serial and parallel loopback) and Normal Operation test modes
- IEEE 1149.1 and 1149.6 (AC JTAG) boundary scan
- Built-in Self-Test (BIST) features for production; at-speed testing on any digital tester
- Advanced, built-in diagnostics including on-chip sampling scope
- Visibility and controllability of hard macro functions through programmable registers in the design:
 - Overrides on all ASIC side functional control inputs for easy debug
 - Access register space through two dedicated APB ports
 - Access register space through a JTAG port
- Configurable raw PCS with extensive debug options:
 - Independent TX and RX control per lane
 - Configurable TX and RX power modes
 - Pseudo-random bit sequence (PRBS) generation and checker (PRBS31, PRBS23, PRBS16, PRBS15, PRBS11, PRBS9, PRBS7)
 - Programmable 10-bit pattern generation with error injection capability
- A wide range of reference clock frequencies: 19.2 MHz, 26 MHz, 38.4 MHz, and 52 MHz with an input high level of 1.2V+10% max
- Built-in, on-chip SSC generation

- A reference clock repeater for other modules
- Supports flip-chip packaging
- Mandatory SRAM size: 20K single-port SRAM, consisting of 16-bit locations

1.4 Applications

The Synopsys MIPI M-PHY Type 1 G5 IP (PHY) is targeted for connectivity in chip-to-chip or card-to-card communication across a combination of printed circuit board, connectors, backplane wiring, or cable.

Designed for low power, efficient area, low test costs, and process centering for optimal yield and interoperability, the PHY enables designers to introduce competitive products using the latest standards.

1.5 Standards Compliance

The PHY is designed to be compliant with the standards listed within the subsections of this section with requisite exceptions listed.

1.5.1 Configuration Settings

For each standard, there are multiple possibilities with respect to configuration based on:

- Baud rate
- Reference clock frequency
- Datapath width

For more information on per protocol support block and lane configuration settings, see [“Support and Lane Configuration Settings”](#).



Note

The customer environment determines some of the configuration settings, such as the TX equalization, in case of protocols that do not adapt the TX.

1.5.2 MIPI MPHY

The PHY is compliant with the required features of the following standards:

- MIPI Alliance Specification for M-PHY, Version 5.0 Revision 05, 31 May 2021

For the supported reference clocks, refer to [“Configuring the PHY”](#) on page 167.

Exceptions to the specification allowed (see [Table 1-2](#)).

Table 1-2 Exceptions to Specification Allowed for MIPI MPHY

Category	Exception Allowed
Reference clock	Unshared clock mode is not supported
Advanced Hibern8/TActivate Capability Support	Advanced Hibern8 or Tactivate capability is not supported

1.6 Process Information

[Table 1-3](#) describes the process information for the PHY. For availability in alternate process nodes or variants, contact your sales representative.

Table 1-3 Process Information

Foundry	Process Technology	Process Core Voltage	Process I/O Voltage	Metal Layers
Intel	3 nm Intel3	0.75 V	1.2 V	m17_6x_2ya_2yb_2yc_2yd_1ye_1ga_mim3x_1gb__bumpi



Note

- A list of GDSII layers used in the IP is included in a text file within the gds sub-directories of the release package. For more information, see [“Deliverables”](#) on page 25.
- For PHY supply voltages, see [“Power Supplies Delivery to Pads”](#) on page 209.
- The hard-macro includes the bump assembly and therefore all metal layers up to and including the bump layer are used.

[Table 1-4](#) lists the primitive devices used by this PHY.

Table 1-4 Primitive Devices Used (not available for this preliminary release)

Devices	Description
-	-

2

Deliverables

This chapter includes the following sections:

- [“Introduction”](#) on page 26
- [“Front-End Views”](#) on page 26
- [“Back-End Views”](#) on page 27
- [“Signal-Integrity Views”](#) on page 27
- [“Test Views”](#) on page 27

2.1 Introduction

The full-package for each Synopsys MIPI M-PHY Type 1 G5 release contains all of the design views and information required to instantiate the PHY in a top-level chip design. It also allows for the simulation of both the digital functionality of the PHY and the signal integrity of the link, and provides the information to enable production testing of the PHY.

For information about the PHY release directory structure, see the top-level Readme file included in the product deliverables.

The deliverables are grouped into four basic categories based on usage:

- Front-end view – Includes deliverables for system simulation and floor planning
- Back-end view – Includes deliverables for physical design and finishing
- Signal-integrity view – Includes deliverables for simulation of link performance at the analog level
- Test view – Includes deliverables for testing

2.2 Front-End Views

Front-end views are provided for both the hard macro portions of the PHY and the raw PCS soft-IP top-level wrapper.

A front-end view includes the following:

- Verilog test bench examples



Note

The packaged testbench is a sample testbench for you to use for reference and pipe-cleaning purposes. The testbench does not exhaustively include all validation tests for all protocols.

- Digital simulation environment
- Synthesis timing models in .db and .lib formats
- Behavioral models for analog blocks
- Library Exchange Format (LEF) file with pin sizes and locations
- Protocol-specific test bench examples
- Synthesis constraints for the PCS
- IP-XACT XML file with register details
- *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Databook*
- *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*

2.3 Back-End Views

Back-end views are provided for the hard macro portions of the PHY. These deliverables tend to be metal stack dependent, so multiple files for each hard macro are provided. The top-level cell name inside all of the files is the same, but the content is not—you must select the appropriate file that matches your metal stack-up.

A back-end view includes the following:

- GDSII layout and layer map files
- Layout-Versus-Schematic (LVS) netlist in SPICE format, and LVS report
- LVS samples
- Design Rule Check (DRC) report
- DRC samples
- Generic technology (GTECH) netlists and models for simulation
- Gate-level Verilog
- Gate-level netlist and Standard Delay Format (SDF) timing file

2.4 Signal-Integrity Views

A signal-integrity view includes the following:

- S-Parameter models of RX and TX to facilitate package design
- IBIS-AMI models of RX and TX for signal integrity simulations
- Current waveforms, along with on-die power grid equivalent capacitance and resistance (C_{die} and R_{die} , respectively) for power delivery network simulations in the package

2.5 Test Views

A test view includes the following:

- Scan test environment with Automatic Test Pattern Generation (ATPG)
- Scan Verilog netlist

3

Functional Description

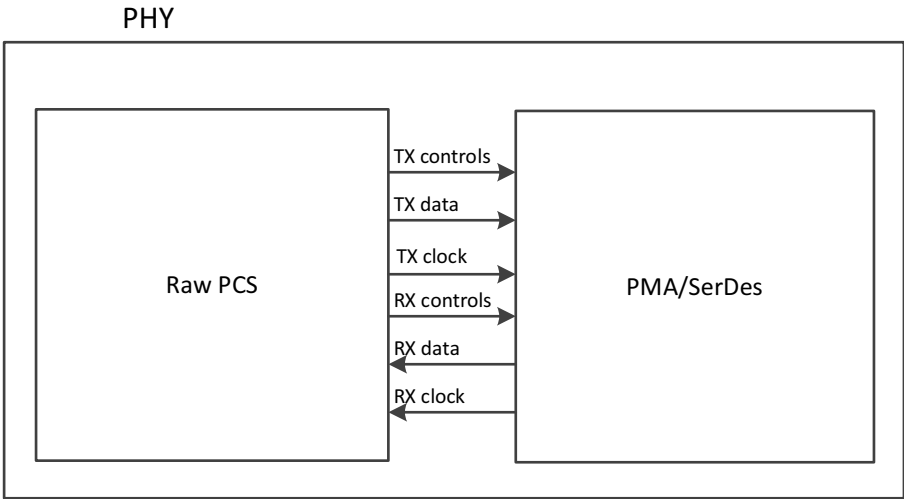
This chapter includes the following sections:

- “PHY” on page [30](#)
- “PMA” on page [31](#)
- “Raw PCS” on page [43](#)

3.1 PHY

The PHY product for the Synopsys MIPI M-PHY Type 1 G5 consists of the hard macro (PMA) and the soft RTL macro (raw PCS). The major blocks are shown in [Figure 3-1](#).

Figure 3-1 PHY Product for the Synopsys MIPI M-PHY Type 1 G5



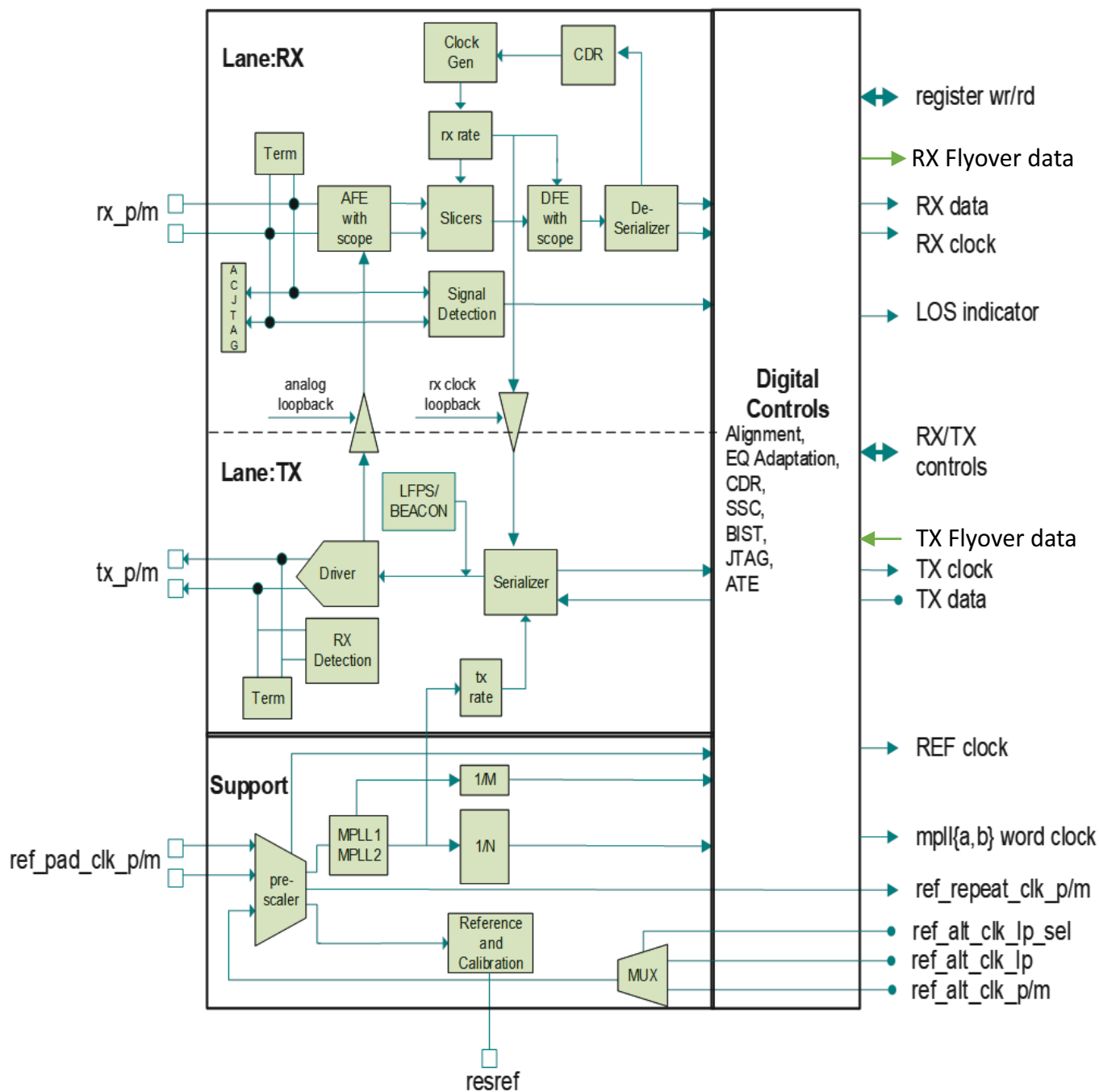
The following sections provide the functional description of the PMA and the Raw PCS.

3.2 PMA

Each PMA is comprised of two transmitters and two receivers.

As shown in [Figure 3-2](#) on page [32](#), the PMA comprises the following main blocks:

- Lane
 - Receiver (RX)
 - Transmitter (TX)
- Support
 - Reference clock
 - MPLLs
 - Termination Resistance Tuning
- Test Features

Figure 3-2 PMA Architecture (Single Lane)

3.2.1 Lane Receiver (RX)

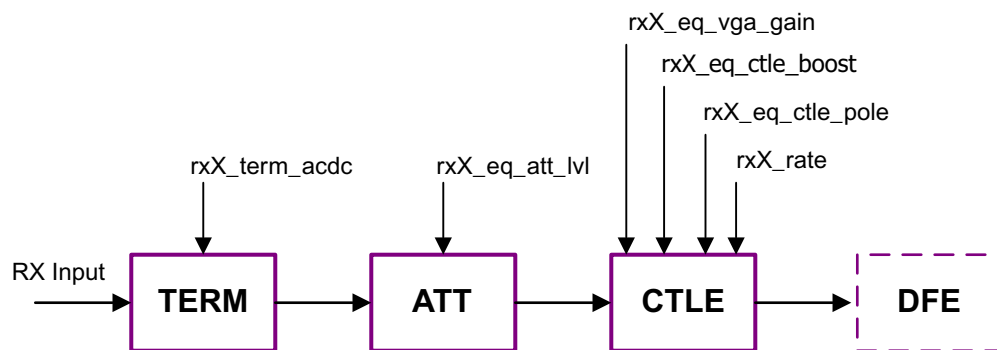
This section describes the receiver (RX) functionality.

On the RX side, the PMA receives differential serial data from a pair of external pads. Receiver characteristics, such as equalization and termination, are user-controllable. A continuous-time linear equalizer (CTLE) is first used to compensate for channel loss. A decision feedback equalizer (DFE) is also implemented after CTLE to optimize the data eye further for high-speed data that exceeds ~6.25 Gbps. The RX output signal is passed to the clock and data recovery (CDR) circuit, which recovers the embedded clock in the data. A dedicated VCO is designed as the source of the recovering clock. The CDR loop adjusts the VCO clock frequency and phase until the frequency matches incoming data and the phase is aligned with incoming data. This recovered clock is used to retime received data and send it to the deserializer, which produces parallel data and a parallel data clock for the relevant PCS lane.

3.2.1.1 RX Analog Front End

The analog front end (AFE) circuitry receives off-chip data and boosts high-frequency components to compensate part of the channel loss with programmable or adaptive CTLE.

Figure 3-3 RX AFE (Analog Front End) Diagram



As shown in [Figure 3-3](#), the AFE consists of the following blocks:

- Termination (TERM)

The TERM block is tuned by the PHY using a reference resistor.

To configure the RX termination in AC or DC mode (see [Figure 3-4](#)), use rxX_term_acdc:

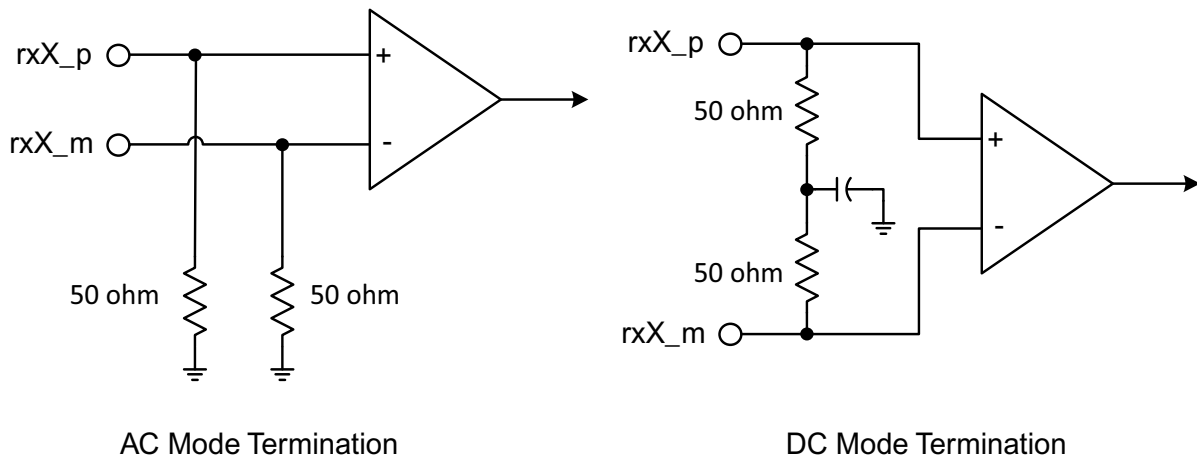
- 0=DC floating mode termination
- 1=AC mode termination to ground

If you use DC floating termination, the common-mode is set by the far end transmitter and must be within 0-300 mV range.



Note

- The PHY supports a DC-coupled link into RX floating termination.
- If you require a DC-coupled link into grounded termination, contact Synopsys support.

Figure 3-4 RX Termination

- Attenuator (ATT)

The ATT block is responsible for attenuating the incoming signal to acceptable levels for the subsequent CTLE block to maintain linearity in the case of a short-reach link. Attenuation is controlled by rxX_eq_att_lvl.

- Continuous time-linear equalizer (CTLE)

The CTLE block has programmable boost level, gain and boost bandwidth (BW) control (provided by rxX_eq_ctle_boost, rxX_eq_vga_gain, and rxX_rate, respectively).

To achieve optimal link performance, the receiver equalization parameters are automatically tuned to their optimal values for protocols that are adapted. For more details about RX AFE equalization adaptation, see [“RX Equalization and Adaptation”](#) on page 36.

3.2.1.2 RX DFE (Decision Feedback Equalizer)

The post-CTLE output is summed by a 5-tap, continuously adaptive Decision Feedback Equalizer (DFE). The output of the DFE is sampled by offset-compensated samplers, de-serialized, and sent to the core along with the recovered clock.

The 5-tap DFE is a half-rate architecture with first tap unrolling. For more details about RX equalization adaptation, see [“RX Equalization and Adaptation”](#) on page 36.

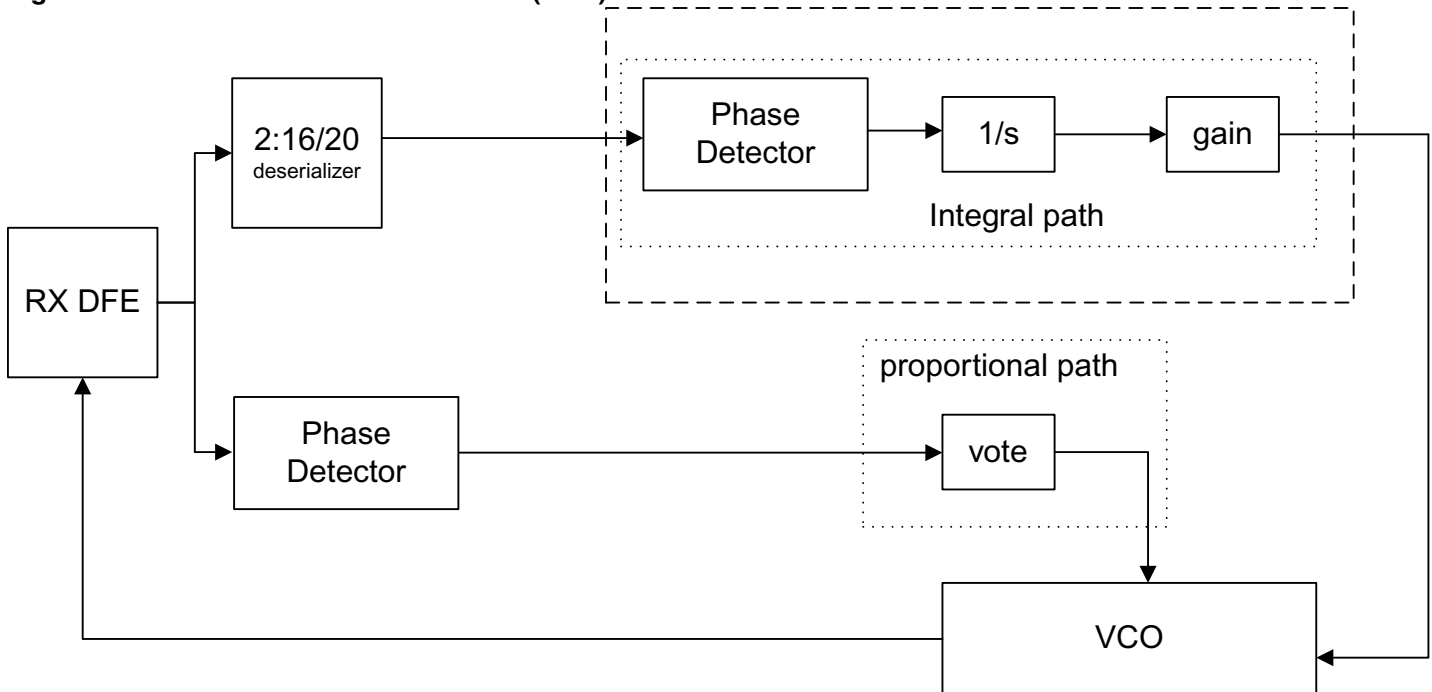
The DFE can be disabled (and DFE_BYPASS enabled) through top-level signal or register control:

- rxX_dfe_bypass
- LANEN_DIG_ANA_RX_CTL_OVRD_OUT.RX_ANA_DFE_BYPASS_EN register bit can be de-asserted to disable DFE and enable DFE_BYPASS
- The DFE and DFE_BYPASS can both be disabled through register LANEN_DIG_ANA_RX_CTL_OVRD_OUT.RX_ANA_DFE_TAPS_EN, which is used to reset tap2 to tap5, therefore virtually disabling the taps and the DFEs

3.2.1.3 RX Clock and Data Recovery

The clock and data recovery (CDR) circuit is implemented as a second-order digital loop. The loop includes a proportional path and an integral path. To reduce proportional path latency, the proportional path is implemented separately from the integral path, with much higher speed, as shown in Figure 3-5.

Figure 3-5 RX Clock and Data Recover (CDR)



The proportional path and integral path outputs control a dedicated CDR voltage-controlled oscillator (VCO), which generates an RX clock aligned with incoming data. To support spread-spectrum clocking (SSC) and to reduce CDR lock time, proportional path gain and integral path gain are dynamically set during the CDR locking process.

3.2.1.4 RX Deserialization

The deserializer converts serial data from RX and sends the data to the PCS as parallel data. The deserializer qualifies serial input data from the CDR using the high-speed serial recovered clock, and deserializes the data using the low-speed parallel recovered clock. The deserializer forwards the deserialized data to the receiver's PCS channel. The deserializer also optionally supports an 16- or 20-bit interface for lower data-rates as specified in Table 1-1 on page 20.

3.2.1.5 RX Internal Eye Monitor

Provided with the PHY is an internal eye monitor, which can provide data to produce a plot of time and voltage axis using internal DFT hooks built into the AFE. The eye monitor is non-destructive and can be run in parallel with live traffic. This procedure can be used to characterize the input RX eye margin as seen after the ATT and after the VGA. In this mode, eye monitor must be always disabled before the decision feedback equalizer (DFE).

3.2.1.6 RX Rate Control

The RX high-speed clock is generated by the CDR loop that controls and aligns the high-speed clock frequency and phase with data received by the RX. The CDR-controlled VCO provides a clock rate that can range from 6–13 GHz. In addition, the RX has a flexible data rate control (configured by rxX_rate) that allows the RX to reliably receive and process incoming data from 2.5 Gbps to a maximum of 16 Gbps.

For example, if the RX VCO is operating at 10 GHz, the RX can operate at 5 Gbps by setting the internal rxX_rate signal to 3'b010, to divide the RX clock rate down from 10–2.5 GHz. For detailed PMA configuration settings for all supported protocols, see [“PHY Usage and Configuration”](#) on page 47.

3.2.1.7 RX Calibration

The PHY has the ability to calibrate the RX AFE and DFE so as to minimize offset. This is run on start up out of reset and may also be run continuously during mission mode.

3.2.1.7.1 Startup Calibration

The PHY runs startup calibration after the RX is powered up. It calibrates the offsets of each stage of the AFE (ATT, CTLE, VGA) in sequence, and then calibrates the offsets of the DFE data, phase, and error slicers.

3.2.1.7.2 Continuous Calibration

The PHY runs continuous calibration in mission-mode. The continuous calibration routine adjusts the offsets of each stage of the AFE (CTLE, VGA, and summer) in sequence. Continuous calibration runs in tandem with continuous adaptation when the latter is supported.

3.2.1.8 RX Equalization and Adaptation

For protocols that require link adaptation, the PHY supports adaptation of the receiver equalization parameters, and requests for incrementing or decrementing the far-end transmitter coefficients. The PHY adapts the AFE equalization and gain, the DFE tap coefficients, and the remote TX equalization. It operates in two steps: startup adaptation and continuous adaptation.

3.2.1.8.1 Startup Adaptation

The PHY runs startup adaptation after startup calibration is complete, when the RX is receiving valid data and the CDR is locked. It first adapts the AFE, where the ATT level and VGA gain are adapted based on measurements of the average eye opening and the CTLE boost is adapted based on data bit correlation. It then adapts the DFE, where the tap coefficients are adapted based on the same correlation algorithm. It calculates a figure-of-merit based on collected statistics. Finally, it evaluates the TX pre-cursor and post-cursor and makes a decision on whether to adjust remote TX equalization.

3.2.1.8.2 Continuous Adaptation

The PHY runs continuous adaptation in mission-mode for protocols that support adaptation. The continuous adaptation routine adjusts the coefficients of DFE taps 1 to 5 and the VGA gain using an algorithm similar to the one used during startup adaptation. Continuous adaptation is run in tandem with continuous calibration.

3.2.2 Lane Transmitter (TX)

This section describes the transmitter (TX) functionality.

On the TX side, the PMA receives parallel data from the relevant lane of the PCS. The data is serialized and sent to the TX driver, which is a high-speed, hybrid-mode (voltage and current), differential-output driver. TX driver characteristics, such as amplitude, pre-emphasis, and termination, are user-controllable. The TX driver output is sent to a pair of external pads.

3.2.2.1 TX Serializer

Parallel data from the PCS is latched using a clock that is synchronous to the lane's PCS TX clock, and then serialized by the Parallel-to-Serial (P2S) converter. The serializer supports a 8-bit or 10-bit interface for lower data-rates as defined in [Table 1-1](#) on page 20.

3.2.2.2 TX Termination

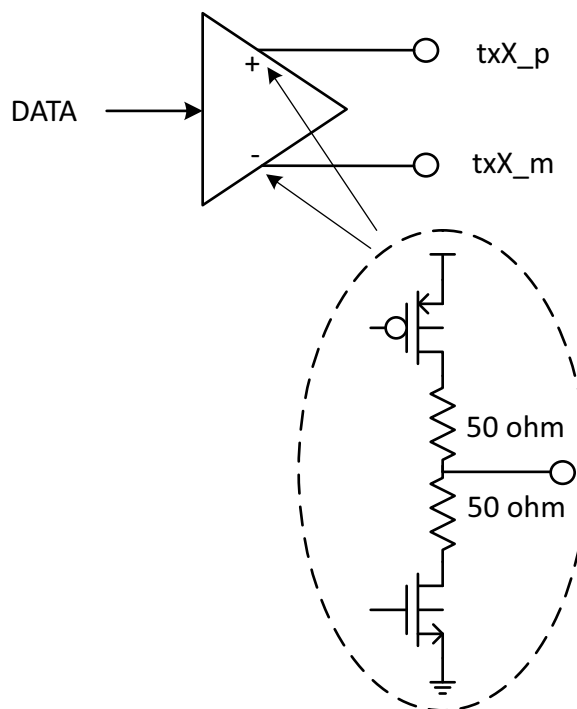
The TX is a voltage mode driver with 50 ohm termination on both txX_p and txX_m pullup and pulldown, as shown in [Figure 3-6](#). The termination is tuned by the PHY using a reference resistor.



Note

- The PHY supports a DC-coupled link into far-end floating termination.
- If you require a DC-coupled link into grounded termination, contact Synopsys support.

Figure 3-6 TX Termination



To be able to achieve low termination impedance of ~40 ohm in extreme PVT conditions, in order to reduce the driver leg impedance, some CREG need to be enabled:

- SUP_DIG_ANA_XF_SUP_ANA_CREG02.SUP_ANA_RT_TX_RBOOST_EN = 0x1
- LANEN_DIG_ANA_XF_TX_ANA_CREG00.RBOOST_TEST_EN = 0x1
- SUP_DIG_RTUNE_CONFIG.SUP_ANA_TERM_CTRL = 0x7

This will allow the calibration procedure to achieve lower calibration impedances that would not be possible otherwise.

3.2.2.3 TX Amplitude Control

The peak-to-peak output swing of the TX, Vswing, is nominally equal to the power supply, vp. However, the TX supports the boost function, which is enabled by the input signal of txX_vboost_en.

$TX_SWING = 4/3 * (24 / (19 - INT(txX_iboost_lvl / 4)) * tx_vboost_vref)$ for $tx_swing > vp$

This formula is only valid if $TX_SWING > VP$, if $TX_SWING \leq VP$ then $TX_SWING = VP$.

INT is the integer function in excel, equivalent to ROUNDDOWN.

Table 3-1 lists typical maximum TX swing values under the listed conditions.

Table 3-1 Typical Maximum TX Swing (Vswing) When txX_iboost_lvl = 4'b1111

Value of tx_vboost_lvl	Amplitude (mVppd)
3'000	719
3'001	781
3'010	843
3'011	905
3'100	967
3'101	1029
3'110	1091
3'111	1153

Table 3-2 lists the typical TX swing with the tx_vboost_vref default setting of "101" and vp = 0.75 V.

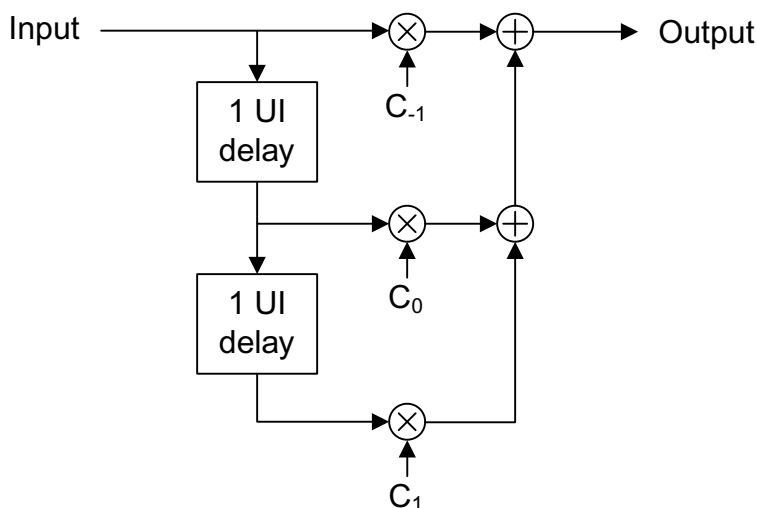
Table 3-2 Typical TX Launch Amplitude Vswing (mVppd)

Value of txX_iboost_lvl	Amplitude (mVppd)
4'b00xx	0.8665
4'b01xx	0.9147
4'b10xx	0.9685
4'b11xx	1.0290

3.2.2.4 TX Equalization

The TX driver is implemented with a 3-tap Feed-Forward Equalization (FFE), as shown in [Figure 3-7](#).

Figure 3-7 TX Driver Architecture



Where:

- C_0 , C_{-1} and C_1 are TX driver main tap, pre-cursor tap and post-cursor tap coefficients
- C_{-1} and C_1 are defined as negative values

The FFE coefficients C_0 , C_{-1} and C_1 are set by three control signals (txX_eq_main, txX_eq_pre, and txX_eq_post) by the following equations:

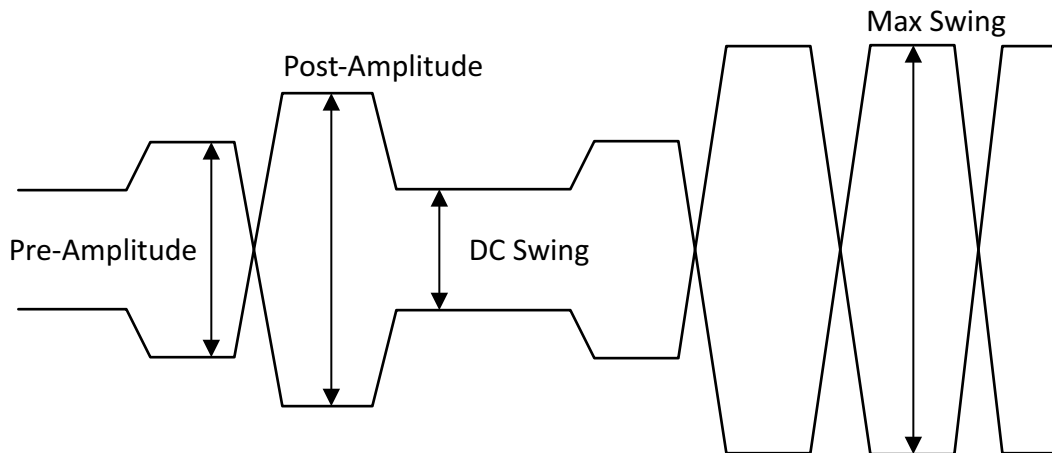
- $C_0 = \text{txX_eq_main}$
- $|C_{-1}| = \text{txX_eq_pre}$
- $|C_1| = \text{txX_eq_post}$



Note

The equations above and below use the absolute values of C_{-1} and C_1 .

TX Equalization with pre-amplitude, post-amplitude, and DC swing is illustrated in [Figure 3-8](#).

Figure 3-8 TX Equalization

To calculate the differential pre-amplitude, post-amplitude, and DC swing, use the following equations:

- $\text{pre_amplitude} = \text{Vswing} * (C_0 + |C_{-1}| - |C_1|)/48$
- $\text{post_amplitude} = \text{Vswing} * (C_0 - |C_{-1}| + |C_1|)/48$
- $\text{DC_swing} = \text{Vswing} * (C_0 - |C_{-1}| - |C_1|)/48$
- $\text{Max_Swing} = \text{Vswing} * (C_0 + |C_{-1}| + |C_1|)/48$

To maximize the output swing, set it so that $C_0 + C_{-1} + C_{+1} = 48$, which results in:

$$\text{Max_Swing} = \text{Vswing}$$

To attenuate output swing, set $C_0 + |C_{-1}| + |C_1|$ to be less than 48. For example, to get half swing output, set $C_0 + |C_{-1}| + |C_1| = 24$.

3.2.2.5 TX Rate Control

The TX high-speed clock is generated by the common clock module in the Support block. As listed in “MPLL” on page 41, the Support block provides a clock rate that can range from 8.0–12.890625 GHz. In addition, the TX has a flexible data rate control (configured by controlling txX_rate) that allows the TX to operate from 1.25 Gbps to a maximum of 24 Gbps.

The TX high-speed clock is always divided down to be half of the baud rate frequency. For example, if the clock provided by the Support block is 5 GHz, by setting the internal txX_rate signal to 3'b010, the clock is divided down by a factor of 4 to 1.25 GHz and the TX operates at 2.5 Gbps. For detailed PMA configuration settings for all supported protocols, refer to “PHY Usage and Configuration” on page 47.

3.2.3 Support Block

The Support block (see Figure 3-2 on page 32) provides all the common functions for the RX/TX link, TX clock generation, TX/RX termination calibration, biasing voltage, and reference clock buffering.

3.2.3.1 Reference Clock Input

The PHY requires a reference clock from either:

- An external pad input: `ref_pad_clk_p` or `ref_pad_clk_m`
- A core-side, on-chip input: `ref_alt_clk_p`

The external or core-side input is selected by `ref_use_pad`. When `ref_use_pad` is asserted, the external reference clock is used; otherwise, the on-chip reference source is used.

For implementation details, see “[Reference Clock Implementation](#)” on page 221.

3.2.3.2 Reference Clock Output

The Support block includes a reference clock repeat function. A current-mode logic (CML) differential reference clock is driven on `ref_repeat_clk_{p,m}` when `ref_repeat_clk_en` is asserted. This clock can be used as the reference clock source for other PHYs. For details, refer to “[Repeat Clock Specification](#)” on page 225.

3.2.3.3 MPLL

The MPLL generates the high-frequency clocks used by the lane TX. There are two MPLLs, MPLL A and MPLL B, which can be either operated individually or combined to support lane bifurcation. The `txX_mpllb_sel` signal allows you to select which MPLL to use.

The following are the key features of the MPLLs:

- For total output frequency ranges refer to “[Support and Lane Configuration Settings](#)” on page 235.
- Support for lane bifurcation (with two MPLLs enabled)
- Support for divide-by-fixed-16.5 in the output clock tree for MPLL A
- Support for fixed-frequency offset compensation

3.2.3.3.1 Configuring the MPLL

You need to configure the MPLL so it runs at the correct frequency for your application and protocol. In particular, you need to set the feedback ratio (Feedback N).

To configure the MPLL, perform the following:

1. Enable the correct MPLL (MPLLA or MPLLB), according to the application and the frequency range. For more about enabling the correct MPLL, see “[MPLL](#)” on page 41.
2. Use the configuration settings in “[PHY Usage and Configuration](#)” on page 47 to determine the correct setting of `mpll{a,b}_multiplier[11:0]`. The divider has a resolution of 0.25 input clock cycles and thus `mpll{a,b}_multiplier[0]` has a binary weighting of 0.25, `mpll{a,b}_multiplier[1]` has a weighting of 0.5, `mpll{a,b}_multiplier[2]` has a weighting of 1, and so on. Note that, in addition to the programmable divider settings, there is an additional divide-by-2 or divide-by-4 in the MPLL feedback path configured by the `mpll{a,b}_fb_clk_div4_en` pin.

$$F_{VCO} = \frac{F_{refclk}}{2^{ref_clk_mpll\{a,b\}_div[2:0]}} 2^{(1+mpll\{a,b\}_fb_clk_div4_en)} \left(mppll\{ab\}_multiplier[11:0] / 4 + \frac{mppll_frac_quot[15:0] + \frac{mppll_frac_rem[15:0]}{mppll_frac_den[15:0]}}{2^{17}} \right)$$

**Attention**

For any use of the fractional component in the MPLL VCO frequency reference equation, contact Synopsys customer support.

**Note**

When `mppll{a,b}_div_clk_en` is enabled, the frequency of `mppll{a,b}_div_clk` clock is equal to the `MPLL{A,B}` frequency divided by `mppll{a,b}_div_multiplier[7:0]`.

3.2.3.4 Termination Resistance Tuning

The PHY uses an external precision resistor to calibrate the termination impedances of the high-speed inputs and outputs of the PHY. This is done automatically by firmware across a range of impedance values such that the termination impedance can be set per-lane of the PHY.

On power-on-reset, firmware is responsible for performing RTUNE 8 times, once for each value of `SUP_ANA_TERM_CTRL`. The `SUP_ANA_TERM_CTRL` register controls the desired termination impedance from 40 to 54 ohm (in increments of 2 ohm). As a result, firmware is essentially performing RTUNE for each of the possible impedance values. The calibrated RTUNE values are then stored in the AON domain of the `RAW_PCS` in the `RAWCMN_DIG_AON_RTUNE_XXX_VAL_N` (where `N = 0 - 7`) registers.

After all of the values are stored, the firmware uses the per-lane overrides to set the impedance on a per-lane basis, according to the value of the `rxX/txX_term_ctrl[2:0]` inputs. The overrides used by firmware here are the `LANEN_DIG_ANA_TX/RX_TERM_CODE_OVRD_OUT` registers to write specific RTUNE values to each lane independently.

Note that there is only a single RTUNE module which sets the calibration codes for all lanes in parallel each time it is run. Lane overrides are available which allows you or the firmware to individually override lanes to the desired calibration values.

In addition, you can request the PHY to perform a re-calibration by asserting `rtune_req`. When this signal is asserted, the firmware will perform the same routine that is done on power-on-reset, and set the impedance on a per-lane basis, according to the value of the `rxX/txX_term_ctrl[2:0]` inputs. When the calibration is completed, the `rtune_ack` will be asserted.

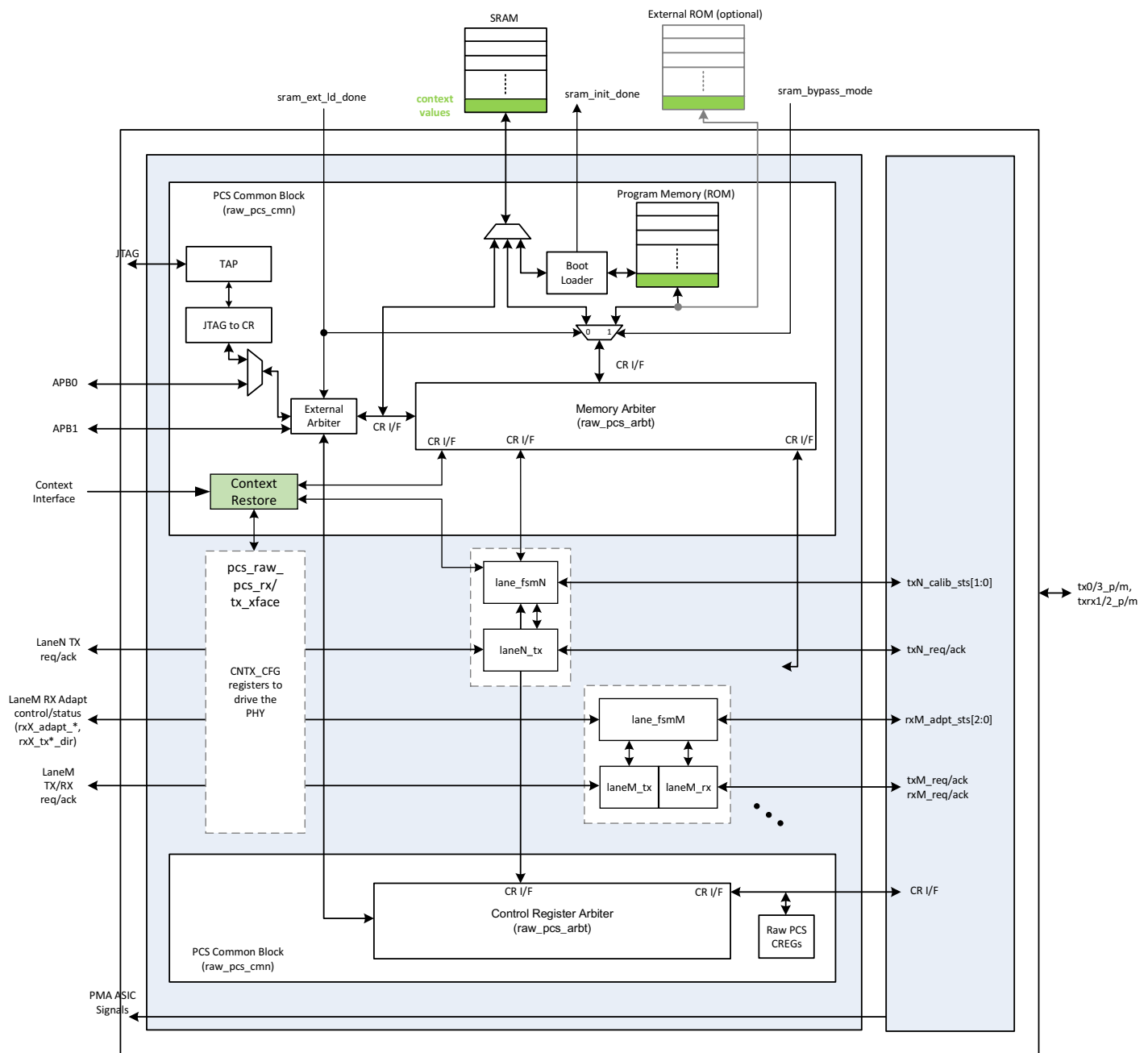
The termination calibration is not continuous, so the PHY has the ability to share a single external resistor using a handshake protocol across several PHYs. This is described in additional details in “[Physical-Level Implementation](#)” on page 183.

3.3 Raw PCS

The Raw PCS is a soft macro provided as Verilog RTL. The following features are implemented in the Raw PCS:

- Microprocessor (lane_fsm) to execute firmware from either an internal ROM, external ROM (optional), or external SRAM (mandatory). The lane_fsm executes the following routines:
 - Power-up calibration algorithms for the PMA analog front-end, for example, RX AFE offset cancellation, RX slicer calibration, MPLL calibrations, RX VCO calibrations, RX/TX duty-cycle correction (DCC) calibrations, and RX data/phase clock (IQC) calibrations.
 - Controls for the adaptation routines in the PMA for modes when adaptation is enabled.
- A context restore engine that reads context information from memory (either ROM or SRAM), and sets the PHY configuration based on the cmn/mplla/mpllb/txX/rxX/_cntx_sel[7:0] inputs.
- MPLL power-up and power down control based upon txX_pstate input
- Programmable RX CDR unlock detector
- JTAG Tap controller for serial register access that can operate at up to 100 MHz.
- Two APB interfaces (apb0/apb1) for parallel register access that can operate at up to 400 MHz. One interface (apb0) provides a high-speed synchronous mode, as well as a pipelined DMA mode for increased throughput.
- Register and memory access arbitration between JTAG and APB0 interfaces. The selection between JTAG and APB interfaces is controlled using primary input, apb0_if_sel (see section “[Control Register Interface](#)” on page 49), which means that both interfaces cannot operate simultaneously. A round-robin arbitration mechanism exists to provide equal priority to different sources requesting access to the shared internal register and memory buses.

The block diagram of the Raw PCS is shown in [Figure 3-9](#).

Figure 3-9 Raw PCS Block Diagram

The RAW_PCS module (dwc_mipi_mphy_type1_g5_pcs_2tx_2rx_ns) contains a microprocessor (lane_fsm) to execute the power-up calibration, runtime calibration, and adaptation algorithms. The algorithms are by default stored in an external ROM. Optionally, the data can be stored in a read-only register bank located in the common module (raw_pcs_cmnn) interface that can also be used to store the FSM firmware code. The external ROM interface is described in “[ROM Interface](#)” on page 69. The PHY also contains an SRAM interface. The firmware code is copied from the internal or external ROM on power-up (after de-assertion of phy_reset) to the SRAM where it can be manipulated before the FSM begins executing firmware. The SRAM interface is described in “[SRAM Interface](#)” on page 70.

The RAW_PCS module also contains finite state machines for powering up the TX and RX lanes and controls for generating interrupts and handshaking with the lane_fsm.

The context restore engine is responsible for reading context information from SRAM/ROM and applying the configuration to the PHY. The context restore engine does this by locking the arbiters to transfer data from memory to the PHY registers. It then passes top-level requests to the TX/RX lanes for processing.

The design includes two register access arbiters. The Memory arbiter is used for accessing the register bank whereas the Control Register Arbiter is used to access all other registers in the Raw PCS and the PMA. This mechanism provides efficient use of the registers and state machines within each lane where each lane can independently execute a different portion of the algorithm while reducing the register access bottleneck.

All control registers in the Raw PCS and PMA can be externally accessed using JTAG or either of the APB Interfaces, with only one interface at any given time.

PHY Usage and Configuration

This chapter explains the usage of PHY features and lists the PHY configuration settings for the protocols and data rates that are supported. It includes the following sections:

- [“PHY Power-Up” on page 48](#)
- [“Control Register Interface” on page 49](#)
- [“Firmware Clock Interface” on page 62](#)
- [“PHY Initialization Sequence” on page 64](#)
- [“ROM Interface” on page 69](#)
- [“SRAM Interface” on page 70](#)
- [“Context Restore” on page 74](#)
- [“MPLL Configuration Update” on page 80](#)
- [“Lane Configuration Update” on page 81](#)
- [“TX Datapath” on page 84](#)
- [“RX Datapath” on page 88](#)
- [“RX CDR PPM Detector” on page 90](#)
- [“RX Margining” on page 91](#)
- [“Fast Context Switching” on page 95](#)
- [“TX and RX Power States” on page 105](#)
- [“TX Equalization and Adaptation” on page 109](#)
- [“RX Equalization and Adaptation” on page 110](#)
- [“Test Modes” on page 112](#)
- [“Power Gating” on page 156](#)
- [“Configuration for a Disabled PHY” on page 165](#)
- [“Notes for Configuring the PHY” on page 166](#)
- [“Configuring the PHY” on page 167](#)

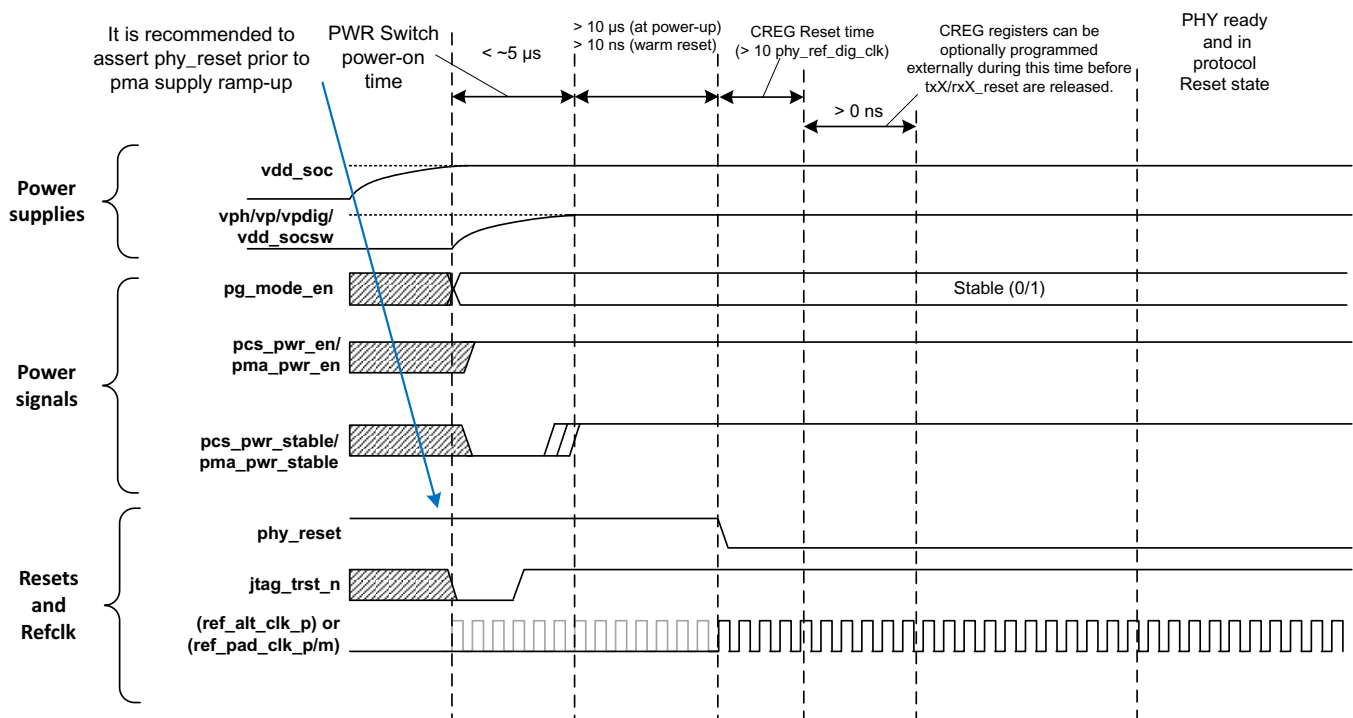
4.1 PHY Power-Up

Power up the vp^* and vph power supplies in any order and then proceed with the PHY reset and initialization sequence (vp^* indicates the grouping of all core supplies; vp and $vpdig$).

Note

- The state of the PHY is unknown after power up but before assertion of reset. The PHY may exhibit unwanted behavior such as TX enabled or RX terminations activated, for example.
- While the unknown state is undesirable, the PHY has been designed to prevent damage if it is left in this state.
- Supply ramp should be monotonic with a ramp time of no faster than $10\ \mu s$ and a rate no faster than $(\text{supply voltage})/10\ \mu s$.

Figure 4-1 Power-up Reset Timing



4.2 Control Register Interface

The control registers in the PHY can be accessed through four APB interfaces (APB0/1/2/3), or a serial test interface (JTAG). These interfaces are defined in the following sections.

The PHY register space supports broadcast writes, as follows:

- If you set `apbX_paddr[15:12] = 4'b0101`, PHY broadcasts to all PMA lanes
- If you set `apbX_paddr[15:12] = 4'b0110`, PHY broadcasts to all RAW PCS lanes
- If you set `apbX_paddr[15:12] = 4'b0111`, PHY broadcasts to all RAW PCS AON lanes

For details, see the “Register Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

Refer to the section “[SRAM Interface](#)” on page 70 for operation during SRAM update.

The address ranges for control registers are provided in [Table 4-1](#).



Note

RO - read-only, RW - read and write. RO/RW - the access varies for each register

Table 4-1 Address Ranges for Control Registers

Address Range	TYPE	Access	Description
0x0000 - 0x0FFF	PMA SUP, RAW CMN, RAW AON CMN	RO/RW	Register space of SUP module in PMA
0x1000 - 0x1FFF	PMA LANE		Register space of LANE instances in PMA
0x2000 - 0x2FFF	RAW LANE		Register space of LANE instances in PCS RAW
0x3000 - 0x3FFF	RAW AON LANE		Register space of AON LANE instances in PCS RAW
0x4000 - 0x4FFF	RESERVED		Reserved
0x5000 - 0x51FF	PMA LANE BCAST		Broadcast address to all LANE instances in PMA
0x6000 - 0x61FF	RAW LANE BCAST		Broadcast address to all LANE instances in PCS_RAW
0x7000 - 0x71FF	RAW AON LANE BCAST		Broadcast address to all AON LANE instances in PCS RAW

Address Range	TYPE	Access	Description
0x8000 - 0x8FFF	ROM_B0	RO	Internal/External ROM space (8 banks of up to 8k locations each). Note: Only one of EITHER ROM or SRAM can be accessed at a time as they share the same memory space. Internal/External ROM Address Range: 0x0000 – 0x7FFF
0x9000 - 0x9FFF	ROM_B1		
0xA000 - 0xAFFF	ROM_B2		
0xB000 - 0xBFFF	ROM_B3		
0xC000 - 0xCFFF	ROM_B4		
0xD000 - 0xDFFF	ROM_B5		
0xE000 - 0xEFFF	ROM_B6		
0xF000 - 0xFFFF	ROM_B7		
0x8000 - 0x8FFF	SRAM_B0	RW	Internal/External ROM space (8 banks of up to 8k locations each). Note: Only one of EITHER ROM or SRAM can be accessed at a time as they share the same memory space. SRAM Address Range: 0x0000 – 0x7FFF
0x9000 - 0x9FFF	SRAM_B1		
0xA000 - 0xAFFF	SRAM_B2		
0xB000 - 0xBFFF	SRAM_B3		
0xC000 - 0xCFFF	SRAM_B4		
0xD000 - 0xDFFF	SRAM_B5		
0xE000 - 0xEFFF	SRAM_B6		
0xF000 - 0xFFFF	SRAM_B7		

CREG Addressing Breakdown

- Within the PMA, RAW, and RAW AON lanes, bits [11:9] of the CREG address represent the lane number
- Bits [8:0] of the CREG address represent the register number within the type/lane

ROM/RAM Addressing

- The first 16k locations of the ROM/SRAM are reserved for firmware (0x0000 – 0x3FFF)
- The next 2k locations are reserved for the adaptation record results (0x4000 – 0x47FF)
- The final 2k locations are reserved for context configuration information (0x4800 – 0x4FFF)

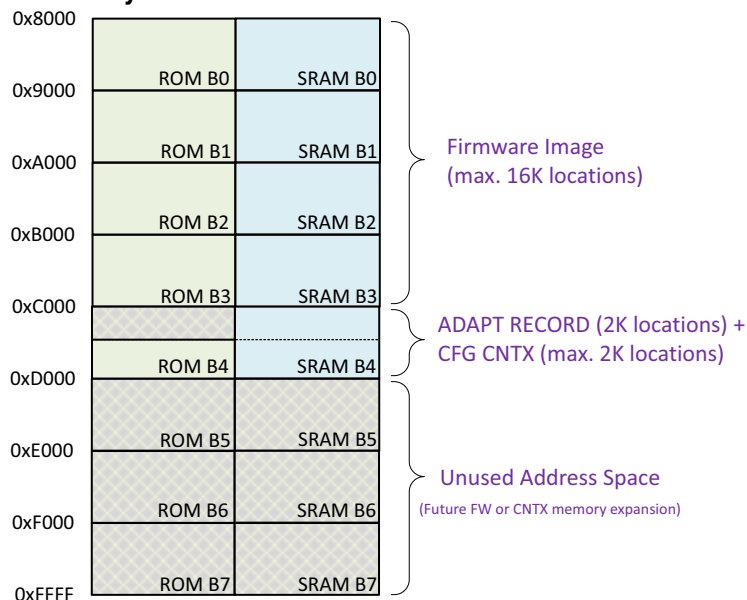
Figure 4-2 ROM and SRAM Memory Locations

Figure 4-2 shows that the internal/external ROM and the SRAM share the same memory locations. As a result, only one of either the internal/external ROM or SRAM can be accessed by either firmware or the external register access interfaces. This is controlled by either the `sram_bypass_mode` input, or the `CREG_ROM_ACCESS_SEL` register in the `RAW_CMN`. If `CREG_ROM_ACCESS_SEL` = 1'b1, the both firmware and context configuration information is read from the internal/external ROM. If `CREG_ROM_ACCESS_SEL` = 1'b0, then the firmware and context configuration information source is based on the `sram_bypass_mode[1:0]` input. If `sram_bypass_mode[0]` = 1'b1, firmware is read from the internal/external ROM. If `sram_bypass_mode[1]` = 1'b1, context configuration information is read from the internal/external ROM. This is summarized in Table 4-2 and Table 4-3.

Table 4-2 Configuration Information if `sram_bypass_mode[0]`

APB Address	<code>sram_bypass_mode[0]</code> or <code>CREG_ROM_ACCESS_SEL</code>	<code>phy0_sram_addr[14:0]</code>	<code>phy0_rom_addr[14:0]</code>
16'h8000 – 16'hC7FF (Firmware + ADAPT Record)	0	15'h0000 – 15'h47FF	NA
	1	NA	15'h0000 – 15'h47FF

Table 4-3 Configuration Information if `sram_bypass_mode[1]`

APB Address	<code>sram_bypass_mode[1]</code> or <code>CREG_ROM_ACCESS_SEL</code>	<code>phy0_sram_addr[14:0]</code>	<code>phy0_rom_addr[14:0]</code>
16'hC800 – 16'hCFFF (CFG CNTX MEM)	0	15'h4800 – 15'h4FFF	NA
	1	NA	15'h4800 – 15'h4FFF

4.2.1 APB Interface

For internal register access the PHY uses the APB interface. The APB interface is aligned to the AMBA 3 APB Protocol Specification v1.0.

The PHY supports up to four APB interfaces:

- APB0 is the primary APB port and it is expected to be connected to a system-level APB bus and used for debug access and for SRAM loading. Note that this is the only APB port that can be used for loading of SRAM during POR, and it is the only one that supports additional access modes (synchronous mode, DMA mode) described in this section.
- APB1 is an additional APB port that can also be used for debug access, but the primary intent of this port is to connect it to a controller so that it can perform protocol/rate specific accesses.
- APB2/3 are intended to be used by a higher level PCS (that is, uPCS) for performing PIPE message bus transactions.

For signals list, refer to “Signal Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

APB0 has two additional signals over and above the signals listed in APB specifications.

apb0_if_sel – As mentioned in signal description, this signal selects whether APB0 interface is used or JTAG

For using the APB0 interface, this signal must be tied to 1'b1.

- **apb0_if_mode** – This is a mode of operation selection signal.
- **apb0_if_mode[0]** – This is a strapping signal that must be set prior to phy_reset de-assertion. This signal indicates to the PHY whether **apb0_pclk** and **fw_clk** inputs to the PHY can be considered synchronous for timing closure perspective. When set to 1'b1, this results in shorter access times when PHY is using **fw_clk** as internal **cr_clk**.
- **apb0_if_mode[1]** – This is a synchronous (to **apb0_pclk**) control to indicate to the PHY whether the current transaction is a normal APB transaction or DMA transaction.

The APB interface signals are converted combinatorially through apb2cr bridge internal to the PHY. Each APB interface has its dedicated apb2cr bridge. The internal CR signals from each bridge are then connected to a dedicated **ext_arbt_ctl** block to convert the signaling from **cr_ext_clk** (**apbX_pclk**) domain to **cr_clk** (can be **phy_ref_dig_clk**, **fw_clk** or **apb0_pclk**) domain. The **ext_arbt_ctl** has two modes of operation: synchronous and asynchronous. For the APB1/2/3 interfaces, the **ext_arbt_ctl** always works in asynchronous mode. For APB0 interface, the mode of operation gets switched dynamically depending on the following specifications

- **cr_clk = phy_ref_dig_clk**
asynchronous mode
- **cr_clk = fw_clk**
 - **apb0_if_mode[0] = 1'b0**
asynchronous
 - **apb0_if_mode[0] = 1'b1**
synchronous
- **cr_clk = apb0_pclk**
synchronous

All control registers can be accessed through either the APB0/1/2/3 interface, internal context restore engine or the internal firmware FSM operating on the `cr_clk` domain. Note that the CR interface cannot be accessed during assertion of `phy_reset`. However, any registers that need to be written before powering up the PHY can be done after de-asserting `phy_reset` but before de-asserting the `rxX_reset`/`txX_reset` signal.

A GLCM exists between the external `apb0_pclk` (or `jtag_tck`) and the internal reference clock for accessing registers inside the PHY. This GLCM is controlled by the `ref_clk_en` signal and always uses the internal reference clock when `ref_clk_en` is asserted. As a result, note that the APB interface latency might change when performing APB interface accesses while changing from one clock source to another.

When accessing the APB interface with the reference clock enabled, you must wait at least 2.4 μ s to allow the prescaler to power up, plus at least 50 reference clock cycles after `phy_reset` is de-asserted.

Alternatively, you can wait for the `phy_ref_dig_clk` clock to start toggling. This waiting time is required for the clock to propagate through the internal PHY de-glitching circuit and to de-assert internal resets.

When accessing the APB interface with the reference clock disabled, you must wait until at least ten external CR clock cycles have elapsed (`apb0_clk` or `jtag_tck`) after `phy_reset` has been de-asserted. In this case, the external clock needs to de-assert only internal resets.

Whenever any APB interface is accessed (regardless of whether or not the reference clock is enabled), the expectation is that the `apbX_pclk` clock is active for at least five cycles before the start of the APB transaction, and for at least five cycles after the end of the APB transaction.

All APB interfaces have a timeout of 1023 `apbX_pclk` cycle. If the transaction is not complete within this time, `apbX_pready` with `apbX_pslverr` will be generated. You can try the same transaction again if a `apbX_pslverr` is detected.

Below is a list of PHY operational states which may have different limitations for APB transactions:

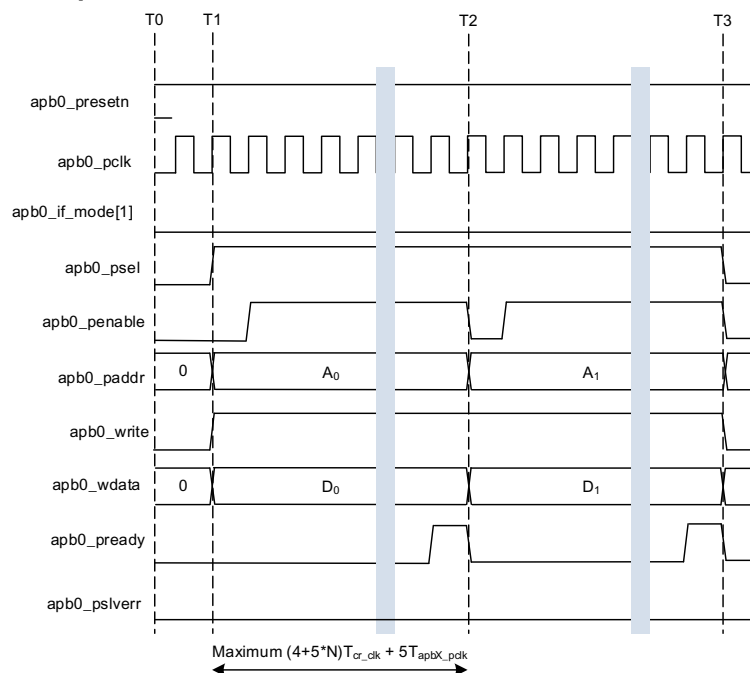
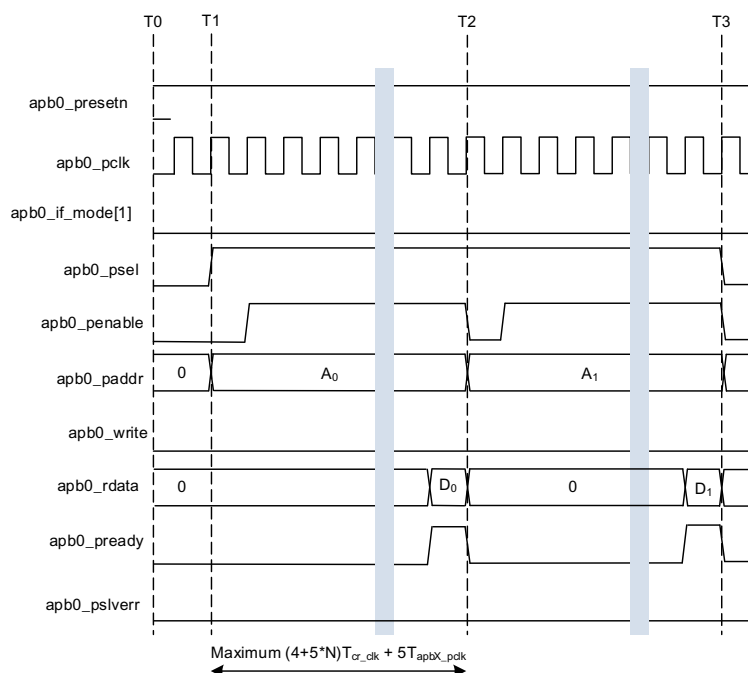
- For modes where `ref_clk_en` is low (except power-gated modes)
 - In this case, the FW and the context restore engine will not be operational
 - `cr_clk` will be sourced from either `apb0_pclk` or `jtag_clk`
 - For accesses through the APB1/2/3 interfaces the `apb0_pclk` or `jtag_clk` must be toggling
 - It is recommended not to perform any APB1/2/3 accesses
 - For APB0, both DMA mode and normal mode accesses can be done
- For power-gated modes
 - In this case, the FW and the context restore engine will not be operational
 - `cr_clk` will be sourced from either `apb0_pclk` or `jtag_clk`
 - No APB accesses can be done in this mode
 - For debug purpose, `pg_mode_en` can be de-asserted to bring PHY out of power-gated mode. Wait for the `pma/pcs_pwr_stable` assertion followed by at least ten `apb0_clk` or `jtag_clk` cycles before initiating a transaction
 - After completing the transactions, the `pg_mode_en` can be asserted again to re-enter PG modes
- At POR, after `ref_clk_en` assertion and before `sram_init_done` assertion
 - In this case, the FW and the context restore engine will not be operational
 - `cr_clk` will be sourced from either `phy_ref_dig_clk` or `fw_clk`
 - No APB accesses can be done in this mode

- The PHY bootloads the SRAM with internal/external ROM contents if the SRAM is present and the bootloading is not bypassed
- At POR, after `sram_init_done` assertion and before `sram_ext_ld_done` assertion
 - In this case, the FW and the context restore engine will not be operational
 - `cr_clk` will be sourced from either `apb0_pclk` or `jtag_clk`
 - APB0 can be used to load SRAM or read and write CREGs. APB1/2/3 cannot be used during this time.
 - For APB0, both DMA mode and normal mode accesses can be done
- During any other mission mode
 - In this case, the FW and the context restore engine will be operational
 - `cr_clk` will be sourced from either `phy_ref_dig_clk` or `fw_clk`
 - APB accesses can be done during this period on all the interfaces
 - For APB0, no DMA mode access is allowed in this period
 - The APB accesses will have the maximum turnaround times in this mode

The timing diagrams of the APB register write and read operations are shown in [Figure 4-3](#) and [Figure 4-4](#), respectively. In these diagrams, "N" corresponds to sum number of lanes in the PHY, number of active APB interfaces (1-4) and one context restore engine; for example, for a four-lane PHY, $N=4+(1\text{to}4)$. Parameters T_{ref} and T_{cr} are defined below:

- T_{ref} : Time Period of `phy_ref_dig_clk`/`fw_clk` clock, with a minimum value of 2.5 ns and a maximum value of 52.08 ns.
- T_{cr} : Time period of the `apb0_pclk`, with a minimum value of 2.5 ns and a maximum value of 52.08 ns.

The completion of a write and read cycle is acknowledged by the assertion of the `apbX_pready` output. The maximum times taken by write and read cycles are provided in [Figure 4-3](#) and [Figure 4-4](#).

Figure 4-3 APB Register Write Operation**Figure 4-4 APB Register Read Operation**

The PHY APB0 interface has a proprietary mode of operation to achieve pipe-lined single cycle accesses for fast register access under certain operational conditions.

These operational conditions can be categorized as:

- SRAM external load phase

■ CPM modes

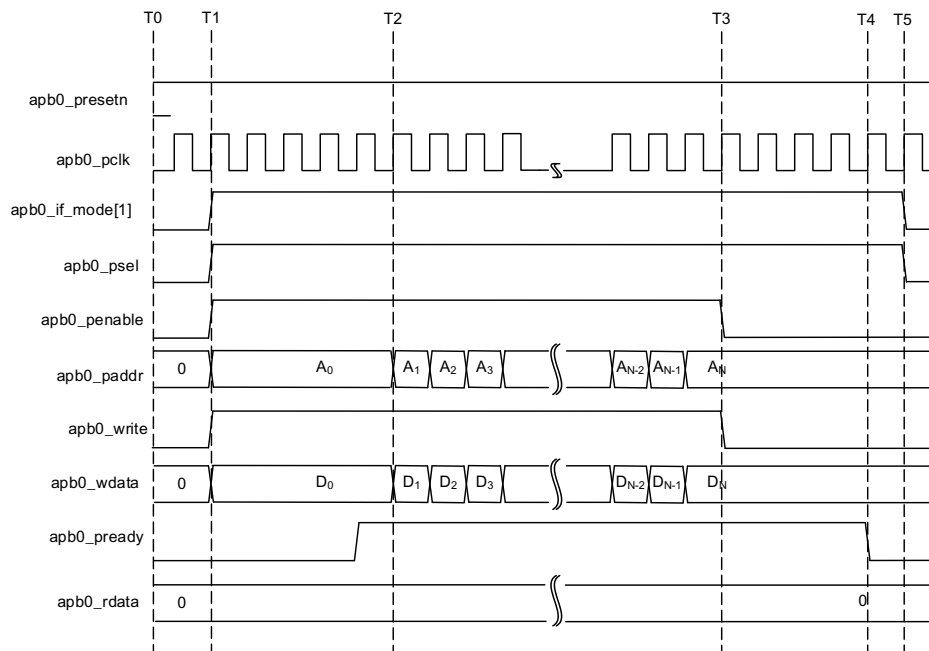
Important Usage Notes

- DMA mode is applicable for APB0 interface only.
- `apb0_if_mode` definition
 - `apb0_if_mode[0]` cannot change dynamically. It is a static control option that must be set correctly before `phy_reset` de-assertion. This signal indicates to the PHY whether `apb0_pclk` and `fw_clk` can be considered synchronous for STA timing closure.
 - `apb0_if_mode[1]` is a dynamic signal that determines the DMA mode (1). This signal shall be stable when `apb0_psel` is high.
 - Setting the `apb0_if_mode[0]` as 1 or 0 has no impact on the DMA mode use case in operational scenarios where DMA mode is used.
- A DMA mode access to the memory address locations and register address locations cannot be done in single access. The memory arbiter and register arbiter have an imbalance of two cycles for access times.

DMA mode memory write access

Figure 4-5 shows the APB0 interface signals for DMA mode memory write access.

Figure 4-5 DMA Mode Memory Write Access



Below is the description of the DMA mode Memory address write sequence:

1. T0: APB0 interface in quiescent state.
2. T1: The SoC logic will assert `apb0_if_mode[1]`, `apb0_psel` and `apb0_penable` and apply the `apb0_paddr` with `apb0_wdata` and `apb0_write` (if a write DMA access) synchronous to `apb0_pclk`.
3. T2: For Memory address DMA mode access at the fifth `apb0_pclk` cycle the SoC logic will detect the `apb0_pready` as high, declaring the success of the first write transaction.

4. T3: SoC logic declares the end of transaction by de-asserting `apb0_penable`. And waits for `apb0_pready` to go low.
5. T4: PHY de-asserts `apb0_pready` after four `apb0_pclk` cycles.
6. T5: SoC logic declares the end of the DMA mode access by de-asserting `apb0_psel` and `apb0_if_mode[1]`.

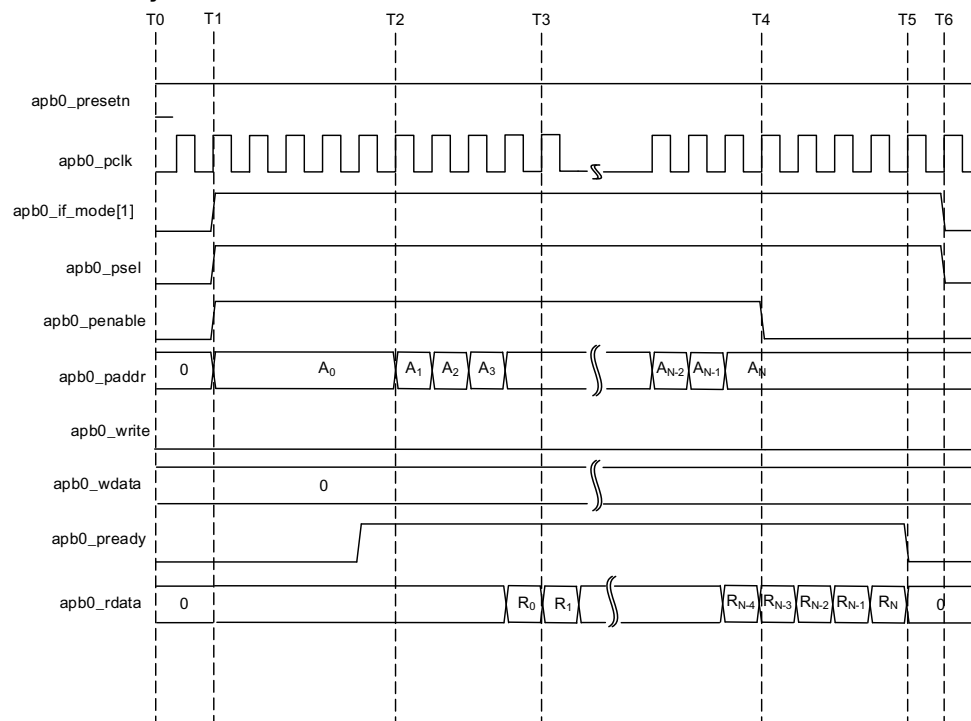
**Note**

The address A0, A1, A2 and so on, shown in the waveform, do not have to be consecutive address and can be random locations in the register address space or memory address space.

DMA mode memory read access

Figure 4-6 shows the APB0 interface signals for DMA mode memory read access.

Figure 4-6 DMA Mode Memory Read Access



Below is the description of the DMA mode Memory address read sequence:

1. T0: APB0 interface in quiescent state.
2. T1: The SoC logic will assert `apb0_if_mode[1]`, `apb0_psel` and `apb0_penable` and apply the `apb0_paddr` with `apb0_wdata` and `apb0_write` (if a write DMA access) synchronous to `apb0_pclk`.
3. T2: For Memory address DMA mode access at the fifth `apb0_pclk` cycle the SoC logic will detect the `apb0_pready` as high, declaring that address for the next read can be updated. The read data on `apb0_rdata` for the A0 address is still not available.
4. T3: SoC logic can start reading the `apb0_rdata` bus. The read data at this edge corresponds to the first address A0.

5. T4: SoC logic declares the end of transaction by de-asserting `apb0_penable`. And waits for `apb0_pready` to go low. Note that valid read data is still being updated by the PHY until de-assertion of `apb0_pready`.
6. T5: SoC logic receives the last valid read data output on `apb0_rdata`.
7. T6: SoC logic declares the end of the DMA mode access by de-asserting `apb0_psel` and `apb0_if_mode[1]`.

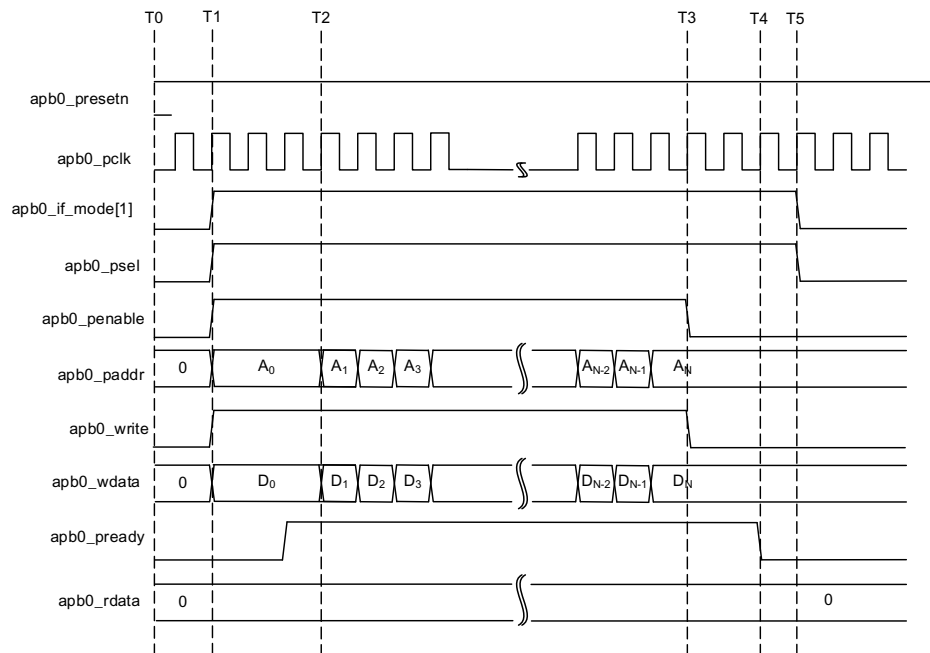
**Note**

The address A0, A1, A2 and so on, shown in the waveform, do not have to be consecutive address and can be random locations in the register address space or memory address space.

DMA mode register write access

Figure 4-7 shows the APB0 interface signals for DMA mode register write access.

Figure 4-7 DMA Mode Register Write Access



Below is the description of the DMA mode Memory register write sequence:

1. T0: APB0 interface in quiescent state.
2. T1: The SoC logic will assert `apb0_if_mode[1]`, `apb0_psel` and `apb0_penable` and apply the `apb0_paddr` with `apb0_wdata` and `apb0_write` (if a write DMA access) synchronous to `apb0_pclk`.
3. T2: For register address DMA mode access at the third `apb0_pclk` cycle the SoC logic will detect the `apb0_pready` as high, declaring the success of the first write transaction.
4. T3: SoC logic declares the end of transaction by de-asserting `apb0_penable`. And waits for `apb0_pready` to go low.
5. T4: PHY de-asserts `apb0_pready` after two `apb0_pclk` cycles.
6. T4: SoC logic declares the end of the DMA mode access by de-asserting `apb0_psel` and `apb0_if_mode[1]`.

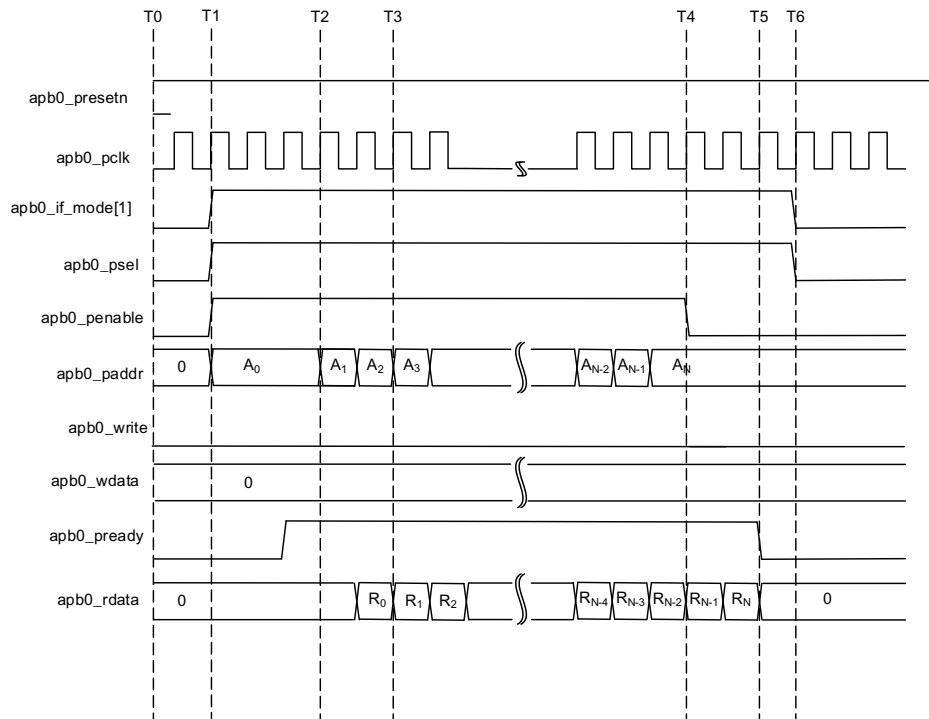
**Note**

The address A0, A1, A2 and so on, shown in the waveform, do not have to be consecutive address and can be random locations in the register address space or memory address space.

DMA mode register read access

Figure 4-8 shows the APB0 interface signals for DMA mode register read access.

Figure 4-8 DMA Mode Register Read Access



Below is the description of the DMA mode Memory register read sequence:

1. T0: APB0 interface in quiescent state.
2. T1: The SoC logic will assert **apb0_if_mode[1]**, **apb0_psel** and **apb0_penable** and apply the **apb0_paddr** with **apb0_wdata** and **apb0_write** (if a write DMA access) synchronous to **apb0_pclk**.
3. T2: For Register address DMA mode access at the third **apb0_pclk** cycle the SoC logic will detect the **apb0_pready** as high, declaring that address for the next read can be updated. The read data on **apb0_rdata** for the A0 address is still not available.
4. T3: SoC logic can start reading the **apb0_rdata** bus. The read data at this edge corresponds to the first address A0.
5. T4: SoC logic declares the end of transaction by de-asserting **apb0_penable**. And waits for **apb0_pready** to go low. Note that valid read data is still being updated by the PHY until de-assertion of **apb0_pready**.
6. T5: SoC logic receives the last valid read data output on **apb0_prdata**.
7. T6: SoC logic declares the end of the DMA mode access by de-asserting **apb0_psel** and **apb0_if_mode[1]**.

 **Note**

The address A0, A1, A2 and so on, shown in the waveform, do not have to be consecutive address and can be random locations in the register address space or memory address space.

4.2.2 JTAG Interface

The JTAG interface in the PHY provides access to an internal TAP controller which implements the CRSEL instruction used for writing and reading registers in the PHY. The JTAG interface is enabled when input cr_para_sel is de-asserted to 0.

There is one JTAG interface per PHY, so for configurations where there are multiple PHYs aggregated (instantiated), there is more than one JTAG interface. To control these with a single TAP controller, these multiple interfaces can be daisy chained externally.

4.2.2.1 IR Codes

Table 4-4 describes the supported JTAG instructions. The LSB of the IR and DR registers is the first bit shifted into jtag_tdi input.

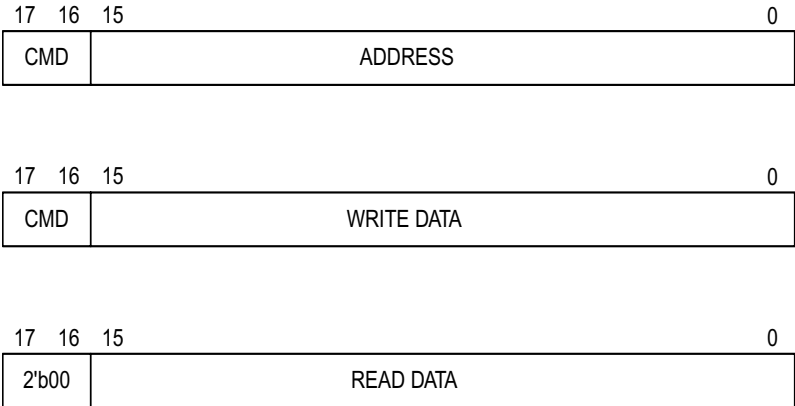
Table 4-4 Supported JTAG Instructions

Instruction	IR	DR	Operation
IDCODE	8'h01	32 bits	Shift out ID code. The 32-bit IDCODE can be found in the following registers: {SUP_DIG_IDCODE_HI, SUP_DIG_IDCODE_LO}.
CRSEL	8'h31	18 bits	Control register access
BYPASS	All others	1 bit	jtag_tdi to jtag_tdo bypass (bit is preloaded to 1)

4.2.2.2 CRSEL Instruction

When operating with the CRSEL instruction, internal registers can be accessed through the JTAG interface. When CRSEL instruction is selected, the shifted-in contents of the DR register are loaded into the CRSEL register during the UPDATE-DR phase of the JTAG state machine. The CRSEL register mapping when CRSEL instruction is selected as shown in Figure 4-9.

Figure 4-9 CRSEL Register Mapping



The CMD field determines what type of access will be performed. [Table 4-5](#) shows the encodings which are defined for CMD field of the CRSEL register.

Table 4-5 Command Field Encodings for the CRSEL Register

CMD	Encoding	Description
ADDR	2'b00	Capture ADDRESS field for use in a subsequent WRITE command. During subsequent SHIFT-DR state, shifts current value from most recently accessed register (via READ or WRITE) out on jtag_tdo.
WRITE	2'b01	Write the value in the WRITE DATA field to the register selected by the address captured during the most recent ADDR or READ command. During subsequent SHIFT-DR state, shifts current value from most recently accessed register (via READ or WRITE) out on jtag_tdo.
DATA	2'b10	During subsequent SHIFT-DR state, shifts current value from most recently accessed register (via READ or WRITE) out on jtag_tdo. WRITE DATA value shifted in is unused.
READ	2'b11	Perform a read operation to the register selected by the ADDRESS field to make its current value available for subsequent CRSEL accesses. During subsequent SHIFT-DR state, shifts current value from most recently accessed register (via READ or WRITE) out on jtag_tdo.

To perform a control register read, shift a READ command into the CRSEL register along with the desired register address in the ADDRESS field. Any subsequent CRSEL access causes the data returned by the READ to be shifted out on jtag_tdo. This allows a limited form of pipelining of READ commands, as the second access may be shifting in the command and address for the subsequent READ while shifting out the data from the previous READ. To ensure reliable read operation, a minimum wait of 100 jtag_tck cycles in the run-test-idle state is needed. This wait is required for the register access to be transferred across clock domains and received by the JTAG controller successfully.

To perform a control register write, the ADDRESS of the register to be written must first be selected by performing either an ADDR or READ command and specifying the register address in the ADDRESS field. A second access using the WRITE command and specifying the data to be written in the WRITE DATA field causes the write to finally be performed to the desired register.

4.3 Firmware Clock Interface

The PHY has a feature to request for a “firmware clock” through the `fw_clk` input that is used by the SRAM bootloader and the internal PHY firmware to reduce latency in those blocks. This is an optional feature that will allow the SRAM bootloader and the internal PHY firmware to run on the `fw_clk` at up to 400 MHz, instead of using the input reference clock which is the default behavior. This will result in reduced latency for the following operations:

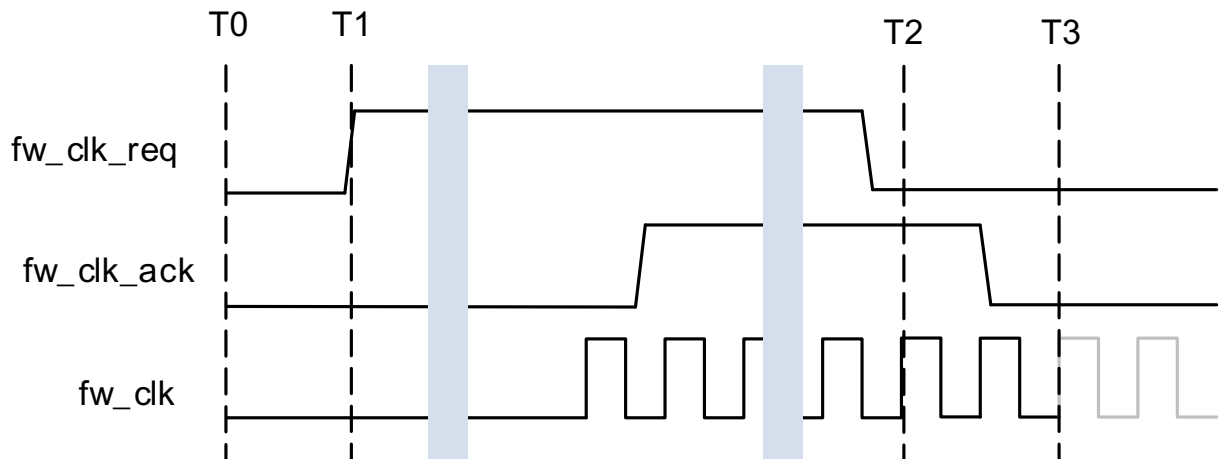
- SRAM bootloading
- POR calibrations
- Power-gating exit
- Initial rate changes to a specific rate that requires TX/RX calibrations
- RX adaptation
- RX margining

The following signals at the PHY interface are used to request a clock on the `fw_clk` input.

- `fw_clk_req`
 - This output port is driven to a value of 1'b1 when the PHY is requesting the `fw_clk` to be available. This is done during SRAM bootloading, or when the PHY is servicing a request.
 - This should be considered as an asynchronous signal.
- `fw_clk_ack`
 - a. This input port declares to the PHY that `fw_clk` is now available and toggling.
 - b. This shall be asserted when `fw_clk` is already toggling.
 - c. This signal shall be de-asserted after `fw_clk_req` is de-asserted.
 - d. The `fw_clk` shall remain toggling until `fw_clk_ack` is kept asserted.
 - e. The `fw_clk_ack` shall be asserted/de-asserted within ten `fw_clk` cycles after assertion/de-assertion of `fw_clk_req`.
 - f. If the feature is not required, then this input can be tied to 1'b0.
- `fw_clk`
 - This input port shall be driven from a clock generator to provide a clock frequency from 100 MHz to 400 MHz.
 - The clock shall always be glitch free when it starts or stops toggling.

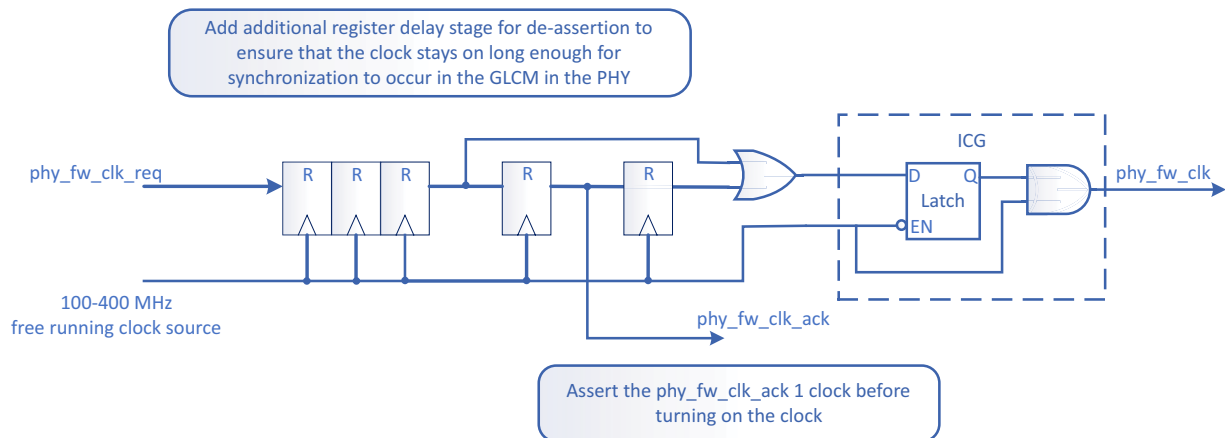
Internally GLCM selects between the `phy_ref_dig_clk` and `fw_clk` to generate `cr_int_clk`. Another GLCM selects between the `cr_int_clk` and `cr_ext_clk` (`apb0_pclk` or `jtag_clk`) to generate `cr_clk`. It is ensured in the design that both GLCMs will not switch together.

Figure 4-10 shows the timing diagram for this interface.

Figure 4-10 Firmware Clock Interface


This feature is optional and is not required for proper PHY operation. The feature has the benefit of reducing latency when the SRAM bootloader or PHY firmware is in use. If this feature is not being used, the `fw_clk_ack` input should be tied to 1'b0 which will ensure that the internal PHY GLCM is never switched to the `fw_clk` input and operates only on the internal reference clock.

If this feature is being used, the expectation is that the `fw_clk` is derived from a free-running clock and the `fw_clk_req`/`fw_clk_ack` signals are used as control signals to gate/ungate the free-running clock. This is because of the low latency desired on the `fw_clk_ack` signal, such that the `fw_clk` will be available when it is needed. Synopsys does not recommend using the `fw_clk_req` signal to power up and down a separate PLL for example, where the latency between `fw_clk_req` and `fw_clk_ack` will be large. [Figure 4-11](#) shows an implementation example for the firmware clock interface feature.

Figure 4-11 Implementation Example for the Firmware Clock Interface Feature


4.4 PHY Initialization Sequence

During PHY reset and initialization sequence, the PHY performs power-up circuit calibrations and enters the configuration mode defined by the primary inputs. The PHY initialization timing diagram when SRAM is disabled is shown in [Figure 4-12](#). The PHY initialization timing diagram when SRAM is enabled is shown in [Figure 4-13](#).

The PHY does not require any clocks to be supplied to enter the reset state (assertion of `phy_reset`). Exit from the reset state is indicated by de-assertion of `txX_ack/rxX_ack`, and requires a reference clock to be supplied and `ref_clk_en` input to be asserted. The maximum time from `phy_reset` de-assertion to de-assertion of `txX_ack/rxX_ack` can be as high as 5 ms in fast simulation mode and as high as 15 ms in normal simulation mode, because offset calibration is not modeled.

When `phy_reset` is asserted, the TX and RX terminations are disabled and `txX_p/m` are tri-stated for all lanes of the PHY. In addition to global `phy_reset`, `txX_reset/rxX_reset` inputs can be used to reset individual lanes in the PHY. When `txX_reset` is asserted, the `txX_pstate` is similar to P2, and `txX_p/m` are tri-stated for lane X. When `rxX_reset` is asserted, the `rxX_pstate` is similar to P2. Keeping either `txX_reset` or `rxX_reset` asserted after `phy_reset` is de-asserted prevents the RAW_PCS FSM from beginning MPLL/TX/RX calibrations. This allows for internal registers to be updated until the `txX_reset` and `rxX_reset` signals are de-asserted.

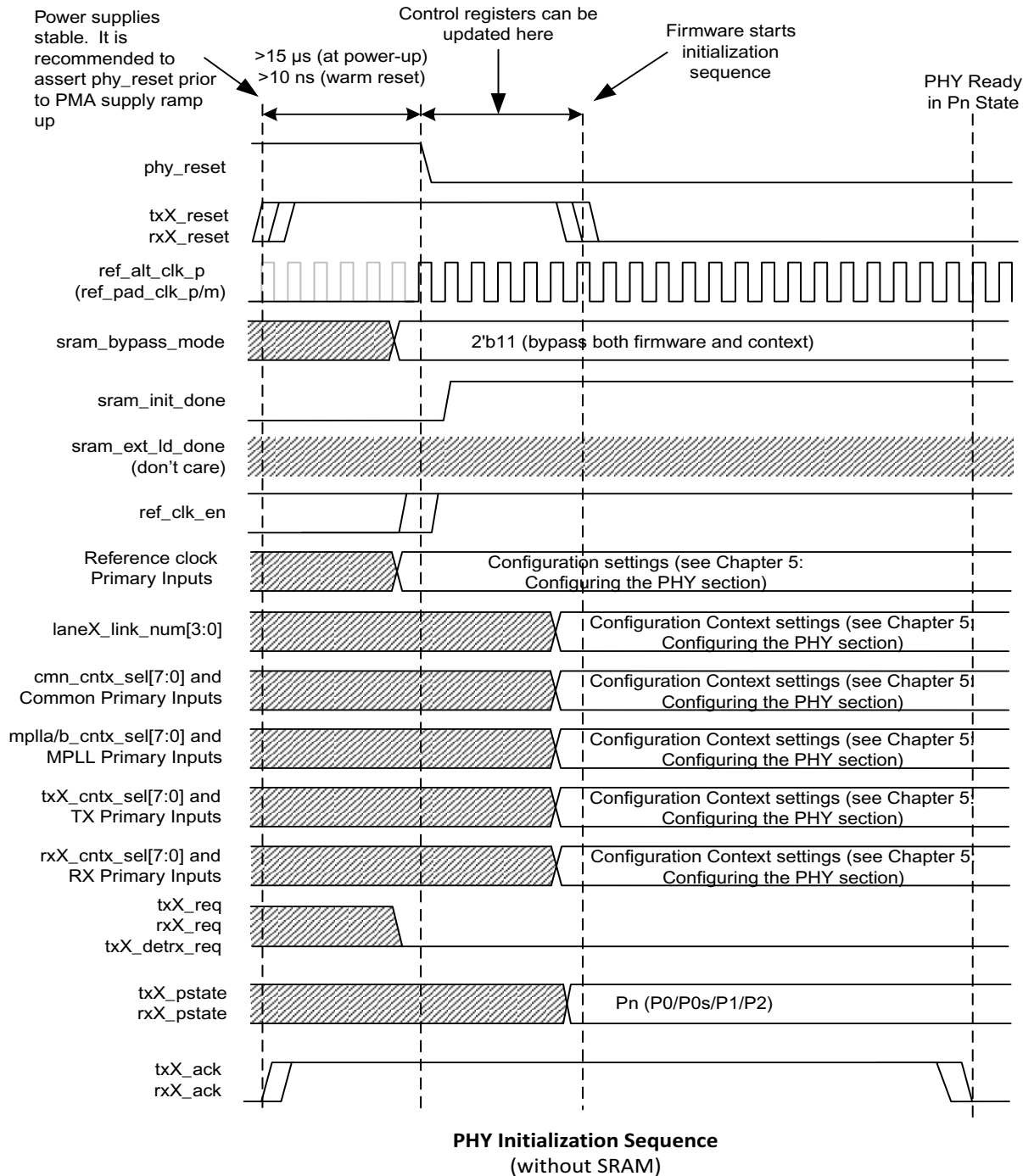
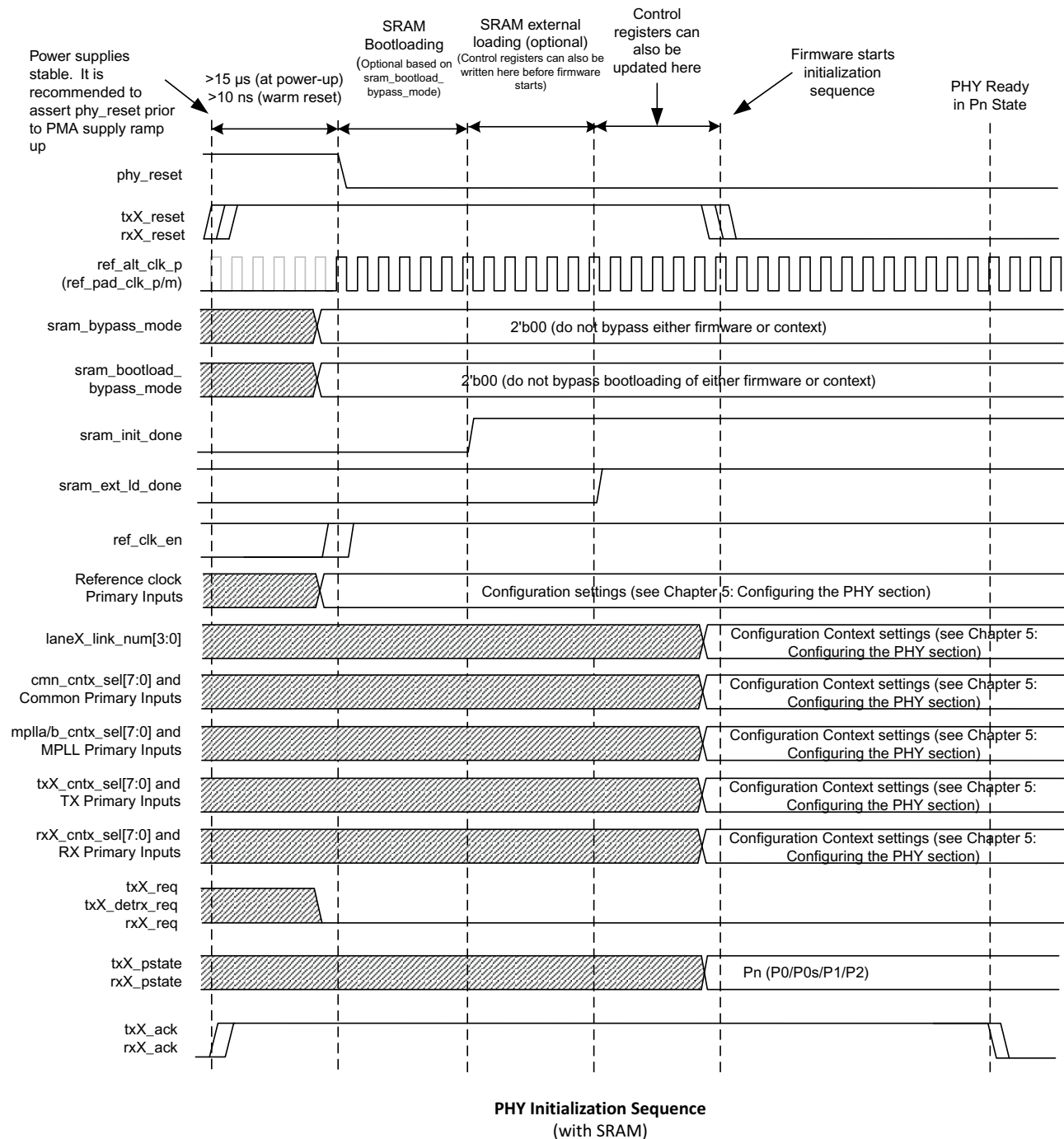
Figure 4-12 PHY Initialization Sequence with SRAM Disabled


Figure 4-13 PHY Initialization Sequence when SRAM is Enabled

Note ref_clk_en must stay asserted, irrespective of txX_reset/rxX_reset, if any of the lanes require the reference clock input to be active.

The PHY reset/initialization steps are as follows:

1. Assert reset (`phy_reset = 1`), and optionally `txX/rxX_reset`.
The lane-acknowledge outputs are asserted during reset (`txX_ack = rxX_ack = 1`).
2. Set the reference clock control signals to the desired configuration as defined in “Configuring the PHY” on page 167. Additional details on setting the reference clock control signals can be found below.
3. De-assert reset (`phy_reset = 0`). The `phy_reset` needs to be asserted for a minimum of 15 μ s after power-supplies are ramped-up (power-on). For any subsequent `phy_reset` assertions (warm-reset), `phy_reset` needs to be asserted for a minimum of 10 ns.
4. Perform optional SRAM loading tasks. These tasks must be performed if SRAM is enabled (for example, `sram_bypass_mode != 2'b11`) and are outlined below:
 - a. The PHY will optionally perform bootloading of the firmware and context memory based on the value of the `sram_bootload_bypass_mode` signal and will assert `sram_init_done`.
 - b. Update SRAM and internal configuration registers as required and assert `sram_ext_ld_done`. Note that it is recommended to perform any configuration register updates at this time before firmware has begun execution after `sram_init_done = sram_ext_ld_done = 1`.



Note

The `sram_init_done` signal will always assert regardless of the value of the `sram_bypass_mode` and `sram_bootload_bypass_mode` signals.

5. Set the common, MPLLA, MPLLB, RX and TX settings to the desired configuration as defined in “Configuring the PHY” on page 167. Additional details on setting the common, MPLLA, MPLLB, RX and TX signals can be found below.
6. Wait for the lane acknowledge outputs to de-assert (`txX_ack = rxX_ack = 0`). At this point, all the desired inputs set in Steps 2-5 have been applied in the PHY.

4.4.1 Reference Clock Control Signals

The reference clock control signals are those signals that control the reference clock into and out of the PHY. These need to be set before the de-assertion of the `phy_reset` (assuming that `ref_clk_en` is asserted) to ensure the reference clock is operating correctly when exiting `phy_reset`. The reference clock control signals are listed below:

- `ref_use_pad`
- `ref_range`

Note that if `ref_clk_en` is not asserted when `phy_reset` is de-asserted, these must be set before asserting `ref_clk_en`.

4.4.2 Common Control Signals

The common control signals are those signals that control features that are common across lanes. These signals must be set before de-asserting the lane reset signals (`txX/rxX_reset`) to ensure they are set prior to the configuration context module being triggered. The common control signals are listed below:

- `laneX_link_num`
- `cmn_cntx_sel`

4.4.3 MPLL Control Signals

The MPLL control signals are those signals that control the configuration of the MPLLs, MPLLA and MPLLB. These signals must be set before de-asserting the lane reset signals (txX/rxX_reset) to ensure they are set prior to the configuration context module being triggered. Note that MPLLA and MPLLB can be controlled independently. The MPLL control signals are listed below:

- mpla/b_cntx_sel
- mpla/b_force_en
- mpla/b_init_cal_disable
- mpla/b_recal_force_en
- mpla/b_recal_skip_en
- mpla/b_recal_bank_sel
- mpla/b_ssc_en

4.4.4 TX Control Signals

The TX control signals are those signals that control the configuration of the transmitter. These signals must be set before de-asserting the lane reset signals (txX/rxX_reset) to ensure they are set prior to the configuration context module being triggered. The TX control signals are listed below:

- txX_cntx_sel
- txX_lpd
- txX_pstate
- txX_mpll_en
- txX_recal_force_en
- txX_recal_skip_en
- txX_lane2lane_dskw_en
- txX_hp_prot_en

4.4.5 RX Control Signals

The RX control signals are those signals that control the configuration of the receiver. These signals must be set before de-asserting the lane reset signals (txX/rxX_reset) to ensure they are set prior to the configuration context module being triggered. The RX control signals are listed below:

- rxX_cntx_sel
- rxX_lpd
- rxX_pstate
- rxX_recal_bank_sel
- rxX_recal_force_en
- rxX_recal_skip_en
- rxX_term_acdc

4.5 ROM Interface

The Raw PCS stores the instruction code to execute the calibration and adaptation algorithms as well as the context configuration for the PHY in an external ROM by default to minimize the area of the Raw PCS. This also permits late-stage GDS swaps or metal-only ECO's to be performed on an external ROM if an update is required, without having to modify the synthesized RAW_PCS RTL.

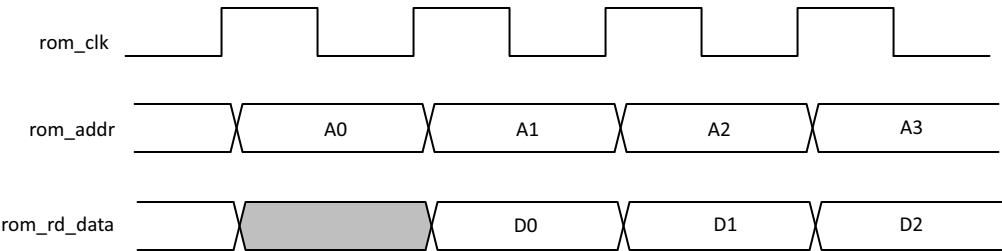
Optionally, this information can be stored in the form of a hard-coded lookup table within the RAW_PCS. To enable the internal lookup table-based ROM with the RAW_PCS, the `DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_NO_EXT_ROM_SUPPORT` macro should be defined. Note that Synopsys recommends using an external ROM.

 **Note**

Mandatory ROM size is a 20K single-port ROM consisting of 16-bit locations.

The following figure provides a timing diagram for the ROM interface.

Figure 4-14 ROM Interface Timing



4.5.1 Unused ROM Locations

There are specific addresses that are intentionally unused by the PHY within the ROM that can be used by the SoC to store certain values for test features such as checksums or MISR (multiple input shift register) values. These locations are provided in [Table 4-6](#).

Table 4-6 Unused ROM Locations

EXT ROM Macro	PHY ROM 15-bit address	APB 16-bit address	Notes
20k x 16-bit	0x3FFF	0xBFFF	When using a single 20k ROM macro, the address location is at the end of the 16k firmware memory map
16k * 16bit	0x3FFF	0xBFFF	When using two separate ROM macros, separate address locations can be used. The first is at the end of the 16k firmware memory map, and the other is at the start of the context memory map
4k * 16-bit	0x4000	0xC000	

Note that when attempting to read from ROM with SRAM enabled, the `RAWCMN_DIG_CREG_ACCESS_CTL` register must be set accordingly. Refer to [“Control Register Interface”](#) on page 49 for more information.

4.6 SRAM Interface

To provide flexibility in updating the firmware code or the contents of the context memory in silicon, an external SRAM interface is provided. The external SRAM is enabled based on the `sram_bypass_mode[1:0]` signal. Details of this signal are listed below:

- `sram_bypass_mode[0]` = Controls the bypassing of SRAM for PHY firmware.
- `sram_bypass_mode[1]` = Controls the bypassing of SRAM for context information.

When either bit is set, the RAW_PCS will use the ROM (either internal or external), instead of the SRAM as its source.

When `sram_bypass_mode` = 2'b00 (default setting), the de-assertion of `phy_reset` will trigger the bootloader in the RAW_PCS to load the code from the internal look up table or external ROM into the external SRAM. Once this process is complete, the PHY will assert the `sram_init_done` signal. Note that for the `sram_init_done` to assert, there are two possible cases:

- `sram_ext_ld_done` = 1'b1. In this case the expectation is that there is no external loading done.
- `sram_ext_ld_done` = 1'b0. In this case the expectation is that there will be external loading performed. As a result, there must also be an external register clock provided to ensure that the `sram_init_done` signal will assert.

After initial bootloading of the SRAM, you can change the SRAM contents through the APB or JTAG interfaces. Note that during this time, any register in the design can also be accessed and Synopsys recommends performing any necessary register writes during this period.

After external loading is complete, the input `sram_ext_ld_done` should be set high, allowing the FSMs in the RAW_PCS to start executing the code from SRAM and fetching the desired context. For a top-level architecture diagram of the RAW_PCS, refer to [Figure 3-9](#) on page 44.



Note

- The ROM/SRAM address space is shared (that is, only one address space is assessable at a time) from "RAWMEM_DIG_MEM_CMN0_B0_R0" through "RAWMEM_DIG_MEM_CMN39_B15_R31".
- When the SRAM is enabled, the memory locations are read/write locations and can be written to and read from. When the internal or external ROM is enabled (that is, SRAM is bypassed with the `sram_bypass_mode[1:0]` input), the memory locations are read-only.
- Mandatory SRAM size is a 20K single-port SRAM consisting of 16-bit locations.

4.6.1 Bypassing SRAM Bootloading

The PHY includes the ability to skip bootloading of either the firmware or context information altogether when it is unnecessary or undesired. The SRAM bootloading can be disabled by asserting the appropriate bits of the `sram_boot_load_bypass_mode[1:0]` signal. Details of this signal are listed below:

- `sram_boot_load_bypass_mode[0]` = Controls the bypassing of bootloading the SRAM for PHY firmware.
- `sram_boot_load_bypass_mode[1]` = Controls the bypassing of bootloading the SRAM for context information.

Similar to the bootloading case, after `sram_init_done` is asserted, the PHY waits for the assertion of the `sram_ext_ld_done` signal in the event that any external loading of SRAM is desired. Some use case examples of where bootloading can be skipped are listed below:

- The SRAM is sideloaded by the SOC before `phy_reset` de-assertion

■ Non-POR phy_reset assertion

In both cases, the SRAM already contains the correct firmware program and context information when phy_reset is de-asserted. As a result, there is no need to go through the bootloading process and then optionally externally load an updated firmware program or context information.

Figure 4-15 and Figure 4-16 on page 72 show the timing diagram of SRAM initialization and loading with and without SRAM bootloading bypassed, respectively.

Figure 4-15 SRAM Initialization and Loading Sequence with SRAM Bootloading Bypassed

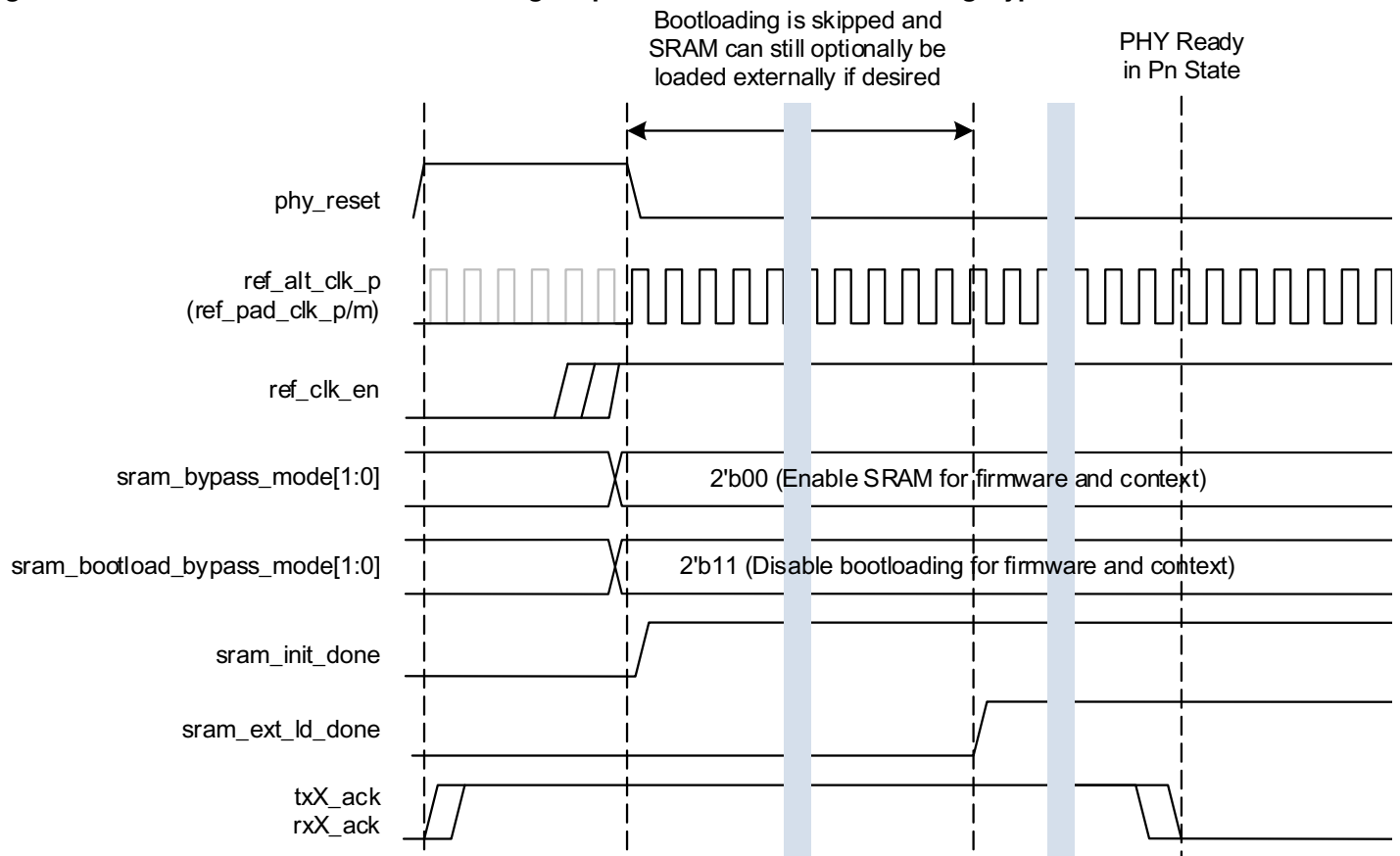
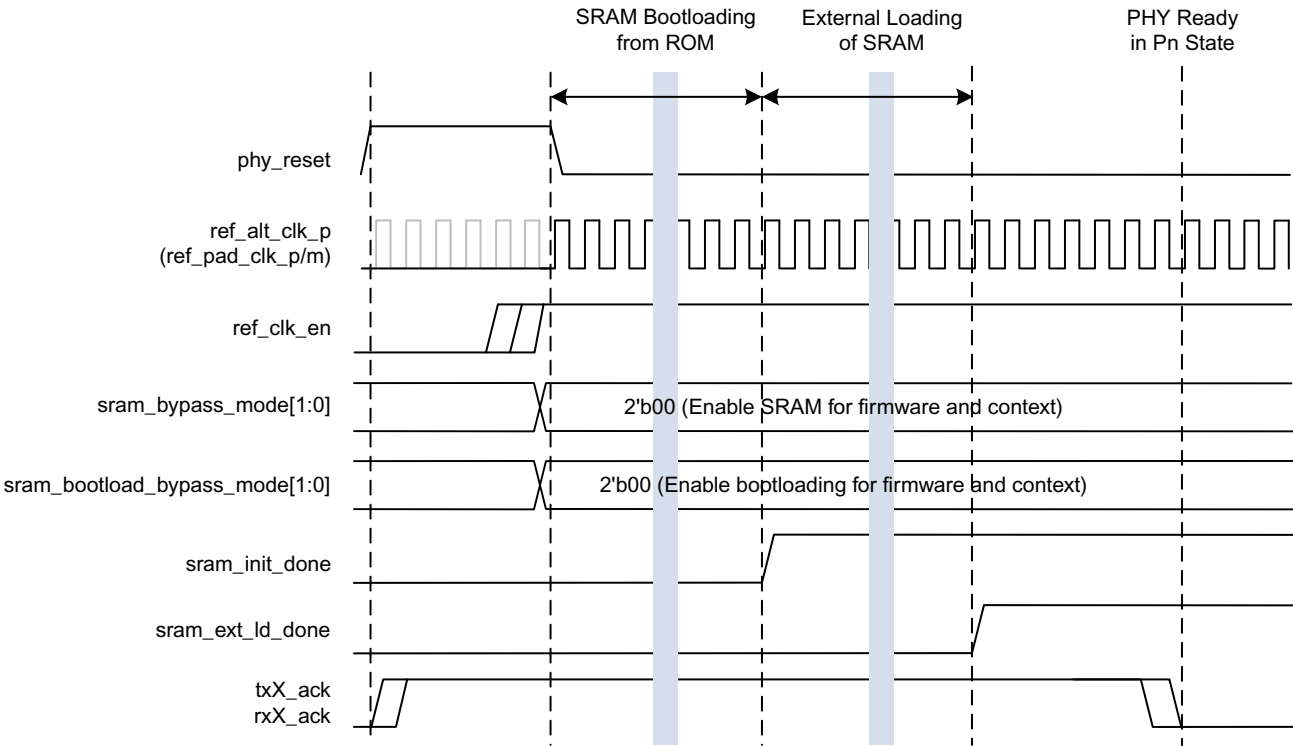


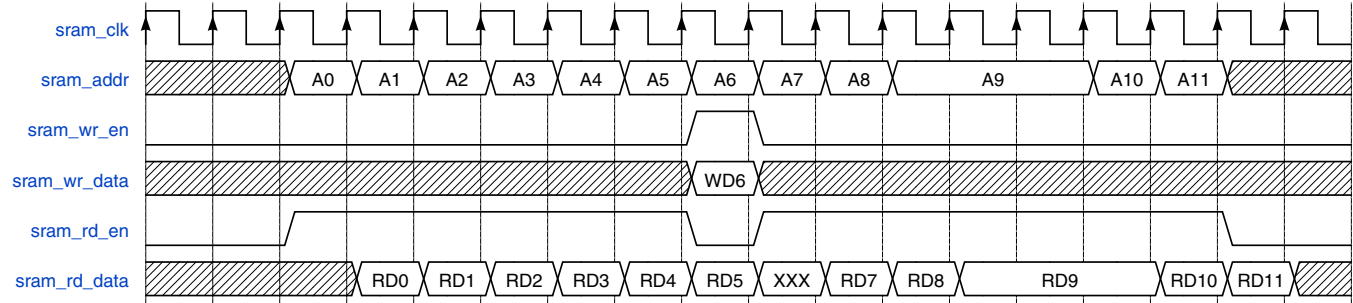
Figure 4-16 SRAM Initialization and Loading Sequence Without SRAM Bootloading Bypassed



When sram_bypass is asserted, the access to the SRAM is bypassed and the lane FSMs execute the code from the hard-coded look up table in the Raw PCS.

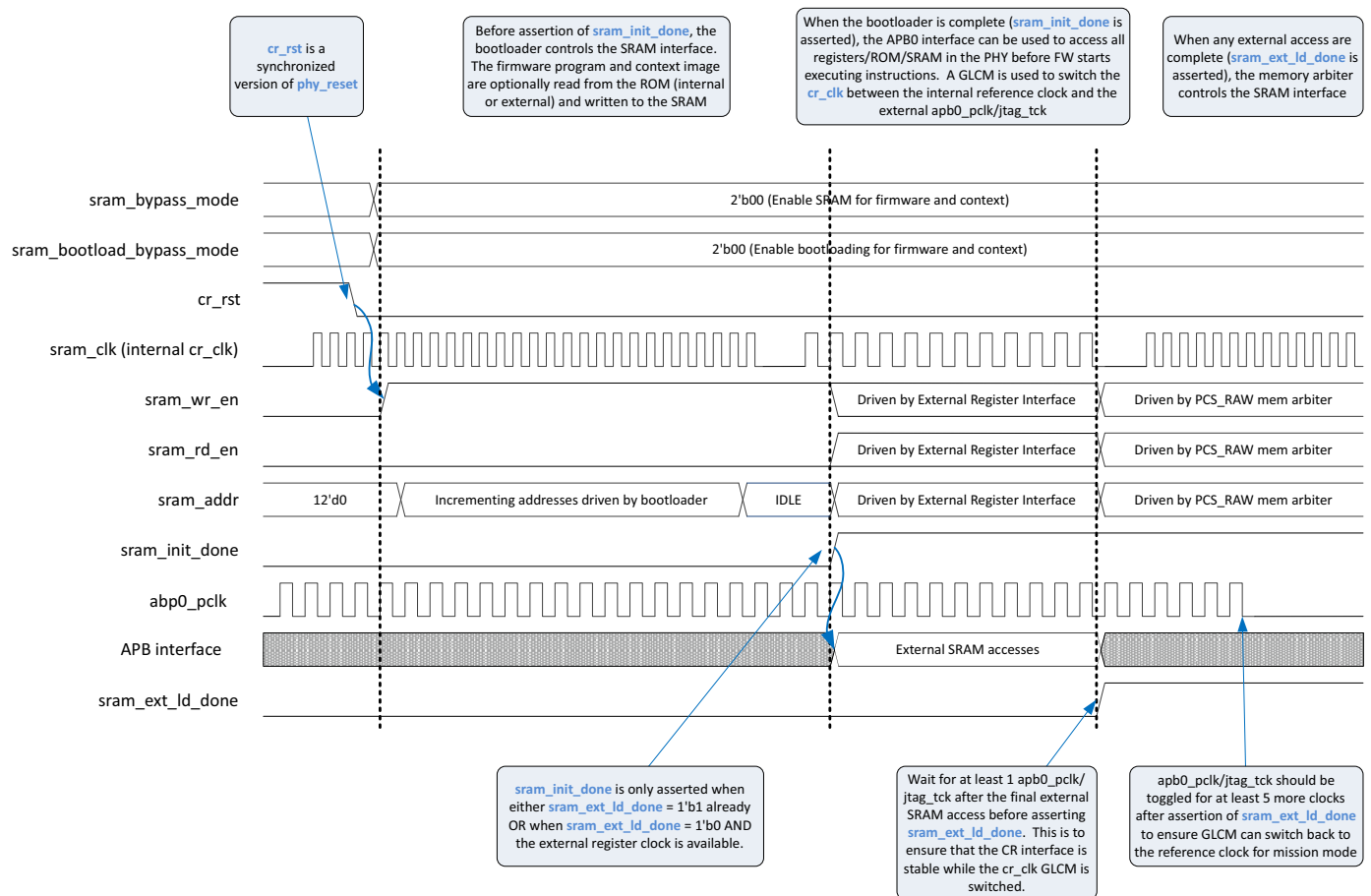
The timing diagram for the SRAM interface is shown in [Figure 4-17](#).

Figure 4-17 SRAM Read/Write Access Timings



Note Maximum clock speed requirement for the SRAM clock is 400 MHz. Depending on which clock is active, the SRAM clock will be derived from either the ref_clk, apb0_clk, or fw_clk.

The sequence of events on the SRAM interface as part of PHY power up are shown in [Figure 4-18](#).

Figure 4-18 Sequence of Events on the SRAM Interface

4.7 Context Restore

4.7.1 Overview

The context restore feature replaces top-level PHY I/O signals with a context memory within the SRAM or ROM that contains all of the necessary configuration information for each supported protocol and rate. Whenever a reset or rate change occurs, the context restore FSM extracts the appropriate context information from memory, and applies the configuration to the PHY.

The mapping between links and lanes can be selected by setting each lanes' `laneX_link_num[3:0]` input. The context restore engine combines all lanes in a link into a single memory request with broadcast addressing to reduce overall latency. The context configuration information is read from the memory through the memory arbiter, and the values are written to the respective internal CREG registers through the control register arbiter. When multiple links are enabled, round robin arbitration is used by the context restore FSM to permit access to the memory and control register arbiters.

There are five individual context settings, one for each configuration settings:

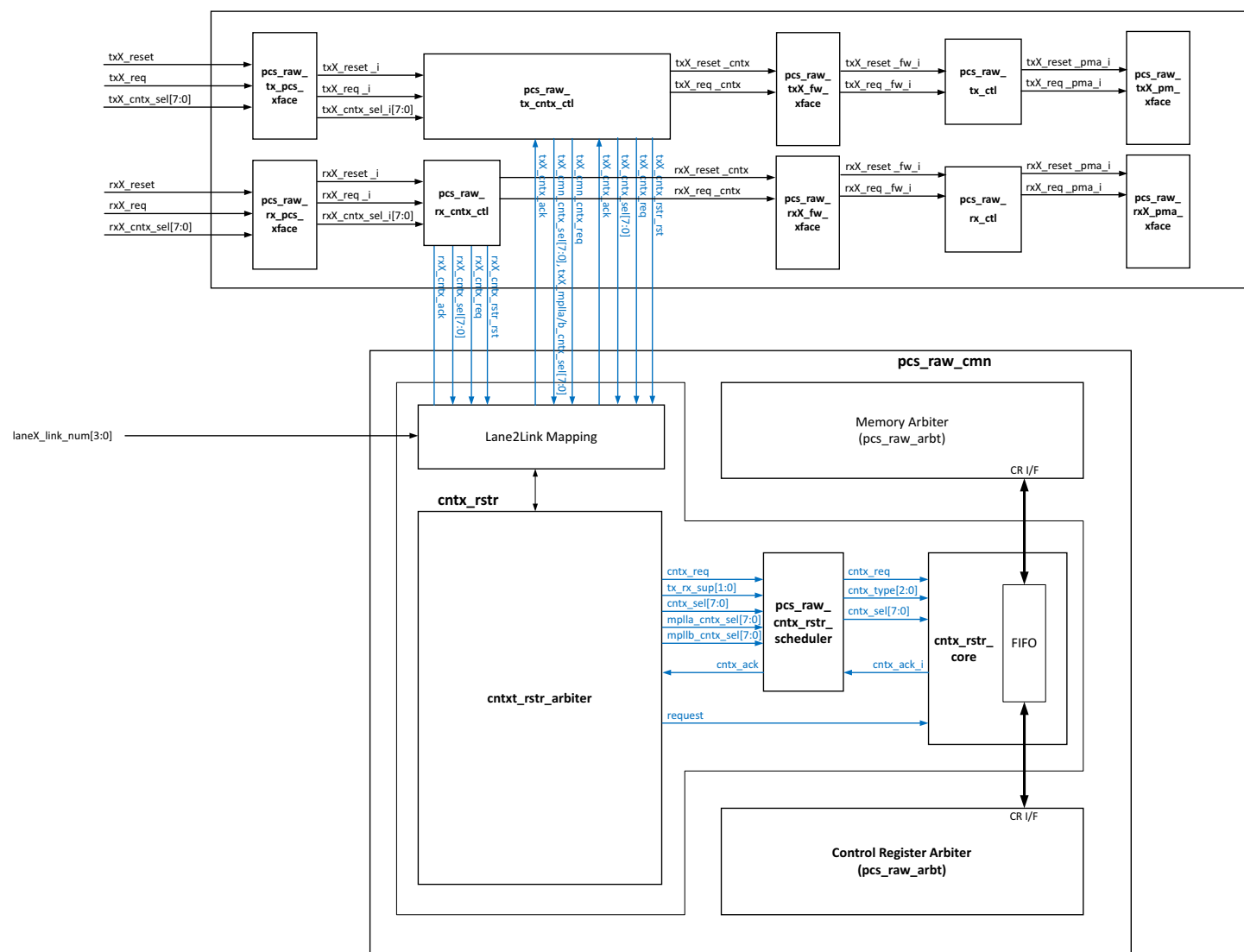
- CMN: `cmn_cntx_sel[7:0]` – Controls common lane-based settings
- MPLLA: `mplla_cntx_sel[7:0]` – Controls MPLLA configuration settings
- MPLLB: `mpllb_cntx_sel[7:0]` – Controls MPLLB configuration settings
- RX: `rxX_cntx_sel[7:0]` – Controls RX configuration settings
- TX: `txX_cntx_sel[7:0]` – Controls TX configuration settings

The expectation is that these context select values match for all lanes within a single link. Similar to all other configuration signals, the values should be stable on the de-assertion of `phy_reset`/`lane_resets`, as well as on the assertion of the `txX/rxX_req` signals. The context select values should then remain stable throughout the four-way handshaking sequence (for example, de-assertion of `txX/rxX_ack` for reset, assertion of `txX/rxX_ack` otherwise).

4.7.2 Context Restore Architecture

The context restore architecture is shown in [Figure 4-19](#).

pcs raw laneX



pcs raw tx/rx cntx

pcs_raw_lane2link_mapper

pcs_raw_cntx_rstr_arbiter

The `pcs_raw_cntx_rstr_arbiter` module performs arbitration between the different requests coming from each lanes' TX and RX, as well as the MPLLA, MPLLB, and common modules which are lumped together into a single common group at this point. This module uses the link information from the `pcs_raw_lane2link_mapper` to combine lane-based requests into link-based requests to reduce the overall number of context restore operations.

`pcs_raw_cntx_rstr_scheduler`

The `pcs_raw_cntx_rstr_scheduler` module contains an FSM that performs context update requests for the three common modules. This module splits the common context requests from the arbiter into separate context requests for MPLLA, MPLLB, and common context updates.

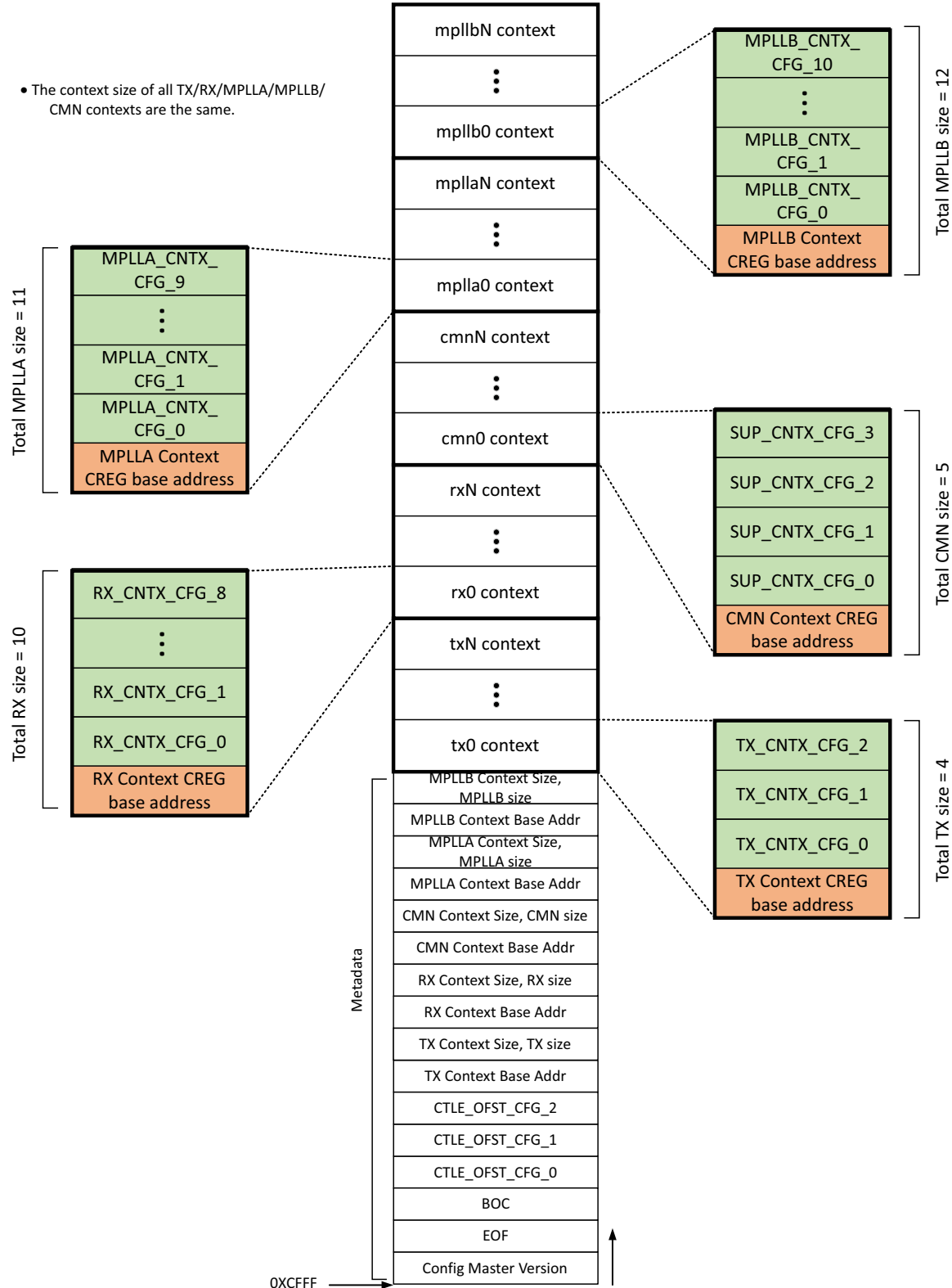
`pcs_raw_cntx_rstr_core`

The `pcs_raw_cntx_rstr_core` module is responsible for transferring the physical information from the context memory within the ROM or RAM and placing it within the `pcs_raw_lane_pcs_xface` registers. This is done through a FSM and a dual-port FIFO. The `pcs_raw_cntx_rstr_core` has the highest priority of all the clients for the memory and CREG arbiters and is guaranteed to be serviced when a request is made.

4.7.3 Context Restore Memory Structure

The context information is stored as CREG data written at the end of the FW from address 0XCFFF upward. The size of the context memory is dependent on the number of contexts that are stored for each of the context types described above. Note that it is possible that the number of contexts can be different for each of the different types and this depends on how they can be shared (for example, same MPLL context for multiple TX/RX contexts). The overall structure of the context restore memory is shown in [Figure 4-20](#).

Figure 4-20 Context Restore Structure



At the start of the context memory is the context metadata. This contains the following information about the context memory:

- Config master version – Internal version number, corresponding to the context restore memory source.
- BOC – Tag ‘Beginning of Context’, denoting the address location of the start of the context memory.
- EOF – Tag the ‘End of Firmware’, denoting the address location where the firmware memory ends.
- CTLE_OFST_CFG_0/1/2 – Configuration settings for RX CTLE offset calibration.
- Context Base Addr – There is a base address for each context type: TX/RX/CMN/MPLLA/MPLLB. This address denotes starting location of the content for each context type.
- Context size – This denotes the number of context configuration registers entries are in a single context.
- Size – This is the memory size used for a single context. This value denotes the sum of the Context Size and additional information for Context restore operation. (Currently, Size = Context Size + 1)

After the metadata, the contexts are stored by context type. Within each context there is a CREG base address pointing to the location in the CREG memory map for the destination registers. After the CREG base address, there are N *_CNTX_CFG registers, containing the context configuration information. These values will be transferred to the internal CREGs during a context restore operation.

The field description of each of the memory spaces can be identified through the corresponding RAWPCS CREG register *_CNTX_CFG_<N> defined in the “Register Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

Table 4-7 shows the mapping of the naming convention between SRAM memory identifier and CREG:

Table 4-7 Naming Convention Mapping

Memory Space Identifier	Register Name
RX_CNTX_CFG_<N>	RAWLANEN_DIG_RX_PCS_XF_CNTX_CFG_<N>
TX_CNTX_CFG_<N>	RAWLANEN_DIG_TX_PCS_XF_CNTX_CFG_<N>
SUP_CNTX_CFG_<N>	RAWCMN_DIG_SUP_CNTX_CFG_<N>
MPLLA_CNTX_CFG_<N>	RAWCMN_DIG_MPLLA_CNTX_CFG_<N>
MPLLB_CNTX_CFG_<N>	RAWCMN_DIG_MPLLB_CNTX_CFG_<N>

4.7.4 Overriding Context Restore Values

Overriding PHY signals that are stored within the context restore memory involves overwriting the context restore memory information directly. This allows for overwriting a specific configuration parameter for a specific rate and protocol. This ensures that the overridden value is used each time a specific rate and protocol configuration is retrieved from the context restore memory. Note also that any updates done to the context restore memory will require a context update (for example, reset or rate change to the desired rate and protocol) to be propagated to the PHY.

Using the pcs_raw_fw_xface module shown in Figure 4-19 on page 75 is not a valid option for providing PHY override control for the following reasons:

- Limited number of signals that actually have overrides within the pcs_raw_fw_xface module.

- The purpose of this module is to allow for firmware control of the PHY when performing calibration and adaptation. Any override done using the pcs_raw_fw_xface module is at risk of being removed by firmware.

The PHY deliverables provide a view “./phy/firmware/<product_name>_context_sel_upcs<_ref_freq>.v”. This view contains the protocol/rate/width information as macros and each macro is assigned an index that can be used to deduce the specific context memory location related to the given protocol/rate/width.

The macro names are self-descriptive and aligned to the column names of Configuration Settings tables provided in “[Support and Lane Configuration Settings](#)”. These can be used to identify the context select ID and thereby the APB address for updating the settings per protocol/rate/width.

The following equation can be used to determine the location of each CNTX_CFG_<N> space for a Context Type (TX/RX/CMN/MPLLA/MPLLB).

APB Address for CNTX_CFG_0

= 16'h8000 + Base Address - [(Context Select ID - 1) * Size] - (Size-Context Size)

APB Address for CNTX_CFG_<N> = APB Address for CNTX_CFG_<N - 1> - 1

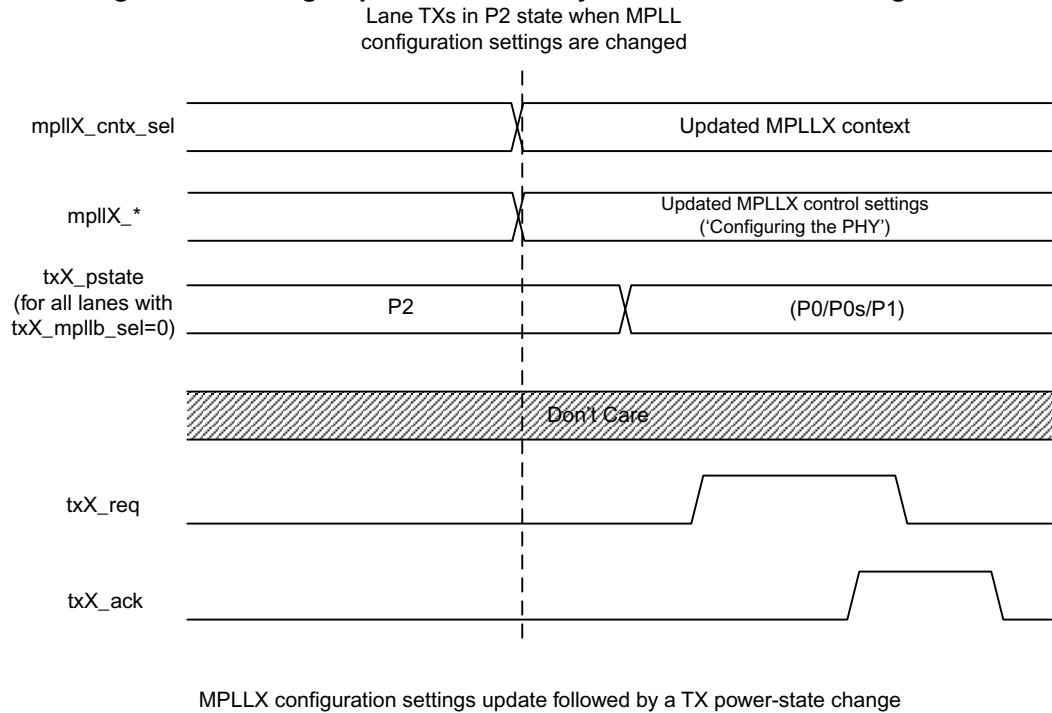
Where:

- Base Address: One of TX/RX/CMN/MPLLA/MPLLB base address read from Metadata
- Context Select ID: Index value for a protocol/rate/width
- Size: One of TX/RX/CMN/MPLLA/MPLLB Size read from Metadata
- Context Size: One of TX/RX/CMN/MPLLA/MPLLB Context Size read from Metadata

4.8 MPLL Configuration Update

The MPLLA and MPLLB in the PHY are powered up and powered down depending upon the txX_pstate and txX_mpllb_sel control inputs on each lane. With the default register settings, the MPLLA is powered down when mplla_force_en input is 0 and for each lane, txX_mpll_en=0 or txX_mpllb_sel=1. Likewise, MPLLB is powered down when mpllb_force_eninput is 0 and, for each lane, txX_mpll_en=0 or txX_mpllb_sel=0. Both MPLLs are calibrated by firmware automatically when phy_reset is de-asserted and the mpll{a,b}_init_cal_disable inputs are de-asserted. Initial power-up calibration of either MPLL can be skipped by asserting the appropriate mpll{a,b}_init_cal_disable input before the de-assertion of the phy_reset and rxX/txX_reset signals. The MPLL configuration settings can only be changed when the corresponding MPLL is powered down. As an example, [Figure 4-21](#) shows the requirements for changing the MPLL configuration settings.

Figure 4-21 MPLL Configuration Settings Update Followed by a TX Power-State Change



Note

For updated MPLLX context, see section “[Configuring the PHY](#)” on page [167](#).

4.9 Lane Configuration Update

Each lane is configured using its TX and RX context input. [Figure 4-22](#) and [Figure 4-23](#) show the configuration change handshake for the TX and RX signals, respectively. These inputs must stay stable from assertion of txX_req/rxX_req until de-assertion of txX_ack/rxX_ack.

Figure 4-22 Lane TX Configuration Change Handshake

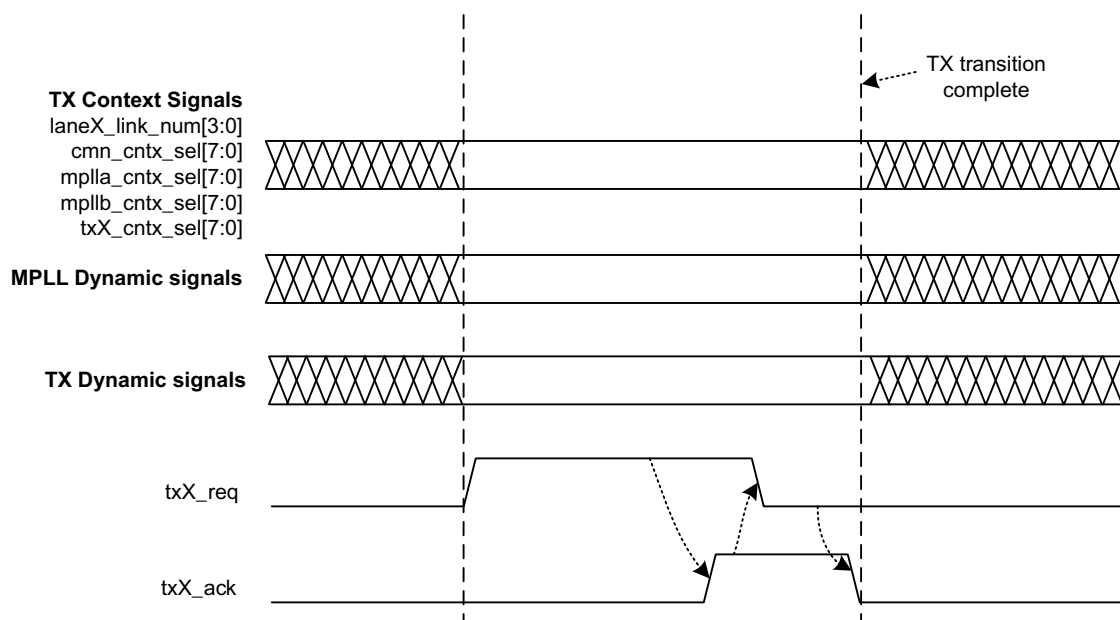
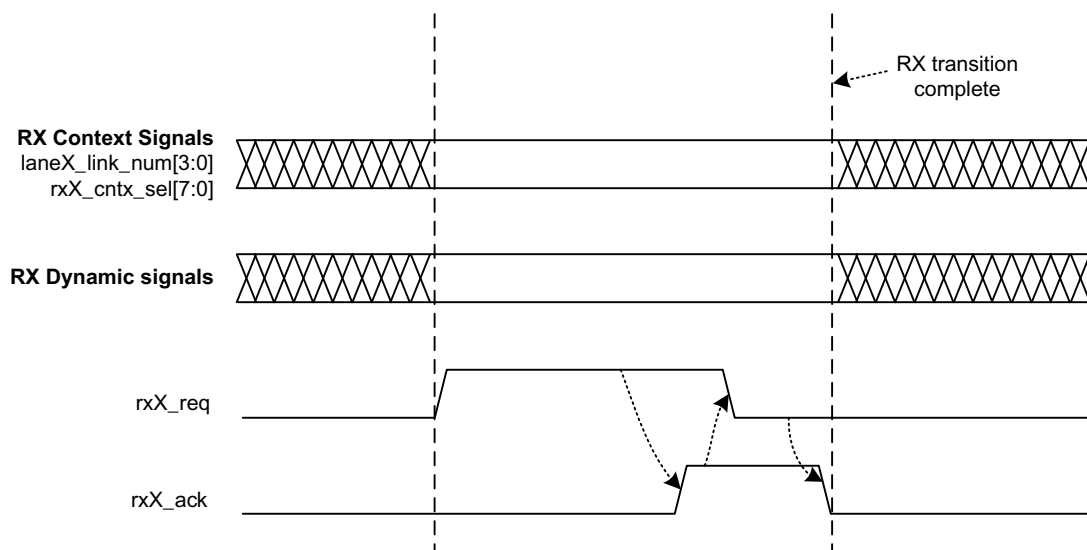


Figure 4-23 Lane RX Configuration Change Handshake



Lanes that are operating as part of one link must comply with the following requirements:

TX context restore inputs and dynamic configuration inputs must be set to the same value for all lanes.

TX Context restore inputs:

- laneX_link_num[3:0]
- cmn_cntx_sel[7:0]
- mpla_cntx_sel[7:0]
- mpllb_cntx_sel[7:0]
- txX_cntx_sel[7:0]

TX Dynamic configuration inputs:

- txX_lpd
- txX_pstate
- txX_mpll_en
- txX_recal_force_en
- txX_recal_skip_en

RX context restore inputs and dynamic configuration inputs must be set to the same value for all lanes.

RX Context restore inputs:

- laneX_link_num[3:0]
- cmn_cntx_sel[7:0]
- rxX_cntx_sel[7:0]

RX Dynamic configuration inputs:

- rxX_lpd
- rxX_pstate
- rxX_recal_bank_sel
- rxX_recal_force_en
- rxX_recal_skip_en
- rxX_term_acdc

A common clock must be supplied to txX_clk inputs for all lanes.

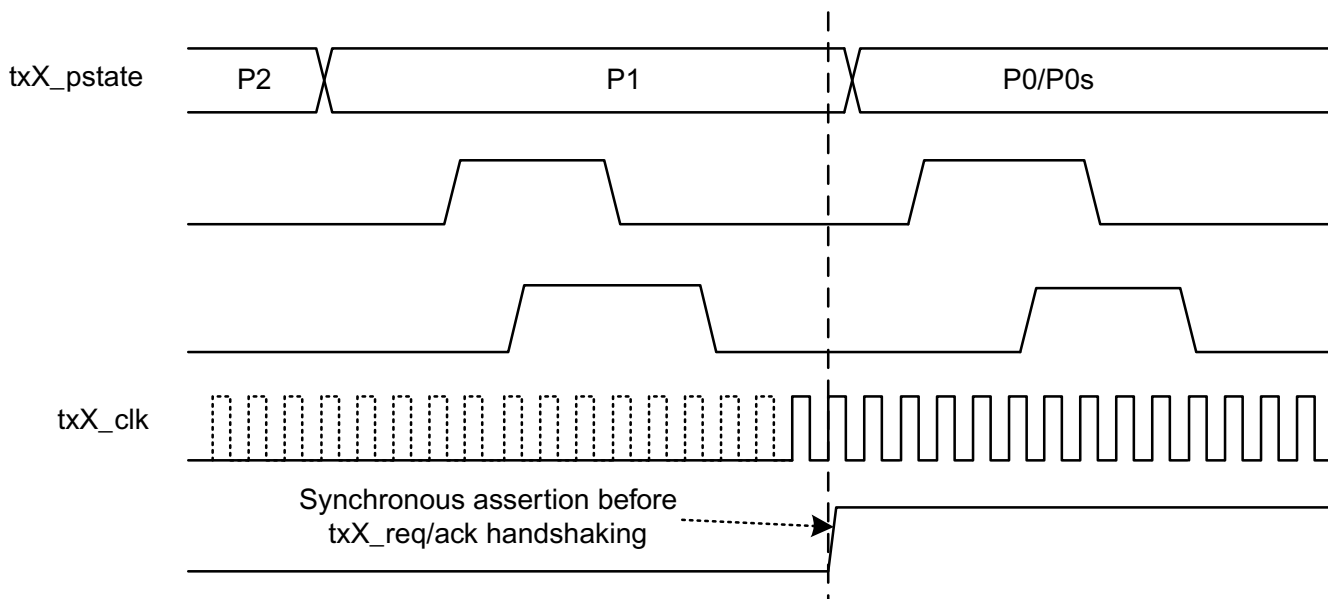
- Assert txX_clk_rdy signal synchronously to txX_clk. This signal denotes that the txX_clk is toggling and the PHY can perform its internal clock alignment procedure to align the internal transmitter clock to the common txX_clk input. This function is performed every time the transmitter in the PHY is powered up to P0/P0s.

There are two options for asserting the txX_clk_rdy signal depending on which power state the PHY is transitioning from when powering up to P0/P0s:

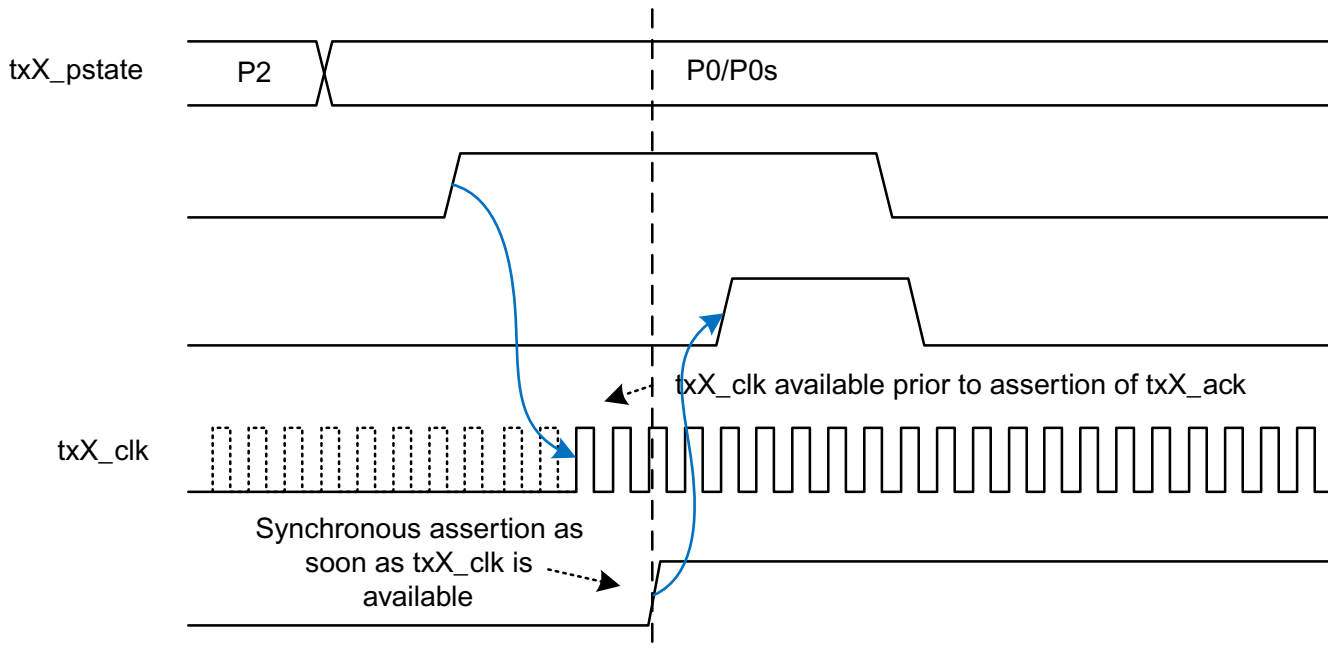
- When powering up from P1, the expectation is that the txX_clk is already available and thus the txX_clk_rdy signal should be asserted synchronously to the txX_clk before the request to P0/P0s. This is illustrated in [Figure 4-24](#).
- When powering up from P2, this will also enable the MPLL which is the source of the txX_clk. The txX_clk_rdy signal should be asserted synchronously once the txX_clk is toggling. This is illustrated in [Figure 4-25](#).

Figure 4-24 TX P1 to P0/P0s Transition

TX P1 to P0/P0s Transition


Figure 4-25 TX P2 to P0/P0s Transition

TX P2 to P0/P0s Transition



4.10 TX Datapath

The TX datapath is active when the TX lane is in P0 state (`txX_pstate=2'b00`) and `txX_data_en` is asserted.

The TX datapath supports serialization of 8/10/16/20/32/40-bit data symbol determined by the setting of the input pin `txX_width`. The valid bits that are transmitted in the 8/10/16/20/32/40-bit mode are defined in the signal description of `txX_data`. The data symbols captured by the PHY on the rising edge of `txX_clk` when `txX_data_en=1` are transmitted on `txX_p/m` serial lines. In addition, there are some dribble (invalid) bits transmitted before the first valid word and after the last valid word. The maximum number of dribble bits at the start of transmission can be up to 16 when operating in 8/10-bit modes, and up to 6 when operating in 16/20/32/40-bit modes. The maximum number of dribble bits at the end of transmission is 2.

For lanes belonging to the same link, ensure that the `txX_clk` inputs are derived from the same source and phase skew balanced to within 200 ps.

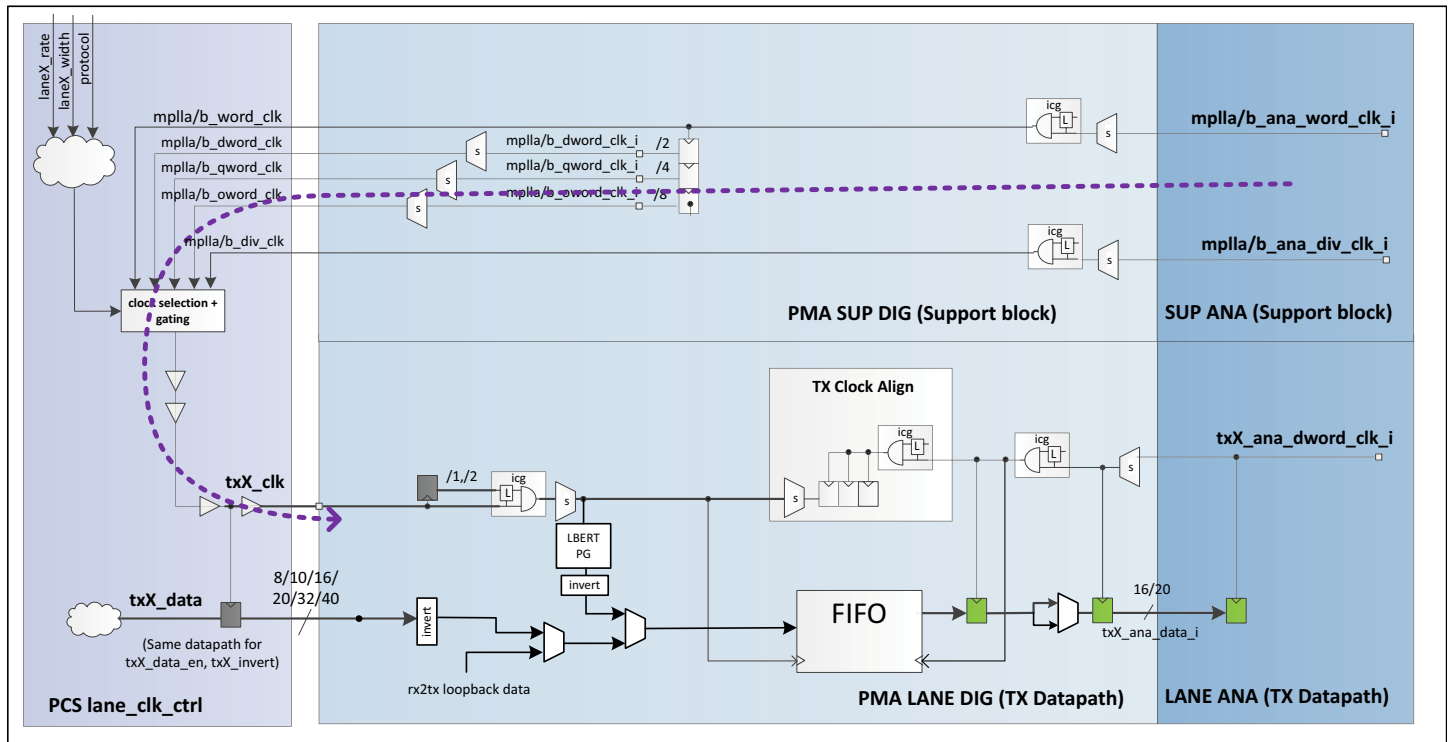
4.10.1 TX Clock Alignment

The PMA contains a circuit that aligns the incoming `txX_clk` with the internal transmit clock used within the PMA. This alignment results in a maximum TX skew on `txX_p/m` across different lanes in the same link to within 2 unit-intervals (UI) + 500 ps. If the `txX_clk` clocks are not aligned between lanes, then the serial skew can be up to 1 `txX_clk` (if 16/20/32/40b mode) or 2 `txX_clk` (if 8/10b mode). This alignment is performed on a 16-bit word when `txX_align_wide_xfer_en=1'b0`. When `txX_align_wide_xfer_en=1'b1`, this alignment is performed on a 32-bit word to decrease the clock frequency of the data transfer from the `txX_clk` domain to the internal transmit clock domain. It is recommended that you set the `txX_align_wide_xfer_en` bit only when the data rate is > 16 Gbps.

4.10.2 TX Skew Requirements

There are multiple types of skew requirements that are placed on the TX datapath inputs to minimize the skew on the high-speed serial transmitter pins. These types can be categorized as static lane-to-lane skew, and dynamic VT skew or variation. The static skew is defined as the skew between the clock pulses that go to different lanes of the PHY. This type of skew can be quantified and constrained during timing closure and does not change. The dynamic skew is defined as the skew or variation across time that can be tolerated by the PHY after it has completed the clock alignment procedure. This type of skew varies according to the voltage and temperature drift of the clock paths in the SOC after the clocks have been aligned by the PHY.

[Figure 4-26](#) shows the TX clock paths within a system.

Figure 4-26 TX Clock Paths

4.10.2.1 TX Static Skew Requirements

For lanes belonging to the same link, the expectation is that they have a common source clock and that the clock signals arriving at the PMA txX_clk pins are balanced to within 200 ps. The static skew should be calculated as the difference between the max and min arrival times of each txX_clk pin, from the MPLL clock source to the txX_clk pin. This should be done after removing any common portions of the path, and should be repeated for each corner. Note that this static skew requirement is constant and does not change based on any of the deskew modes outlined below. The equation for static skew calculation is as follows:

$$[\max(\text{arrival, corner}) - \min(\text{arrival, corner})] - \text{common_path} < 200 \text{ ps}$$

4.10.2.2 TX Dynamic Skew Requirements

There also exists a dynamic skew requirement for each txX_clk input pin independently, due to the fact that VT will drift over time and the optimal alignment of the external and internal TX clocks may change relative to each other. There exists a FIFO along the TX parallel data path that can absorb up to 1 symbol clock (based on the txX_width input) of variation after TX clock alignment has been completed. Any additional variation on the input TX clock path can cause this FIFO to over/underflow and data can be lost. The dynamic skew can be calculated as the difference between the max and min arrival times for each txX_clk pin across extreme VT corners. The equation for dynamic skew calculation is as follows:

$$\max(\text{arrival, max_VT}) - \min(\text{arrival, minVT}) < 1 \text{ txX_clk}$$

Note that when clock deskewing is enabled, this dynamic skew requirement is increased. This is described at “TX Clock De-skew” on page 86.

4.10.3 TX Lane-to-Lane Deskewing Modes

There are three deskewing modes available in the PHY. The deskewing modes determine the amount of skew that will be present on the high-speed serial TX output pins based on the parallel input stream to the lanes of the PMA. The deskewing modes are controlled by two signals:

- `txX_lane2lane_dskw_en`
- `txX_clk_dskw_en`

Regardless of deskewing mode, the signals listed above must be stable prior to powering up the PHY to P0 to ensure the internal logic with the transmitter is configured properly. Also note that deskewing can only be done on a single link when bifurcating multiple links in the PHY.



Note

- The TX lane and clock de-skewing features are only available on MPLLB and are not supported on MPLLA.
- The TX lane and clock de-skewing features can only be enabled on MPLLB if it was previously off (that is, these features cannot be enabled while the MPLLB is already enabled).

4.10.3.1 No De-skewing (Default)

The default deskewing mode is sufficient for the majority of use cases for the C20 PHY. In this mode, `txX_lane2lane_dskw_en=1'b0` and `txX_clk_dskw_en=1'b0` for all lanes of a given link. In this case the high-speed serial output TX pins can have a skew of up to 2 UI across different lanes of a link, assuming the TX parallel inputs are aligned.

4.10.3.2 TX Lane De-skew

In lane de-skew mode, all lanes of a link are balanced to 0 UI. This mode is enabled with `txX_lane2lane_dskw_en=1'b1` and `txX_clk_dskw_en=1'b0` for all lanes of a given link. In this mode, all lanes of the link are aligned to a master lane (that is, the lowest lane number of link) during clock alignment. When this mode is enabled, the high-speed serial output TX pins can have a skew of 0 UI across the different lanes of a link, assuming the TX parallel inputs are aligned.

4.10.3.3 TX Clock De-skew

In clock de-skew mode, all lanes of a link are still balanced to 0 UI, but the internal FIFO depth is extended to tolerate additional dynamic skew of up to a total of 2.4 ns (translates to up to 3 symbol clocks of skew at the highest supported data rate). This mode is enabled with `txX_clk_dskw_en=1'b1` for all lanes of a given link (`txX_lane2lane_dskw_en` is internally enabled in this case so the input becomes a don't-care). In this mode, all lanes of the link are still aligned to a master lane (that is, the lowest lane number of link) during clock alignment. When this mode is enabled, the high-speed serial output TX pins can have a skew of 0 UI across the different lanes of a link, assuming the TX parallel inputs are aligned to within the maximum of 1 symbol clock and 2.4 ns. The equation for dynamic skew in TX clock de-skew mode calculation is as follows:

$$\max(\text{arrival}, \text{max_VT}) - \min(\text{arrival}, \text{minVT}) < \text{MAX}(1 \text{ tx_clk}, 2.4 \text{ ns})$$

Table 4-8 De-skew Mode Summary

De-skew mode	txX_clk_dskw_en	txX_lane2lane_dskw_en	Output TX skew + clock skew across lanes	Dynamic skew budget due to txX_clk VT skew
Default (no deskew)	1'b0	1'b0	0 UI to 2 UI + clock skew	1 symbol clock
Lane deskew mode	1'b0	1'b1	0 UI + clock skew	1 symbol clock
Clock deskew mode	1'b1	1'bx	0 UI + clock skew	MAX (1 symbol clock, 2.4 ns)

Table 4-9 provides the datapath latency values.

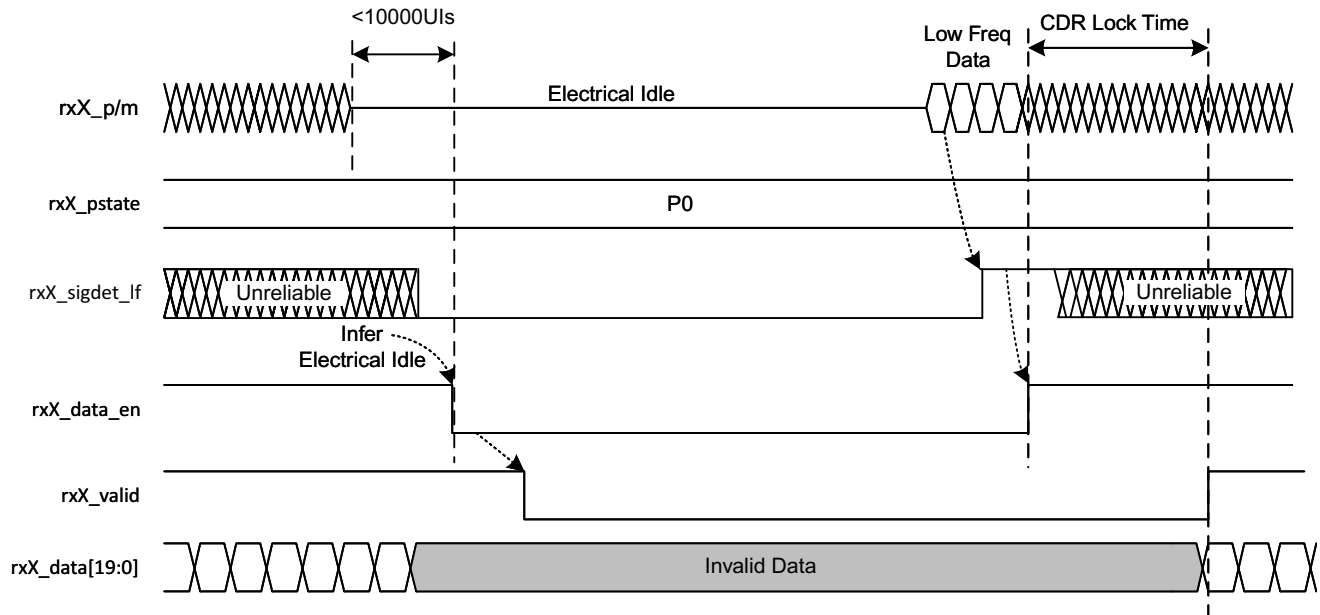
Table 4-9 TX Datapath Latency

Parameter	Min (UI)	Max (UI)	Avg (UI)
tx_datapath_latency_8b	59.50	63.23	61.50
tx_datapath_latency_10b	69.58	73.40	71.19
tx_datapath_latency_16b	53.25	61.25	57.66
tx_datapath_latency_20b	65.17	69.38	67.29
tx_datapath_latency_32b	78.07	85.81	81.89
tx_datapath_latency_40b	96.23	99.06	97.65

4.11 RX Datapath

The RX datapath is active when the RX lane is in P0 state ($rxX_pstate=2'b00$) and rxX_data_en is asserted. You should only assert rxX_data_en when the RX lane is in P0 state and the RX input signal is present (indicated by the assertion of the rxX_sigdet_lf output). If a loss of signal is detected either by the de-assertion of rx_sigdet_lf or inferred using higher layer protocol-specific mechanisms, the clock-and-data-recovery (CDR) must be disabled by setting $rxX_data_en = 0$ and re-enabled by setting $rxX_data_en = 1$ after a signal is detected using $rxX_sigdet_lf = 1'b1$. A typical timing diagram of RX datapath for baud rates greater than 2.5 Gbps is shown in Figure 4-27.

Figure 4-27 RX Datapath and Control Timing for Baud Rate Greater than 2.5 Gbps



The output rxX_valid is asserted when a pre-determined bit-locking timer expires after assertion of rxX_data_en . rxX_valid is de-asserted when rxX_data_en is de-asserted to 0 or when the CDR frequency drift goes beyond that set by the input $rxX_cdr_ppm_max[4:0]$. The default settings for $rxX_cdr_ppm_max[4:0]$ values provided in this chapter set the drift threshold to 15000 ppm. If the CDR frequency goes beyond this threshold, the rxX_data_en is internally overridden to 0 to enable reference tracking mode until the CDR frequency returns to the ideal frequency. When the CDR has returned to a nominal frequency, locking to incoming data resumes and rxX_valid re-asserts after the bit-locking timer expires.

When the input rxX_data_en is asserted, the CDR's data tracking mode is enabled. When $rxX_data_en=0$, the RX VCO frequency is calibrated using the reference clock inputs ($ref_alt_clk_p$, $ref_pad_clk_p/m$) to ensure that the VCO frequency stays within the precision required for the CDR to lock to the received incoming data when rxX_data_en is re-asserted.

The rxX_data_en signal must be de-asserted while performing rate changes in P0 state. rxX_data_en should be de-asserted before asserting the rxX_req signal for the rate change and asserted only after the rxX_req/rxX_ack handshaking is completed for the rate change.

If rxX_data_en is enabled during invalid data, the fastest recovery will be to change the rxX_pstate from P0 to P1 to P0. This action takes approximately 30 μs and is much faster than negating rxX_data_en for an extended time.

The maximum CDR lock times as measured in [Figure 4-27](#) are provided in [Table 4-10](#) for both Spread-Spectrum-Clocking (SSC) and non-SSC modes.

Table 4-10 CDR Lock Times

CDR Mode (SSC/non-SSC)	Maximum CDR Lock Time (UIs)
Non-SSC (rxX_cdr_ssc_en=0)	6,560
SSC (rxX_cdr_ssc_en=1)	36,800

The RX datapath supports de-serialization of 8/10/16/20/32/40-bit data symbol determined by the setting of the input pin rxX_width. The valid bits on rxX_data in the 8/10/16/20/32/40-bit modes are defined in the signal description of rxX_data.

The RX datapath latency for the PHY is measured as per [Figure 4-28](#). The latency numbers are provided in [Table 4-11](#).

Figure 4-28 Receive Datapath Latency Measurement In the PHY

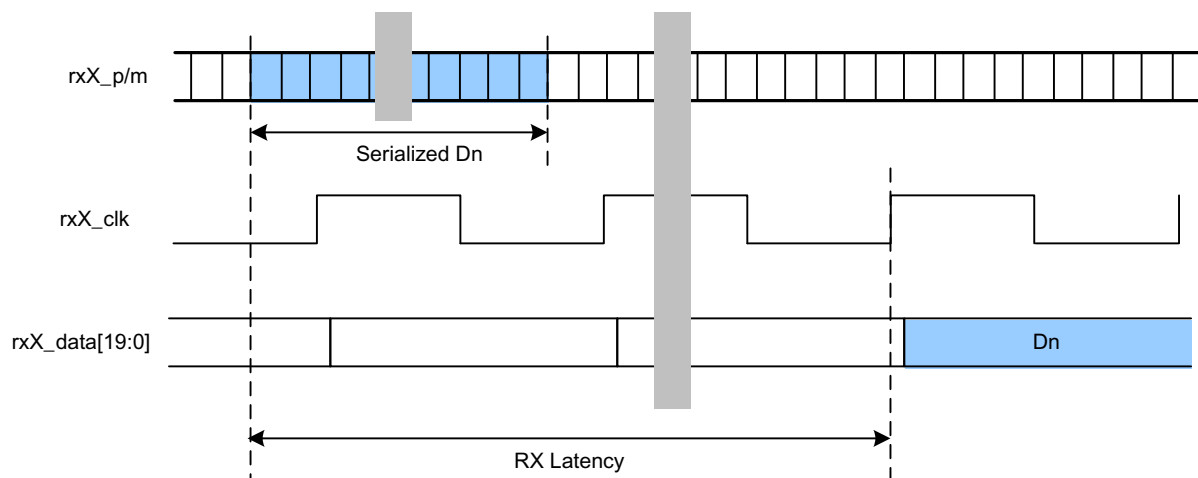


Table 4-11 RX Datapath Latency

Parameter	Min (UI)	Max (UI)	Avg (UI)
rx_datapath_latency_8b	41.94	42.25	42.02
rx_datapath_latency_10b	52.93	53.29	53.02
rx_datapath_latency_16b	50.91	51.29	51.01
rx_datapath_latency_20b	62.92	63.34	63.02
rx_datapath_latency_32b	82.90	83.45	83.00
rx_datapath_latency_40b	102.91	103.00	102.95

4.12 RX CDR PPM Detector

The PHY includes a CDR PPM detection circuit that contains the following features:

- Programmable PPM offset
- Current status of PPM offset relative to nominal frequency
- Ability to operate simply as a PPM monitor without intervening
- Ability to reset the CDR back to a nominal state when the clock drifts past the programmable PPM offset

The CDR PPM detector contains running counters and compares the incoming `rxX_clk` to a nominal clock based on the `rxX_ref_ld_val` and `rxX_vco_ld_val` inputs. If the difference exceeds the programmable PPM offset, the CDR unlock detector forces the `rxX_data_ensignal` low to the hard macro PMA and enables the PMA to track back to the nominal frequency.

The RX CDR PPM detector is enabled by default but can be disabled by setting the `RAWLANEA0NN_DIG_RX_CDR_DETECTOR_CTL.RX_CDR_DETECTOR_EN` register to 1'b0. If you want to disable the correction mechanisms of the RX CDR PPM detector, you can also enable it in monitoring mode by setting the `RAWLANEA0NN_DIG_RX_CDR_DETECTOR_CTL.RX_CDR_PPM_MONITOR_MODE` register to 1'b1.

The `rxX_ppm_drift` signal is the difference between the current running counter value relative to the programmed `rxX_vco_ld_val`. The output is a signed value that ranges from -32 (6'b100000) to +31 (6'b011111). This value is valid when `rxX_ppm_drift_vld` is asserted. The `rxX_ppm_drift_vld` signal is periodically asserted when the FSM has completed an iteration of comparing the ppm drift.

Note that the `rxX_ppm_drift` signal must not be used to control an ESTORE module in the protocol PCS. This signal provides only a difference from the NOMINAL frequency and does not take into account the actual transmitter.

4.12.1 Configuring the RX CDR PPM Detector

The RX CDR PPM detector can be programmed by the changing the `rxX_cdr_ppm_max[4:0]` input signals. This signal represents a difference from the nominal `rxX_vco_ld_val` that represents the appropriate PPM setting. This value is available in the PHY configuration section for 15,000 ppm (default setting). However, the value can be updated for each configuration based on the following equation (the example is based on 15,000 ppm for MPHY Gear1 @19.2 MHz):

$$\text{FLOOR}(\text{rxX_vco_ld_val} * (\text{ppm_value} / 1\text{e}6)) - 2 \text{ (for margin)} = (1,560 * 0.015) - 2 = 21$$

4.13 RX Margining

For more information contact Synopsys Customer Support.

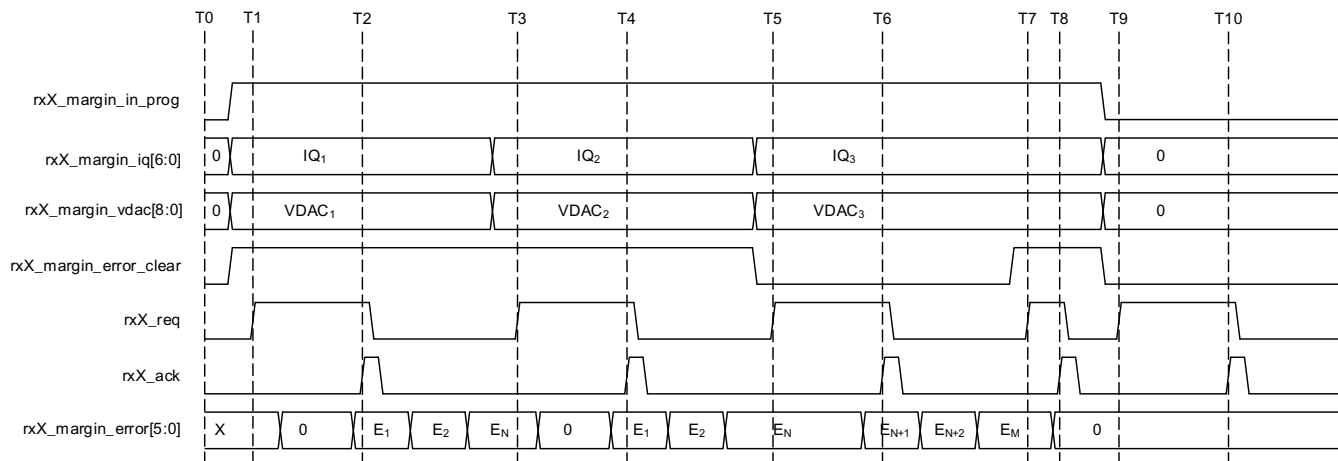
The PHY provides the ability to perform horizontal and vertical RX margining. The RX phase mixer (PMIX) has a nominal step size of $1/35$ UI. Note that the nominal step size is an absolute delay and is based on the PHY operating in a 'full-rate' protocol (that is, when the RX VCO operates at $1/2$ the baud rate). This resolution is increased as the ratio of RX VCO/baud rate increases. The RX slicer voltage DACs have a nominal step size of 1.6 mV. Note that the nominal step size is based on average Typical PVT conditions.

The PHY supports two modes of margining:

- Non-destructive
 - Timing only
 - Voltage only
 - Concurrent Timing and Voltage
- Destructive
 - Timing only

Figure 4-29 shows the timing waveforms for performing non-destructive margining:

Figure 4-29 Non-destructive Margining Modes



Note

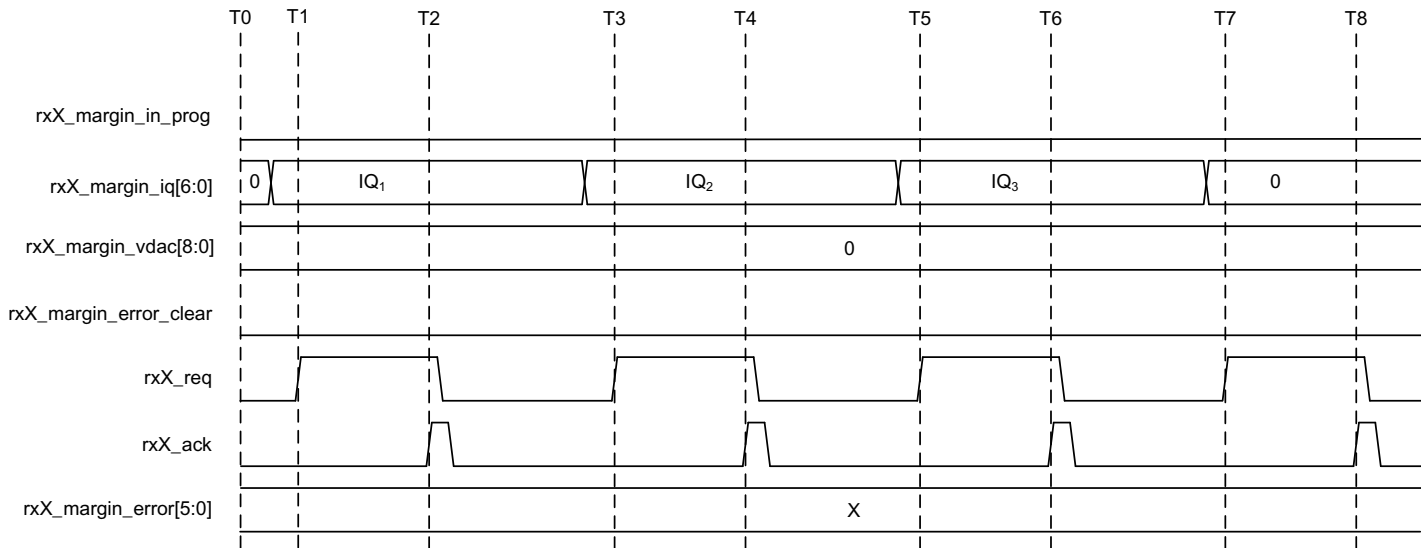
When completing an RX margining sequence, the final request to the PHY must set `rxX_margin_in_prog`, `rxX_margin_iq` and `rxX_margin_vdac` to 0. If either `rxX_margin_iq` or `rxX_margin_vdac` are kept at non-zero values while de-asserting `rxX_margin_in_prog`, the PHY will enter into destructive margining mode.

Below is the explanation of the sequence:

1. T0: PHY in P0 receiving high speed data.
2. T1: `rxX_req` assertion with `rxX_margin_in_prog` and `rxX_margin_error_clear` with either a new value on the `rxX_margin_iq` and/or `rxX_margin_vdac`.

3. T2: PHY clears the internal error counter; applies the requested `rxX_margin_iq/rxX_margin_vdac` to the samplers and initiates margining and the error counter. It responds with `rxX_ack`. The `rxX_margin_error` will have first result of the error counter (E1 it can be 0 or a non-zero value at this time).
 - a. The PHY keeps on updating the `rxX_margin_error` approximately every 20-30 `ref_clk` cycles. The upper layer logic can detect a change on the `rxX_margin_error` to update any register at the upper layer which can be read by the software to check the current error count during margining.
4. T3: The upper layer logic applies a newer value of the `rxX_margin_iq` and/or `rxX_margin_vdac` with `rxX_margin_error_clear` as 1'b1 and assert `rxX_req`.
5. T4: PHY clears the internal error counter; applies the newly requested `rxX_margin_iq/rxX_margin_vdac` to the samplers and initiates margining and the error counter. It responds with `rxX_ack`. The `rxX_margin_error` will have first result of the error counter (E1 it can be 0 or a non-zero value at this time).
 - a. The PHY keeps on updating the `rxX_margin_error` approximately every 20-30 `ref_clk` cycles. The upper layer logic can detect a change on the `rxX_margin_error` to update any register at the upper layer which can be read by the software to check the current error count during margining.
6. T5: The upper layer logic applies a newer value of the `rxX_margin_iq` and/or `rxX_margin_vdac` with `rxX_margin_error_clear` as 1'b0 and assert `rxX_req`.
7. T6: PHY stalls the internal error counter; applies the newly requested `rxX_margin_iq/rxX_margin_vdac` to the samplers and initiates margining and the error counter. It responds with `rxX_ack`. The `rxX_margin_error` will have first result of the error counter (EN+1 it can be 0 or a non-zero value at this time).
 - a. The PHY keeps on updating the `rxX_margin_error` approximately every 20-30 `ref_clk` cycles. The upper layer logic can detect a change on the `rxX_margin_error` to update any register at the upper layer which can be read by the software to check the current error count during margining.
8. T7: The upper layer logic applies `rxX_margin_error_clear` as 1'b1 and assert `rxX_req`.
9. T8: PHY clears the internal error counter. The `rxX_margin_error` become 0.
10. T9: The upper layer logic applies a value of 0 on the `rxX_margin_iq` and `rxX_margin_vdac` with `rxX_margin_in_prog` as 1'b0, and asserts `rxX_req` to complete the margining sequence.
11. T10: PHY applies the newly requested `rxX_margin_iq/rxX_margin_vdac` to the samplers and finishes the margining routine. It responds with `rxX_ack`.

Figure 4-30 shows the timing waveforms for performing destructive margining:

Figure 4-30 Destructive Margining Modes**Note**

rxX_margin_in_prog, rxX_margin_error_clear and rxX_margin_vdac must be kept 0 for destructive margining.

Below is the explanation of the sequence:

1. T0: PHY in P0 receiving high speed data.
2. T1: rxX_req assertion with a new value on the rxX_margin_iq.
3. T2: PHY applies the requested rxX_margin_iq to the data samplers. It responds with rxX_ack. The upper layer logic can perform the data sanity check on the rxX_data output to check for errors.
4. T3/T5: rxX_req assertion with a new value on the rxX_margin_iq.
5. T4/T6: PHY applies the requested rxX_margin_iq to the data samplers. It responds with rxX_ack. The upper layer logic can perform the data sanity check on the rxX_data output to check for errors.
6. T7: rxX_req assertion with a new value on the rxX_margin_iq.
7. T8: PHY applies the requested rxX_margin_iq to the data samplers. It responds with rxX_ack. The upper layer logic can perform the data sanity check on the rxX_data output to check for errors.

**Note**

- The destructive margining uses the actual data samplers. If the clock phase for the data sampler is deviated too far from the center of the eye, the CDR may unlock and may not be able to lock again even if the clock phase is reset to the center of the eye.
- Only half of the bit transitions are considered for margining due to the unrolled nature of the DFE architecture.
- Since there is no Sample counter support the upper layer must estimate the number of bits sampled for a given BER measurement.
- Interrupting the FW during the margining in progress before the initial calibration stage is completed may move the PHY to an unexpected state.
- Every time the rx_margin_in_prog is toggled and a new rxX_req handshake is initiated the error counter is reset to 0 and a new calibration will be performed (resetting the margining offsets back to the center of the Eye).

4.13.1 RX Margin Stop

The option exists to stop RX margining if required. To stop the RX margining sequence while the rxX_req signal is asserted to the PHY, you can de-assert the rxX_margin_in_prog signal, along with setting the rxX_margin_iq and rxX_margin_vdac values to 0. Note that during this stop sequence, the rxX_req signal should remain asserted, and the PHY will assert the rxX_ack signal to acknowledge the stop request.



Note

The gap between the assertion of rxX_req and the de-assertion of rxX_margin_in_prog can not be less than six reference clock cycles. This is the amount of time required to ensure that the PHY detects this as a stop event and not as a valid RX margining completion request.

4.13.2 Margining Capability

Table 4-12 shows the margining capability.

Table 4-12 M-PHY Margining Capability

Field Name	Description	PHY Capability	
		Non-destructive Margining	Destructive Margining
RX_EYEMON_Capability (0xF1)	Specifies support for eye monitor function.	1 (Supported)	0 (Not Supported)
RX_EYEMON_Timing_MAX_Steps_Capability[5:0] (0xF2)	Timing step maximum from center sampling point. Monotonicity and same step size toward both positive and negative side. Existence depends on: RX_EYEMON_Capability	6'd17	Not Supported
RX_EYEMON_Timing_MAX_Offset_Capability[5:0] (0xF3)	Timing MAX offset. Minimum ± 0.15 UI, value 15 to Maximum ± 0.50 UI, value 50. Existence depends on: RX_EYEMON_Capability	6'd49	Not Supported
RX_EYEMON_Voltage_MAX_Steps_Capability[5:0] (0xF4)	Voltage step maximum from center sampling point. Monotonicity and same step size toward both positive and negative side. Existence depends on: RX_EYEMON_Capability	6'd53 (the value must be multiplied by 2 at the RawPCS 9-bit interface)	Not Supported
RX_EYEMON_Voltage_MAX_Offset_Capability[4:0] (0xF5)	Voltage MAX offset. Minimum ± 30 mV, value 3 to Maximum ± 310 mV, value 31. Existence depends on: RX_EYEMON_Capability	5'd17	Not Supported

**Note**

For more information regarding the terminology used in [Table 4-12](#) refer to reference documentation *MIPI Alliance Specification for M-PHY, Version 5.0 Revision 05, 31 May 2021*.

4.14 Fast Context Switching

This section describes the Fast Context Switching feature, including initial calibration and rate changes.

4.14.1 Usage Overview

The PHY implements a feature for fast context switching during a rate change, which enables performing calibrations only once for a given PHY TX and RX clock configuration. The calibration results are stored within the always-on (AON) power domain in the RAW_PCS, and subsequent rate changes simply restore the previously calibrated values for that given rate instead of performing a full re-calibration.

Context switching uses the concept of “banks” to store the calibrated values for MPLLA, TX duty cycle correction (DCC), RX duty cycle correction and IQ correction (IQC). The MPLLA calibrations are stored in banks controlled by the `mplla_recal_bank_sel` signal. The TX DCC results are stored in banks controlled by the `txX_recal_bank_sel` signal. The RX DCC and IQC results are both stored in the same bank controlled by the `rxX_recal_bank_sel` signal. In the case of `mplla_recal_bank_sel`, a bank selection is defined by a change in the MPLL clock context configuration settings. In the case of `txX/rxX_recal_bank_sel`, a bank is defined by the TX and RX clock context configuration settings respectively. The following table lists the signals that define a bank selection.

Table 4-13 Context Configuration Settings that Define Bank Selection

TX Signals	RX Signals
<code>mpll{a,b}_multiplier[11:0]</code>	<code>rxX_ref_id_val[6:0]</code>
<code>mpll{a,b}_fb_clk_div4_en</code>	<code>rxX_vco_id_val[12:0]</code>
<code>mpll{a,b}_tx_clk_div[2:0]</code>	<code>rxX_cdr_vco_config[11:0]</code>

There are two primary usage cases for fast context switching. The first use case is the initial calibration of a given bank. In this case, the firmware performs all the necessary MPLL, DCC, and IQC calibrations for the given rate and stores the information within the AON domain in the RAW_PCS so it can be used in subsequent rate changes. The second use case is a subsequent rate change either within or between already calibrated banks. In this case, the firmware restores the relevant information into the PMA, but no calibrations takes place. The following sections describe both use cases in detail.

4.14.2 Initial Calibration

An initial calibration is defined as the first time that the PHY is being configured for a new bank. The initial calibration ensures that all calibrations for the given clock configuration are performed up-front, ensuring that subsequent transitions to this bank do not require re-calibrations. All initial calibrations are performed in PHY state P1 and are triggered when the MPLL is powered up and the corresponding bank has not been previously calibrated.

To perform an initial calibration for a given rate, perform the following procedure.

1. Power the PHY TX and RX into P1 state, and perform four-way request-ack handshaking using txX_req/ack and rxX_req/ack (described in “MPLL Configuration Update” on page 80).
`rxX_pstate = 2'b10`
`txX_pstate = 2'b10`
2. Power down the MPLL, and perform four-way request-ack handshaking using txX_req/ack (described in “MPLL Configuration Update” on page 80).
`txX_mpll_en = 1'b0`
3. Set up the PHY context configuration inputs to the appropriate values based on the desired protocol/rate for the targeted bank, power up the MPLL, and perform four-way request-ack handshaking using txX_req/ack (for MPLL and TX DCC) and rxX_req/ack (for RX DCC and IQC) (described in “MPLL Configuration Update” on page 80).



Note

When changing a bank using the `mplla_recal_bank_sel` signals, you must power up MPLLA using the `mplla_cntx_sel` which configures `txX_mpllb_sel` appropriately for the firmware to properly detect the calibration event. For example, if only the `mplla_recal_bank_sel` is changed, the `txX_mpllb_sel` configuration setting must be set to 1'b0 to denote that MPLLA is to be powered up during the txX_req/ack handshaking.

For this initial calibration phase, firmware checks the internal calibration done flag and determines that the selected bank has not been calibrated before, then triggers a calibration.

- If the `mplla_recal_force_en`, `txX_recal_force_en`, or `rxX_recal_force_en` signals are asserted, the appropriate bank is always re-calibrated.
- If the `mplla_recal_skip_en`, `txX_recal_skip_en`, or `rxX_recal_skip_en` signals are asserted, the appropriate bank is not re-calibrated, and any previously stored values are used.

For the following sequences, it is assumed that the bank had not been previously calibrated.

For MPLL/TX DCC calibrations:

- a. MPLLA calibration is performed (if selected).
- b. TX DCC calibration is performed at the current rate.
- c. Restore the recently calibrated values for the MPLLA and TX DCC based on the `txX_mpllb_sel` and `txX_rate` context configuration settings, and release overrides to perform the initial requested TX rate change.

For RX calibrations:

- a. RX DCC calibration is performed at the current rate.
- b. RX IQC calibration is performed at the current rate.
- c. Restore the recently calibrated codes for RX DCC and IQC based on the `rxX_rate` and `rxX_d-fe_bypass` context configuration settings, and release overrides to perform the initial requested RX/TX rate change.

4. Power the PHY TX and RX back to P0 state.

Figure 4-31 Initial Calibration of MPLL and TX DCC

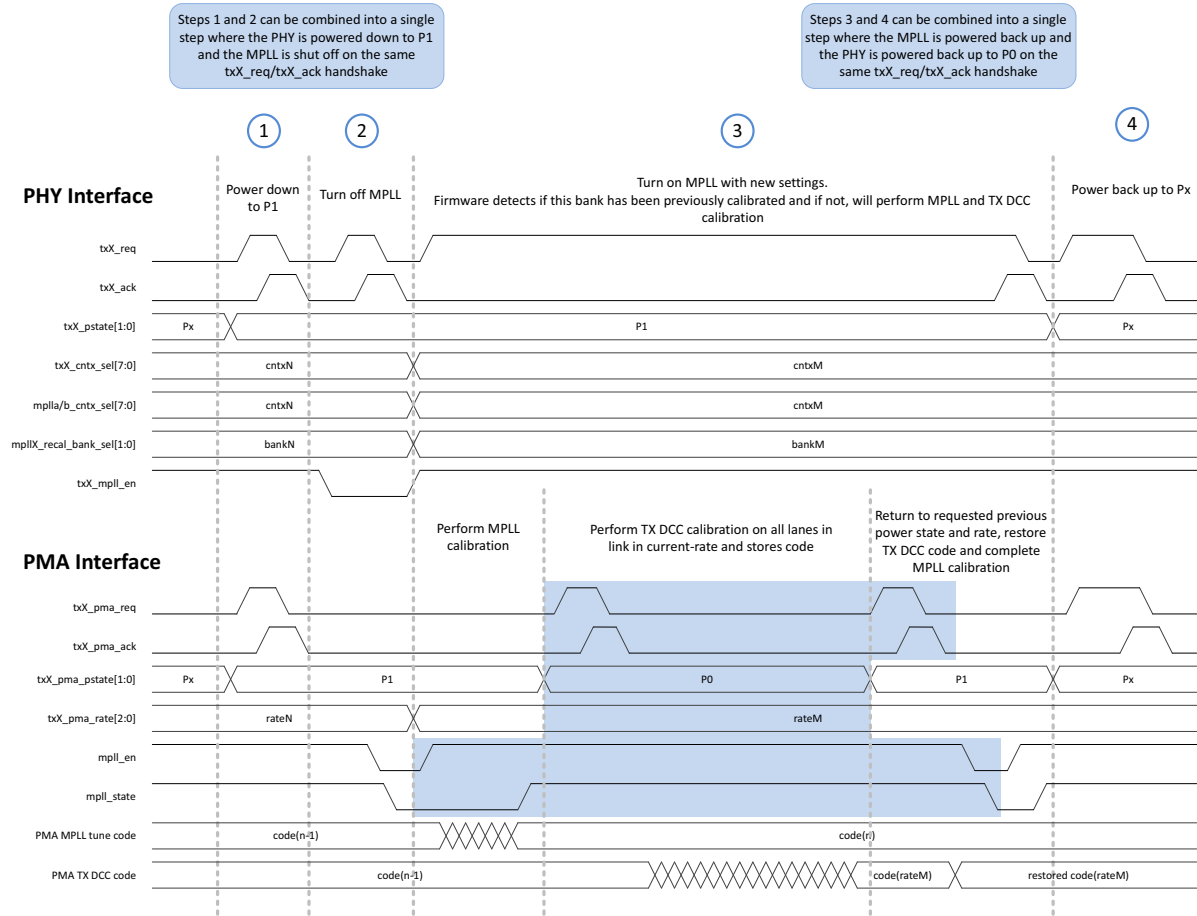
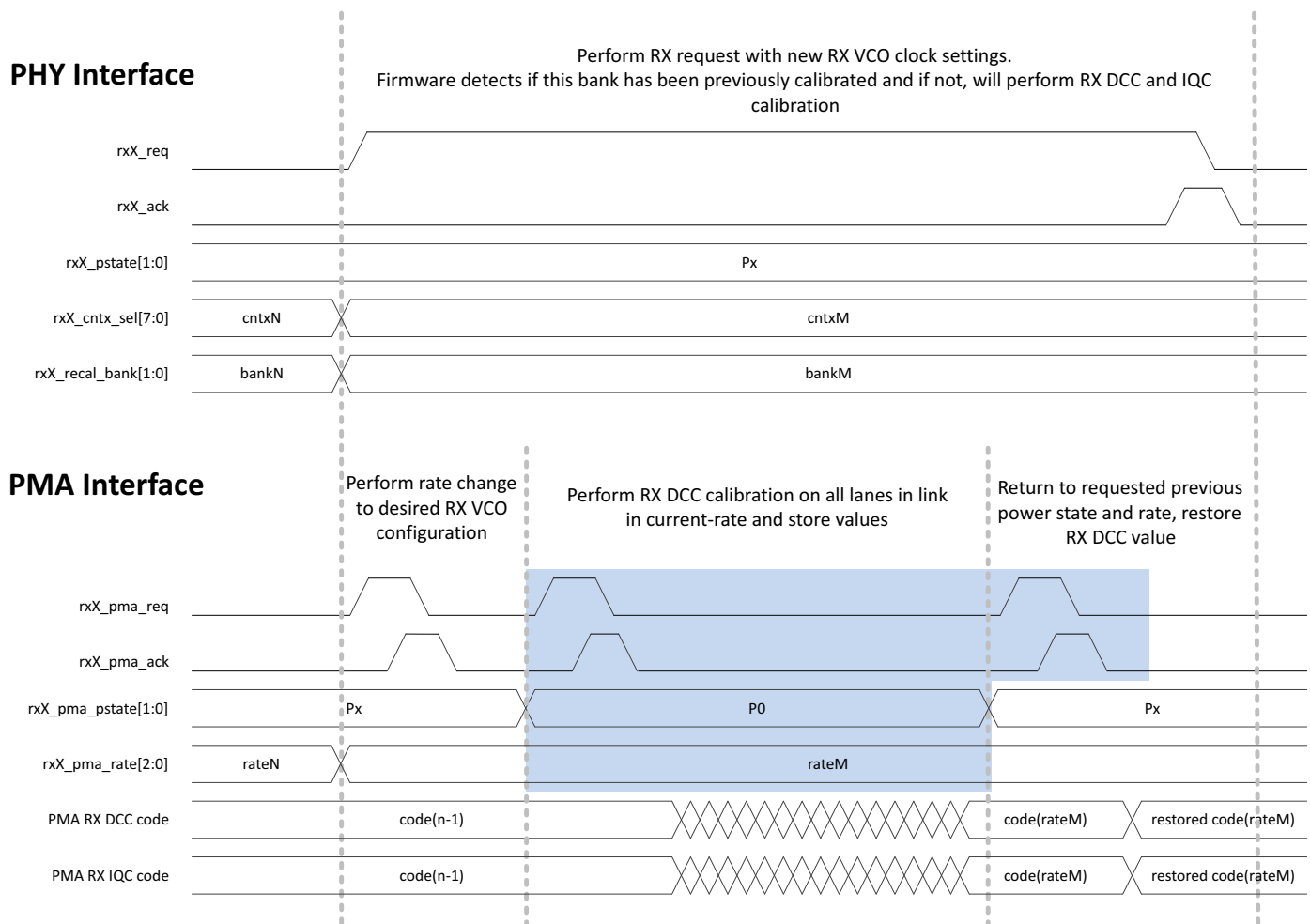


Figure 4-32 Initial Calibration of RX DCC and RX IQC

4.14.3 Subsequent Rate Changes

When performing subsequent rate changes, you have two options depending on whether the destination rate is within the same bank or in another bank.

4.14.3.1 Rate Change to a Different Bank

When performing a rate change to a different bank, a similar procedure to the one described in [“Initial Calibration”](#) on page 95 must be followed. The difference is that because the destination bank has already been calibrated, the firmware does not perform any further calibrations, but only restores the appropriate settings for the MPLL, TX DCC, RX DCC and RX IQC.

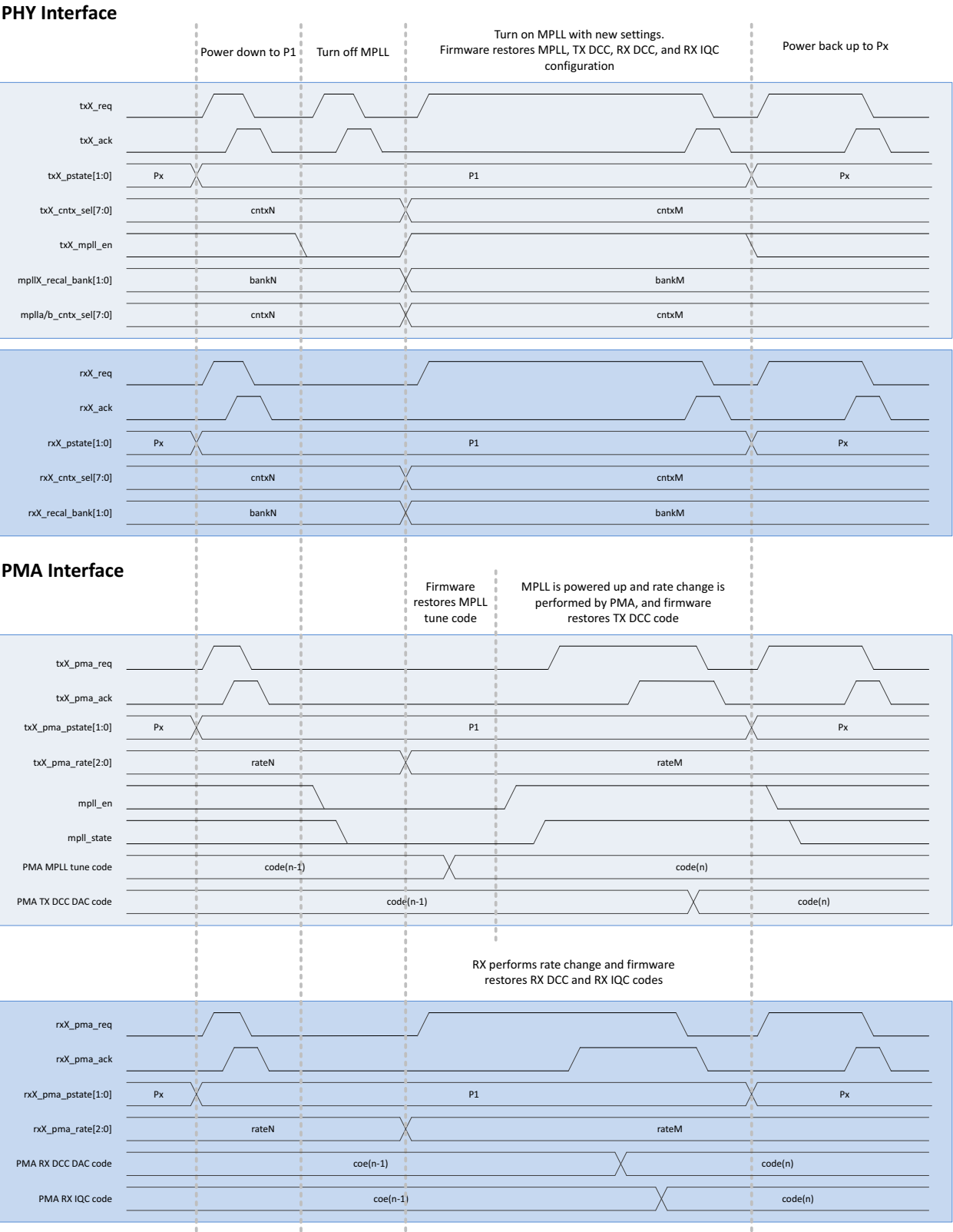
To perform a rate change to a different bank, perform the following procedure.

1. Power the PHY TX and RX into P1 state, and perform four-way request-ack handshaking using txX_req/ack and rxX_req/ack (described in [“MPLL Configuration Update”](#) on page 80).
`rxX_pstate = 2'b10`
`txX_pstate = 2'b10`
2. Power down the MPLL, and perform four-way request-ack handshaking using txX_req/ack (described in [“MPLL Configuration Update”](#) on page 80).
`txX_mpll_en = 1'b0`
3. Set up the PHY context configuration inputs to the appropriate values based on the desired protocol/rate for the targeted bank, power up the MPLL, and perform four-way request-ack handshaking using txX_req/ack (for MPLL and TX DCC) and rxX_req/ack (for RX DCC and IQC) (described in [“MPLL Configuration Update”](#) on page 80).

The firmware determines that the appropriate bank has been previously calibrated and therefore, performs the following tasks.

- a. Restore the calibrated value for the MPLL.
 - b. Perform the requested TX rate change, and restore the previously calibrated values for TX DCC based on the txX_mpllb_sel and txX_rate context configuration settings.
 - c. Perform the requested RX rate change, and restore the previously calibrated values for RX DCC and IQC based on the rxX_rate and rxX_dfe_bypass context configuration settings.
4. Power the PHY TX and RX back to P0 state.

Figure 4-33 Rate Change to a Different Bank



4.14.3.2 Rate Change (txX_rate/rxX_rate) Within the Same Bank

When performing a rate change to the same bank, there is no need to power the MPLL down and back up.

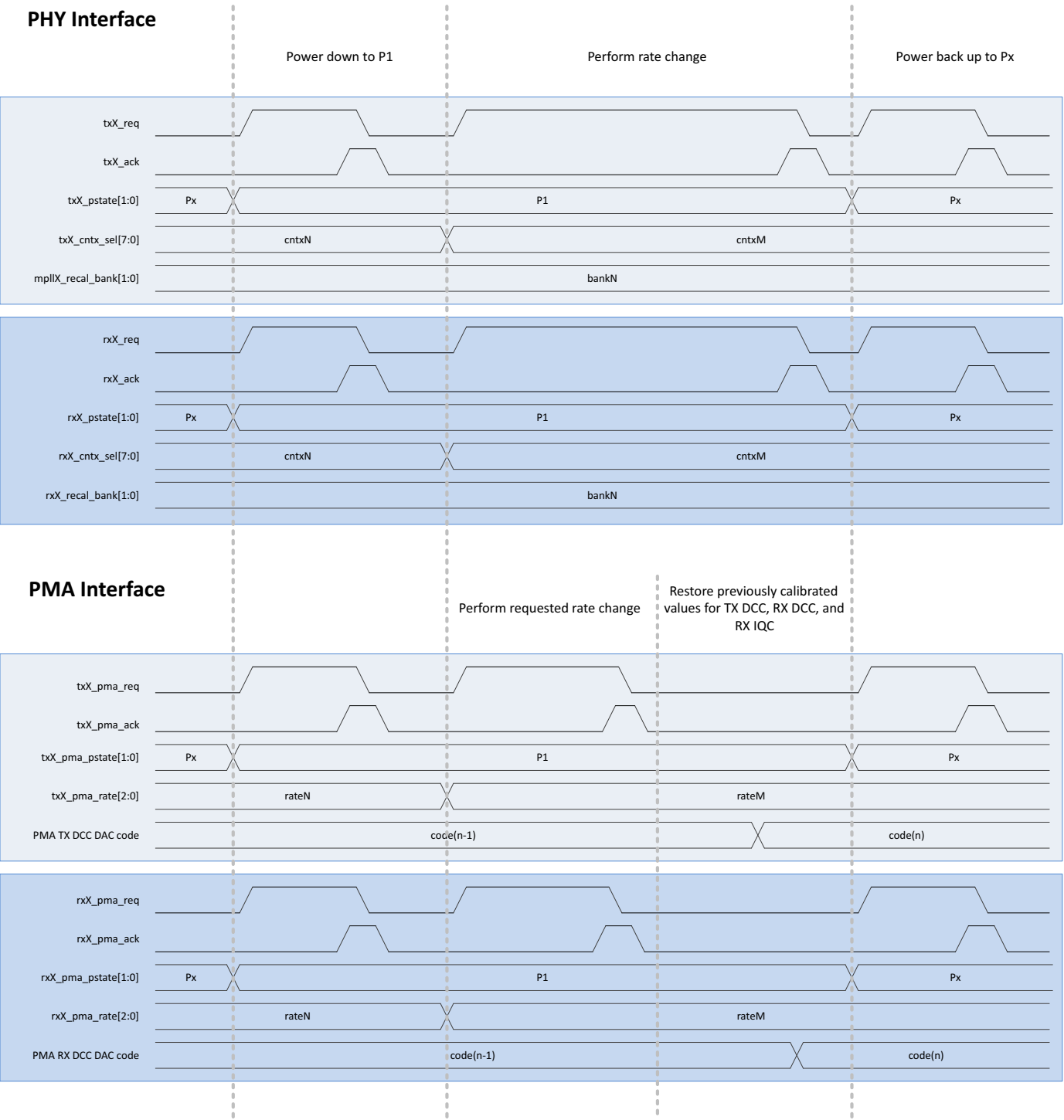
To perform a rate change to the same bank, perform the following procedure.

1. Power the PHY TX and RX into P1 state, and perform four-way request-ack handshaking using txX_req/ack and rxX_req/ack (described in [“MPLL Configuration Update”](#) on page 80).
rxX_pstate = 2'b10
txX_pstate = 2'b10
2. Set up PHY context configuration inputs to appropriate values based on the desired protocol/rate for the targeted bank, and perform four-way request-ack handshaking using txX_req/ack and rxX_req/ack (described in [“MPLL Configuration Update”](#) on page 80).

The firmware determines that the appropriate bank has been previously calibrated and therefore, performs the following tasks.

- a. Perform the requested TX rate change, and restore the previously calibrated values for TX DCC based on the txX_rate context configuration setting.
 - b. Perform the requested RX rate change, and restore the previously calibrated values for RX DCC and IQC based on the rxX_rate context configuration setting.
3. Power the PHY TX and RX back to P0 state.

Figure 4-34 Rate Change Within the Same Bank



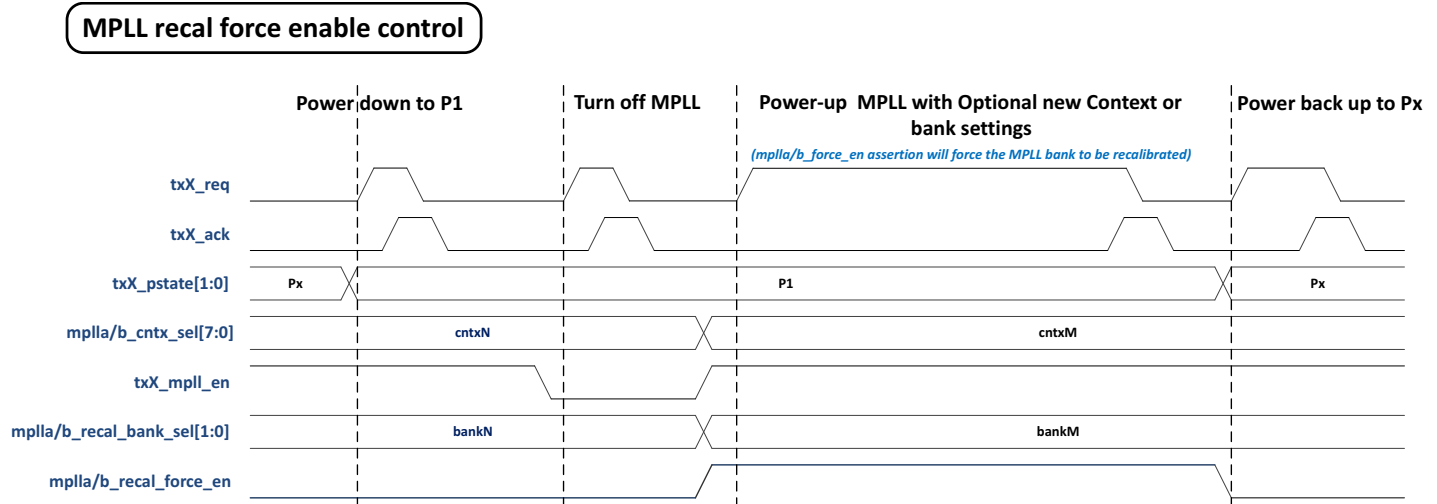
4.14.4 Force/Skip Re-Calibration

This section describes forcing MPLL, TX, and RX re-calibrations. The PHY contains a mechanism to allow for calibrations to be forced by the user, to “replace” a previous calibration result stored within a specific bank for MPLLA/B, TX or RX.

4.14.4.1 Force MPLL Re-Calibration

When using `mplla/b_recal_force_en` to force a re-calibration of an MPLL, this signal should be asserted as part of a TX request using the `txX_req/ack` handshaking signals. The MPLL will only be re-calibrated if it is the active MPLL (that is, based on the `tx_mpllb_en` context configuration setting), and if it is currently OFF. The `mplla/b_recal_force_en` is sampled on each TX request, so keeping it asserted will cause a re-calibration to be forced each time the conditions previously described are met. Figure 4-35 illustrates this sequence.

Figure 4-35 Force MPLL Re-Calibration Sequence



Note that MPLLB does not require calibration but the same logic exists for MPLLB for consistency and future use.

4.14.4.2 Force TX/RX Re-Calibration

Similar to the MPLL force re-calibration feature, the TX and RX also have the ability to force a re-calibration. When forcing a TX re-calibration, the `txX_recal_force_en` signal should be asserted as part of a TX request using the `txXrxX_req/ack` handshaking signals. When forcing an RX re-calibration, the `rxX_recal_force_en` signal should be asserted as part of an RX request using the `rxX_req/ack` handshaking signals.

Note that the `txX/rxX_force_recal_en` signals have a lower priority than the `txX/rxX_dcc_bypass` context configuration signal. If a specific protocol/rate has this context signal asserted, this implies that calibrations will always be skipped for this specific protocol/rate.

4.14.4.3 Skip Re-Calibration

The PHY also contains a mechanism to skip calibrations for the MPLLA/B, TX, or RX. By asserting the `mplla/mpllb/txx/rxx_recal_skip_en` signal, the associated calibrations will be skipped for the specific bank. When skipping a calibration, the PHY will use whatever previously calibrated (or uncalibrated) code exists within the AON domain storage. If a bank was never previously calibrated, this implies that a default mid-code value will be used. Synopsys does not recommend skipping any calibrations, unless it is guaranteed that the PHY will not be used to transmit/receive data in that appropriate protocol/rate.

4.15 TX and RX Power States

The PHY supports independent power state controls for each lane and TX and RX within each lane. There are four power states supported for the TX and RX within each lane: P0, P0s, P1, and P2.

4.15.1 TX and RX Power State Description

The operational modes of the lane in each power state are provided in [Table 4-14](#) and [Table 4-15](#).

Table 4-14 TX Power State Descriptions

TX Power State	Description
P0	P0 is the fully operational power state for the TX. In the state: 1. txX_pstate is set to P0. 2. txX_reset, txX_disable inputs must be de-asserted. txX_lpd can be asserted to disable a lane in a link. 3. txX_mpll_en must be set to 1. 4. If tx_data_en is asserted, TX driver is enabled and serial data is transmitted. 5. txX_clk input must be valid and of correct frequency prior to de-asserting txX_req input when requesting entry into P0 state (shown in Figure 4-25 on page 83). 6. Appropriate MPLL clocks, TX clocks, and TX datapath are enabled. 7. MPLL is powered up. 8. When MPLL{A/B} is powered up (as described in “TX Datapath” on page 84), the following clocks are enabled and valid: ■ mpla_div16p5_clk, mpla{a/b}_div33_clk, mpla{a/b}_div66_clk, when mpla_div16p5_clk_en is asserted ■ mpla{a/b}_div_clk, when mpla{a/b}_div_clk_en is asserted ■ mpla{a/b}_word_clk, mpla{a/b}_dword_clk, mpla{a/b}_qword_clk, mpla{a/b}_oword_clk
P0s	Same state as P0 with the following exceptions: 1. txX_pstate is set to P0s 2. tx_data_en must be de-asserted
P1	Same state as P0s with the following exceptions: 1. txX_pstate is set to P1. 2. txX_mpll_en/txX_disable may be set to 0/1, respectively. 3. MPLLs may be powered down if txX_mpll_en=0 and no other lane is requesting MPLLs to be powered up
P2	Same state as P1, with following exceptions: 1. txX_pstate is set to P2. 2. MPLL{A/B} is powered down if txX_mpll_en=0 and no other lane is requesting MPLLs to be powered up. In this case, MPLL clock outputs may be disabled. 3. MPLL{A/B} is powered up if txX_mpll_en=1. In this case, MPLL clock outputs are disabled.

Table 4-15 RX Power State Descriptions

RX Power State	Description
P0	All RX circuits are powered up. - When rxX_data_en is asserted, the RX data recovery and de-serialization is enabled. - rxX_reset and rxX_disable inputs must be de-asserted. rxX_lpd can be asserted for disabling lane while link in P0. rxX_term_en must be asserted. - rxX_clk is valid.
P0s	Same state as P0 with the following exceptions: - RX CDR and datapath are disabled. - rx0_lpd and rxX_disable inputs may be asserted. - rxX_data_en must be de-asserted. - rxX_clk is disabled.
P1	Same state as P0s with the following exceptions: - RX VCO is powered down. - rx_reset, rx0_lpd and rxX_disable inputs may be asserted.
P2	Same state as P1, with following exceptions: RX analog front end is powered down.

4.15.2 Power State Transition Times for TX and RX

The time taken for the PHY to assert txX_ack and rxX_ack in response to a txX_req and rxX_req corresponding to a txX_pstate and rxX_pstate transition is referred to as the power state transition time. The power state transition times are dependent upon the reference clock frequency, txX_clk/rxX_clk frequency, and ref_range input. In addition, the power state transition times are dependent upon a range of input and parameters. These include, but are not limited to: register settings, reference clock frequency, MPLL clock frequency, RX VCO clock frequency, lane divider settings.

[Table 4-28](#) lists the normal and fast-sim power state transition times for M-PHY modes at 19.2 MHz, 26 MHz, 38.4 MHz, 52 MHz and 104 MHz reference clocks. For other reference clocks contact Synopsys.

Table 4-16 Power State Transition and Receiver Detection Times in M-PHY Mode

Transition	Latency 19.2 MHz (us)		Latency 26 MHz (us)		Latency 38.4 MHz (us)		Latency 52 MHz (us)		Latency 104 MHz (us)	
	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim
P0 -> P0S	2.00	2.00	1.50	1.50	1.50	1.50	1.50	1.50	1.00	1.00
P0 -> P1	2.50	2.50	2.00	2.00	2.00	2.00	1.50	1.50	1.50	1.50
P0 -> P2	2.50	2.50	2.00	2.00	2.50	2.50	2.00	2.00	2.00	2.00
P0S -> P0	4.00	2.50	3.00	2.00	4.00	2.00	3.00	1.50	2.50	1.50
P0S -> P1	2.50	2.50	2.00	2.00	2.00	2.00	1.50	1.50	1.50	1.50
P0S -> P2	2.50	2.50	2.00	2.00	2.50	2.50	2.00	2.00	2.00	2.00

Transition	Latency 19.2 MHz (us)		Latency 26 MHz (us)		Latency 38.4 MHz (us)		Latency 52 MHz (us)		Latency 104 MHz (us)	
	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim	Normal	Fast-Sim
P1 -> P0	25.00	22.00	23.50	21.50	24.50	21.50	23.00	21.00	23.00	20.50
P1 -> P0S	24.50	22.00	22.00	21.50	23.00	21.50	22.00	21.00	21.50	20.50
P1 -> P2	2.00	2.00	2.00	2.00	2.00	2.00	1.50	1.50	1.50	1.50
P2 -> P0	56.00	26.50	46.50	24.50	55.00	25.00	46.00	24.00	45.50	23.50
P2 -> P0S	54.50	26.50	45.00	24.50	53.00	25.00	44.50	24.00	44.00	23.50
P2 -> P1	32.00	5.50	24.00	4.00	31.50	4.50	23.50	4.00	23.50	3.50
P1.CPM->P0	63.00	30.50	52.00	27.50	60.00	27.50	50.50	26.00	49.50	24.50
P1.1->P0	63.00	30.50	52.00	27.50	60.00	27.50	50.50	26.00	49.50	24.50
P1.2->P0	70.50	30.50	58.00	27.50	67.50	27.00	56.00	25.50	55.00	24.50
P1.2->P0 (PG)	79.00	45.50	68.00	42.00	76.00	42.50	66.00	40.50	65.00	39.50

Table 4-17 details the exit latencies for transition times between L1 substates power states.

Table 4-17 Substate Exit Latencies and Timing Information

Latency/Transition	Description	Value	Unit
T_COMMON_MODE	Worst-case time for the Common mode voltage to be reached on txX_{p,m} pins while transmitting in Recovery mode. This parameter applies to an exit from P1.2 through P1 to P0 (Recovery) and represents the extra time in which the MAC is required to send TS1 ordered sets.	60	μs

Table 4-18 provides the power state mapping.

Table 4-18 Power State Mapping

Protocol	PHY Power state	TX				RX							SUP	PGATE
		tx_reset	tx_pstate	tx_mpll_en	tx_disable	rx_reset	rx_pstate	rx_disable	rx_term_en	rx_sigdet_lf_en	rx_sigdet_hf_en	rx_sq_en	ref_clk_en	pg_reset
MPHY	HS-BURST	0	0	1	0	0	0	0	1	0	0	0	1	0
	STALL	0	0	1	0	0	0	0	1	0	0	0	1	0
	SLEEP-1 ^a	0	3	0	0	0	1	0	0	0	0	0	1	0
	SLEEP-2 ^a	0	3	0	0	0	2	0	0	0	0	0	0	0
	SLEEP-3 ^a	0	3	0	0	0	1	0	0	0	0	0	1	0
	H8	0	3	0	1	0	3	1	0	0	0	1	0	0
	H8ULP	0	3	0	1	0	3	1	0	0	0	1	0	1

a. Three options are available for SLEEP: SLEEP-1: Host/SLEEP/PWM-BURST; SLEEP-2: Device/SLEEP/PWM-BURST/Before PMC-HS; SLEEP-3: Device/SLEEP/PWM-BURST/After PMC-HS

[Table 4-14](#) and [Table 4-15](#) show the settings of PHY inputs after the PHY has entered the corresponding power-state. While transitioning between power states, the PHY inputs require the appropriate sequencing in order to meet the following PHY requirements:

- txX_disable and rxX_disable must not be asserted before the txX_pstate and rxX_pstate transition into the P1 state is completed (indicated by the assertion of txX_ack and rxX_ack).
- ref_clk_en must not be de-asserted before txX_mpll_en is de-asserted and the four-way txX_req/ack handshaking has completed.
- In P0 state, the TX datapath and RX CDR are enabled by asserting txX_data_en and rxX_data_en inputs. These can be enabled only in the active power state and must be de-asserted in all other power states.

4.16 TX Equalization and Adaptation

TX equalization (EQ) and transmit levels are controlled by the txX_eq_post[5:0], txX_eq_pre[5:0], and txX_eq_main[5:0] input signals. These inputs are derived from the coefficients C_{-1} , C_1 , and C_0 using the following equations.

Equation:

TX Equalization input mapping to specification parameters

$$\text{txX_eq_pre}[5:0] = |C_{-1}|$$

$$\text{txX_eq_post}[5:0] = |C_1|$$

$$\text{txX_eq_main}[5:0] = C_0$$

The txX_eq_post[5:0], txX_eq_pre[5:0], and txX_eq_main[5:0] inputs can be updated in a single txX_clk cycle to a known valid value. Alternately, the inputs can be sequenced to free up resources in the following order: Increment C_1 , Increment C_{-1} , Decrement C_0 , Decrement C_1 , Decrement C_{-1} and then Increment C_0 . An invalid value might cause TX serial data corruption in simulation.

4.16.1 TX Equalization Coefficient Mapping

Contact Synopsys for more information about Equalization Coefficient Mapping.

4.16.2 TX Equalization, Initialization and Presets

The information in this section is preliminary and subject to change. Contact Synopsys for more information.

The TX equalization input settings and TX margining settings are provided in [“TX Equalization Values”](#).

The PMA hard-macro has a total of 96 legs for TX Equalization distributed into the pre, main, and post cursors.

The mapping of 48 legs to 96 legs is translated in the Raw PCS soft macro. The PHY phy_txX_eq_pre/main/post[5:0] (6-bit) inputs from the protocol PCS soft macro are LSB padded with a 1'b0 before driving the PMA pma_txX_eq_pre/main/post[6:0] (7-bit) inputs.

4.16.3 TX Equalization and Adaptation

This section describes how the TX equalization inputs to the PHY should be updated when a coefficient update request is received from the link partner (far-end receiver) as part of link adaptation.

4.17 RX Equalization and Adaptation

The PHY supports receiver equalization, adaptation of receiver parameters, and commands for incrementing or decrementing the transmitter coefficients of the link partner. For protocols that require DFE and link adaptation, the receiver adaptation is triggered by asserting the rxX_adapt_req input. Upon completion of the receiver adaptation, the PHY asserts rxX_adapt_ack and uses Figure of Merit (FOM) based link adaptation/direction change to adapt the transmitter coefficients of the link partner:

- Figure of Merit (FOM) – An 8-bit rxX_adapt_fom[7:0] output indicating the quality of the received data eye. A higher value of rxX_adapt_fom [7:0] indicates better link equalization, with 8'd0 indicating worst equalization setting and 8'd255 indicating the best equalization setting.



Note

The PHY does not support direction change based adaptation of TX equalization.

Receiver adaptation is supported only when the PHY is in P0 state, electrical idle exit is detected on the RX serial lines, rxX_data_en is asserted, and the clock and data recovery (CDR) circuitry is nominally locked (indicated by the assertion of rxX_valid output. When rxX_data_en is enabled during the equalization phase, for some equalization settings the CDR might drift and not be able to recover. As a result, it is recommended to assert rxX_reset for at least 8 ns between performing RX adaptation requests.

The recommended sequence is as follows.

1. Assert rxX_reset to the PHY.
2. Wait for de-assertion of rxX_ack from the PHY.
3. Ensure the PHY lane receiver is in P0 state. Transition to P0, if not already in P0 out of reset.
4. Wait for detection of electrical idle exit condition on rxX_sigdet_lf (for low-frequency data).
5. Assert rxX_data_en to the PHY.
6. Wait for assertion of rxX_valid from the PHY.
7. Perform an RX adaptation request and assert rxX_adapt_req.

The PHY performs RX adaptation, then signals the completion by asserting rxX_adapt_ack.

8. De-assert rxX_adapt_req to the PHY.



Note

- rxX_adapt_req should not be asserted unless rxX_dfe_bypass = 0
- Aborting rxX_adapt_req by negating before rxX_adapt_ack is asserted or by asserting rxX_reset can leave the receiver with a less than optimal settings.
- No requests should be sent to the PHY using rxX_req/rxX_ack while rxX_adapt_req is asserted.
- The internal RX CDR PPM detector is disabled during adaptation requests. Any external PPM detectors must also be turned off during the adaptation process.

For protocols that require link adaptation, the PHY supports receiver equalization, adaptation of receiver equalization parameters, and requests for incrementing or decrementing the far-end transmitter coefficients.

The PHY adapts RX AFE parameters, DFE coefficients, and remote TX equalization in two steps – startup adaptation and activation mode adaptation.

Startup Adaptation

Startup adaptation starts when the RX is powered up with valid data received and with CDR locked. First, AFE gain, including ATT and GAIN parameters, are adapted based on a measurement of the average eye opening. Then, the CTLE BOOST is adapted using correlations. After CTLE is adapted, DFE tap coefficients are adapted using correlations. Finally, the Figure of Merit is measured. Directions for the pre-cursor and post-cursor TX equalization are directed, and this information is returned to the controller, along with the Figure of Merit, to enable back-channel remote TX adaptation.

Mission Mode Adaptation

The goal of mission mode adaptation is to continuously fine-tune the system. For AFE parameters (such as gain control) during mission mode adaptation, CTLE boosting is not changed unless the eye opening exceeds some preset range. Also during mission mode adaptation, DFE coefficients are adapted continuously by the least mean squares (LMS) algorithm.

4.18 Test Modes

The test modes supported in the PHY and the input configuration settings for each mode are provided in [Table 4-19](#).

Table 4-19 PHY Test Mode Selection

PHY Mode of Operation	bs_ce	test_burnin	test_powerdown	test_flyover_en	test_tx_ref_clk_en	test_stop_clk_en	scan_mode	scan_shift*
“Boundary Scan (ACJTAG)”	1	X	X	X	X	X	X	X
“Burn-In”	0	1	0	0	0	0	0	0
“IDDQ”	0	0	1	0	0	0	0	0
“Flyover”	0	0	0	1	0	0	0	0
“Reference Clock Over TX”	0	0	0	0	1	0	0	0
“Scan Dump”	0	0	0	0	0	1	0/1	0/1
“Scan (At-Speed, Stuck-At)”	0	0	0	0	0	0	1	0/1
“Diagnostics” (serial and parallel loopback)	0	0	0	0	0	0	0	0
Normal Operation	0	0	0	0	0	0	0	0

The following sections describe each test mode in more detail.

4.18.1 Boundary Scan (ACJTAG)

The PHY includes boundary scan registers per lane, which provide direct controllability and observability of the txX_p/m and rxX_p/m lines. The boundary scan instructions supported in the PHY are SAMPLE, PRELOAD, BYPASS, EXTEST_PULSE and EXTEST_TRAIN as defined in IEEE Std 1149.6-2003.

The states of all inputs of the PHY except bs_* and power supplies are ignored in boundary scan mode.

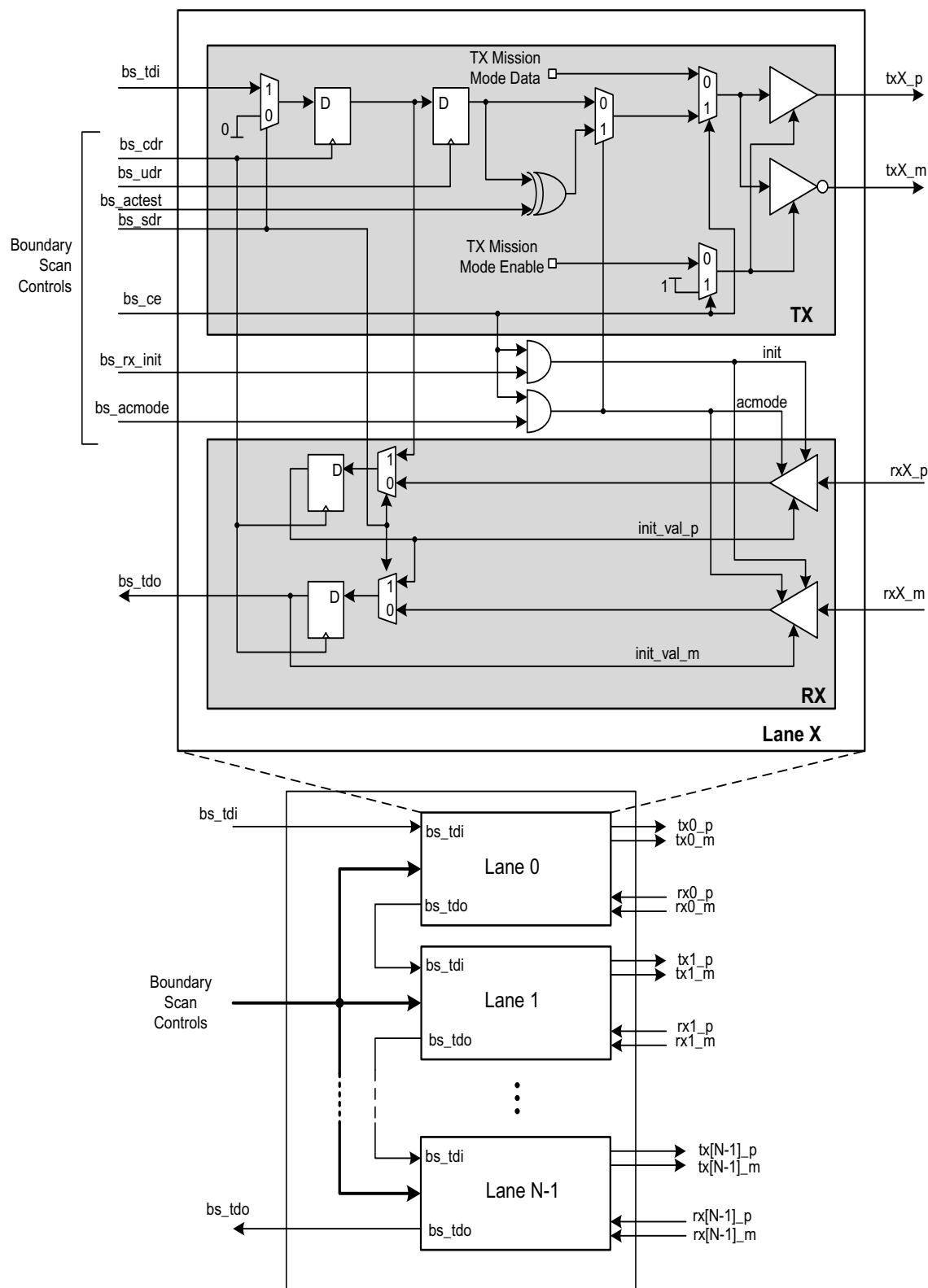
[Figure 4-36](#) shows the logical structure of the boundary scan implementation in the N-lane PHY. The TX driver control logic is similar to the Output Driver Example defined in Sec 6.3.2. IEEE Std 1149.6-2003. The TX common mode is enabled in boundary scan mode. There is no single bit register bypass logic within the boundary scan logic of the PHY, but it can be added externally if required.

[Table 4-20](#) and [Table 4-21](#) on page 114 show the low and big swing mode trip levels for the ACJTAG Input Receiver. These tables provide the minimum and maximum trip levels over PVT corners for each setting. The low swing mode and big swing mode are intended for half swing (that is, 400-600 mVppd) and full swing (800-1200 mVppd) protocols, respectively. These tables are only a reference, and the choice of trip level is determined based on customer's boundary scan implementation.



Note

- For ACJTAG DC mode, use bs_rx_level[2:0]=3'b101(decimal 5) for a DC trip level between 85 mV and 115 mV.
- ACJTAG DC mode is supported only for common-mode of 0 V.

Figure 4-36 Boundary Scan Structure

**Note**

ACJTAG low and big swing mode trip levels will be available in a future release.

Table 4-20 ACJTAG - Low Swing Mode Trip Levels (Preliminary)

bs_rx_level[2:0]	bs_rx_bigswing	Minimum Trip Level (mVppd)	Maximum Trip Level (mVppd)
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-

Table 4-21 ACJTAG - Big Swing Mode Trip Levels (Preliminary)

bs_rx_level[2:0]	bs_rx_bigswing	Minimum Trip Level (mVppd)	Maximum Trip Level (mVppd)
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-

4.18.2 Burn-In

In burn-in mode, all analog blocks are powered up, all clocks are enabled, the analog front end is put into TX-to-RX Loopback mode, and the RX pattern matcher is enabled. In this mode, the TX datapath is driven by a fixed high-frequency Nyquist pattern (40'hAA_AAAA_AAAA). This mode turns on an on-chip oscillator that is used as a reference clock for the MPLLs and the RX VCO. Note that the RX pattern matcher is enabled only to generate additional internal toggling of signals and is set to an LFSR31 pattern. It is expected that the RX pattern matcher generates errors in this mode due to the lack of precision in the on-chip oscillator. This mode is intended for use during high-temperature operating life (HTOL) or Burn-in, and sets up the PHY to toggle as much circuitry as possible (for realistic hot carrier injection degradation - HCI), while unused circuitry is carefully biased in its power-down mode to ensure realistic degradation for NBTI (negative bias temperature instability).



Note

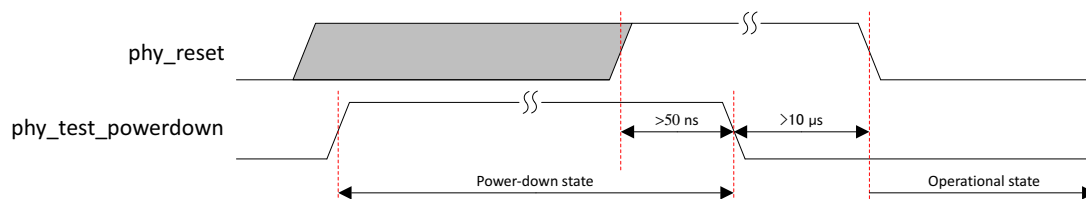
Prior to enabling the burn-in mode by asserting test_burnin, phy_reset must be asserted and de-asserted to ensure the states of internal registers are reset.

4.18.3 IDDQ

In IDDQ test mode, all the analog blocks are powered down to allow leakage measurement. Input settings for enabling the IDDQ mode are provided in [Table 4-19](#) on page 112. States of all other inputs are ignored.

It is recommended that phy_reset be asserted for at least 50 ns before test_powerdown is negated and remain asserted for at least 10 μ s.

Figure 4-37 PHY Power-Down Sequence



4.18.4 Flyover

Settings to enable flyover mode are provided in [Table 4-19](#) on page 112.

The flyover mode can be used for the following functions:

- Send scan data on and off die through the PHY TX and RX IO pins
- Output of high frequency clocks and data for observation and debug purposes

On the TX-side, the flyover mode enables you to asynchronously send an on-die signal off-chip from the txX_flyover_data_p and txX_flyover_data_m ASIC-side inputs to the txX_p and txX_m outputs. In this mode, a reference clock is not required.

The txX_flyover_data_p and txX_flyover_data_m inputs support up to 1 GHz differential. The output stage at the TX pads is common to the mission mode data path; the speed limitation of differential signaling is not in the analog driver. To ensure you are driving the analog driver differentially, keep the skew between the

complementary signals from SOC lower than 15 ps, or they end up being single-ended signaling and speed will be limited to a maximum of 200 MHz.

On the RX-side, the flyover mode enables you to asynchronously send an off-chip signal on-die through the rxX_p and rxX_m inputs to the rxX_flyover_data_p and rxX_flyover_data_m ASIC-side outputs. In this mode, a reference clock is not required as the signal is received asynchronously using the test receiver.

The flyover mode supports up to 200 MHz single-ended data on both the RX and the TX data paths and up to 1 GHz TX output when driven with a complementary signal. Each of the independent RX inputs on rxX_p and rxX_m should have a common mode of 0 V and a signal swing of 400–600 mV.

4.18.5 Reference Clock Over TX

It is possible to send the reference clock out of the differential TX high-speed serial output pins to observe the quality of the reference clock reaching the PHY. This is done by asserting the test_tx_ref_clk_en input pin.

To use this feature, the PHY should be placed in power state P1, and the txX_beacon_en input signal should be set to 1'b0 to ensure the PHY is not transmitting low frequency signaling. This is to ensure that all of the required analog blocks are powered up to transmit the reference clock over the TX high-speed serial pins.

Alternatively, the reference clock over TX feature can be enabled by setting the following registers and pins:

- LANEN_DIG_ANA_XF_TX_OVRD_OUT_1.TX_ANA_REFGEN_EN_OVRD_EN[1] = 1'b1
- LANEN_DIG_ANA_XF_TX_OVRD_OUT_1.TX_ANA_REFGEN_EN[0] = 1'b1
- ref_clk_en = 1'b1 to ensure the prescaler is powered up
- ref_use_pad = A fixed value to ensure that the reference clock source is constant
- txX/rxX_disable = 1'b0 to ensure the bandgap is powered up
- txX_beacon_en = 1'b0 to ensure that the PHY is not transmitting LFPS
- txX_async_en = 1'b0 to ensure that the PHY is not transmitting async data



Note

For additional information refer to “Signal Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

4.18.6 Scan (At-Speed, Stuck-At)

Settings to enable scan mode are provided in [Table 4-19](#) on page 112. To achieve the coverage metrics provided below, all other inputs must be controllable and all outputs must be observable.



Note

PMA .lib ETM modes:

- NORMAL—For Functional mode
- SCAN—For Stuck-at Shift and Capture mode
In this mode, Shift and Capture happens at Scan Clock Frequency.
- SCAN_ASST—For At Speed Scan Testing
In this mode, Shift occurs at Scan Clock Frequency, whereas Capture occurs at Functional mode frequency.

Note the following scan features and requirements in the PHY:

- The PHY provides independent scan clock inputs per internal clock domain.
- Primary clock inputs to the PHY must have scan clocks supplied to them externally in scan mode.
- All clock outputs from the PHY, except `ref_repeat_clk_p/m`, have a scan clock input muxed on them.
- Synchronous inputs and outputs are at-speed scannable.
- All scan chains can be shifted in/out at the same time. By default, only one-hot scan clock capture is supported. Customers have the option to externally group the scan clocks, which are logically independent and share the same frequency. The ATPG flow in the deliverables provide an optional static clock grouping of the scan clocks. By default, no clock grouping is enabled for the vectors and scripts that are provided with the product. For more information on enabling clock grouping, see the `./pma/atpg/<metal-stack>/readme_tmax.txt` file.
- All resets (primary input resets or internally generated resets) are over-ridden to primary scan reset input `scan_set_rst` in scan mode.
- Maximum scan shift frequency is 200 MHz.
- Maximum scan chain length is 100 flops.
- All scan chains are positive-edge triggered.
- All scan chains are terminated with lockup latches.
- All analog blocks are powered down in scan mode. To prevent propagation of logic X's in the scan chain, the outputs of all the analog blocks feeding the digital scannable logic are over-ridden to known states.
- Stuck-At-Fault (SAF) coverage of the PMA is greater than 98%.
- Transition-Delay-Fault (TDF) coverage of the PMA is greater than 85%.
- Coverage of the soft-macro (Raw PCS) is place-and-route implementation dependent.

4.18.6.1 On-Chip Clocking Generation

The hard macro PMA supports on-chip clocking (OCC) generation, allowing the internal PLL to be powered up and used as a high-speed clock source for the PHY input scan clocks. However, make sure that the external clock dividers are added outside of the PHY to generate the additional divided frequencies. Also, ensure that you add OCC controllers during DFT insertion to control the scan clocks before they propagate back to the hard macro PMA.

The sequence to enable OCC for TDF testing is as follows:

1. Power up PHY to P0 to calibrate and power up MPLLA (`txX_mpllb_sel = 1'b0` and `txX_mpll_en = 1'b1`) using the `txX_req/txX_ack` handshake.
2. Set `scan_pma_occ_en` to `1'b1`, which latches the MPLLA configuration.
3. Wait for 100 `ref_clks`, which is required to allow internal modulated MPLL signals to stabilize.
4. Set `scan_mode` to `1'b1`.
5. Perform TDF testing by controlling `scan_shift` and `scan_shift_cg`.

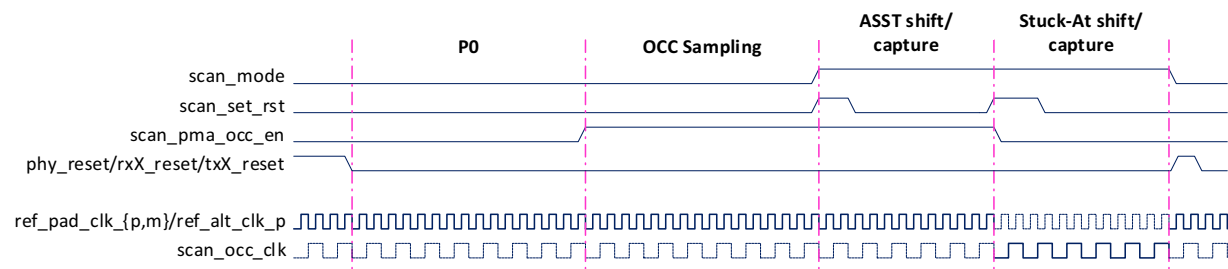
The sequence to perform SAF testing is as follows:

1. Set `scan_pma_occ_en` to `1'b0`, which disables the OCC latching.
2. Set `scan_mode` to `1'b1`.

3. Perform SAF testing by controlling scan_shift and scan_shift_cg.

A waveform outlining the sequence for OCC with TDF testing, followed by SAF testing is shown in [Figure 4-38 \(“OCC_Waves”\)](#).

Figure 4-38 OCC_Waves



The OCC logic cannot be tested at-speed. For SAF testing, the OCC hold logic is on a separate scan chain so it can be shifted and captured independently. This occurs when the scan_pma_occ_en signal = 1'b0, and scan_mode = 1'b1.

4.18.6.2 At-Speed Scan Clock Input Frequencies

Table 4-22 outlines the maximum supported clock frequencies for the various scan input clock ports of the PHY. The expectation is that all of these scan clock input frequencies can be derived from the provided on-chip clock generation clocks described in “On-Chip Clocking Generation” on page 117. Table 4-23 provides details of the scan clock outputs during at-speed scan.

Table 4-22 Scan Clock Inputs (Max)

Name	Max At-speed Scan Frequency (MHz)	Max Scan Shift Frequency (MHz)	Max Stuck-At Scan Frequency (MHz)	Notes
scan_clk	200	200	100	Primary single scan clock input that can be used for scan shift and stuck-at capture for all scan chains. Up to 200 MHz supported for scan shift. Up to 100 MHz supported for scan capture.
scan_ref_clk	104	200	100	At-speed scan capture group. Derived from ref_dig_fr_clk
scan_mpll_dword_clk	1167.36	200	100	At-speed scan capture group. Derived from mp1la/b_word_fr_clk
scan_rx_dword_clk	1167.36	200	100	At-speed scan capture group. Derived from mp1la/b_word_fr_clk
scan_mpll_qword_clk	583.68	200	100	At-speed scan capture group. Derived from mp1la/b_word_fr_clk divided by 2
scan_cr_clk	400	200	100	At-speed scan capture group. Derived from fastest available source between scan_ref_clk, fw_clk, apbN_pclk, jtag_tck
scan_occ_clk	N/A	200	100	Not supported for at-speed scan capture. Scan clock for on-chip-clock generation latch-and-hold circuits.
fw_clk	400	200	100	Top-level input clock
apbN_pclk	400	200	100	Top-level input clock
jtag_tck	100	200	100	Top-level input clock

Table 4-23 Scan Clock Outputs (Max)

Name	Source Scan Clock	Max Frequency (MHz)
phy_ref_dig_clk	scan_ref_clk	104
ref_dco_clkcal	scan_mpll_dword_clk	1167.36
txX_pwm_clk	scan_mpll_dword_clk	1167.36

Name	Source Scan Clock	Max Frequency (MHz)
txX_pwm_bit_clk	scan_ref_clk	104
txX_pwm_word_clk	scan_ref_clk	104
mplla/b_word_clk	scan_mpll_dword_clk	1167.36
mplla/b_dword_clk	scan_mpll_dword_clk	1167.36
mplla/b_qword_clk	scan_mpll_qword_clk	583.68
mplla/b_oword_clk	scan_mpll_qword_clk	583.68
mplla/b_div_clk	scan_mpll_dword_clk	1167.36
mplla_div16p5_clk	scan_mpll_dword_clk	1167.36
mplla_div33_clk	scan_mpll_qword_clk	583.68
mplla_div66_clk	scan_mpll_qword_clk	583.68
mpllb_hdmi_pixel_clk	scan_mpll_dword_clk	1167.36
txX_word_clk	scan_mpll_dword_clk	1167.36
rxX_clk	scan_rx_dword_clk	1167.36
rxX_div16p5_clk	scan_mpll_qword_clk	583.68
rxX_pwm_clk	scan_ref_clk	104
rxX_pwm_word_clk	scan_ref_clk	104

4.18.7 Scan Dump

The hard macro PHY provides a scan-dump feature which allows the internal clocks in the design to be stopped and the current state of the PHY to be shifted out and observed. Usage of the scan-dump feature is described below:

1. Assert test_stop_clk_en to stop the clocks.
2. Wait for 250 ns to allow the internal clocks to be gated. Note that this wait time is based on the slowest supported reference clock.
3. Assert scan_mode to switch the clock muxes and make clock-gating latches transparent.
4. Assert scan_shift_cg and scan_shift signals.
5. Toggle scan clocks to shift data out of scan outputs.

The PHY must be reset by pulsing the phy_reset signal to exit from scan-dump mode.



Note

For additional information refer to the “Signal Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

4.18.8 Diagnostics

The PHY provides the following diagnostic features to enhance test bench characterization and ATE testing.

- Loopback functions (independent per-lane)
- TX-to-RX serial (pre-driver) loopback
- RX-to-TX parallel loopback
- BERT (independent per-lane)
- 7th, 9th, 11th, 15th, 16th, 23rd, and 31st order polynomial pattern generation and recognition
- Generation of simple test patterns
- Error insertion and detection

4.18.8.1 Loopback Functions

Each lane in the PHY supports a TX-to-RX and an RX-to-TX loopback mode. Only one loopback mode can be enabled at any given time. The following conditions must be met prior to enabling the loopback modes:

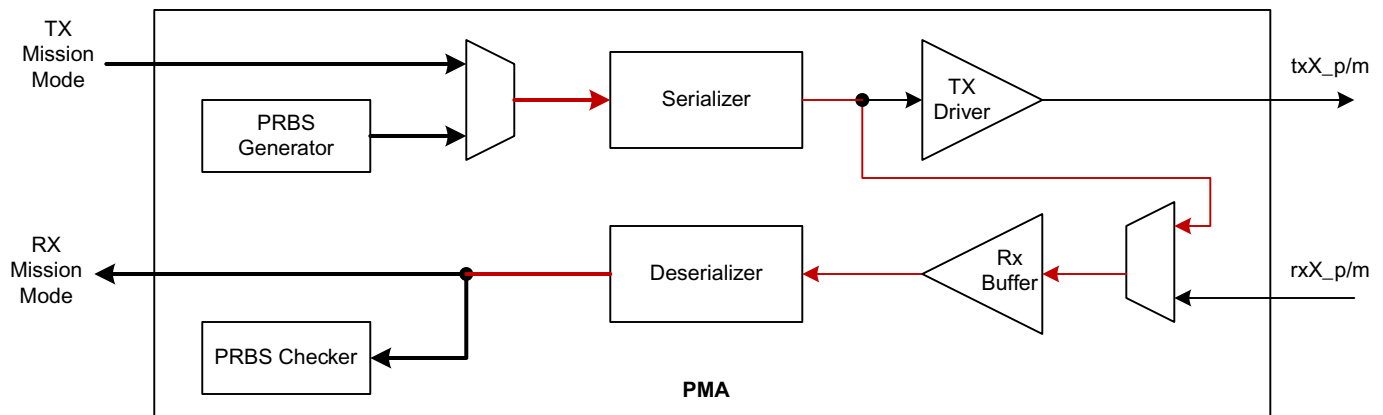
- Input settings must be set to values provided in [Table 4-19](#) on page 112
- phy_reset must be asserted and then de-asserted
- Reference clock input to the PHY must be supplied
- MPLLA/B configuration settings must correspond to the required baud rate (for more information, see “[Configuring the PHY](#)” on page 167); this can be done by controlling primary inputs or accessing registers.
- Set the signals in [Table 4-24](#) accordingly; this can be done by controlling primary inputs or accessing registers.

Table 4-24 Loopback Function Settings

Signal	Setting
txX_reset rxX_reset txX_lpd rxX_lpd	De-assert for the corresponding lane X.
txX_pstate rxX_pstate	Set to P0.
cmn_cntx_sel mpllx_cntx_sel rxX_cntx_sel txX_cntx_sel	Set to required baud rate (for more information, see “ Configuring the PHY ” on page 167).

4.18.8.1.1 TX-to-RX Serial (Pre-Driver) Loopback

The TX-to-RX loopback path is shown in [Figure 4-39](#).

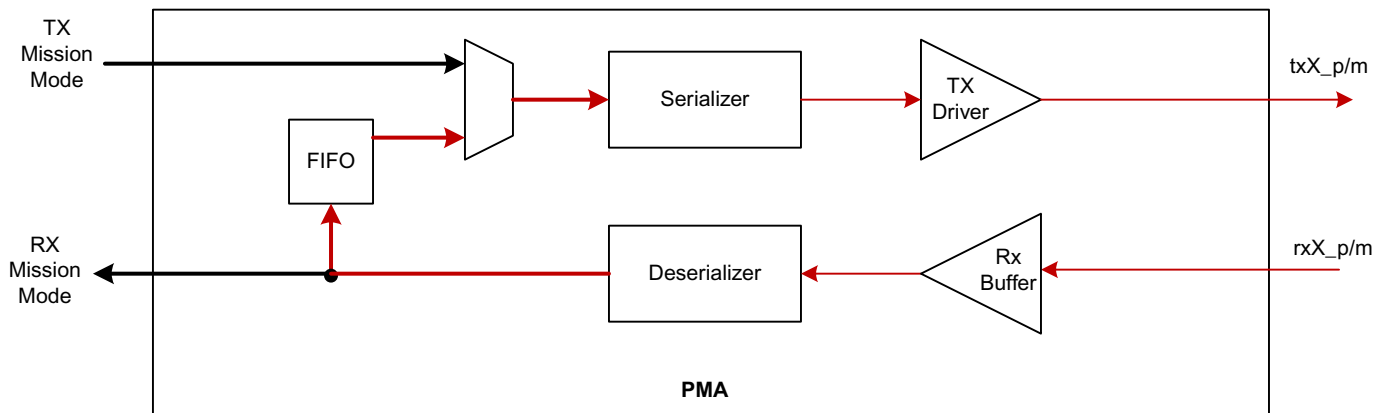
Figure 4-39 TX-to-RX Serial (pre-driver) Loopback

This mode uses either the user logic or the built-in PRBS Generator/Checker to generate and check the test data. The serial loopback path to the RX Buffer is after the TX pre-driver (before the TX driver). Using this mode, you can verify the operation for all enabled PCS and PMA functional blocks in the transmitter and receiver channel.

The TX-to-RX loopback mode is enabled by asserting the input `laneX_tx2rx_ser_lb_en`, and enabling the TX and RX datapaths by asserting `txX_data_en` and `rxX_data_en` input (also accessible through over-ride registers `LANEx_DIG_ASIC_LANE_OVRD_IN`, `LANEx_DIG_ASIC_TX_OVRD_IN_0`, and `LANEx_DIG_ASIC_RX_OVRD_IN_0`).

4.18.8.1.2 RX-to-TX Parallel Loopback

The RX-to-TX parallel loopback path is shown in [Figure 4-40](#).

Figure 4-40 RX-to-TX Parallel Loopback

The RX-to-TX parallel loopback mode is enabled by asserting the input `laneX_rx2tx_par_lb_en`, and enabling the TX and RX datapaths by asserting `txX_data_en` and `rxX_data_en` (also accessible through over-ride registers `LANEx_DIG_ASIC_LANE_OVRD_IN`, `LANEx_DIG_ASIC_TX_OVRD_IN_0`, and `LANEx_DIG_ASIC_RX_OVRD_IN_0`). In this loopback mode, the received recovered data `rxX_p/m` is transmitted back on the `txX_p/m` outputs. To prevent errors that may get introduced in the RX to TX loopback data because of the frequency deviation between the local MPLLs and the external transmitter, the reference clock input to the PHY must be derived from the same source as the reference clock used by the external transmitter feeding data on `rxX_p/m`.

During RX to TX loopback, the TX clock path is sourced from the RX clock path and as a result must be calibrated for TX DCC. Upon entry into RX-to-TX parallel loopback, the firmware will re calibrate the TX DCC for the new clock path. This process takes approximately 1ms to complete. The expectation is that loopback data will only be valid after this 1ms has elapsed. Upon exiting RX-to-TX parallel loopback, the firmware will restore the previous TX DCC calibration code. This process takes approximately 50 μ s to complete. The system should not make any new requests to the PHY until after the 50 μ s has elapsed.

4.18.8.2 BERT

The Byte Error Rate Tester (BERT) comprises two independently programmed modules: a pattern generator and a pattern matcher/error counter.

4.18.8.2.1 BERT Pattern Generator

[Table 4-25](#) provides a list of patterns that can be generated with the built-in pattern generator.

Table 4-25 Pattern Generator Mode Control

LANEx_DIG_TX_LBERT_CTL[3:0]	Description
4'b0000	Pattern generator is disabled
4'b0001	LFSR31: $x^{31} + x^{28} + 1$
4'b0010	LFSR23: $x^{23} + x^{18} + 1$
4'b0011	LFSR23: $x^{23} + x^{21} + x^{16} + x^8 + x^5 + x^2 + 1$
4'b0100	LFSR16: $x^{16} + x^5 + x^4 + x^3 + 1$
4'b0101	LFSR15: $x^{15} + x^{14} + 1$
4'b0110	LFSR11: $x^{11} + x^9 + 1$
4'b0111	LFSR9: $x^9 + x^5 + 1$
4'b1000	LFSR7: $x^7 + x^6 + 1$
4'b1001	Fixed word (PAT0)
4'b1010	DC balanced word (PAT0, ~PAT0)
4'b1011	Fixed pattern (000, PAT0, 3ff, ~PAT0)

The desired 10-bit pattern PAT0 can be set by writing to the pattern generator control register LANEx_DIG_TX_LBERT_CTL[14:5]. Register bits LANEx_DIG_TX_LBERT_CTL[14:5] are used for 20-bit mode and bits LANEx_DIG_TX_LBERT_CTL[12:5] are used for 16-bit mode. The pattern generator only operates in either 20-bit or 16-bit mode. The mode selected depends on the transmit data width, which is selected using input txX_width or by writing into the control register LANEx_DIG_ASIC_TX_OVRD_IN_0. When the transmit data is set to 20-bit or 10-bit, then the pattern generator is set to 20-bit mode. When the transmit data is set to 16-bit or 8-bit, then the pattern generator is set to 16-bit mode.

**Note**

The PAT0 data pattern has the following restriction:
Patterns of all 0's or all 1's should not be used for “Fixed word” and “Fixed pattern” modes

A single-bit error can be inserted into the LSB of a transmitted word by writing a 1 on the register LANEx_DIG_TX_LBERT_CTL[4].

4.18.8.2.2 BERT Pattern Matcher

The pattern matcher can detect and synchronize to errored bytes in multiple types of patterns. Errored symbols are counted in the error counter. The pattern matcher only operates in either 20-bit or 16-bit mode. The mode selected depends on the receive data width which is selected using input rxX_width or by writing into the control register LANEx_DIG_ASIC_RX_OVRD_IN_0. When the receive data is set to 20-bit or 10-bit, then the pattern matcher is set to 20-bit mode. When the receive data is set to 16-bit, then the pattern matcher is set to 16-bit mode.

For 20-bit mode, error detecting word size is 10 bits; bits 0 to 9 and bits 10 to 19. For 16-bit mode, the error detecting word size is 8 bits; bits 0 to 7 and bits 8 to 15.

Note that the controls for the pattern generator and pattern matcher within a lane are independent.

Table 4-26 describes the LANEx_DIG_RX_LBERT_CTL[3:0] field, which selects the expected pattern and operating mode.

Table 4-26 Pattern Matcher Mode Control

LANEx_DIG_RX_LBERT_CTL[3:0]	Description
4'b0000	Disabled
4'b0001	LFSR31: $x^{31} + x^{28} + 1$
4'b0010	LFSR23: $x^{23} + x^{18} + 1$
4'b0011	LFSR23: $x^{23} + x^{21} + x^{16} + x^8 + x^5 + x^2 + 1$
4'b0100	LFSR16: $x^{16} + x^5 + x^4 + x^3 + 1$
4'b0101	LFSR15: $x^{15} + x^{14} + 1$
4'b0110	LFSR11: $x^{11} + x^9 + 1$
4'b0111	LFSR9: $x^9 + x^5 + 1$
4'b1000	LFSR7: $x^7 + x^6 + 1$
4'b1001	$d[n] = d[n-10]$
4'b1010	$d[n] = !d[n-10]$
4'b1011	$d[n] = !d[n-20]$

The pattern matcher generates the expected pattern and synchronizes the generated pattern to the incoming pattern. Synchronization and error counting are initiated by asserting the sync bit (LANEx_DIG_RX_LBERT_CTL[4]).

The error counter in the pattern matcher is 22 bits wide. You can read the error count result from the LANEx_DIG_RX_LBERT_ERR register, the content of which is {lbert_err_ovbit14, lbert_err_cnt[14:0]}. The contents of LANEx_DIG_RX_LBERT_ERR are presented according to the following rules:

- When the error count is $< 2^{15}$, the bottom 15 bits are presented in the count field (lbert_err_cnt[14:0]) and the lbert_err_ovbit14 field is de-asserted.
- When the error count is $< 2^{22}$ but $\geq 2^{15}$, the top 15 bits are presented in the count field (lbert_err_cnt[14:0]) and the lbert_err_ovbit14 field is set. This result effectively scales the contents down by a factor of 2^7 .
- When the 22-bit error counter overflows, the count field (lbert_err_cnt[14:0]) is set to 0x7FFF, and the lbert_err_ovbit14 field is asserted.

The pattern generator and pattern matchers are extensively used as part of ATE testing. Although all of the functional ATE tests are performed using register accesses, some inputs need to be driven to specific values as shown below:

- phy_reset = 0 (after subjecting the DUT though POR sequence)
- scan_mode = 1'b0
- rxX_adapt_req = 1'b0
- rxX_disable = 1'b0
- rxX_req = 1'b0
- txX_disable = 1'b0
- txX_req = 1'b0
- txX_detrx_req = 1'b0
- res_req_in = 0
- phy*_bs* inputs = 0
- phy*_jtag_trst_n = 1
- phy_test_powerdown = 0
- phy_test_burnin = 0

4.18.8.3 Registers Used In Debug Analysis

The following table provides a list of registers that can be used as a sanity check for the calibrated and receiver adapted PHY.

Table 4-27 Debug Key Registers

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
COMMON								
RAWCMN.AON_RTUNE_RX_VAL_0.RTUNE_RX_VAL_0 ^a	COMMON	Resistor Tune RX Value 0	16'h015e[5:0]	5	0	0-635	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_0.RTUNE_TXDN_VAL_0 ^a	COMMON	Resistor Tune TX Down Value 0	16'h015f[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_0.RTUNE_TXAVG_VAL_0 ^a	COMMON	Resistor Tune TX Average Value 0	16'h0160[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_1.RTUNE_RX_VAL_1 ^a	COMMON	Resistor Tune RX Value 1	16'h0161[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_1.RTUNE_TXDN_VAL_1 ^a	COMMON	Resistor Tune TX Down Value 1	16'h0162[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_1.RTUNE_TXAVG_VAL_1 ^a	COMMON	Resistor Tune TX Average Value 1	16'h0163[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_2.RTUNE_RX_VAL_2 ^a	COMMON	Resistor Tune RX Value 2	16'h0164[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_2.RTUNE_TXDN_VAL_2 ^a	COMMON	Resistor Tune TX Down Value 2	16'h0165[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_2.RTUNE_TXAVG_VAL_2 ^a	COMMON	Resistor Tune TX Average Value 2	16'h0166[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_3.RTUNE_RX_VAL_3 ^a	COMMON	Resistor Tune RX Value 3	16'h0167[5:0]	5	0	0-63	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWCMN.AON_RTUNE_TXDN_VAL_3.RTUNE_TXDN_VAL_3 ^a	COMMON	Resistor Tune TX Down Value 3	16'h0168[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_3.RTUNE_TXAVG_VAL_3 ^a	COMMON	Resistor Tune TX Average Value 3	16'h0169[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_4.RTUNE_RX_VAL_4 ^a	COMMON	Resistor Tune RX Value 4	16'h016a[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_4.RTUNE_TXDN_VAL_4 ^a	COMMON	Resistor Tune TX Down Value 4	16'h016b[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_4.RTUNE_TXAVG_VAL_4 ^a	COMMON	Resistor Tune TX Average Value 4	16'h016c[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_5.RTUNE_RX_VAL_5 ^a	COMMON	Resistor Tune RX Value 5	16'h016d[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_5.RTUNE_TXDN_VAL_5 ^a	COMMON	Resistor Tune TX Down Value 5	16'h016e[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_5.RTUNE_TXAVG_VAL_5 ^a	COMMON	Resistor Tune TX Average Value 5	16'h016f[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_6.RTUNE_RX_VAL_6 ^a	COMMON	Resistor Tune RX Value 6	16'h0170[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_6.RTUNE_TXDN_VAL_6 ^a	COMMON	Resistor Tune TX Down Value 6	16'h0171[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_TXAVG_VAL_6.RTUNE_TXAVG_VAL_6 ^a	COMMON	Resistor Tune TX Average Value 6	16'h0172[9:0]	9	0	0-1023	0	-
RAWCMN.AON_RTUNE_RX_VAL_7.RTUNE_RX_VAL_7 ^a	COMMON	Resistor Tune RX Value 7	16'h0173[5:0]	5	0	0-63	0	-
RAWCMN.AON_RTUNE_TXDN_VAL_7.RTUNE_TXDN_VAL_7 ^a	COMMON	Resistor Tune TX Down Value 7	16'h0174[9:0]	9	0	0-1023	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWCMN.AON_RTUNE_TXAVG_VAL_7.RTUNE_TXAVG_VAL_7 ^a	COMMON	Resistor Tune TX Average Value 7	16'h0175[9:0]	9	0	0-1023	0	-
RAWCMN.AON_MPLLA_TUNE_BANK_0.MPLLA_TUNE_BANK_0 ^{a,b}	COMMON	MPLLA Coarse Tune value Bank 0	16'h0142[11:0]	11	0	0-4095	0	-
RAWCMN.AON_MPLLA_TUNE_BANK_1.MPLLA_TUNE_BANK_1 ^{a,b}	COMMON	MPLLA Coarse Tune value Bank 1	16'h0143[11:0]	11	0	0-4095	0	-
RAWCMN.AON_MPLLA_TUNE_BANK_2.MPLLA_TUNE_BANK_2 ^{a,b}	COMMON	MPLLA Coarse Tune value Bank 2	16'h0144[11:0]	11	0	0-4095	0	-
RAWCMN.AON_MPLLA_TUNE_BANK_3.MPLLA_TUNE_BANK_3 ^{a,b}	COMMON	MPLLA Coarse Tune value Bank 3	16'h0145[11:0]	11	0	0-4095	0	-
TX - CALIBRATION								
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_0.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for MPLLA Bank 0	16'h3021[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_0.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLLA Bank 0	16'h3021[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_1.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for MPLLA Bank 1	16'h3024[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_1.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLLA Bank 1	16'h3024[7:4]	7	4	0-15	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_2.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for MPLLA Bank 2	16'h3027[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_2.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLLA Bank 2	16'h3027[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_3.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for MPLLA Bank 3	16'h302a[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_CTRL_RANGE_BANK_3.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLLA Bank 3	16'h302a[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_0.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 0	16'h3022[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_0.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 0	16'h3022[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_1.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 1	16'h3025[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_1.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 1	16'h3025[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_2.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 2	16'h3028[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_2.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 2	16'h3028[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_3.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 3	16'h302b[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_FULL_BANK_3.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLLA Bank 3	16'h302b[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_0.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 0	16'h3023[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_0.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 0	16'h3023[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_1.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 1	16'h3026[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_1.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 1	16'h3026[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_2.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 2	16'h3029[7:0]	7	0	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_2.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 2	16'h3029[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_3.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 3	16'h302c[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLLA_DCC_HALF_BANK_3.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLLA Bank 3	16'h302c[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLLB_DCC_CTRL_RANGE_BANK_0.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for M PLLB Bank 0	16'h302d[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLB_DCC_CTRL_RANGE_BANK_0.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for M PLLB Bank 0	16'h302d[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLLB_DCC_CTRL_RANGE_BANK_1.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for M PLLB Bank 1	16'h3030[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLLB_DCC_CTRL_RANGE_BANK_1.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for M PLLB Bank 1	16'h3030[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLLB_DCC_CTRL_RANGE_BANK_2.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for M PLLB Bank 2	16'h3033[3:0]	3	0	0-15	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLL_B_DCC_CTRL_RANGE_BANK_2.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLL Bank 2	16'h3033[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLL_B_DCC_CTRL_RANGE_BANK_3.FULL_VAL ^a	0	Full-Rate TX DCC CTRL range Code Value for MPLL Bank 3	16'h3036[3:0]	3	0	0-15	0	-
RAWLANEAONX.TX_MPLL_B_DCC_CTRL_RANGE_BANK_3.HALF_VAL ^a	0	Half-Rate TX DCC CTRL range Code Value for MPLL Bank 3	16'h3036[7:4]	7	4	0-15	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_0.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 0	16'h302e[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_0.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 0	16'h302e[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_1.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 1	16'h3031[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_1.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 1	16'h3031[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_2.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 2	16'h3034[7:0]	7	0	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_2.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 2	16'h3034[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_3.CM_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 3	16'h3037[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_FULL_BANK_3.DIFF_VAL ^a	0	TX DCC Full-Rate Code Value for MPLL Bank 3	16'h3037[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_0.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 0	16'h302f[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_0.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 0	16'h302f[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_1.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 1	16'h3032[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_1.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 1	16'h3032[15:8]	15	8	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_2.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 2	16'h3035[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_2.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 2	16'h3035[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_3.CM_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 3	16'h3038[7:0]	7	0	0-255	0	-
RAWLANEAONX.TX_MPLL_B_DCC_HALF_BANK_3.DIFF_VAL ^a	0	TX DCC Half-Rate Code Value for MPLL Bank 3	16'h3038[15:8]	15	8	0-255	0	-
TX - ADAPTATION								
LANEN_DIG_ASIC_TX_ASIC_IN_1.TX_M_AIN_CURSOR ^{c,d}	0	Value from ASIC for tx_eq_main	16'h100A[15:0]	14	8	32-96	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_1.TX_M_AIN_CURSOR ^{c,d}	1	Value from ASIC for tx_eq_main	16'h120A[15:0]	14	8	32-96	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_1.TX_M_AIN_CURSOR ^{c,d}	2	Value from ASIC for tx_eq_main	16'h140A[15:0]	14	8	32-96	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_1.TX_M_AIN_CURSOR ^{c,d}	3	Value from ASIC for tx_eq_main	16'h160A[15:0]	14	8	32-96	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_POST_CURSOR ^{d,e}	0	Value from ASIC for tx_eq_pre	16'h100B[13:0]	6	0	0-32	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_POST_CURSOR ^{d,e}	1	Value from ASIC for tx_eq_pre	16'h120B[13:0]	6	0	0-32	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_POST_CURSOR ^{d,e}	2	Value from ASIC for tx_eq_pre	16'h140B[13:0]	6	0	0-32	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_POST_CURSOR ^{d,e}	3	Value from ASIC for tx_eq_pre	16'h160B[13:0]	6	0	0-32	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_PRE_CURSOR ^{d,f}	0	Value from ASIC for tx_eq_post	16'h100B[13:0]	13	7	0-16	1	Final result = Decimal / 2

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_PRE_CURSOR ^{d,f}	1	Value from ASIC for tx_eq_post	16'h120B[13:0]	13	7	0-16	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_PRE_CURSOR ^{d,f}	2	Value from ASIC for tx_eq_post	16'h140B[13:0]	13	7	0-16	1	Final result = Decimal / 2
LANEN_DIG_ASIC_TX_ASIC_IN_2.TX_PRE_CURSOR ^{d,f}	3	Value from ASIC for tx_eq_post	16'h160B[13:0]	13	7	0-16	1	Final result = Decimal / 2
RX - CALIBRATION								
LANEX.RX_DPLL_FREQ.VAL ^g	0	Current VCO Frequency Integrator Value	16'h12bb[13:0]	13	0	0-16383	0	-
LANEX.RX_VCOCAL_RX_VCO_STAT_0.RX_ANA_CDR_FREQ_TUNE ^a	0	RX VCO Coarse Calibration Status Register	16'h10af[9:0]	9	0	0-1023	0	-
RAWLANEAONX.RX_AFE_CTL_EIDAC_OFST_BANK_0.AFE_CTL_EIDAC_OFST ^a	0	Offset value for AFE CTLE iDAC for bank 0	16'h'311c[7:0]	7	1	0-128	0	-
RAWLANEAONX.RX_AFE_CTL_EIDAC_OFST_BANK_1.AFE_CTL_EIDAC_OFST ^a	0	Offset value for AFE CTLE iDAC for bank 1	16'h'311d[7:0]	7	1	0-128	0	-
RAWLANEAONX.RX_AFE_CTL_EIDAC_OFST_BANK_2.AFE_CTL_EIDAC_OFST ^a	0	Offset value for AFE CTLE iDAC for bank 2	16'h'311e[7:0]	7	1	0-128	0	-
RAWLANEAONX.RX_AFE_CTL_EIDAC_OFST_BANK_3.AFE_CTL_EIDAC_OFST ^a	0	Offset value for AFE CTLE iDAC for bank 3	16'h'311f[7:0]	7	1	0-128	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_0.FULL_VAL ^a	0	Full-Rate RX DCC CTRL RANGE calibrated Value for Bank 0	16h'3135[3:0]	3	0	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_0.HALF_VAL ^a	0	Half-Rate RX DCC CTRL RANGE calibrated Value for Bank 0	16h'3135[7:4]	7	4	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_1.FULL_VAL ^a	0	Full-Rate RX DCC CTRL RANGE calibrated Value for Bank 1	16h'313e[3:0]	3	0	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_1.HALF_VAL ^a	0	Half-Rate RX DCC CTRL RANGE calibrated Value for Bank 1	16h'313e[7:4]	7	4	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_2.FULL_VAL ^a	0	Full-Rate RX DCC CTRL RANGE calibrated Value for Bank 2	16h'3147[3:0]	3	0	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_2.HALF_VAL ^a	0	Half-Rate RX DCC CTRL RANGE calibrated Value for Bank 2	16h'3147[7:4]	7	4	0-15	0	-
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_3.FULL_VAL ^a	0	Full-Rate RX DCC CTRL RANGE calibrated Value for Bank 3	16h'3150[3:0]	3	0	0-15	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_CTRL_RANGE_BANK_3.HALF_VAL ^a	0	Half-Rate RX DCC CTRL RANGE calibrated Value for Bank 3	16h'3150[7:4]	7	4	0-15	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_0.CM_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 0	16h'3137[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_0.DIFF_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 0	16h'3137[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_1.CM_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 1	16h'3140[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_1.DIFF_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 1	16h'3140[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_2.CM_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 2	16h'3149[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_2.DIFF_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 2	16h'3149[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_3.CM_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 3	16h'3152[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_BYP_BANK_3.DIFF_VAL ^a	0	RX DCC Full-Rate Bypass Code Value for Bank 3	16h'3152[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_0.CM_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 0	16h'3136[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_0.DIFF_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 0	16h'3136[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_1.CM_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 1	16h'313f[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_1.DIFF_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 1	16h'313f[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_2.CM_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 2	16h'3148[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_2.DIFF_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 2	16h'3148[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_3.CM_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 3	16h'3151[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_DATA_BANK_3.DIFF_VAL ^a	0	RX DCC Full-Rate Data Code Value for Bank 3	16h'3151[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_0.CM_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 0	16h'3138[7:0]	7	0	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_0.DIFF_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 0	16h'3138[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_1.CM_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 1	16h'3141[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_1.DIFF_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 1	16h'3141[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_2.CM_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 2	16h'314a[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_2.DIFF_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 2	16h'314a[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_3.CM_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 3	16h'3153[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_FULL_PHASE_BANK_3.DIFF_VAL ^a	0	RX DCC Full-Rate Phase Code Value for Bank 3	16h'3153[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_BANK_0.CM_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 0	16h'313a[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_BANK_0.DIFF_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 0	16h'313a[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_1.CM_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 1	16h'3143[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_1.DIFF_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 1	16h'3143[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_2.CM_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 2	16h'314c[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_2.DIFF_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 2	16h'314c[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_3.CM_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 3	16h'3155[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_BYP_B ANK_3.DIFF_VAL ^a	0	RX DCC Half-Rate Bypass Code Value for Bank 3	16h'3155[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_B ANK_0.CM_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 0	16h'3139[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_B ANK_0.DIFF_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 0	16h'3139[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_B ANK_1.CM_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 1	16h'3142[7:0]	7	0	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_HALF_DATA_BANK_1.DIFF_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 1	16h'3142[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_BANK_2.CM_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 2	16h'314b[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_BANK_2.DIFF_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 2	16h'314b[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_BANK_3.CM_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 3	16h'3154[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_DATA_BANK_3.DIFF_VAL ^a	0	RX DCC Half-Rate Data Code Value for Bank 3	16h'3154[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_0.CM_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 0	16h'313b[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_0.DIFF_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 0	16h'313b[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_1.CM_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 1	16h'3144[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_1.DIFF_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 1	16h'3144[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_2.CM_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 2	16h'314d[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_2.DIFF_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 2	16h'314d[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_3.CM_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 3	16h'3156[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DCC_HALF_PHASE_BANK_3.DIFF_VAL ^a	0	RX DCC Half-Rate Phase Code Value for Bank 3	16h'3156[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_DFE_BYPASS_EVEN_VDAC_OFST ^a	0	Offset value for DFE Bypass Even vDAC	16h'312c[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_BYPASS_ODD_VDAC_OFST ^a	0	Offset value for DFE Bypass Odd vDAC	16h'312d[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_DATA_EVEN_HIGH_VDAC_OFST ^a	0	Offset value for DFE Data Even High (DEH) vDAC	16h'3128[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_DATA_EVEN_LOW_VDAC_OFST ^a	0	Offset value for DFE Data Even Low (DEL) vDAC	16h'3129[7:0]	7	0	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_DFE_DATA_ODD_HIGH_VDAC_OFST.DFE_DATA_ODD_HIGH_VDAC_OFST ^a	0	Offset value for DFE Data Odd High (DOH) vDAC	16h'312a[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_DATA_ODD_LOW_VDAC_OFST.DFE_DATA_ODD_LOW_VDAC_OFST ^a	0	Offset value for DFE Data Odd Low (DOL) vDAC	16h'312b[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_ERROR_EVEN_VDAC_OFST.DFE_ERROR_EVEN_VDAC_OFST ^a	0	Offset value for DFE Error Even vDAC	16h'312e[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_ERROR_ODD_VDAC_OFST.DFE_ERROR_ODD_VDAC_OFST ^a	0	Offset value for DFE Error Odd vDAC	16h'312f[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_PHASE_EVEN_VDAC_OFST.DFE_PHASE_EVEN_VDAC_OFST ^a	0	Offset value for DFE Phase Even vDAC	16h'3122[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_DFE_PHASE_ODD_VDAC_OFST.DFE_PHASE_ODD_VDAC_OFST ^a	0	Offset value for DFE Phase Odd vDAC	16h'3123[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_0.FULL_RATE_VAL ^{h,i}	0	Full-Rate RX IQ Calibration Codes Value for Bank 0	16h'313c[15:8]	15	8	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_0.HALF_RATE_VAL ^{h,i}	0	Half-Rate RX IQ Calibration Codes Value for Bank 0	16h'313c[7:0]	7	0	0-79	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_IQ_CAL_BANK_1.FULL_RATE_VAL ^{h,i}	0	Full-Rate RX IQ Calibration Codes Value for Bank 1	16h'3145[15:8]	15	8	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_1.HALF_RATE_VAL ^{h,i}	0	Half-Rate RX IQ Calibration Codes Value for Bank 1	16h'3145[7:0]	7	0	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_2.FULL_RATE_VAL ^{h,i}	0	Full-Rate RX IQ Calibration Codes Value for Bank 2	16h'314e[15:8]	15	8	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_2.HALF_RATE_VAL ^{h,i}	0	Half-Rate RX IQ Calibration Codes Value for Bank 2	16h'314e[7:0]	7	0	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_3.FULL_RATE_VAL ^{h,i}	0	Full-Rate RX IQ Calibration Codes Value for Bank 3	16h'3157[15:8]	15	8	0-79	0	-
RAWLANEAONX.RX_IQ_CAL_BANK_3.HALF_RATE_VAL ^{h,i}	0	Half-Rate RX IQ Calibration Codes Value for Bank 3	16h'3157[7:0]	7	0	0-79	0	-
RAWLANEAONX.RX_REF_EE_VDAC_OFFSET ^a	0	Value for reference level error even slicer	16h'3113[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_REF_EO_VDAC_OFFSET ^a	0	Value for reference level error odd slicer	16h'3114[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_SIGDET_CAL.SIGDET_LF_CAL ^a	0	SigDet low-frequency calibration value	16h'3109[6:0]	6	0	0-128	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_VGEN_VDAC_OFST. VGEN_VDAC_OFST ^a	0	Offset value for VGEN VDAC	16h'3108[7:0]	7	2	0-63	0	-
RX - ADAPTATION								
RAWLANEX.RX_FW_XF_ADAPT_FOM.AD APT_FOM ^j	0	RX Adaptation Figure of Merit	16h'20a7[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_ADPT_ATT_BANK_0 .ATT_ADPT_VAL ^a	0	Stored RX adapted ATT value. Bits 4:0 are dither bits Bank 0	16h'316a[7:0]	7	5	0-7	0	-
RAWLANEAONX.RX_ADPT_ATT_BANK_1 .ATT_ADPT_VAL ^a	0	Stored RX adapted ATT value. Bits 4:0 are dither bits Bank 1	16h'317e[7:0]	7	5	0-7	0	-
RAWLANEAONX.RX_ADPT_CTLE_BANK_0. CTLE_BOOST_ADPT_VAL ^a	0	Stored RX adapted CTLE boost value. Bits 4:0 are dither bits Bank 0	16h'316c[9:0]	9	5	0-31	0	-
RAWLANEAONX.RX_ADPT_CTLE_BANK_1. CTLE_BOOST_ADPT_VAL ^a	0	Stored RX adapted CTLE boost value. Bits 4:0 are dither bits Bank 1	16h'3180[9:0]	9	5	0-31	0	-
RAWLANEAONX.RX_ADPT_DFE_TAP1_B ANK_0.DFE_TAP1_ADPT_VAL ^a	0	Stored RX adapted DFE TAP1 value. Bits 4:0 are dither bits Bank 0	16h'316d[12:0]	12	5	-128 to 127	2	if (MSB == 1) Final Result = Decimal result-256) else Final Result = Decimal result

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_ADPT_DFE_TAP1_B ANK_1.DFE_TAP1_ADPT_VAL ^a	0	Stored RX adapted DFE TAP1 value. Bits 4:0 are dither bits Bank 1	16h'3181[12:0]	12	5	-128 to 127	2	if (MSB == 1) Final Result = Decimal result-256) else Final Result = Decimal result
RAWLANEAONX.RX_ADPT_DFE_TAP2_B ANK_0.DFE_TAP2_ADPT_VAL ^a	0	Stored RX adapted DFE TAP2 value. Bits 4:0 are dither bits Bank 0	16h'316e[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP3_B ANK_0.DFE_TAP3_ADPT_VAL ^a	0	Stored RX adapted DFE TAP3 value. Bits 4:0 are dither bits Bank 0	16h'316f[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP3_B ANK_1.DFE_TAP2_ADPT_VAL ^a	0	Stored RX adapted DFE TAP3 value. Bits 4:0 are dither bits Bank 1	16h'3182[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP3_B ANK_1.DFE_TAP3_ADPT_VAL ^a	0	Stored RX adapted DFE TAP3 value. Bits 4:0 are dither bits Bank 1	16h'3183[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP4_B ANK_0.DFE_TAP4_ADPT_VAL ^a	0	Stored RX adapted DFE TAP4 value. Bits 4:0 are dither bits Bank 0	16h'3170[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP4_B ANK_1.DFE_TAP4_ADPT_VAL ^a	0	Stored RX adapted DFE TAP4 value. Bits 4:0 are dither bits Bank 1	16h'3184[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_ADPT_DFE_TAP5_BANK_0.DFE_TAP5_ADPT_VAL ^a	0	Stored RX adapted DFE TAP5 value. Bits 4:0 are dither bits Bank 0	16h'3171[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_DFE_TAP5_BANK_1.DFE_TAP5_ADPT_VAL ^a	0	Stored RX adapted DFE TAP5 value. Bits 4:0 are dither bits Bank 1	16h'3185[11:0]	11	5	-64 to 63	3	Final Result = Decimal - 64
RAWLANEAONX.RX_ADPT_IQ_BANK_0.RX_ADPT_IQ ^{h,k}	0	RX adapted value of IQ for Bank 0	16h'317b[6:0]	6	0	0-79	0	-
RAWLANEAONX.RX_ADPT_IQ_BANK_1.RX_ADPT_IQ ^{h,k}	0	RX adapted value of IQ for Bank 1	16h'318f[6:0]	6	0	0-79	0	-
RAWLANEAONX.RX_ADPT_REF_ERR_BANK_0.RX_ADPT_REF_ERR_EVEN ^a	0	RX Adapted value of reference level for error even slicer for Bank 0	16h'317c[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_ADPT_REF_ERR_BANK_0.RX_ADPT_REF_ERR_ODD ^a	0	RX Adapted value of reference level for error odd slicer for Bank 0	16h'317c[15:8]	15	8	0-255	0	-
RAWLANEAONX.RX_ADPT_REF_ERR_BANK_1.RX_ADPT_REF_ERR_EVEN ^a	0	RX Adapted value of reference level for error even slicer for Bank 1	16h'3190[7:0]	7	0	0-255	0	-
RAWLANEAONX.RX_ADPT_REF_ERR_BANK_1.RX_ADPT_REF_ERR_ODD ^a	0	RX Adapted value of reference level for error odd slicer for Bank 1	16h'3190[15:8]	15	8	0-255	0	-

Register	Lane	Description	Address_bits	MSB	LSB	Expected_Range	Post-process_to_be_applied	Post-processing_Formula
RAWLANEAONX.RX_ADPT_VGA_BANK_0.VGA_ADPT_VAL ^a	0	Stored RX adapted VGA value. Bits 4:0 are dither bits Bank 0	16h'316b[8:0]	8	5	0-15	0	-
RAWLANEAONX.RX_ADPT_VGA_BANK_1.VGA_ADPT_VAL ^a	0	Stored RX adapted VGA value. Bits 4:0 are dither bits Bank 1	16h'317f[8:0]	8	5	0-15	0	-

a. Maximum limit: No condition; Minimum limit: No condition; Condition: No condition; Remark: Normal; Else: Normal

b. The higher 3 bits correspond to the fine tune code and the lower 9 bits correspond to the coarse tune code.

c. Maximum limit: 96; Minimum limit: 32; Condition: If <=32 or >=96; Remark: MAIN; Else: MAIN

d. Input from PCS LSB padded with 0 before driving PMA

e. Maximum limit: 32; Minimum limit: 0; Condition: No condition; Remark: POST; Else: POST

f. Maximum limit: 16; Minimum limit: 0; Condition: No condition; Remark: PRE; Else: PRE

g. Maximum limit: 16383; Minimum limit: 0; Condition: If 0 or 16383; Remark: CDR unlocked; Else: Normal

h. Maximum limit: 79; Minimum limit: 0; Condition: If > 79; Remark: Outside expected bounds; Else: Normal

i. Condition only applies if the calibration is performed, otherwise the register contents are set to its defaults.

j. Maximum limit: 255; Minimum limit: 0; Condition: If < 50; Remark: Close to Extreme Range; Else: Normal

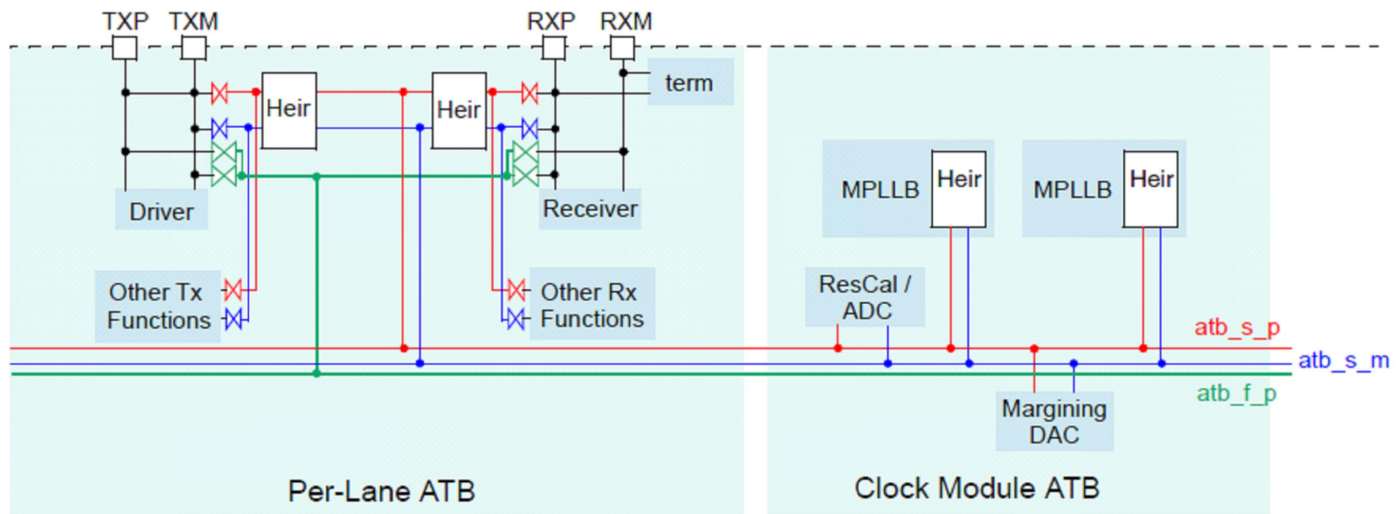
k. Condition only applies if the valid bit [7] is set, otherwise the register contents are set to its defaults.

4.18.9 Analog DC Test Capabilities

This section describes the MIPI M-PHY Type 1 G5 PMA's analog DC test capabilities.

4.18.9.1 Analog Test Bus Overview

The following figure shows a high-level view of the analog test bus (ATB).

Figure 4-41 High-Level View of Analog Test Bus

The analog test bus (ATB) comprises three signals routed throughout the Active section, through the Clock module and each lane. Some of these signals can be brought to the internal ADC, while others can be brought to any TX or RX pin. The following table describes the ATB signals.

Table 4-28 Analog Test Bus Signals

Signal	Description	Can be Brought to
atb_s_p	Zero current “sense” connections	ADC, TXP, TXM
atb_s_m	Zero current “sense” connections	ADC, TXP, TXM
atb_f_p	Large current (up to ~1 mA) “force” connection	TXM

Many internal circuits have connections that can be enabled to these buses. Note that these connections are intended to be used in a very-low-frequency (near DC) situation. Series resistance between connections on the bus are typically > 30 kohm, while bus parasitic capacitance can be multiple picoFarads, yielding bandwidths below 2 MHz. When connections are made to nodes carrying high-frequency signals, such as TX or RX serial pins, explicit 10-kohm resistors are used, so that connecting the ATB bus does not impair the lane’s operation.

Bringing a selected internal signal to either a package pin or the internal ADC is a matter of activating the correct combination of hierarchical switches and leaf-level switches. Control of these switches is distributed across registers in the analog circuitry, as detailed in the following section.



Note ATB measurements are unreliable if measured voltage is above 95% of VP level at bumps.

For details on ATB registers, refer to the “Register Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*. For more information, contact Synopsys.

4.18.9.2 ATB Access Through RX

To bring ATB signals out on the RX pins, disable the resistive terminations and the LOS circuit that are otherwise present at these pins.

The TX pins must not be used for this purpose, because the driver contains some impedances and leakage that cannot be completely eliminated.

To disable the RX termination and LOS circuit and connect the ATB to RX pins:

1. Override the RX termination by setting LANEN_DIG_ASIC_RX_OVRD_IN_3.EN to a value of 1.
2. Disable the RX termination by setting LANEN_DIG_ASIC_RX_OVRD_IN_3.TERM_EN to a value of 0.
3. Override the LOS control by setting LANEN_ANA_RX_PWR_CTRL1.OVRD_LOS_EN to 1.
4. Disable the LOS by setting LANEN_ANA_RX_PWR_CTRL1.LOS_EN_REG to 0.
5. Enable connection of atb_s_p to the RXP pin by setting LANEN_ANA_RX_CDR_AFE.MEAS_ATB_RX to a value of 0010.
6. Enable connection of atb_s_m to the RXM pin by setting LANEN_ANA_RX_ATB_MEAS1.MEAS_ATB_RX to a value of 010000.

In addition, you must set hierarchical and leaf ATB switches based on the source of voltage or current being output through the RX.

4.18.9.3 ATB Measurement Using 10-Bit ADC

The 10-bit ADC uses circuit components that are already present in the Clock module—the 10-bit DAC required for voltage margining and the offset calibrated comparator present for resistor calibration.

To use the 10-bit ADC:

1. Setup the IP ADC.
 - a. Place the DAC in ADC mode by writing a 0 to SUP_ANA_RTUNE_CTRL.RT_DAC_MODE.
 - b. Connect the ADC to the ATB by writing a 1 to SUP_ANA_RTUNE_CTRL.RT_ATB.
 - c. Sense the ATB sense lines by writing a 0 to SUP_ANA_RTUNE_CTRL.RT_SEL_ATBF.
 - d. Sense on atb_s_p (1) and atb_s_m (0) by writing a 1 or a 0 to SUP_ANA_RTUNE_CTRL.RT_SEL_ATBP.
 - e. Override the power-on signal to force the IP ADC on by writing a 1 to SUP_ANA_RTUNE_CTRL.RT_EN_FRCON.
 - f. Signal RTUNE that you are doing an ADC run by writing a 00 to SUP_DIG_RTUNE_DEBUG.TYPE.
2. Select ATB node in the PHY to sense.
 - a. Enable the block-level switch (if it exists) for the block using CREG.
 - b. Enable the specific ATB switch using CREG.
3. Trigger the ADC run.
 - a. Trigger ADC operation by writing a 1 to SUP_DIG_RTUNE_DEBUG.MAN_TUNE.
 - b. Read the 10-bit result from SUP_DIG_RTUNE_STAT.STAT.

- c. Reset the ADC engine prior to subsequent use by writing a 0 to SUP_DIG_R-TUNE_DEBUG.MAN_TUNE.

4. Calculate voltage.

$$\text{Voltage} = \text{result} * \text{VPH} / 1,023$$

4.18.9.4 ATB Access Through Top-Level Pins

You can access the ATB through top-level PMA pins.

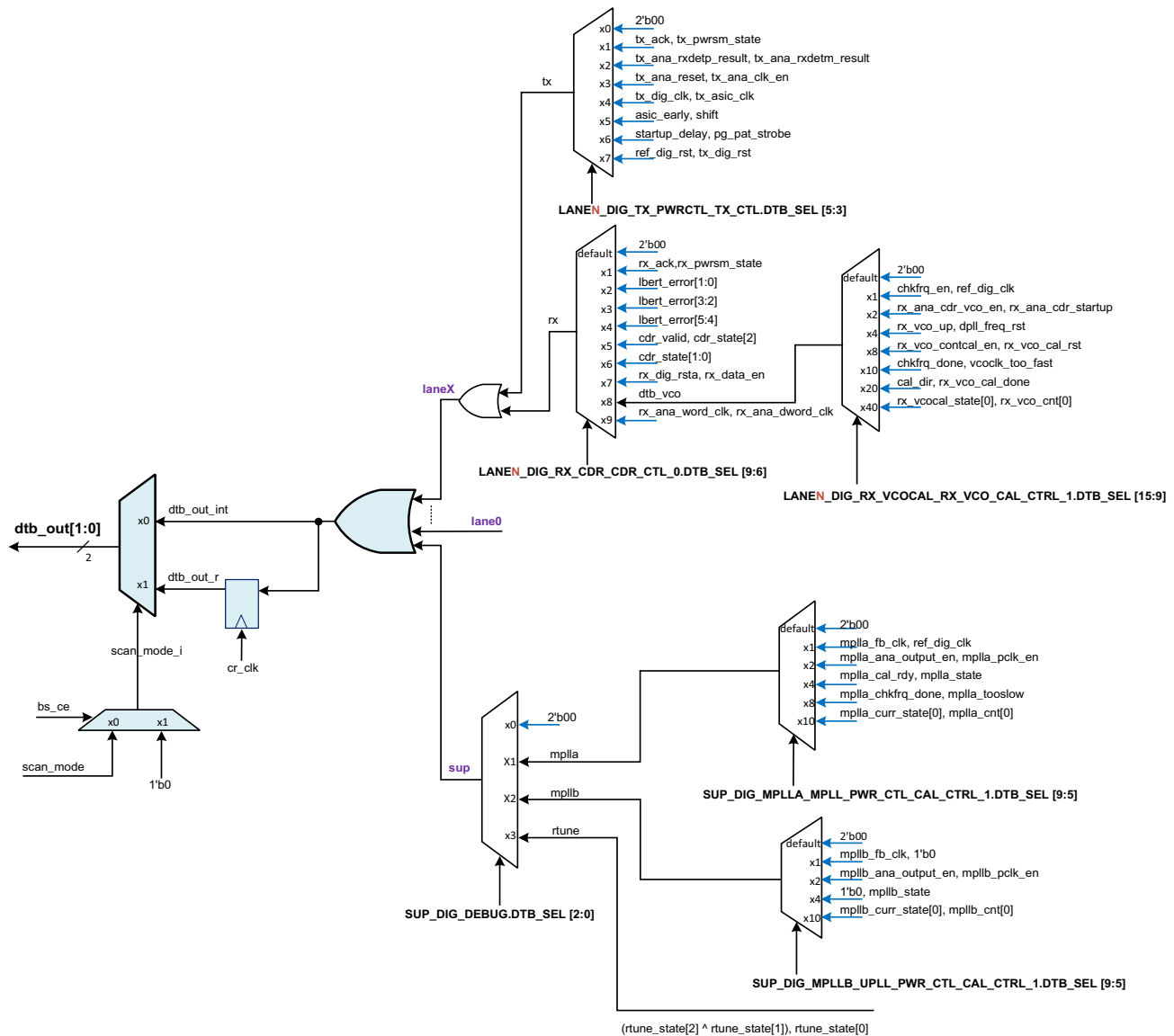
Due to contention, these pins cannot be wired together. You should isolate the atb* pins from multiple PMA instances from one another with an appropriate pass gate. Doing so ensures that leakage and IR drop from the different instances are isolated from one another.

These top-level pins are not ESD-protected. If they are connected to a bump, use an appropriate I/O cell. Any leakage from the I/O cell can corrupt the ATB voltage.

4.18.10 Digital Test Bus (DTB)

The digital test bus (DTB) is provided as a means of accessing internal digital signals for debug purposes. If this bus is not needed, when integrating the PHY, you can leave the dtb_out[1:0] pins unconnected or connect to external pins to facilitate PHY debug. Since the debug signals come from different clock domains, the dtb_out[1:0] must not be latched internally, but be muxed out to the top-level pins. The dtb_out[1:0] pin is a false path during the synthesis.

The DTB diagram is shown in [Figure 4-42](#).

Figure 4-42 Digital Test Bus Diagram

The list of dtb_out related registers is shown in [Table 4-29](#).

Table 4-29 DTB_SEL Registers

Register Name	Bits	Offset	Description
SUP_DIG_DEBUG	[2:0]	0x0023	Select Module Register: mplla/b clock or rtune status
SUP_DIG_MPLLA_MPLL_PWR_CTL_CAL_CTRL_1	[9:5]	0x0041	Select mplla dtb signal
SUP_DIG_MPLLB_UPLL_PWR_CTL_CAL_CTRL_1	[9:5]	0x0051	Select mpllb dtb signal
LANEN_DIG_TX_PWRCTL_TX_CTL	[5:3]	0x1N1A	Select TX Lanes signal
LANEN_DIG_RX_VCOCAL_RX_VCO_CAL_CTRL_1	[15:9]	0x1NAB	Select RX Lanes probe signal

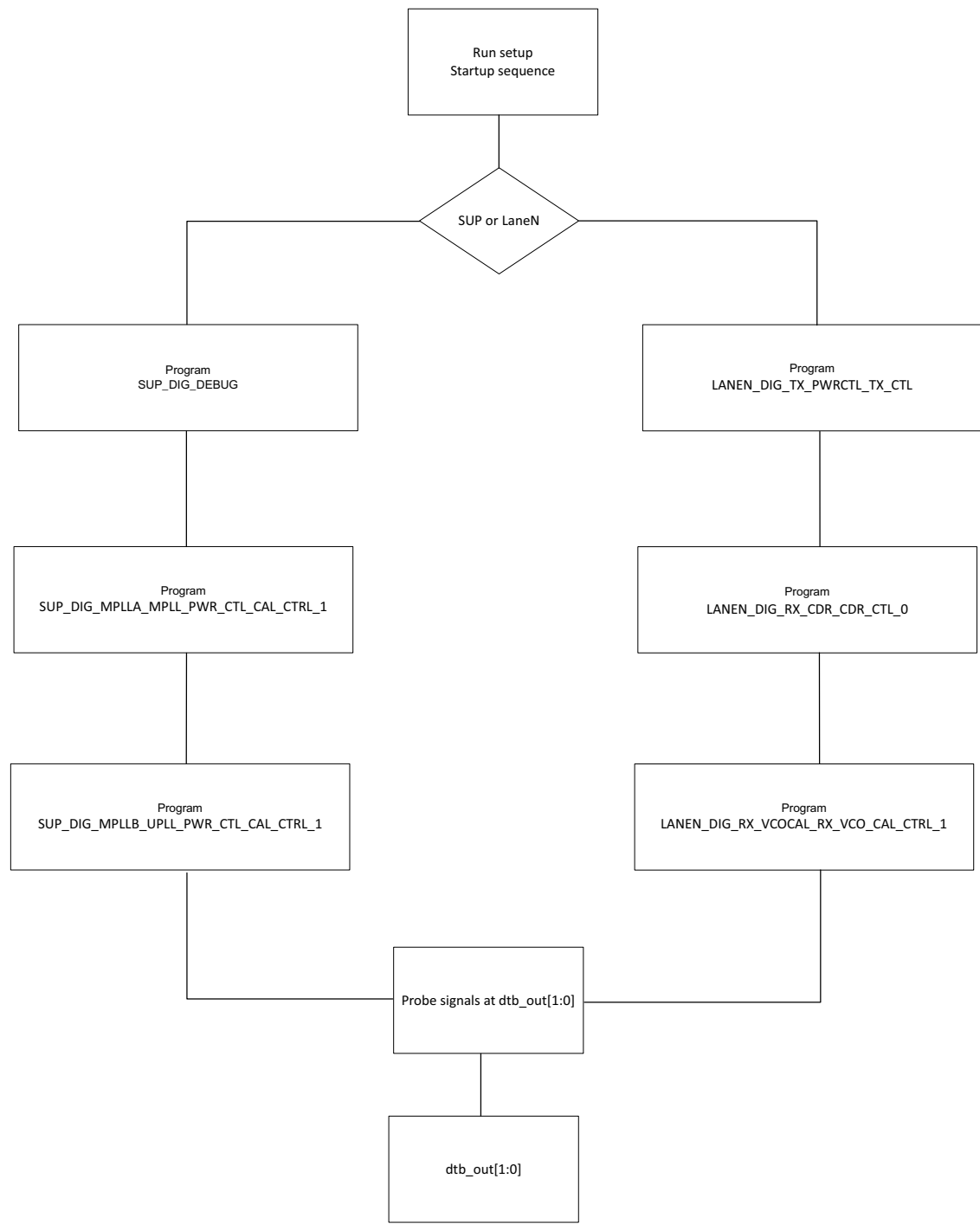
Register Name	Bits	Offset	Description
LANEN_DIG_RX_CDR_CDR_CTL_0	[9:6]	0x1NB5	Select RX Lanes probe signal

**Note**

All DTB muxed signals are connected to each other through a logic OR gate. Hence, only one DTB_SEL register can be enabled at a time, while other DTB_SEL registers must be set to 0.

The DTB access flow chart is shown in [Figure 4-43](#).

Figure 4-43 DTB Access Flow Chart



Mapping information on dtb_out[1:0] versus probing signals is shown in [Table 4-30](#). All signals listed in this table are mapped from the most significant bit (MSB) to the least significant bit (LSB) on the dtb_out[1:0] bus.

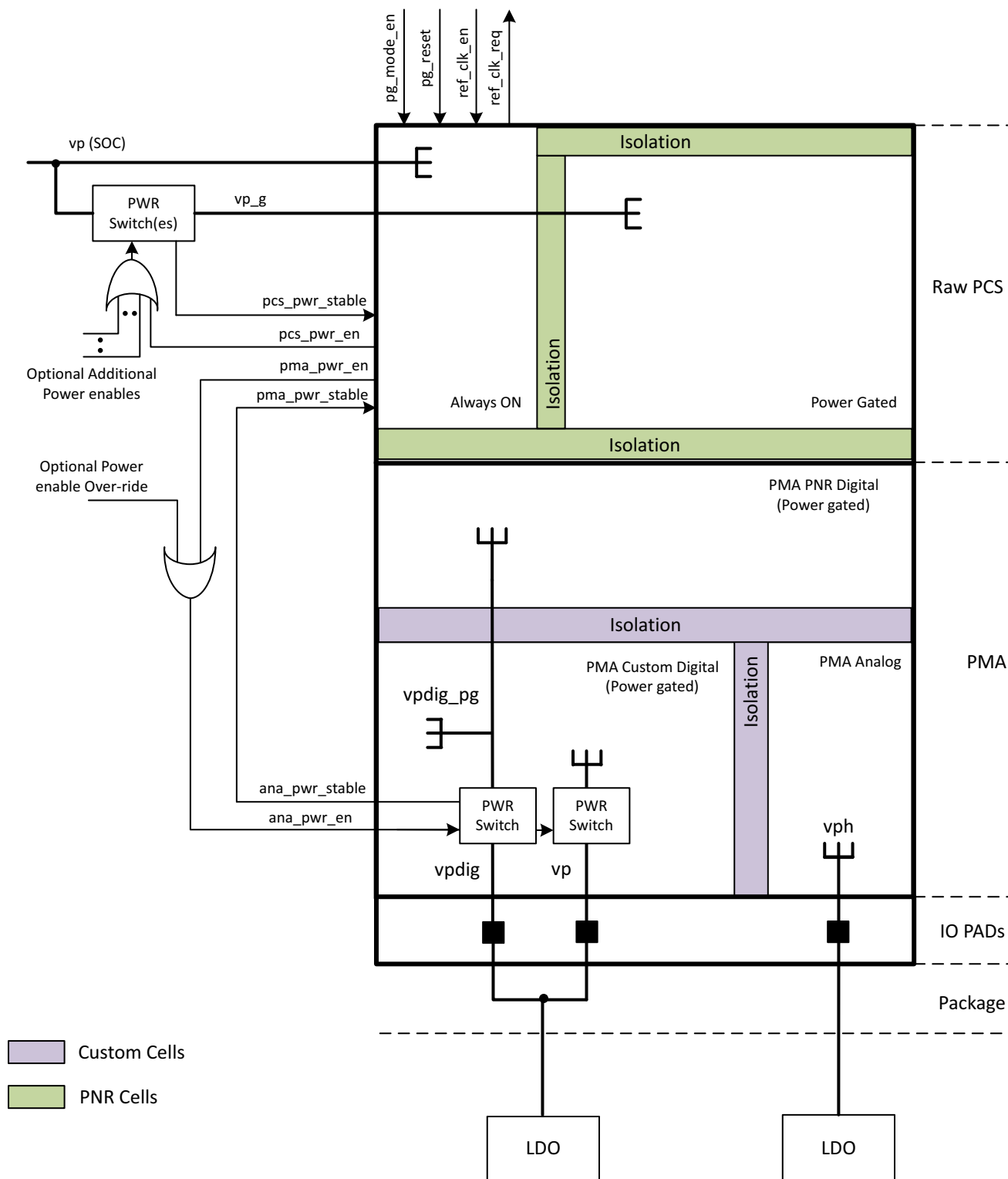
Table 4-30 dtb_out[1:0] Versus Probing Signals

Register Name	Probe Signals
SUP_DIG_DEBUG[2:0]	0x1: mpla_PWR_CTL_CAL_CTRL_1 0x2: mpllb_PWR_CTL_CAL_CTRL_1 0x3: rtune (rtune_state[2] XOR rtune_state[1]), rtune_state[0]
SUP_DIG_MPLLA_MPLL_PWR_CTL_CAL_CTRL_1[9:5]	0x1: mpla_fb_clk, ref_dig_clk 0x2: mpla_ana_output_en, mpla_pclk_en 0x4: mpla_cal_rdy, mpla_state 0x8: mpla_chkfrq_done, mpla_tooslow 0x10: mpla_curr_state[0], mpla_cnt[0]
SUP_DIG_MPLLB_UPLL_PWR_CTL_CAL_CTRL_1[9:5]	0x1: mpllb_fb_clk, 1'b0 0x2: mpllb_ana_output_en, mpllb_pclk_en 0x4: 1'b0, mpllb_state 0x10: mpllb_curr_state[0], mpllb_cnt[0]
LANEN_DIG_TX_PWRCTL_TX_CTL[5:3]	0x1: tx_ack, tx_pwrsrm_state 0x2: tx_ana_rxdetp_result, tx_ana_rxdetm_result 0x3: tx_ana_reset, tx_ana_clk_en 0x4: tx_dig_clk, tx_asic_clk 0x5: asic_early, shift 0x6: startup_delay, pg_pat_strobe 0x7: ref_dig_rst, tx_dig_rst
LANEN_DIG_RX_VCOCAL_RX_VCO_CAL_CTRL_1[15:9]	0x1: chkfrq_en, ref_dig_clk 0x2: rx_ana_cdr_vco_en, rx_ana_cdr_startup 0x4: rx_vco_up, dpil_freq_rst 0x8: rx_vco_contcal_en, rx_vco_cal_rst 0x10: chkfrq_done, vcoclk_too_fast 0x20: cal_dir, rx_vco_cal_done 0x40: rx_vcocal_state[0], rx_vco_cnt[0]
LANEN_DIG_RX_CDR_CDR_CTL_0[9:6]	0x1: rx_ack, rx_pwrsrm_state 0x2: lbert_error[1:0] 0x3: lbert_error[3:2] 0x4: lbert_error[5:4] 0x5: cdr_valid, cdr_state[2] 0x6: cdr_state[1:0] 0x7: rx_dig_rst, rx_data_en 0x8: dtb_vco 0x9: rx_ana_word_clk, rx_ana_dword_clk

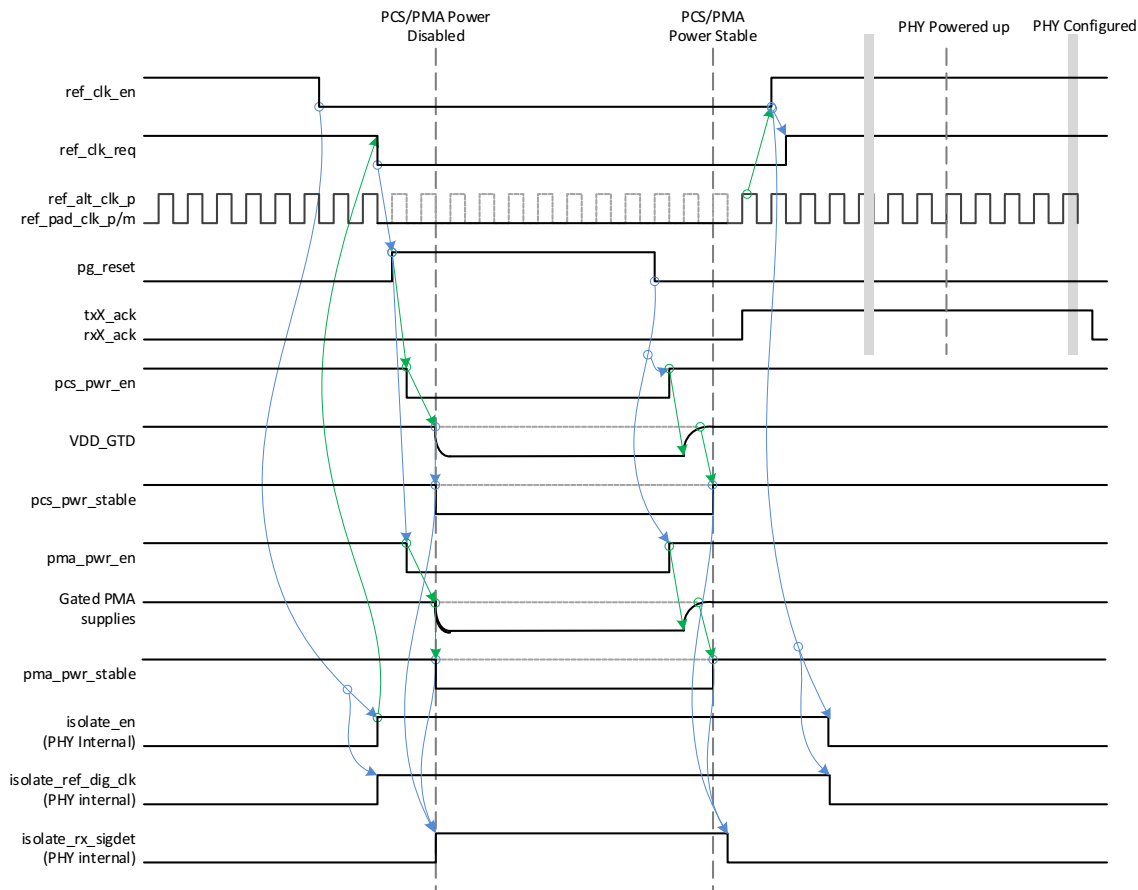
4.19 Power Gating

The PHY supports an optional power-gating feature through control and status signals for external power switches for the Raw PCS logic and the PMA vp and vpdig supplies.

[Figure 4-44](#) shows the control signal connectivity for power-gating with external power switch for the PMA. The PHY generates power-gating enables for the PCS switch (or multiple switches) as well as the PMA switch external to the PHY. The inputs `pcs_pwr_stable` and `pma_pwr_stable` must only be asserted after the supplies ramp up and are at 90% of nominal or higher. The power-gating mode is enabled when `pg_mode_en` input is 1. When `pg_mode_en` is 0, the power-gating control/status inputs to the PHY (except `ref_clk_en`, `ref_clk_req`, and `txX/rxX_ack`) shown in [Figure 4-44](#) are ignored.

Figure 4-44 Power-Gating Connectivity for External PMA Switch

The sequence of powering gating is shown in [Figure 4-45](#).

Figure 4-45 Power-Gating Sequence for the PHY (Internal or External PMA Switch) With pg_mode_en=1

The entry into power gated state starts by the de-assertion of `ref_clk_en`. The PHY performs isolation sequencing in preparation of power-gating by retaining states of some calibration registers and de-asserts the `ref_clk_req`. At this point, you can:

- Disable the reference clock inputs to the PHY.
- Assert the `pg_reset` input, which causes the `pcs_pwr_en` and `pma_pwr_en` signals to de-assert.
- Turn off the power supplies to PMA (`vp`, `vp_dig`, and `vp_txX`) and Raw PCS (`VDD_GTD`). However, it is allowed that these power supplies can stay ON because of any other SOC component requirement.

The exit from power gated state is initiated by de-assertion of `pg_reset`. The PHY requests power by assertion of `pcs_pwr_en` and `pma_pwr_en`. It is required that the `ref_clk_en` input be asserted after the PMA and Raw PCS power supplies are stable (`pcs_pwr_stable/pma_pwr_stable=1/1`). The PHY resets the internal power gated logic, disables the isolation on power gated signals, and asserts `txX_ack/rxX_ack` outputs. After the reference clock inputs are supplied, the PHY performs the fast power-up by recovering states of the retained calibration registers, and de-asserts `txX_ack/rxX_ack` outputs. At this point, the PHY is configured into the state defined by its primary configuration inputs.

4.19.1 Isolation Between VP and VDD Domains

The <PREFIX>_SEPARATE_VP_VDD macro can be used when there are steady state times, while VDD is powered OFF (that is, entire RAW_PCS/RMMI including the AON domain) but the VP is powered ON (that is, the hard-macro PMA). When this macro is enabled, there are specific isolation requirements to ensure that signals going to the hard-macro PMA are driven to inactive states. The list of required isolation signals are shown in [Table 4-31](#):

Table 4-31 Required Isolation Signals

Isolation strategy	Element	Clamp value	Isolation control
pma_vp_0_vdd_1_iso_rule	pma/ana_pwr_stable	0	phy_vpoff_vddon_iso_en
pma_vp_0_vdd_1_iso_rule	pma/rxX_sigdet_lf_ao	0	phy_vpoff_vddon_iso_en
pma_vp_0_vdd_1_iso_rule	pma/rxX_sigdet_hf_ao	0	phy_vpoff_vddon_iso_en
pma_vp_0_vdd_1_iso_rule	pma/rxX_sq_output_ao	0	phy_vpoff_vddon_iso_en
pma_vp_1_vdd_0_iso_cl1_rule	pma/ana_isolation_en_ao	1	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl1_rule	pma/phy_reset	1	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/ana_pwr_en	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/ref_alt_clk_lp	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/ref_repeat_clk_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_sigdet_hf_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_sigdet_lf_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_sq_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_sq_inv_polarity_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_sq_weakkeep_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_term_dc_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_term_gd_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/rxX_vrefgen_en_ao	0	phy_vpon_vddoff_iso_en
pma_vp_1_vdd_0_iso_cl0_rule	pma/txX_hp_prot_en_ao	0	phy_vpon_vddoff_iso_en

These isolation rules are included in the PHY-level UPF and are enabled when the macro <PREFIX>_SEPARATE_VP_VDD is enabled in the <prefix>_phy_defaults.v file.

When enabling this macro, there are specific requirements for the power domain supplying the isolation cells listed in [Table 4-31](#). These do not need to be explicitly supplied from the VPDIG power domain in the PMA, because this voltage domain may not be accessible outside of the hard-macro PMA. However, any other always-on power domain can be used as long as it meets the following requirements:

- DC voltage range is within the requirements specified in [Table 6-2](#) on page 188.

- Max voltage difference between the power domain supplying isolation cells and PHY vpdig is less than 100 mV.



Note

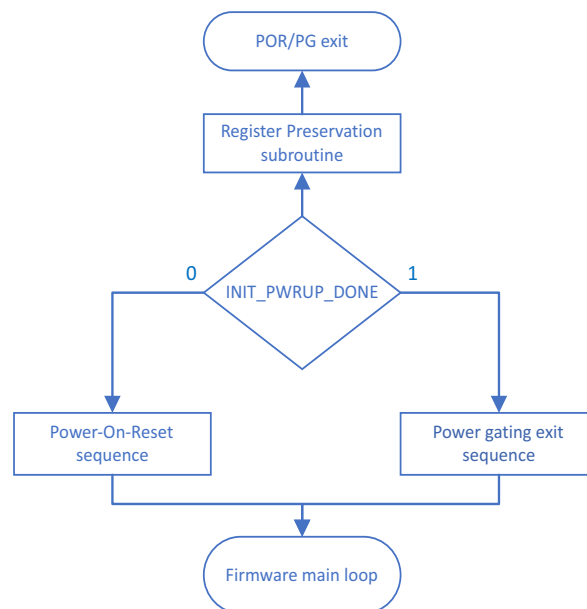
- Minimum list of pins that must have isolated cells while VDDSOC is OFF to avoid reliability issues is one of the following:
 - “phy_reset” =1 is applied by PHY by an isolation cell
 - “pma/ana_pwr_en”=0 is applied by PHY by an isolation cell. In this case, it is required to apply “phy_reset” =1 when vdd_soc is powered on and before asserting “pma/ana_pwr_en”=1
- In addition to the minimum list of pins, all the isolation cells listed in [Table 4-31](#) are added to ensure that the PHY remains in the lowest power consumption state. However, they do not have an impact on reliability.
- To enable the <PREFIX>_SEPARATE_VP_VDD macro, the phy_reset input to the PHY must be asserted when the VDD is ramped back up to fully reset the PHY.

4.19.2 Register Preservation

When performing power gating within the PHY, the state of all registers in the power gated domains will be lost and they will be set to their default reset value. As a result, a mechanism is required to be able to restore these values quickly and efficiently without having to interrupt the power gating exit sequence and without any intervention from the SoC.

The register preservation feature allows for storing a specific programming sequence that will be automatically executed by firmware during the power-on-reset (POR) and power-gated exit (PG exit) sequences as depicted in the following diagram:

Figure 4-46 POR and PG Exit Programming Sequence



The feature relies on available unused memory addresses within the SRAM or ROM to instruct firmware to configure PHY registers upon POR and PG exit. At the end of the firmware memory map there are five memory banks reserved with 64 16-bit locations (one each for lane0, lane1, lane2, lane3 and common) to

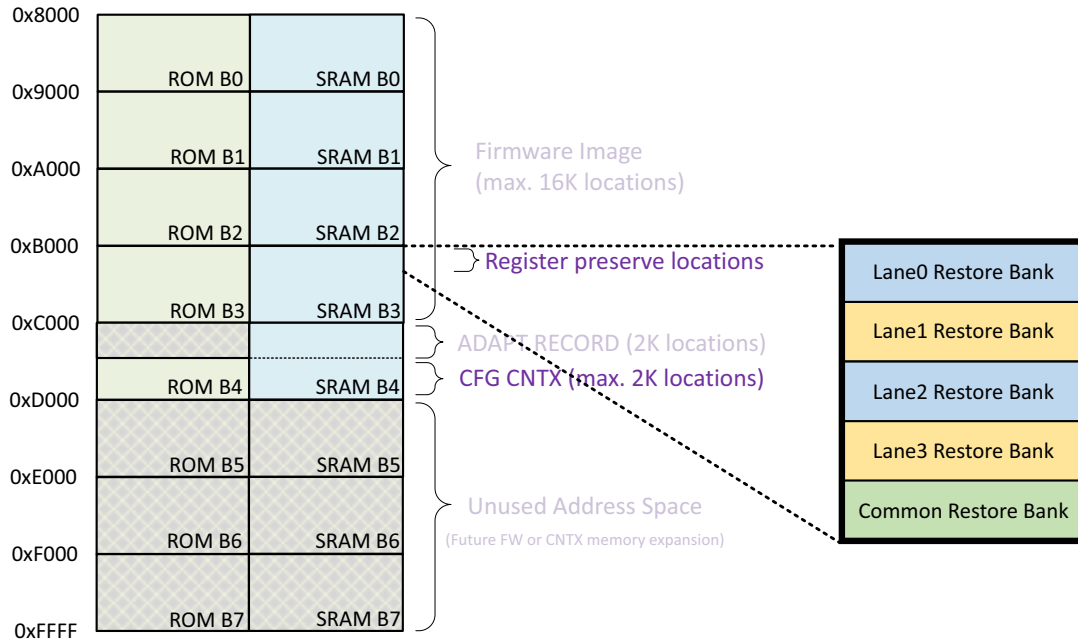
implement this feature. The Memory locations allocated for this feature are from 0xB000 to 0xB13F for a total of 320 16-bit spaces. This is shown in Table 4-32 below:

Table 4-32 Memory Bank Address Offset Descriptions

Memory Bank Address Offset	Description
0xB000-0xB03F	Memory offset location for Lane0 [Address, Data] pairs
0xB040-0xB07F	Memory offset location for Lane1 [Address, Data] pairs
0xB080-0xB0CF	Memory offset location for Lane2 [Address, Data] pairs
0xB0D0-0xB0FF	Memory offset location for Lane3 [Address, Data] pairs
0xB100- 0xB13F	Memory offset location for Common [Address, Data] pairs

Figure 4-47 shows the overall structure of this firmware instruction memory space.

Figure 4-47 Firmware Instruction Memory Space Structure



Since the firmware instruction execution is per-lane, each lane's firmware executes instructions in its corresponding lane's memory bank. The instructions in the common bank are executed by all firmware lanes before each lane's memory bank is accessed. Out of the 64 memory locations available, the first and last memory locations are reserved. The firmware uses the first location to store a pointer to exit this function and return to the normal processing sequence. By default, an end tag of "0x9016" is present at the next location. This tag informs firmware that it has executed all of the valid configurations on that memory bank and that it can now return to the normal processing sequence. As a result, if content is to be added, this end tag needs to be shifted down to the end of the added instructions up to the end of the memory bank address offset. The remaining 62 locations are empty spaces and allow for up to 31 STORE instructions to be executed per lane/common module.

The following table depicts the default memory content. If there is no update to this default content, the firmware moves to the respective bank and moves back without performing any actions.

Table 4-33 Default Memory Content

Memory Block Name	Memory Address	Memory Content	Comment
Memory Bank Lane 0	0xB000	0x33DF	Reserved instruction for FW operation
	0xB001	0x9016	End tag
	0xB002-0xB03F	Empty	Free space with zeros
Memory Bank Lane 1	0xB040	0x33DF	Reserved instruction for FW operation
	0xB041	0x9016	End tag
	0xB042-0xB07F	Empty	Free space with zeros
Memory Bank Lane 2	0xB080	0x33DF	Reserved instruction for FW operation
	0xB081	0x9016	End tag
	0xB082-0xB0BF	Empty	Free space with zeros
Memory Bank Lane 3	0xB0C0	0x33DF	Reserved instruction for FW operation
	0xB0C1	0x9016	End tag
	0xB0C2-0xB0FF	Empty	Free space with zeros
Memory Bank Common	0xB100	0x33DF	Reserved instruction for FW operation
	0xB103	0x9016	End Tag
	0xB104-0xB13F	Empty	Free space with zeros

4.19.2.1 Instructions to Fill the Memory Banks

- The memory banks are always filled with [Address, Data] pairs where the first memory address is the register pointer and the second is the register content.
 - Memory Space N - Address
 - IPXACT_CREG_ADDRESS[15:14] are dropped.
 - IPXACT_CREG_ADDRESS[11:9] are dropped. Those bits are used to decode the lane number. This feature uses the memory bank as lane decoder, each lane has its own memory bank reserved.
 - Address = {5'b00000, IPXACT_CREG_ADDRESS[13:12], IPXACT_CREG_ADDRESS[8:0]}
 - Memory Space N - Data
 - IPXACT_CREG_DATA_UPDATE[15:0]
- The bit-wise content of the IPXACT_CREG_DATA_UPDATE should be updated only for the bits that need a change. Care should be taken to ensure that other bits are kept at their reset values.
- The end tag must always be placed after the last valid [Address, Data] pair.

Any CREG that does not have the *AON* naming convention in the IPXACT is to be treated as a power-gated domain register. Any configuration updates to CREGs in the power-gated domain shall be

done through this feature to ensure that the configuration is reloaded any time PHY exits from a power-gated state.

Refer to “Register Descriptions” chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual* or the IPXACT views provided for identifying the address value (IPXACT_CREG_ADDRESS) and the default reset value of any registers that are being configured using the register preserve feature.

The comments and following table provide an example for how to update the memory content to modify the following registers:

1. Lane0
 - a. CREG A1 – Data C1
2. Lane1
 - a. CREG A2 – Data C2
 - b. CREG A3 – Data C3
3. Lane2
 - a. No updates
4. Lane3
 - a. No updates
5. Common
 - a. CREG A4 – Data C4

Table 4-34 Example of Memory Content

Memory Block Name	Memory Address	Memory Content	Comment
Memory Bank Lane 0	0xB000	0x33DF	Reserved instruction for FW operation
	0xB001	A1	Write on register A1 value C1
	0xB002	C1	
	0xB003	0x9016	End tag
	0xB004-0xB03F	Empty	Leave free space with zeros
Memory Bank Lane 1	0xB040	0x33DF	Reserved instruction for FW operation
	0xB041	A2	Write on register A2 value C2
	0xB042	C2	
	0xB043	A3	Write on register A3 value C3
	0xB044	C3	
	0xB045	0x9016	End tag
	0xB046-0xB07F	Empty	Leave free space with zeros

Memory Block Name	Memory Address	Memory Content	Comment
Memory Bank Lane 2	0xB080	0x33DF	Reserved instruction for FW operation
	0xB081	0x9016	End tag
	0xB082-0xB0BF	Empty	Leave free space with zeros
Memory Bank Lane 3	0xB0C0	0x33DF	Reserved instruction for FW operation
	0xB0C1	0x9016	End tag
	0xB0C2-0xB0FF	Empty	Leave free space with zeros
Memory Bank Common	0xB100	0x33DF	Reserved instruction for FW operation
	0xB101	A4	Write on register A4 value C4
	0xB102	C4	
	0xB103	0x9016	End Tag
	0xB104-0xB13F	Empty	Leave free space with zeros.

4.20 Configuration for a Disabled PHY

There are several methods to disable a PHY, if the PHY is not being used in your SOC implementation (that is, SOC that may have several PHY instantiations where depending on an application, one or more PHYs are disabled). To disable a PHY, follow the methods outlined in [Table 4-35](#).

Table 4-35 Signal Connections If the PHY is Not Used

Signal	Method 1: All Power Supplies (vp, vph, vpdig, gd) at Nominal Voltage and IDDQ = 1	Method 2: All Power Supplies (vp, vph, vpdig, gd) Connected to Ground	Method 3: All Power Supplies (vp, vph, vpdig, gd) Left Unconnected (Floating)
PMA inputs from Raw PCS	Not applicable ^a	Tie all inputs to ground—recommended	Tie all inputs to ground—recommended
PMA outputs to Raw PCS	Not applicable ^a	Insert isolation gate	Insert isolation gate
rxX_p, rxX_m	Leave floating or tie low	Leave floating or tie low	Leave floating or tie low
txX_p, txX_m	Leave floating	Leave floating	Leave floating
resref	Leave floating	Leave floating	Leave floating
ref_pad_clk_p, ref_pad_clk_m	Tie low	Tie low	Tie low
ref_alt_clk_p	Tie low	Tie low	Tie low
Note: ■ For methods 2 and 3, to ensure that the outputs do not float high nor drive any active logic, you must use isolation gates on PHY outputs. Furthermore, to prevent the PHY circuitry from inadvertently toggling, it is good practice to tie all the inputs to the ground. ■ If REXT is being shared with other PHYs, use only method 1. For REXT sharing, the request and ack handshake logic within each PHY must be powered and should allow the handshake signals from other PHYs to propagate through. For more information about resistor sharing, see Figure 6-9 on page 196 (“Reference Resistor Content - Multiple PHYs Sharing a Single Resistor”). ■ If clock is being repeated, ensure that you do not power down the PMAs in the chain. ■ For more information about IDDQ mode, see Table 4-19 on page 112 (“PHY Test Mode Selection”).			

a. Assuming that VDD_SOC is supplied and at nominal voltage

4.21 Notes for Configuring the PHY

For the configuration tables that follow, note the following important information:

- All values are in decimal
- The following inputs should be stable before assertion of `ref_clk_en` and de-assertion of `phy_reset`:
 - `ref_range[2:0]`
 - `ref_clk_div2_en`
 - `ref_clk_mpll{a,b}_div[2:0]`
 - `ref_use_pad`
 - `rx_vref_ctrl[4:0]`
 - `tx_vboost_lvl[2:0]`
 - `txX_iboost_lvl[3:0]`
 - `rxX_term_ctrl[2:0]`
 - `txX_term_ctrl[2:0]`
 - `rxX_term_acdc`
 - `pg_mode_en`
 - `cr_para_sel`
 - `sram_bypass`
- The following inputs should be stable before the de-assertion of `phy_reset` when powering up the PHY. The MPPLL-specific inputs can also be changed only when the corresponding MPPLL is powered down:
 - `mppll{a,b}_word_clk_div[1:0]`
 - `mppll{a,b}_fb_clk_div4_en`
 - `mppll{a,b}_ssc_en`
 - `mppll{a,b}_ssc_peak[19:0]`
 - `mppll{a,b}_ssc_step_size[20:0]`
 - `mppll{a,b}_ssc_up_spread`
 - `mppll{a,b}_frac_en`
 - `mppll{a,b}_div16p5_clk_en`
 - `mppll{a,b}_frac_den[15:0]`
 - `mppll{a,b}_frac_quot[15:0]`
 - `mppll{a,b}_frac_rem[15:0]`
 - `mppll{a,b}_div_clk_en`
 - `mppll{a,b}_div_multiplier[7:0]`
 - `mppll{a,b}_multiplier[11:0]`
 - `mppll{a,b}_bw_high[15:0]`
 - `mppll{a,b}_bw_low[15:0]`
 - `mppll{a,b}_ctl_buf_bypass`

- rxX_sigdet_lf_threshold[2:0]
- The following input can be changed only when the TX clock source is disabled (txX_pstate[1:0] = P1/P2, or txX_reset = 1):
 - txX_disable
- The following inputs can be changed only when the RX clock source is disabled (rxX_pstate[1:0] = P1/P2; or rxX_reset = 1):
 - rxX_div16p5_clk_en
 - rxX_cdr_ssc_en
 - rxX_disable

4.22 Configuring the PHY

For support and lane configuration values, see [“Support and Lane Configuration Settings”](#).

5

Verilog Model Description

This chapter includes the following sections:

- [“Overview” on page 170](#)
- [“Verilog Function” on page 171](#)
- [“Serial Line Modeling” on page 172](#)
- [“Verilog Model Files and Simulators” on page 173](#)
- [“Fast Simulation Mode” on page 174](#)
- [“Simulation Environment” on page 181](#)

5.1 Overview

The product deliverables include Verilog RTL for the Raw PCS and a Verilog behavioral model of the PMA. Contrary to the Verilog RTL for the Raw PCS, which can be synthesized into a gate-level netlist by a synthesis tool, the Verilog behavioral model of the PMA includes a netlist of generic GTECH gates and a non-synthesizable behavioral model of the analog circuits.

In the set of deliverables, the Raw PCS Verilog RTL and the PMA Verilog behavioral model reside at:

- Raw PCS: ./Latest/phy/rtl
- PMA Verilog model: ./Latest/pma/behavior

Together, they form the Verilog model of the PHY (dwc_mipi_mphy_type1_g5_phy_2tx_2rx_ns.v; N=Number of lanes). The PHY model allows you to run functional verification of your logic along with the PHY.

**Note**

This PHY model and test bench do not represent an example of exhaustive PHY testing.

5.2 Verilog Function

The Verilog model simulates using a four-state (0, 1, X, Z) simulator. Therefore, several features that require analog voltage and current values transmitted over wires cannot be simulated with this model.

Features that are not accurately modeled in the PHY model are as follows:

- Receive and transmit I/O terminations
- Termination resistor tuning and measurement
- Voltage-level setting for the Receiver Electrical Idle Detector
- AC signal characteristics of the ACJTAG receiver
- Voltage margining
- Analog test buses; the PHY pins atb_f_p, atb_s_m, atb_s_p are inaccurate during simulations
- Receiver detection

The receiver detection status output from the PHY txX_detrx_result is inaccurate during simulations. To simulate the receiver detection feature, you should force the receiver detection status on txX_detrx_result when txX_ack is asserted in response to the assertion of txX_detrx_req, and release the force after txX_ack is de-asserted.

- Transmitter equalization settings (pre-shoot, swing, and post-cursor) on the transmit serial lines txX_p/m
- Receiver equalizer settings (AFE CTLE boost and VGA gain, and DFE taps)

The RX Evaluation outputs rxX_adapt_fom, rxX_txmain_dir, rxX_txpost_dir, and rxX_txpre_dir (described in “[RX Equalization and Adaptation](#)” on page 110) are inaccurate during simulations. To simulate link adaptation, you should force the required settings at these PHY outputs when rxX_adapt_ack is asserted, and release the forces after rxX_adapt_ack is de-asserted.

5.3 Serial Line Modeling

The serial lines (txX_p/m, rxX_p/m) are fundamentally analog signals. Because each wire is limited to the simulator's four states, not all aspects of the analog signals can be reflected. [Table 5-1](#) provides encodings of the analog transmitter model outputs.

Table 5-1 Transmitter Model Serial Encodings

Output	txX_p	txX_m
Differential 1	1	0
Differential 0	0	1
No signal, holding common mode	0	0
No signal, not holding common mode	Z	Z

The analog receiver model interprets only differential 0 or differential 1 encodings as valid signal levels.

5.4 Verilog Model Files and Simulators

The Verilog model is generated for simulations and can be simulated using Synopsys VCS.

[Table 5-2](#) describes each portion of the Verilog model and indicates the product deliverables directory where the portion is stored.

Table 5-2 PHY Model Database Structure

Directory	Description
./Latest/phy/include	Contains macros used in the Raw PCS, PMA and simulation files.
./Latest/phy/rtl	Contains Verilog RTL for the Raw PCS files ■ <code>dwc_*_pcs_raw_xN.v</code>: Top-level RTL file for the xN Raw PCS ■ <code>dwc_*.v</code>: PHY top-level wrapper instantiating PMA and Raw PCS
./Latest/pma/behavior	Contains Verilog RTL for the behavioral models for the PMA. ■ <code>dwc_*_gtech.v</code>: Top-level file for the xN PMA instantiating GTECH model of the digital and behavioral model of analog sub-blocks.

Instructions for the PHY simulations are provided in the TESTBENCH.README file under `./Latest/phy/testbench` directory. The simulation database structure for PHY simulations is provided in [Table 5-3](#).

Table 5-3 Simulation Database Structure

Directory	Description
./Latest/phy/sim	VCS and NC Verilog regression scripts for GTECH and gate-level simulations of the PHY, which includes the following: ■ <code>run_<simulator>_gtech</code>: Simulation run script for PMA GTECH model and Raw PCS RTL (simulator = vcs, ncx) ■ <code>run_<simulator>_gates</code>: Simulation run script for PMA technology specific gates model and Raw PCS RTL (simulator = vcs, ncx) ■ <code>regression_gtech.csh</code>: Regressions script with multiple calls to <code>run_<simulator>_gtech</code> for different test cases, where <code><simulator></code> is passed as input argument to <code>regression_gtech.csh</code> ■ <code>regression_gates.csh</code>: Regressions script with multiple calls to <code>run_<simulator>_gates</code> for different test cases, where <code><simulator></code> is passed as input argument to <code>regression_gates.csh</code>
./Latest/phy/testbench	Sample test-cases for the behavioral and gate-level simulations. ■ TESTBENCH.README: Run instructions for the PHY test bench.



Note

To simulate the PHY model, define the following macros:

- `DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_SIM_MODE` - Sets the model to operate in simulation mode
- `DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_RANDOM_SEED` - Sets the seed for randomization
- `DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_*_PMA` - Sets the appropriate PHY mode.

5.5 Fast Simulation Mode

Some features, such as power-state transitions, receiver-detect, and RX adaptation, may take long simulation times in normal operating modes. These simulation times can be reduced for simulation purposes by either using fast simulation macros or by writing into the PHY control registers through the Parallel Control Register Interface or through JTAG (for information, see “[Control Register Interface](#)” on page 49). In addition, there are certain Verilog force statements that need to be included in the customer testbench to make simulations work.

The list of forces needed for normal simulation mode as well as fast simulation mode are listed in file “./phy/testbench/phy_forces.v.” For more information, see “./phy/testbench/TESTBENCH.README” file.

Within the PHY RTL wrappers ./phy/rtl/<prefix>.v, there exist two modules that are used to force certain values when operating in fast simulation mode. The two modules are described below:

- <phy_prefix>_force_freq_tune.v
Forces the RX VCO calibration code based on the reference clock frequency and the RX VCO configuration inputs
- <phy_prefix>_mpll_coarse_tune.v
Forces the MPLL calibration code based on the reference clock frequency and the MPLL configuration inputs

These modules are automatically loaded when running RTL simulations because they exist in the PHY RTL wrapper. However, if the RAW_PCS has been synthesized, these modules will be removed, and as a result must be instantiated and connected as they are in the PHY RTL wrapper.

The fast simulation mode can be enabled by defining the macro “DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_SHORT_RESET.” The macro based fast simulation mode is only supported for the RTL model of the Raw PCS. The fast simulation mode is supported for both GTECH and gate-level models of the PMA. In real silicon or with synthesized Raw PCS, the following sequence can be used to modify the internal Raw PCS FSM operation:

1. Set phy_reset and all of rxX_reset and txX_reset to '1'.
2. Wait for 10 ns or a minimum of 10 μ s after the power-supplies are ramped-up (at power-up).
3. Set phy_reset to '0'.
4. Wait for bootloading to complete (if enabled) and wait for sram_init_ld_done to assert.
5. Write a 1'b1 to the appropriate register fields described in the table below.
6. Optionally perform SRAM loading (if desired) and assert the sram_ext_ld_done.
7. Set all of rxX_reset and txX_reset to '0' for lanes, which are to be operational.



Note

If the FAST simulation registers cannot be written in step 5 (that is, during the SRAM external loading phase), they can optionally be written before step 7.

Table 5-4 Registers for Fast Simulation Mode

Register	Register Field	Description
SUP_DIG_RTUNE_FAST_FLAGS	FAST_RTUNE	Fast resistor tuning

Register	Register Field	Description
SUP_DIG_CLK_RST_BG_PWRUP_TIME_0	FAST_BG_WAIT	Fast BG times
SUP_DIG_CLK_RST_REF_PWRUP_TIME_0	FAST_REF_WAIT	Fast pre-scaler times
SUP_DIG_MPLLA_MPLL_PWR_CTL_CAL_CTRL_1	FAST_MPLL_PWRUP	Fast MPLL power-up
	FAST_MPLL_LOCK	Fast MPLL locking
SUP_DIG_MPLLB_UPLL_PWR_CTL_CAL_CTRL_1	FAST_MPLL_PWRUP	Fast MPLL power-up
	FAST_MPLL_LOCK	Fast MPLL locking
LANEN_DIG_TX_PWRCTL_TX_PWRUP_TIME_0	FAST_TX_REFGEN_EN_TIME	Fast TX ana refgen enable
	FAST_TX_CLK_EN_TIME	Fast TX ana clock enable
LANEN_DIG_TX_PWRCTL_TX_PWRUP_TIME_1	SKIP_TX_VCM_HOLD_WAIT	Skip wait for TX common mode hold power up
LANEN_DIG_TX_PWRCTL_TX_PWRUP_TIME_3	FAST_TX_RXDET	Fast TX RX-detection
LANEN_DIG_TX_PWRCTL_TX_PWRUP_TIME_4	FAST_TX_SERIAL_EN_TIME	Fast TX ana serial enable
LANEN_DIG_RX_PWRCTL_RX_PWRUP_TIME_0	FAST_RX_VREG_EN	Fast RX VREG enable
LANEN_DIG_RX_PWRCTL_RX_PWRUP_TIME_1	FAST_RX_CLK_DCC_EN	Fast RX clock dcc enable
LANEN_DIG_RX_VCOCAL_RX_VCO_CAL_CTRL_2	SKIP_RX_VCO_FREQ_TUNE_CAL	Skip RX VCO coarse calibration
	SKIP_RX_VCO_CAL	Skip RX VCO calibration altogether
LANEN_DIG_RX_VCOCAL_RX_VCO_CAL_TIME_0	FAST_RX_VCO_WAIT	Fast RX VCO wait times
LANEN_DIG_RX_ADPTCTL_RX_FAST_FLAGS	FAST_AFE_DFE_SETTLE	Fast AFE and DFE settling time
	FAST_SSM_DAC_SETTLE	Fast SSM DAC settling time

Register	Register Field	Description
RAWLANEAONN_DIG_RX_STARTUP_CAL_ALGO_CTL_0	SKIP_RX_AFE_CAL_STARTUP	Skip RX AFE DAC start-up calibration
	SKIP_RX_REF_CAL_STARTUP	Skip RX start-up reference level calibration
	SKIP_RX_REF_CAL_EXT_STARTUP	Skip RX start-up reference level calibration in extended mode
	SKIP_RX_ATT_CAL_STARTUP	Skip RX ATT start-up calibration
	SKIP_RX_VGA_CAL_STARTUP	Skip RX VGA1 start-up calibration
	SKIP_RX_VGA_CAL_EXT_STARTUP	Skip RX start-up VGA calibration in extended mode
	SKIP_RX_CTLE_CAL_STARTUP	Skip RX CTLE start-up calibration
	SKIP_RX_IQ_CAL_STARTUP	Skip RX IQ start-up calibration
	SKIP_RX_IQ_DELTA_STARTUP	Skip RX IQ fixed offset
	SKIP_RX_PHS_CAL_STARTUP	Skip RX phase slicer startup calibration
	SKIP_RX_PHS_CAL_EXT_STARTUP	Skip RX extended phase slicer startup calibration
	SKIP_RX_DFE_CAL_STARTUP	Skip RX DFE slicer start-up calibration
	SKIP_RX_DFE_CAL_EXT_STARTUP	Skip RX DFE slicer extended start-up calibration
	SKIP_RX_ERROR_CAL_STARTUP	Skip RX error slicer start-up calibration
	SKIP_RX_BYPASS_CAL_STARTUP	Skip RX bypass slicer start-up calibration
	SKIP_RX_SCOPE_CAL_STARTUP	Skip RX scope slicer start-up calibration

Register	Register Field	Description
RAWLANEAONN_DIG_RX_STARTUP_CAL_ALGO_CTL_1	SKIP_VGEN_CAL_STARTUP	Skip VGEN start-up calibration
	SKIP_SIGDET_CAL_STARTUP	Skip SIGDET start-up calibration
	SKIP_RX_VGA_SLICER_CAL_STARTUP	Skip RX VGA slicer start-up calibration
	SKIP_RX_BUF_CAL_STARTUP	Skip RX buffer calibration
	SKIP_RX_DCC_CAL_STARTUP	Skip RX DCC start-up calibration
	SKIP_RX_DCC_RANGE_CAL_STARTUP	Skip RX DCC ctrl range startup calibration
	SKIP_RX_FULL_RATE_CAL_STARTUP	Skip RX full-rate start-up calibration (DCC and IQ)
	SKIP_RX_HALF_RATE_CAL_STARTUP	Skip RX half-rate start-up calibration (DCC and IQ)
	SKIP_RX_DCC_DATA_CAL_STARTUP	Skip RX DCC DATA start-up calibration
	SKIP_RX_DCC_PHASE_CAL_STARTUP	Skip RX DCC PHASE start-up calibration
	SKIP_RX_DCC_BYPASS_CAL_STARTUP	Skip RX DCC BYPASS start-up calibration
	SKIP_RX_BIAS_CAL_STARTUP	Skip RX BIAS start-up calibration
	SKIP_RX_DCC_CAL_RATE	Skip RX DCC rate-change calibration
	SKIP_RX_DCC_RANGE_CAL_RATE	Skip RX DCC ctrl range rate-change calibration
	SKIP_RX_IQC_CAL_RATE	Skip RX IQC rate-change calibration
	SKIP_RX_BYP_EXT_STARTUP_CAL	Skip RX Bypass slicer start-up

Register	Register Field	Description
RAWLANEAONN_DIG_RX_STARTUP_ADAPT_ALGO_CTL_0	SKIP_RX_AFE_ADAPT_BANK_0_STARTUP	Skip RX AFE DAC start-up adaptation bank0
	SKIP_RX_DFE_ADAPT_BANK_0_STARTUP	Skip RX DFE DAC start-up adaptation bank0
	SKIP_RX_DFE_COARSE_ADAPT_BANK_0_STARTUP	Skip RX DFE coarse start-up adaptation for Bank0
	SKIP_RX_DFE_FINE_ADAPT_BANK_0_STARTUP	Skip RX DFE fine start-up adaptation for Bank0
	SKIP_RX_IQ_ADAPT_BANK_0_STARTUP	Skip RX IQ DAC start-up adaptation bank0
	SKIP_RX_AFE_ADAPT_BANK_1_STARTUP	Skip RX AFE DAC start-up adaptation bank1
	SKIP_RX_DFE_ADAPT_BANK_1_STARTUP	Skip RX DFE DAC start-up adaptation bank1
	SKIP_RX_DFE_COARSE_ADAPT_BANK_1_STARTUP	Skip RX DFE coarse start-up adaptation for Bank1
	SKIP_RX_DFE_FINE_ADAPT_BANK_1_STARTUP	Skip RX DFE fine start-up adaptation for Bank1
	SKIP_RX_IQ_ADAPT_BANK_1_STARTUP	Skip RX IQ DAC start-up adaptation bank1
	SKIP_RX_ADAPT_RELOAD	Skip RX adaptation initialization
	SKIP_RX_TX_INC_DEC_STARTUP	Skip RX continuous phase calibration
	SKIP_RX_FOM_STARTUP	Skip RX start-up FOM measurement
	SKIP_RX_MARGINING	Skip RX margining

Register	Register Field	Description
RAWLANEAONN_DIG_RX_CONT_ALGO_CTL	SKIP_RX_AFE_CAL_CONT	Skip RX AFE DAC continuous calibration
	SKIP_RX_PHS_CAL_CONT	Skip RX phase slicer offset calibration
	SKIP_RX_DFE_CAL_CONT	Skip RX DFE slicer continuous calibration
	SKIP_RX_DCC_CAL_CONT	Skip RX DCC continuous calibration
	SKIP_RX_REFLVL_ADAPT_CONT	Skip RX REF continuous adaptation
	SKIP_RX_VGA_ADAPT_CONT	Skip RX VGA continuous adaptation
	SKIP_RX_DFE_ADAPT_TAP1_CONT	Skip RX DFE DAC continuous adaptation for tap1
	SKIP_RX_DFE_ADAPT_TAP25_CONT	Skip RX DFE DAC continuous adaptation for tap2 - tap5
	SKIP_RX_FOM_CONT	Skip RX continuous FOM measurement
RAWLANEAONN_DIG_RX_FAST_FLAGS	FAST_RX_STARTUP_CAL	Fast RX start-up calibration
	FAST_RX_ADAPT	Fast RX adaptation
	FAST_RX_CONT_CAL_ADAPT	Fast RX continuous calibration/adaptation
	FAST_RX_CONT_ADAPT	Fast RX continuous adaptation
	FAST_RX_PWRUP	Fast RX power-up (LOS,VREG/AFE and DCC)
	FAST_RX_VCO_CAL	Fast RX VCO Calibration
	FAST_RX_VCO_WAIT	Fast RX VCO wait times
RAWLANEAONN_DIG_RX_STARTUP_CAL_ALGO_CTL_2	SKIP_VCM_CAL_STARTUP	Skip VCM start-up calibration

Register	Register Field	Description
RAWLANEAONN_DIG_TX_STARTUP_ALGO_CTL_0	SKIP_TX_DCC_CAL_STARTUP	Skip TX DCC start-up calibration
	SKIP_TX_DCC_RANGE_CAL_STARTUP	Skip TX DCC ctrl range startup calibration
	SKIP_TX_DCC_CAL_RATE	Skip TX DCC rate-change calibration
	SKIP_TX_DCC_RANGE_CAL_RATE	Skip TX DCC ctrl range rate-change calibration
RAWLANEAONN_DIG_TX_CONT_ALGO_CTL_0	SKIP_TX_DCC_CAL_CONT	Skip TX DCC continuous calibration
RAWLANEAONN_DIG_TX_FAST_FLAGS_0	FAST_SUP	Fast Support block (MPLL and Rtune)
	FAST_TX_CMN_MODE	Fast TX Common Mode Charge-up
	FAST_TX_RXDET	Fast TX Detect RX
	FAST_TX_STARTUP_CAL	Fast TX start-up calibration
	FAST_TX_PWRUP	Fast TX power-up (refgen, serial_en, clk_en)

**Note**

- Asserting phy_reset resets the registers to default values. To preserve fast simulation mode, the registers enabling the fast simulation mode must be written after every phy_reset de-assertion.
- The DAC behavior is not modeled in the PMA digital model. Therefore, the DAC offset calibration can take a long time, up to 15 ms. For digital simulations, to speed up runtime, it is recommended to define the FAST_RX_STARTUP_CAL macro or write "1" to the FAST_RX_STARTUP_CAL register field.
- All FAST simulation registers within the PHY are provided for completeness. Only those located in the RAW_PCS (that is, RAWLANEAONN) need to be written for FAST simulations since the FW will copy the FAST simulation flags from the RAW_PCS to the hard-macro PMA during boot up.
- For more details on the registers for fast simulation mode described in the previous table, refer to "Register Descriptions" chapter in the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3 Reference Manual*.

5.5.1 Interrupt Disable Mode

Even when running in fast simulation mode, the firmware is interrupted any time there is a rate change request to the PHY. The firmware checks each incoming rate change request to determine if any restorations are required (in fast simulation mode, all calibrations are skipped) and performs them accordingly. This process can take up to approximately 10 μ s per request. To disable the interrupts, define the `DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_DISABLE_RATE_IRQ` macro in addition to the

DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_SHORT_RESET macro. Defining this macro improves the rate change times in DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_SHORT_RESET mode by disabling firmware interaction during rate change requests to the PHY.

Note that defining DWC_MIPI_MPHY_TYPE1_G5_2TX_2RX_NS_DISABLE_RATE_IRQ removes the following functionality:

TX/RX DCC calibrations/restorations on rate changes

The firmware does not perform any calibrations or restorations during rate changes. The incoming request to change MPLL/TX/RX banks is ignored. Calibration codes for the MPLL are still updated, because they are driven from the `dwc*_phy_x2_ns_force_mpll_coarse_tune.v` file included in the PHY deliverables.

Note that when performing rate changes, the correct MPLL settings must still be applied to the PHY inputs to obtain the correct MPLL clock frequency.

5.6 Simulation Environment

The deliverables include a simulation environment including the `dwc_mipi_mphy_type1_pcs_2tx_2rx_ns` and the MIPI M-PHY Type 1 G5 as the Design Under Test (DUT).

6

Physical-Level Implementation

This chapter includes the following sections:

- [“PMA Assembly”](#) on page 184
- [“PMA Placement”](#) on page 185
- [“PMA Fill”](#) on page 188
- [“Operating Conditions”](#) on page 188
- [“Power Supply Connectivity”](#) on page 190
- [“ESD”](#) on page 191
- [“Reference Clock”](#) on page 194
- [“Reference Resistor”](#) on page 194
- [“ATB and DTB Signals”](#) on page 197
- [“Raw PCS Synthesis”](#) on page 197

6.1 PMA Assembly

6.1.1 Flip-Chip Pinout



Note

Refer to LEF file for minimum bump pitch used in the PHY.
Diagram below is preliminary and subject to change.
[Figure 6-1](#) is for illustration purposes only.
Actual bump locations may be different.
For accurate details, refer to the LEF view.

The assembled PMA pinout is shown in [Figure 6-1](#).

Figure 6-1 Bump Map Diagram (not available for this preliminary release)

6.1.2 PMA Size

The size of the PMA is determined by the size of the following:

- PMA hard macro
- Flip-chip pads

[Table 6-1](#) shows the size of the hard macro building blocks for the PMA.



Note

All dimensions in this table are preliminary and subject to change.

Table 6-1 PMA Size

Block	Drawn X (um)	Drawn Y (um)	Drawn Area (mm ²)	Shrink Factor	Actual Area (mm ²)
PMA	556.20	1388.16	0.772094592	1	0.772094592

6.2 PMA Placement

This section describes things to consider when placing the assembled PMA.

6.2.1 PMA Rotation and Mirroring

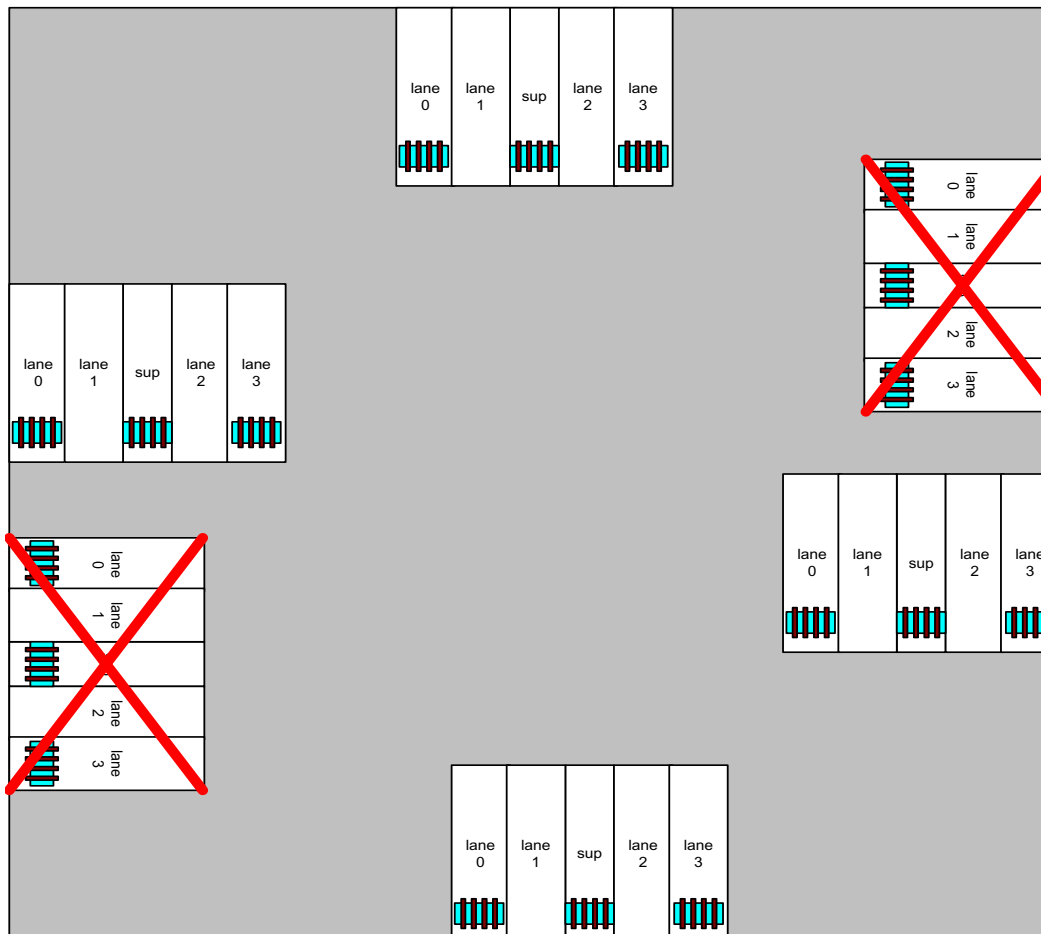
The FET POLY gates within the PMA must be vertical according to foundry rules.

The PMA hard macro is designed to be placed with preference for the north or south side of the die, as shown in Figure 6-2.

In order to meet the PAD to chip-edge spacing rule, the SoC level designer may need to place the PMA with an offset from the chip edge. SoC level designer must ensure that this placement follows the foundry rule and pad rule verification decks.

It is recommended that you DO NOT place the PMA hard macro on the east or west (non-preferred) side of the die, because doing so will complicate on-die interface routing as well as package routing of the TX and RX pairs. If placing the PMA on the east/west side is required, contact Synopsys support.

Figure 6-2 PMA Rotation and Mirroring



The PMA may be rotated by 180°, but not 90° or 270° due to the vertical POLY requirement.

The PMA may be mirrored in either the X or Y direction as this does not change POLY orientation.

6.2.2 PMA Abutment

The following requirements must be met when placing the PMA adjacent to other circuitry:

- Spacing Requirements
 - PMA to PMA:
 - When multiple PHYs are placed adjacent to each other, place them at 5 μm or at a spacing which satisfies the minimum bump pitch rule, whichever is larger, as shown in [Figure 6-3](#).

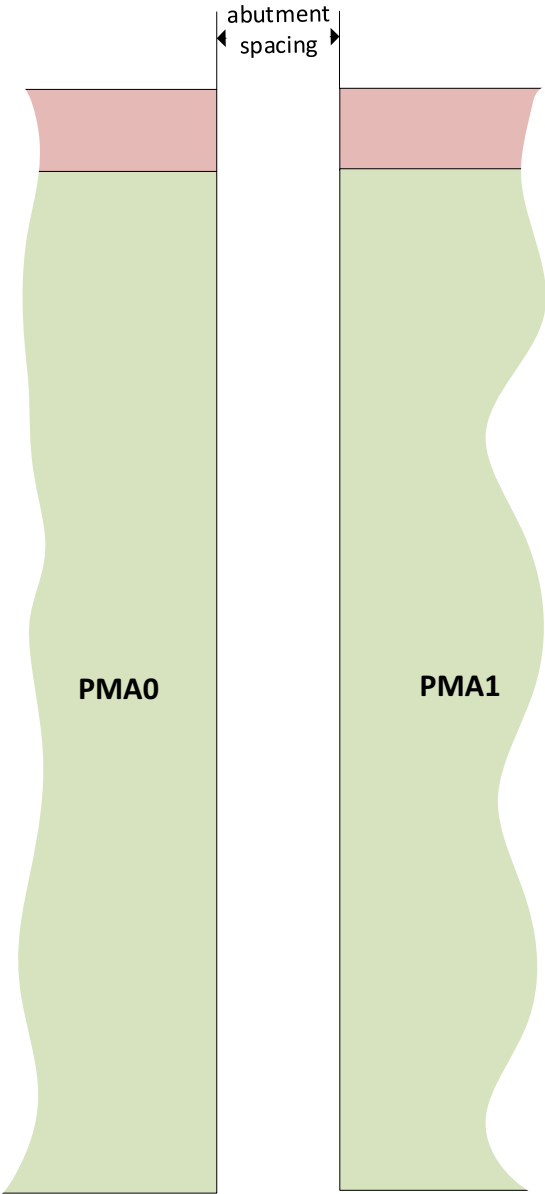


Note

[Figure 6-3](#) on page 187 is for illustration purposes only. Actual bump locations may be different. For accurate details, refer to the LEF view.

- Place a column of GD pads to isolate TX and RX routes of the adjacent PMAs. To accommodate smaller spacing, you can remove the column of GD pads. However, minimum bump pitch requirements must be met.
- Do not place the x1 and x2 PMA such that the clock support block is adjacent to the clock support block of the next PMA (that is, next PMA is instantiated in a flip-X)
- PMA to core logic:
 - Spacing for PMA requires a 5- μm "moat" on east and west edge
 - Spacing for x2 and x1 configuration requires a 20- μm moat on the edge of the PMA that has the clock support block exposed to the edge – on the unflipped/rotated PMA, this is the east edge. The other side of the PMA still has the 5- μm requirement.
- Moat requirements:
 - The "moat" cannot contain any core logic circuitry including cells, signal routes, and power meshes.
 - The "moat" cannot contain any substrate taps.

Figure 6-3 PMA Abutment



6.3 PMA Fill

The PMA comes pre-filled. When you run dummy fill at the chip level, no fill shapes are added within the PMA.

6.4 Operating Conditions

This section details the operating conditions of the PMA.

6.4.1 Power Supply Limits

The PMA supports the power supply voltages in [Table 6-2](#).

Table 6-2 Supported Power Supply Limits at the PMA Macro Bumps

Supply	Description	Minimum	Typical (Nominal)	Maximum	Unit
vph	High Voltage IO supply	1.2 -7%	1.2	1.2 + 10%	V
vp/vpdig	Core supply	0.75 - 7%	0.75	0.75 + 10%	V



Note

Allowed supply variation in [Table 6-2](#) might be lower than the supply variation found in the PMA libs.

6.4.2 Temperature

The PMA supports the following operating temperature:

- -40 ~ 125°C

Temperature drift after initial startup adaptation and calibration can affect the performance of PMA, particularly the receiver. The amount of temperature drift that the PMA can withstand before violating the protocol bit error rate limit is dependent on the receiver, the data rate, and external channel impairments. If there is a temperature drift, it may be necessary to re-adapt and re-calibrate the receiver. Sensitivity to this effect can be reduced by implementation of continuous offset correction and adaptation (CCA).

This PHY implements continuous offset calibration and adaptation (CCA). This reduces but does not zero out the sensitivity of the receiver to changes in temperature. Changes in temperature may still result in reduced eye opening as seen by the receiver and will reduce margin for jitter tolerance.

6.4.3 Electromigration

The electromigration limit of the PMA is 88KPOH@105°C (accounting for localized self heating - SHE).



Note

Operating at conditions that exceed the electromigration limits of PMA reduces the lifetime of the PMA. While the PMA can meet performance requirements at higher temperature, this is expected to be a short term condition, such as temporary failure in cooling, and so on.

6.4.4 Aging Specification

Table 6-3 shows the aging specification.

 **Note**

 Aging Specification values will be available in a future release.

Table 6-3 Aging Specification

Parameter	Value
-	-

6.5 Power Supply Connectivity

6.5.1 Power and Ground Isolation

Power supply connectivity must be designed accounting for all noise sources, decap, parasitics, package, and so on. You must perform a power integrity analysis.

To prevent power supply noise from inducing jitter on the transmitted and received waveforms, ensure that the PHY power supplies are isolated from the power supplies used on the rest of the ASIC. For more information about supply connectivity within the package and on the board, see “[Supplies Requirements](#)” on page 209.

In case of PI challenges, additional on-die decap could be supplied by customer to PHY vp/vph bumps via package.

The PHY ground, gd, is isolated on the die from the ASIC core ground vsscore_X. Back-to-back diodes between the ASIC ground (connected to vsscore_X pins) and the PHY ground, gd, are provided within the PHY macro as part of the ESD protection strategy (described in “[ESD Protection Strategy](#)” on page 192), but no direct metal-metal connection is permitted.

Isolation of all power and ground on the die implies the only connection between the PHY hard macro and the rest of the ASIC are for signal wires. The signal integrity of this interface is maintained using a fully clock-forwarded architecture where both clock and data cross the power domains together, so that subtle differences in supply voltages have no impact on timing.

6.6 ESD

This section describes ESD considerations.

6.6.1 ESD Requirements

ESD performance is evaluated by the amount of voltage and current that the protection circuits can absorb during an ESD event while keeping voltages below damaging levels. The protection circuits are designed to meet the protection levels provided in [Table 6-4](#).

Table 6-4 Minimum ESD Protection Levels

ESD Category Stress Type	Minimum Protection Level	JEDEC Class
Human Body Model (HBM) ANSI/ESDA/JEDEC [JS-001-2023]	2 kV	2
Charged Device Model (CDM) ANSI/ESDA/JEDEC [JS-002-2022]	6A peak discharge current	C2a/C1 (500 V/250 V) ^a
Latch Up (LU) JEDEC [JESD78F]	Signal Pin Test: 100 mA (125°C) Supply Test: 1.5x V _{maxSUP} (125°C)	II.A

a. Support for both 250 V or 500 V CDM target level is dependent on maximum discharge current generated in final SOC/package implementation.

All high speed IOs (such as rxX_p/rxX_m and txX_p/txX_m) have 2 kV Human Body Model (HBM) protection for optimized performance at 8 Gbps and above.

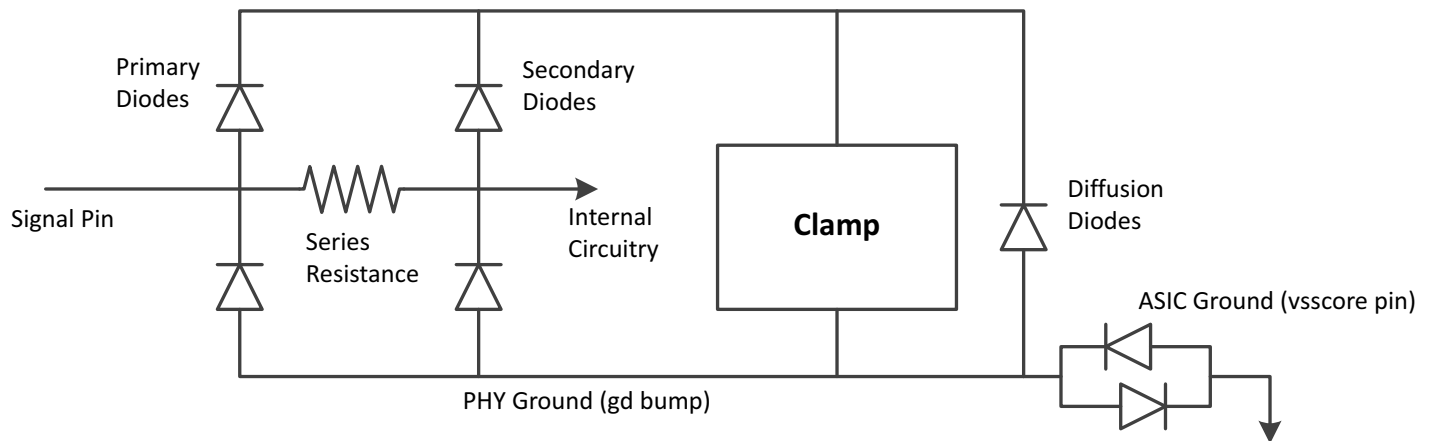
6.6.2 ESD Protection Strategy

The PHY's ESD protection strategy provides the following features:

- Large primary diodes between signal pad and PHY power and ground supplies
- A series resistor and secondary diodes between signal pad and PHY power and ground supplies
- An active clamp between the PHY power and ground supplies
The clamp and primary diodes are placed close to each other within the PHY, minimizing the resistance between them and maximizing the effectiveness of the ESD circuits.
- Back-to-back diodes between PHY ground, gd, and interfacing (ASIC) ground, vsscore_X
- The interface between the ASIC and hard macro is protected through the use of NAND/NOR CDM protection within the hard macro on all inputs and outputs
- Sufficient metal width in all ESD paths

Figure 6-4 shows the recommended ESD strategy to implement in the MIPI M-PHY Type 1 G5 .

Figure 6-4 ESD Protection Strategy

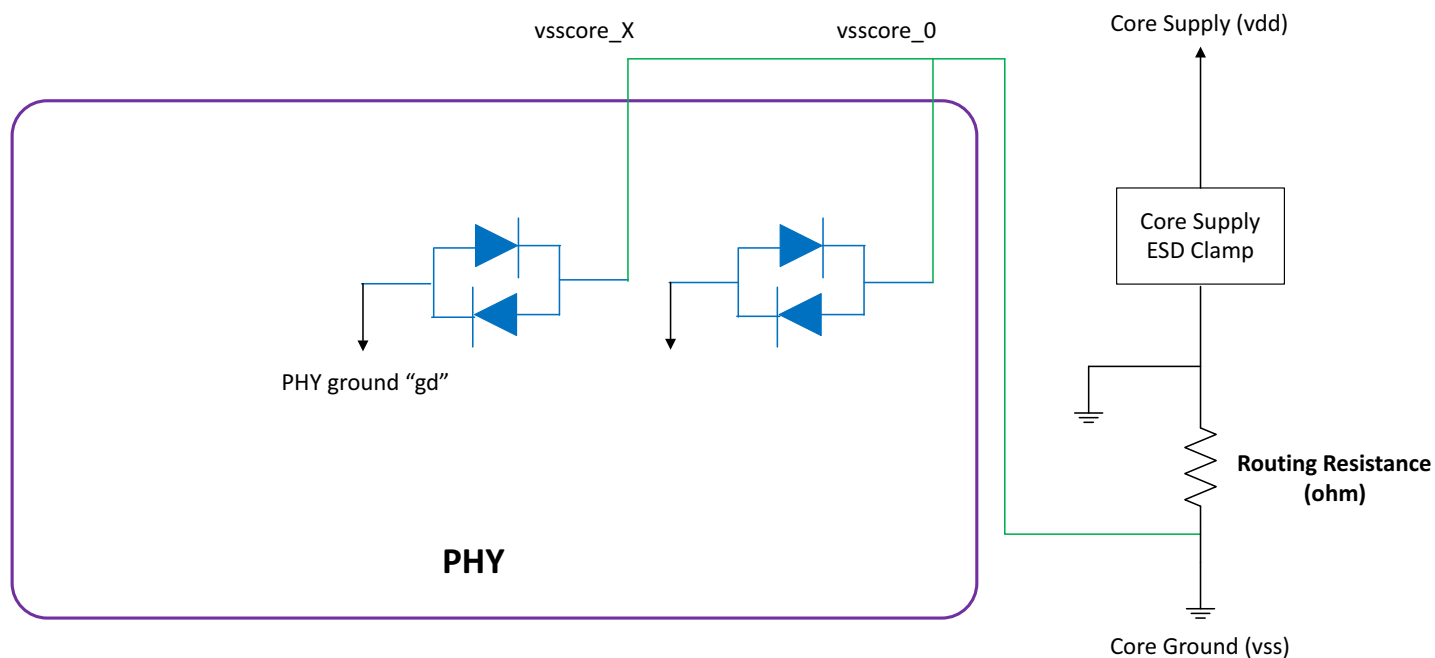


During ESD events, the primary diodes direct most of the current to the PHY power/ground buses and through the clamp. This path has minimal resistance, keeping the signal pad voltage and voltage across the PHY power buses low. Some current flows through the secondary diodes and develops a further voltage drop across the series resistor from the signal pad to the internal circuits. Back-to-back diodes enable current flow between ground domains-preventing large differences from developing between ground domains. The back-to-back diodes prevent damage between interfacing circuits due to power bus differences.

6.6.3 ESD Back-to-Back Diode Connection to Core VSS

Although vsscore and gd are to be connected together within the package, for ESD protection of the PHY/ASIC interface a back to back diode connection between vsscore and gd is provided within the PHY.

Each of the vsscore pins on the PHY must have a strong connection to the ASIC vsscore supply grid. The vsscore pins provided in the IP should be connected with a solid metal. Follow Foundry Design Rule Manual (DRM) ESD guidelines for IP integration for routing resistance between each vsscore pin on the PHY and the closest clamp within the ASIC (see [Figure 6-5](#) on page 193).

Figure 6-5 Back-to-Back Diodes in PHY**Note**

- The x2 configuration only has vsscore_0
- Add a vdd-to-vss clamp close to the back-to-back diodes with routing resistance from the diodes to the clamp with a value following the recommendations by the foundry Design Rule Manual (DRM).

Additional back-to-back diodes outside of the PHY (or other vss coupling devices) can be used to connect ASIC ground and PHY gd for additional ESD protection if desired.

If non-ground supplies are connected together for ESD protection through back-to-back diodes, special care must be taken to check for, and guard against latch-up, power-up sequencing and leakage issues.

6.7 Reference Clock

For details about reference clock physical implementation, see [“Reference Clock Implementation”](#) on page 221.

6.8 Reference Resistor

A reference resistor is used for calibration of the RX and TX termination. The reference resistor can be either external or internal (on-die) and is used when the PHY comes out of reset and when a manual tuning request is made. When a PHY is using the reference resistor (internal or external), it forces current across the resistor. When a PHY is not using the reference resistor, the current is disabled.

When internal reference resistor is used, it is required to access **resref** PAD for ATE calibration and e-fuse the calibration word. Calibration word must then be written to pma pins **res_ext_rcal[5:0]** for normal operation of the PHY. For ATE calibration procedures refer to section [“Reference Resistor”](#) on page 216. ATE calibration or normal usage of internal reference resistance requires **res_ext_en** to be set to '1'.

It is not allowed to use both internal and external reference resistors simultaneously. When internal reference resistor is being used (calibration or normal operation), external cannot be connected. When external reference resistance is being used, internal must be disabled (**res_ext_en** = '0').

Figure 6-6 External Reference Resistor Physical Implementation

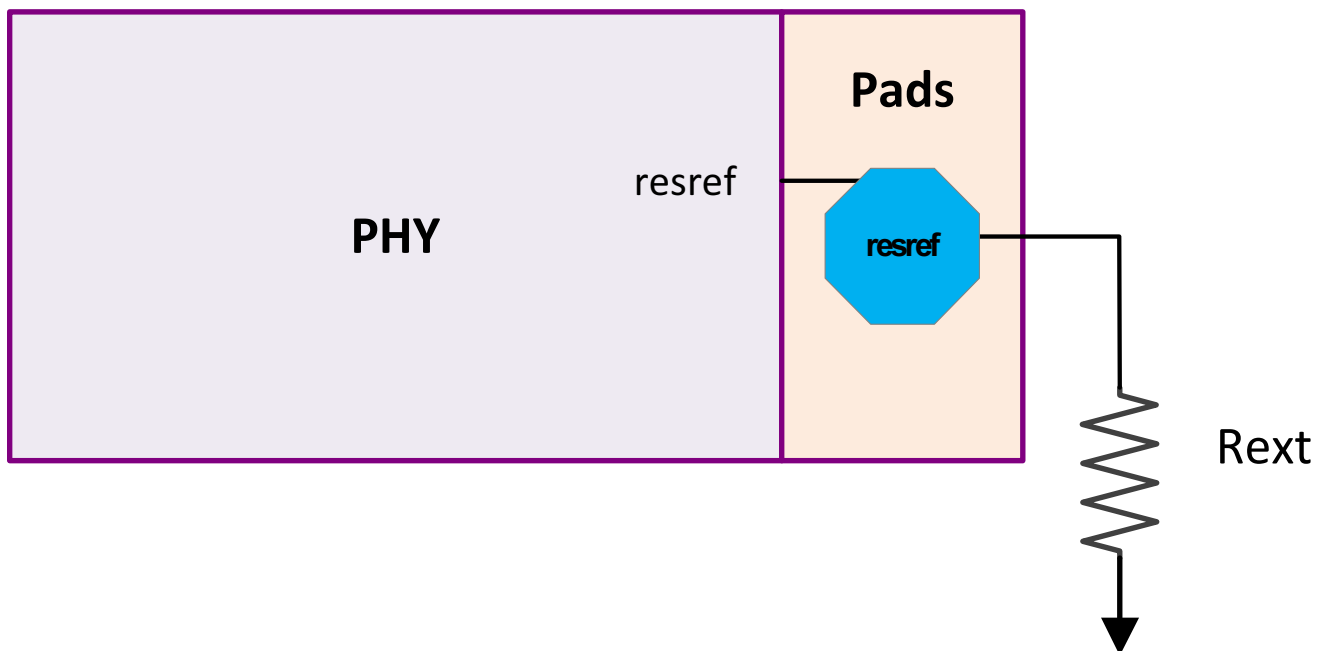
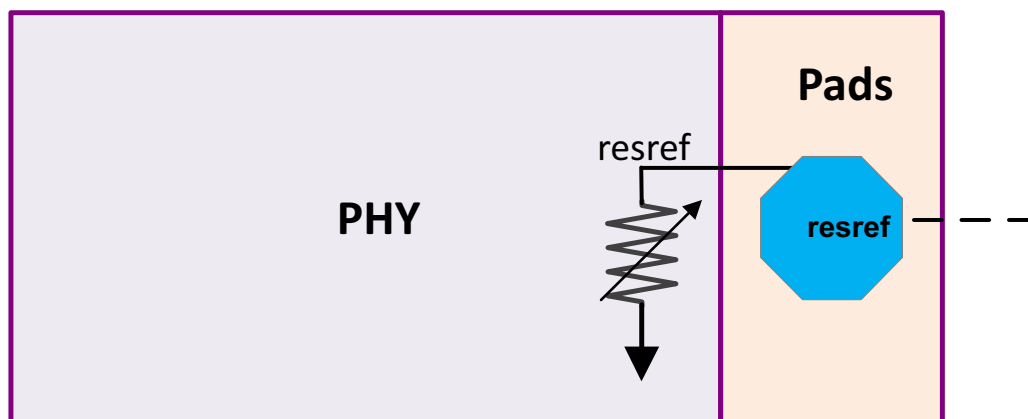


Figure 6-7 Internal Reference Resistor Physical Implementation

6.8.1 Reference Resistor Contention Bus

Each PHY requires access to a reference resistor. This access may be per PHY or it may be shared. A contention bus is used to control PHY access to the external resistor using `res_req_in`, `res_req_out`, `res_ack_in`, `res_ack_out`. Sharing a reference resistor is only allowed when using an external reference resistor.

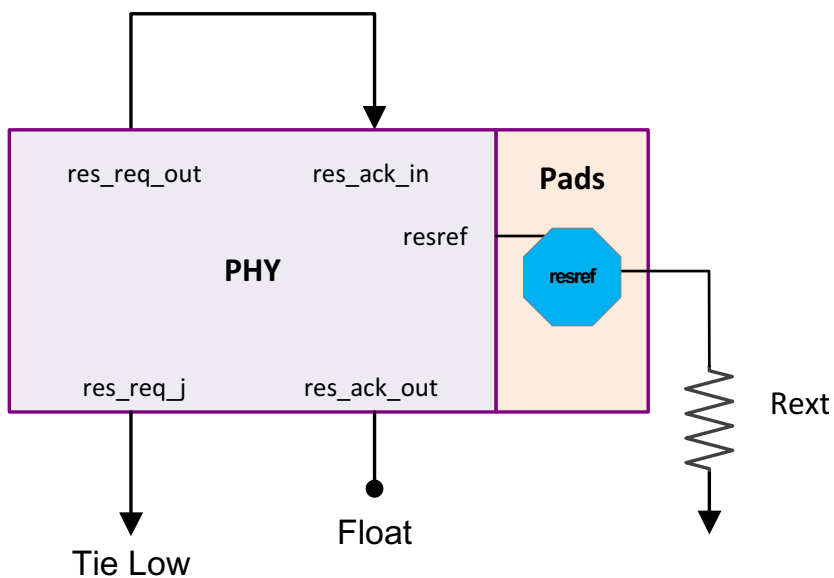
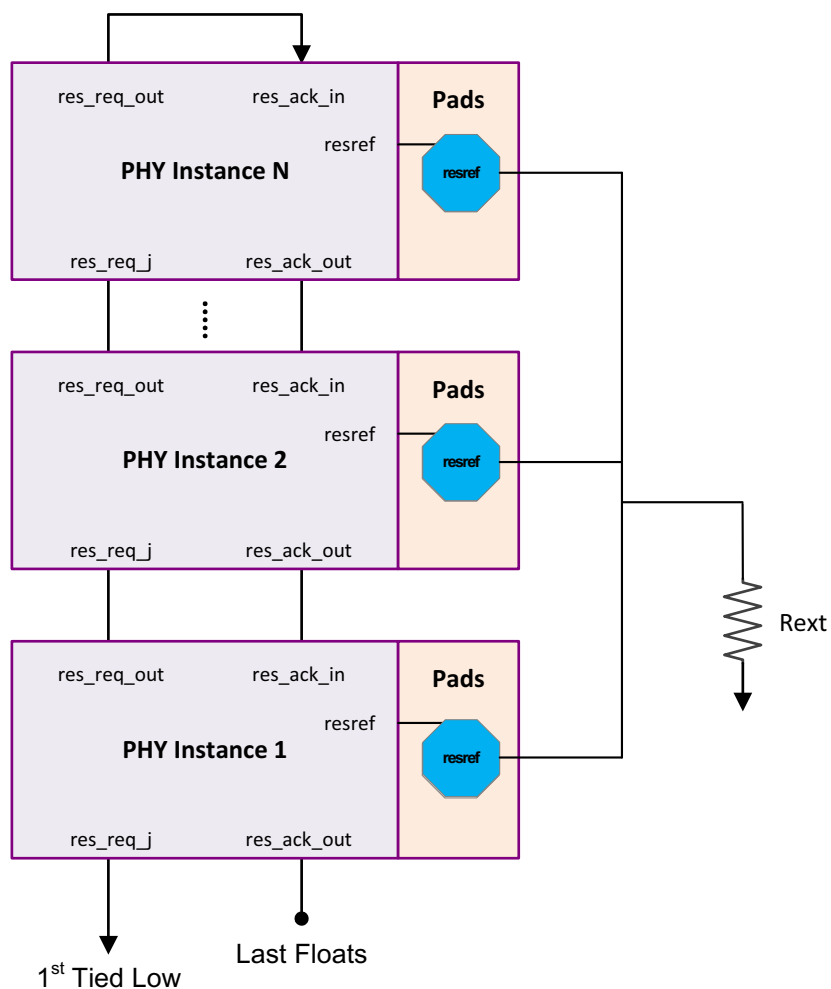
For a single-PHY implementation, connect the signals as shown in [Figure 6-8](#) on page 196.

For sharing a reference resistor among several PHYs, connect the signals as shown in [Figure 6-9](#) on page 196.



Note

- When sharing a reference resistor amongst several PHYs, it may take up to 30 μ s per PHY to sequentially calibrate each PHY.
- Before choosing to share a reference resistor, make sure to be aware of how it will be connected in the package and on the board. For information on package and board implementation, see [“Board- and Package-Level Implementation”](#) on page 207.

Figure 6-8 Reference Resistor Contention Bus - Single PHY**Figure 6-9 Reference Resistor Contention Bus - Multiple PHYs Sharing a Single Resistor**

6.9 ATB and DTB Signals

The analog test bus (ATB) is reserved. When integrating the PHY, leave the atb_s_p, atb_s_m, and atb_f_p pins unconnected.

The digital test bus (DTB) is provided as a means of accessing internal digital signals for debug purposes. If this bus is not needed, when integrating the PHY, you can leave the dtb_out[1:0] pins unconnected.

6.10 Raw PCS Synthesis

The Raw PCS is delivered as soft RTL, meaning that it will be hardened along with the rest of the RTL of the protocol-PCS controller and ASIC. Synthesizing the Raw PCS requires using a Verilog-2001 compatible synthesis tool.

6.10.1 Synthesis Database

The deliverables include the Design Compiler synthesis scripts in TCL format that can be used to synthesize the Raw PCS logic using the timing model of the PMA. The synthesis database structure is shown in [Table 6-5](#) and the ATPG deliverables are listed in [Table 6-6](#).

Table 6-5 Synthesis Database Structure for PHY

Directory	Description
./Latest/phy/synth	Synthesis directory for PHY ■ run_synth.sh: Shell script to run synthesis using Design Compiler TCL scripts. ■ run_formal.sh: Shell script to run logic equivalence checks between RTL and synthesized netlist of the Raw PCS using Formality. You need to update the variables in the scripts to match the installed technology library location (TECH_DIR, STD_DB). These scripts must be run in the directory ./Latest/phy/synth.
./Latest/phy/synth/scripts	Synthesis and Formal scripts ■ dwc_*_synth.tcl: Design Compiler Synthesis script to synthesize dwc_*_pcs_raw_xN using RTL files and libs for dwc_mipi_mphy_type1_g5_pma_2tx_2rx_ns PMA. ■ dwc_*_con.tcl: Design Compiler synthesis constraint file for the Raw PCS in TCL format. ■ dwc_*_synth_formal.tcl: Formality script for running logic equivalence check on the synthesized netlist of Raw PCS.

Table 6-6 ATPG Deliverables for the PHY

Directory	Description
./Latest/pma/atpg/<metal_stack>/ctl	CTL and SPF model
./Latest/pma/atpg/<metal_stack>/models	Verilog scan model of the PMA; only to be used by TetraMax (not to be used for simulations)
./Latest/pma/atpg/<metal_stack>/rpt	TetraMax log file
./Latest/pma/atpg/<metal_stack>/tetramax	run_tmax.sh: Top-level shell script to run TetraMax TCL script (tmax_atpg.tcl) for generating vectors

6.10.2 Raw PHY (PCS and PMA) Constraints

The synthesis constraints are provided in the form of TCL script used by Synopsys Design Compiler. This file (`dwc_*_con.tcl`) describes the design intent and surrounding constraints for synthesis, clocking, timing, power, test, and environmental and operating conditions. The constraints include the input and output delays of interface pins, clock definitions, and any timing path exceptions in the design, such as false paths or multicycle paths.

In most cases, the Raw PCS is synthesized in conjunction with other logic that resides on the opposite side of the ASIC interface boundary. In this situation, timing constraints can be automatically inferred from the adjoining logic and from the PMA timing model to which the Raw PCS is connected. When synthesizing the Raw PCS as a standalone block, signals on the ASIC interface must be constrained so that sufficient time is budgeted for the adjoining logic to be connected later.

6.10.2.1 Hand-Placed Logic Constraints

There are generic, digital-cell, primitive modules within the Raw PCS that are used to instantiate a specific digital cell that must not be logically optimized during synthesis. These modules are typically used around asynchronous clock domain crossing (CDC) points within the design. It is essential that these modules are processed correctly, not just synthesized as usual with the Raw PCS logic. Failure to do so will likely create glitch-generating and reconvergent logic that can cause problems on the ASIC interface.

BCM (basic core modules) are used for hand-instantiated logic within the PCS_RAW. This is a standard library of common cells used across Synopsys products.

[Table 6-7](#) provides a list of hand-instantiated BCM cells within the RAW_PCS.

Table 6-7 Hand-Instantiated BCM Cells

DesignWare Library Part Name	Description
*BCM_bcm21.v	2 stage data synchronizer asynchronous reset (active low)
*BCM_bcm21.v	3 stage data synchronizer asynchronous reset (active low)
*BCM_bcm21.v	4 stage data synchronizer asynchronous reset (active low)
*BCM_bcm00_ck_gt_lat.v	Clock Gating Latch
*BCM_bcm00_ck_buf.v	Pass-Through Buffer for Clock Trees
*BCM_bcm00_and.v	2-Input AND Gate
*BCM_bcm00_or.v	2-Input OR Gate
*BCM_bcm00_mx.v	2-Input Multiplexer Cell
*BCM_bcm00_ck_and.v	AND Block for Clock Trees
*BCM_bcm00_ck_or.v	2-Input Clock OR Gate
*BCM_bcm00_ck_mx.v	2-Input Clock Multiplexer
*BCM_bcm00_buf.v	Non-inverting Buffer with Random DELAY Sim Model

DesignWare Library Part Name	Description
*BCM_bcm00_ck_dff_prstn.v	Positive-edge-triggered D Flip-Flop Dedicated for Clock Path with Active-Low Asynchronous Preset
*BCM_bcm00_ck_dff_clrn.v	Positive-edge-triggered D Flip-Flop Dedicated for Clock Path with Active-Low Asynchronous Clear
*BCM_bcm00_dff_prstn.v	General Positive-edge-triggered D Flip-Flop with Active-Low Asynchronous Preset
*BCM_bcm00_dff_clrn.v	General Positive-edge-triggered D Flip-Flop with Active-Low Asynchronous Clear

**Note**

- There are two additional cells called `hand_ana_mux` and `hand_ana_buf` in the design. Do not hand instantiate during place and route because they reside within the analog GDS. They exist as analog leaf cells in the behavioral netlist.
- Note that the modules listed above are a superset of all the hand-instantiated modules that can be used by the design. As a result, if a module does not get instantiated it simply does not need to be replaced in the customer design.

To prevent these modules from being optimized during synthesis, customers can either replace the contents of these modules with an instantiation of the appropriate standard cell from a digital library, or synthesize each module stand-alone with a restricted library, so that only the correct cell is selected by the tool. In both cases, a “set_size_only” attribute must be applied to prevent reoptimization of the instance during regular Raw PCS synthesis.

6.10.2.2 BCM Technology Mapping

BCM cells support technology mapping which allows you to provide standard cell information to replace the BCM cells with technology specific cells during synthesis. The BCM cell technology mapping feature is enabled with the `ENABLE_Tech_CELL_MAPPING` define. Technology mapping is done through verilog `defines and tcl variables in the following files:

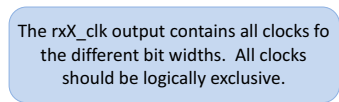
- `phy/include/*_bcm_tech_map.v`
 - This file provides the standard cell mapping `define macros to instantiate the technology specific cells.
- `phy/synth/constraints/*_bcm_pin_map.tcl`
 - This file provides the pin mapping variables that are used in the synthesis flow.

The above files should be modified by you to match the specific standard cell library being used. Additional details can be found in [Appendix D, “Cell Mapping for Hand-Placed Cells”](#).

6.10.2.3 Clock Constraints

External scan clock muxes should be added to all primary clock inputs of the PHY. This includes the `txX_clk`, `jtag_tck`, `fw_clk`, and `apbX_pclk`. Note that when `scan_mode = 1`, the input `apb0_pclk` is used to clock the external CR interface logic.

Figure 6-10 Functional Clock Definition Diagram for Raw PCS Synthesis



The clock definitions and associated false paths for the PHY are provided in the constraints file `dwc*_con.tcl` delivered as part of the synthesis deliverables.

6.10.2.4 Synchronizers

Soft IP (Raw PCS) instantiates synchronizer modules used for clock domain crossing protection. There are two types of synchronizer modules used in the design.

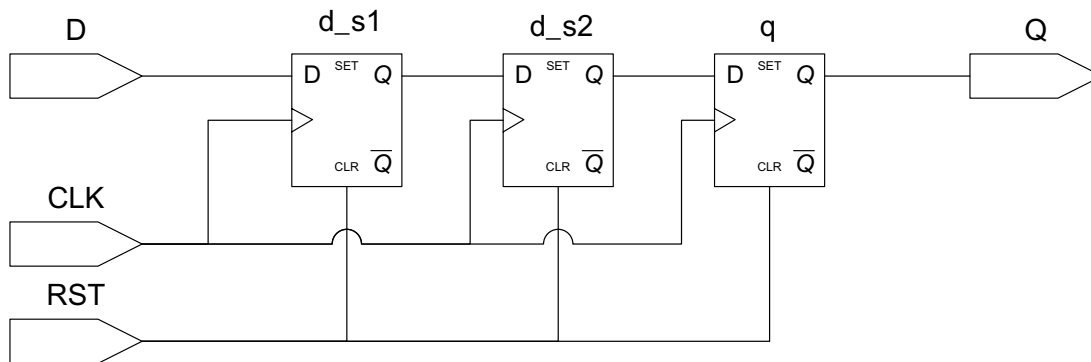
6.10.2.4.1 Data Synchronizers

For single-bit signals that must be passed from one clock domain to another, an instance of the module `"dwc*_gen_sync"` is used to synchronize the source signal into the receiving clock domain. The structure of a single-bit data synchronizer is shown in [Figure 6-11](#).

The expectation is that the customer will replace these synchronizer cells with an internal library synchronizer cell that has the same functional behavior. Regardless of whether these synchronizer cells are replaced, the following constraints should still be applied where applicable.

In case the `"dwc*_gen_sync"` module is synchronizing a data bus, the `false_path` should be replaced by a `set_max_delay` of 80% of the capture clock.

Figure 6-11 `dwc*_gen_sync` Module



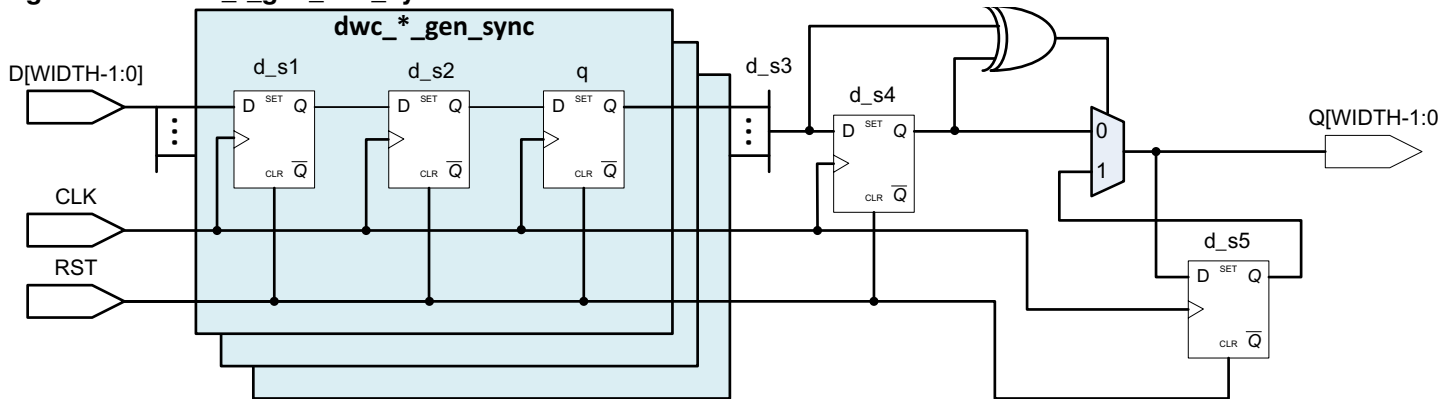
The input to the first stage of the synchronizer can be considered asynchronous when the source comes from an asynchronous clock domain. However, it is still advised to constrain the maximum delay to the input of the first stage of the synchronizer cell to a maximum of 80% of the capture clock period.

The second and third registers in the `dwc*_gen_sync` module are `d_s2` and `q`. To ensure stable synchronization, constrain the maximum delay to these cells, from `d_s1/CLK` to `d_s2/D` and from `d_s2/CLK` to `q/Q` to 20% of the sampling clock frequency.

Metastability behavior is modeled in the synchronizers and is activated by defining the macro `<PREFIX>_SIM_MODE` during simulations.

6.10.2.4.2 Bus Synchronizers

For multi-bit buses that need to be passed from one clock domain to another, an instance of the module `dwc*_gen_bus_sync` is used to synchronize the bus into the receiving clock domain. The structure of a bus synchronizer is shown in [Figure 6-12](#).

Figure 6-12 dwc*_gen_bus_sync Module

The input to the first stage of the data-bus synchronizer can be considered asynchronous when the source comes from an asynchronous clock domain. However, since multiple bits are being synchronized, each bit of the bus going to the data-bus synchronizer input stage must have a maximum delay constraint of up to 80% of the capture clock. This is required to ensure that all bits in the receiving domain are captured in either the same clock period or the subsequent clock period, which is necessary to ensure that the data-bus synchronizer functions correctly.

**Note**

Note that the maximum delay requirement of 80% of the capture clock period can be relaxed as long as the skew across the bits of the bus going to the bus synchronizer cell is constrained to less than 80% of the capture clock period.

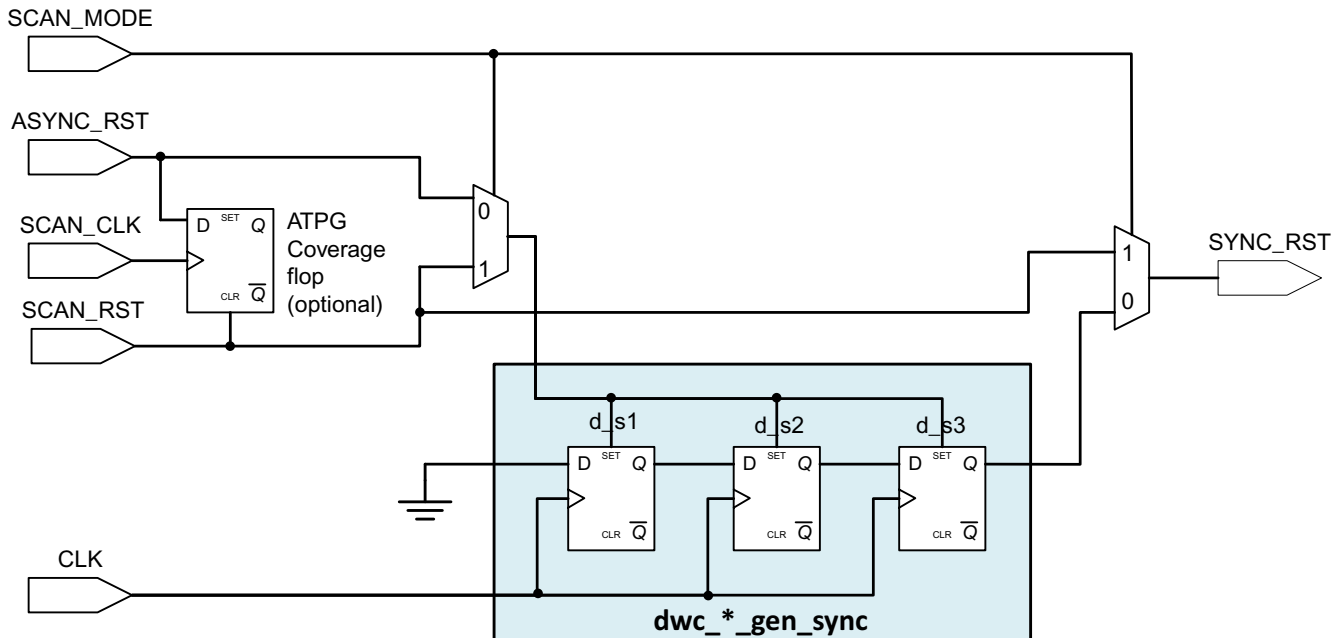
The dwc*_gen_bus_sync module instantiates a dwc*_gen_sync module to synchronize each bit of the data bus into the destination clock domain and as a result the same macro settings and timing constraints described in section “Data Synchronizers” on page 201 will apply.

Once the data bus has been synchronized, there is a single-cycle filter that filters out unstable intermediate data values.

6.10.2.4.3 Reset Synchronizers

For synchronizing an asynchronous reset signal, an instance of the module, "dwc*_gen_rst_sync," is used. As shown in Figure 6-13 on page 203, any time an external or internal reset condition is passed from one clock domain to another domain, the reset condition must pass through a dwc*_gen_rst_sync module to do the following:

- Asynchronously capture the reset signal
- Stretch the reset pulse to be active for at least one full clock cycle in the receiving domain
- Synchronously de-assert the reset signal with respect to the receiving domain

Figure 6-13 dwc_*_gen_rst_sync Module

Because assertion of a reset will cause all flip-flops in the synchronizer module to reset the pulse, the de-assertion edge of the asynchronous reset input has no effect on the final flip-flop that drives the reset into the receiving domain. Therefore, all asynchronous de-assertion paths through this module can be considered false.

The dwc_*_gen_rst_sync module instantiates a dwc_*_gen_sync module to synchronize the reset signal into the destination clock domain and as a result the same macro settings and timing constraints described in section “[Data Synchronizers](#)” on page 201 will apply.

The dwc_*_gen_rst_sync module also instantiates an optional (enabled by default) ATPG coverage flip-flop. This flip-flop is used to capture the incoming async_rst signal so it can be observed for the purposes of ATPG. The module also contains a scan mux to bypass the functional reset and use the top-level scan_set_rst reset signal directly in scan mode.

6.10.2.4.4 Synchronizer Constraints

Care must be taken when defining constraints related to data synchronizer cells. It is recommended that clock groups or false paths across asynchronous clock domains are used instead of globally adding false path constraints to the inputs of data synchronizer cells. Doing so ensures that valid intra-clock domain paths are still timed as synchronous paths. For example, the internal FSM synchronizes all read data, but in the case where the read data is coming from the internal cr_clk, the data is expected to be synchronous. Only timing paths from different asynchronous clock domains should be false paths through the data synchronizer cells.

6.10.2.5 txX_clk Constraints

TX parallel data to the PHY is transferred from the external ASIC txX_clk domain to the internal txX_ana_dword_clk domain. These two clocks are generated from the same MPLL source clock and are the same frequency, but asynchronous in phase. As a result, the clocks are aligned within the PMA by the TX Clock Alignment module to be able to accurately transfer data from one domain to the other. The TX Clock

Alignment function is triggered whenever the PHY enters P0/P0s state from a low-power state (P1/P2). The ASIC's txX_clk is aligned to the internal txX_ana_dword_clk to provide $\frac{1}{2}$ cycle of setup and hold margin between the clocks. However, it is possible for the incoming ASIC txX_clk to drift relative to the internal txX_ana_dword_clk domain as a result of voltage and temperature variation. Because TX clock alignment is performed only upon entry to P0/P0s state, the variation on the txX_clk must be bound to ensure that this drift does not cause txX_clk to misalign relative to the txX_ana_dword_clk domain, resulting in the PHY's inability to capture incoming data. Because the ASIC txX_clk is derived from the output PMA MPLL clocks, the entire path from the PMA lib outputs, through the external PCS logic and back to the PMA, the txX_clk input must be analyzed to determine the worst-case tree latency variation across all voltage/temperature (VT) corners.

Refer to section “TX Skew Requirements” on page 84 for additional details and equations for the TX skew requirements.

6.10.2.6 rxX_clk Constraints

To support 8/10/16/20/32/40-bit interfaces in synthesis, there are internal generated clocks within the .lib model of the PMA as shown in Figure 6-10 on page 200. The PHY RX interface can operate at up to:

In 8/10-bit mode:

- For M-PHYv5: 1.0 GHz

In 16/20-bit mode:

- For M-PHYv5: 1.168 GHz (23.3472Gbps@20bit)

In 32/40-bit mode:

- For M-PHYv5: 584 MHz (23.3472Gbps@40bit)

To maintain a common .lib model for all supported modes, there are internal pins for primary clock definitions for 8/10-bit operation and for 16/20/32/40-bit operation in the same .lib model:

8/10-bit operation:

- pma/rxX_ana_word_clk_i =
 - For M-PHYv5: 1.0 GHz

16/20/32/40-bit operation:

- pma/rxX_ana_dword_clk_i =
 - For M-PHYv5: 1.168 GHz (23.3472Gbps@20bit)

The result is three functional output clocks at the rxX_clk output pin of the PMA .lib model:

8/10-bit operation:

- pma/rxX_asic_wclk =
 - For M-PHYv5: 1.0 GHz

16/20-bit operation:

- pma/rxX_asic_dwclk =
 - For M-PHYv5: 1.168 GHz (23.3472Gbps@20bit)

32/40-bit operation:

- `pma/rxX_asic_qwclk =`
 - For M-PHYv5: 584 MHz (23.3472Gbps@40bit)

It is recommended to perform functional timing analysis for all three clocks coming out of the `rxX_clk` output because the clocks have different paths inside of the PMA and as a result, have slightly different delays to the `rxX_clk` output pin. If only a single mode is being used (for example, always operating in 32-bit mode), then it is possible to only consider timing analysis results from a single clock (that is, for a 32-bit interface, only the `rxX_asic_qwclk` is necessary), and ignore the other clocks.

6.10.2.7 ATE Clock Constraints

In the `RAW_PCS` constraints, there are a number of ATE clocks defined that are used only for ATE mode. These clocks are taken from the PMA and looped back through a mux so that they can be used to drive the LBERT module in the PMA for ATE testing. Note that when in LBERT mode, the PMA needs a `txX_clk` input toggling to drive the logic and generate the data.

These clocks should not be connected to anything in the SOC. They are only used to logically break timing paths between the clock looped back to the PMA and the logic outside of the PHY. This is done to prevent the PnR tools from attempting to balance the clock trees for these paths.

6.10.2.8 Reference Clock Constraints

No input-to-output arc exists for the `ref_ana_clk_i` in the PMA .lib file. This is an internal node within the PMA .lib file that is used to generate the `ref_dig_clk` output of the PMA. The reference clock should be defined on this internal `ref_ana_clk_i` pin of the PMA, not on `ref_*_clk_p/m`, because these are dummy pins that are not used within the PMA .lib file. Also note that in the PMA .lib file, the `ref_dig_clk` output clock is always a divide-by-2 clock with respect to the `ref_ana_clk_i` (regardless of the setting of the `ref_clk_div2_en` signal). This division is done during PMA .lib generation to ensure that the highest supported clock frequencies are propagated. As a result, if the target `ref_dig_clk` frequency is 24 MHz, `ref_ana_clk_i` should always be defined as $2 * 24 = 48$ MHz.

6.10.2.9 Scan Clocking Constraints**6.10.2.9.1 .lib ETM Timing Modes**

The PHY contains a multi-mode .lib file for the PHY module that provides timing information for various modes of operation. The following table outlines the modes.

Table 6-8 .lib ETM Timing Modes

Timing Mode	ETM Mode in .lib	Additional Constraints
Functional	NORMAL	N/A
Scan Shift	SCAN	False path all non scan_*_in/out ports
Stuck-At Scan Capture	SCAN	False path all scan_*_in/out ports
At-Speed Scan Capture	SCAN_ASST	False path all scan_*_in/out ports

**Note**

The hard macro PMA .lib file contains generated clock definitions for various functional clocks. If timing is performed using .lib ETM timing modes for SCAN/SCAN_ASST modes without defining the functional clocks, it may be necessary to remove these generated clocks (using `remove_clock` constraints) to prevent clock propagation errors.

6.10.2.9.2 Unused Signals

- Tie to ground potential any unused inputs to the Raw PCS.
- Leave floating any unused outputs from the Raw PCS.

7

Board- and Package-Level Implementation

This chapter includes the following sections:

- [“Package Type”](#) on page 208
- [“RX and TX Differential Signal Guidelines”](#) on page 208
- [“Supplies Requirements”](#) on page 209
- [“Reference Resistor”](#) on page 216
- [“Reference Clock”](#) on page 220
- [“Board Requirements – External Components”](#) on page 220

7.1 Package Type

The PMA is designed to support flip-chip packaging technology. For PMA bump-map diagrams, see “[Flip-Chip Pinout](#)” on page 184.

7.2 RX and TX Differential Signal Guidelines

You must design and simulate each of the RX and TX differential routes to ensure that signal integrity requirements are met.

**Note**

For more information, see the requirements listed in the “SerDes Differential Channel Guidelines” chapter of the *Synopsys PHY IP Cores SERDES PCB and Packaging Design Guide Application Note*.

7.2.1 Package Guidelines—RX and TX Maximum Allowed Package Loss

In general, the package insertion loss at Nyquist is 1.5 dB as a guideline.

For most protocols, the TX eye requirements must be met at the package pins. Both TX output amplitude and equalization are affected by package loss. These considerations can limit the amount of package loss for certain protocols.

7.2.2 Signal Integrity Analysis

To enable signal integrity (SI) analysis of the RX and TX routes, s-parameter models as well as an AMI model is provided with the PHY deliverables.

7.2.2.1 S-Parameter Model

To assist in package design, an S-parameter model of the RX input and TX output are provided. These models should be simulated along with your extracted package model to ensure that the protocol specific requirements for return loss and other signal integrity requirements are being met.

7.2.2.2 AMI Model

To assist link modeling, an IBIS Algorithmic Modeling Interface (IBIS-AMI) model of the PHY transmitter and receiver is provided along with a representative channel model.

The provided channel model is simply an example, and customers should measure the system board and connectors they are using to get a more accurate representation of the link performance.

7.3 Supplies Requirements

You must design and simulate the Power Distribution Network (PDN) for each of the PHY supplies to ensure that power integrity requirements are met.



Note

For more information, see the requirements listed in the “Supply Guidelines” chapter of the *Synopsys PHY IP SerDes PCB and Packaging Design Guide Application Note*.

7.3.1 Power Supplies Delivery to Pads

A power integrity (PI) analysis must be performed to ensure that the DC and AC noise supply requirements in [Table 7-1](#) are met on-die.

Table 7-1 Design Guidelines for Power Supplies

Signal	Supply Voltage at PHY bumps (DC)	Noise specification	Design Guidelines
gd	0 V	N/A	Dedicated full or partial plane Keep a sufficient distance from high speed signals / CLKs to ensure 30 dB isolation
vph	Absolute min DC value (V): See Minimum at Table 6-2 on page 188 Typical DC value (V): See Typical at Table 6-2 on page 188 Absolute MAX DC value (V): See Maximum at Table 6-2 on page 188	See section “ vph Supply Requirements ” on page 211	
vp/vpdig	Absolute min DC value (V): See Minimum at Table 6-2 on page 188 Typical DC value (V): See Typical at Table 6-2 on page 188 Absolute MAX DC value (V): See Maximum at Table 6-2 on page 188	See section “ vp/vpdig Supply Requirements ” on page 210	



Note

For the low noise vph analog supply, an LDO is recommended in order to minimize supply noise. If a switching supply is used, follow these additional recommendations:

- Do not share directly with other noisy circuitry
- Use an inductor or ferrite bead to isolate supply noise and filter edge rate at the package signals
- Share with Synopsys the actual power integrity waveforms for further analysis

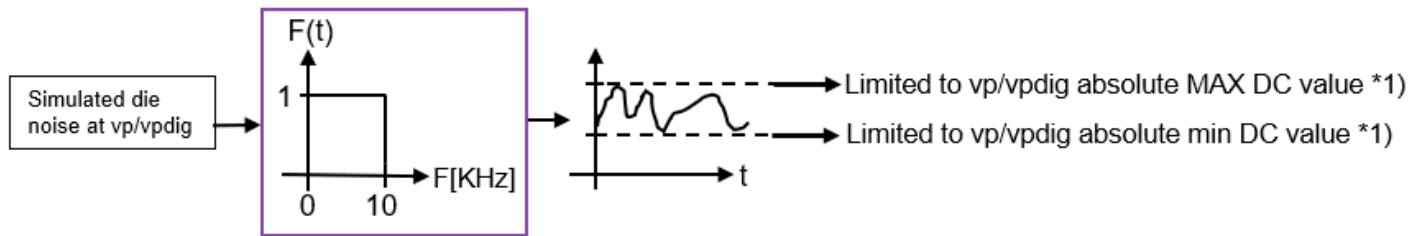
Current waveforms along with C_{die} and R_{die} are provided with the PHY deliverables. Simulate the supply noise using this PI enablement along with a model of your package and board to ensure that on-die supply requirements are being met.

The power supply requirements are on-die at the PHY pads.

7.3.1.1 vp/vpdig Supply Requirements

The total amount of tolerated noise in this band depends on the applied DC level, being maximized when DC level is centered within vp/vpdig DC range limits. DC limits are specified in [Figure 7-1](#).

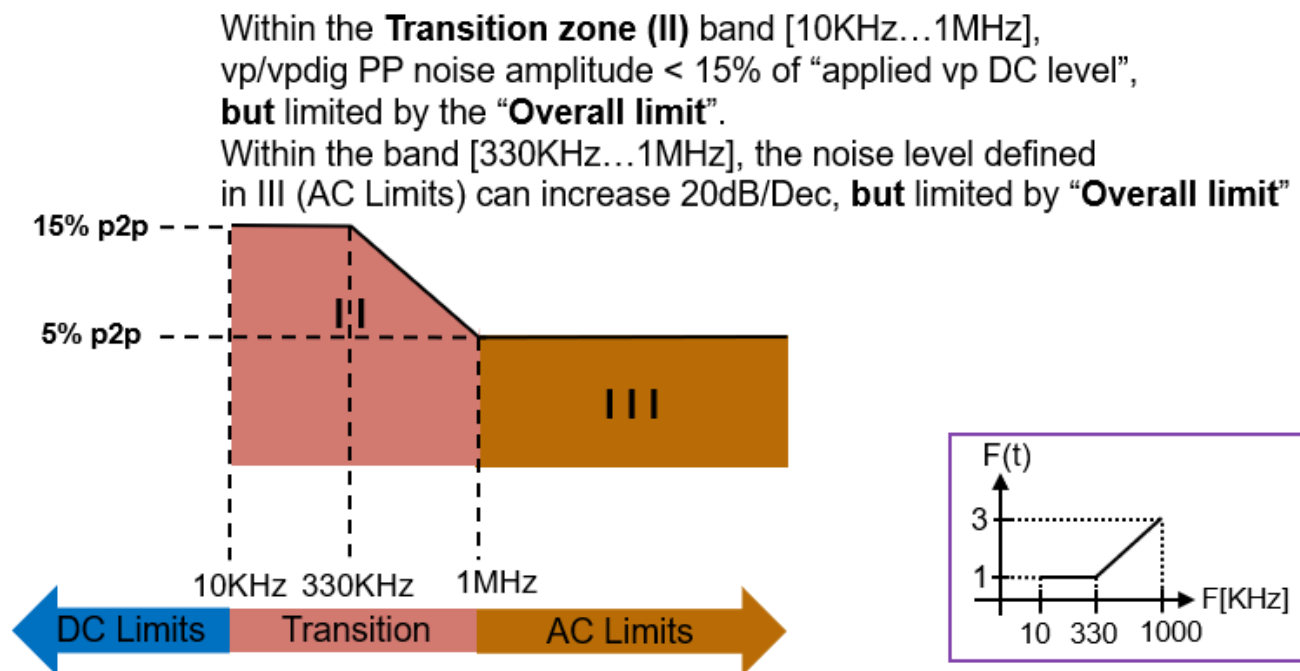
Figure 7-1 Specified DC Limits



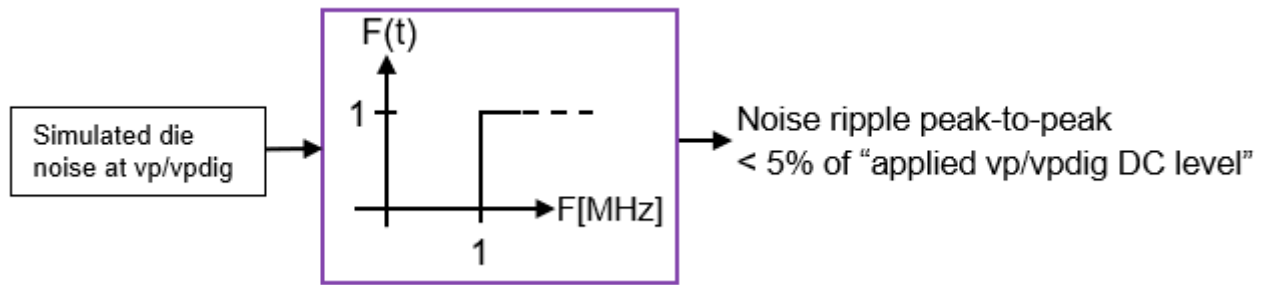
1) As defined in [Table 6-2](#) on page 188.

See [Figure 7-2](#) for specified vp/vpdig AC limits at the die.

Figure 7-2 vp/vpdig AC Limits at the Die



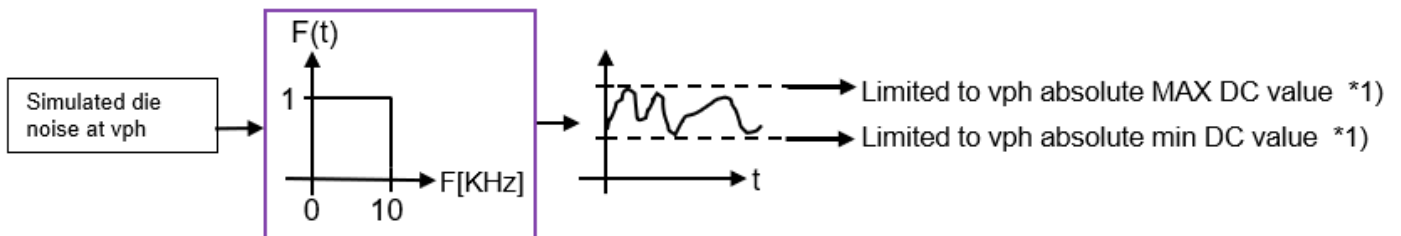
The maximum allowed peak-to-peak voltage is relative to the “applied vp/vpdig DC level. AC limits are specified in [Figure 7-2](#) (“III”).

Figure 7-3 Specified AC Limits**Overall limits:**

In addition to the previously defined DC/Transition/ AC limits, there is also a limit to the minimum allowed instantaneous simulated vp/vpdig voltage, that must be always above "Absolute min vp/vpdig DC value" - 2.5%.

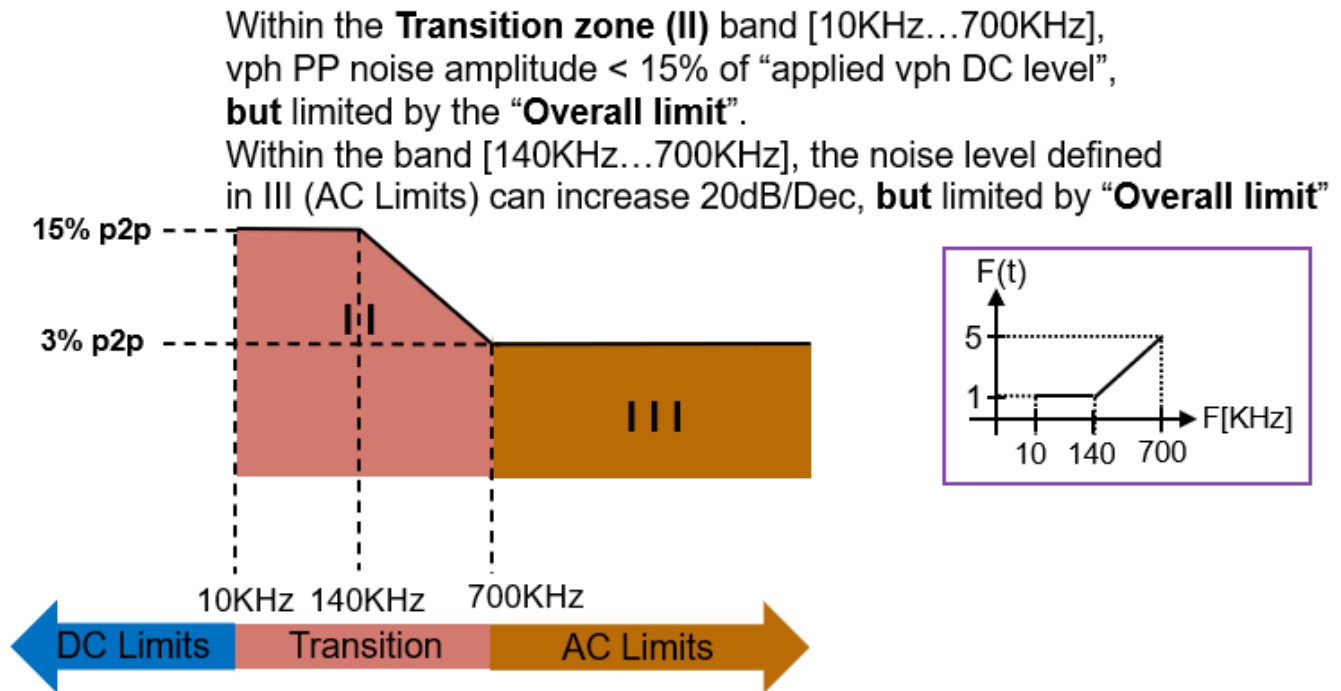
7.3.1.2 vph Supply Requirements

The total amount of tolerated noise in this band depends on the applied DC level, being maximized when DC level is centered within vph DC range limits. DC limits are specified in [Figure 7-4](#).

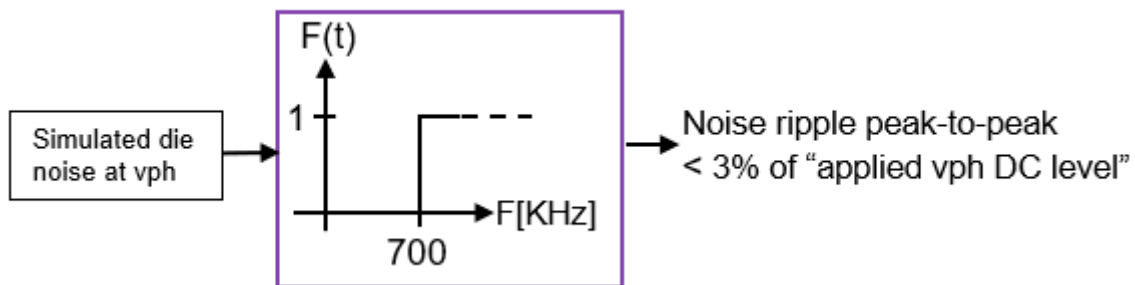
Figure 7-4 Specified DC Limits

1) As defined in [Table 6-2](#) on page 188.

See [Figure 7-5](#) for specified vph AC limits at the die.

Figure 7-5 vph AC Limits at the Die

The maximum peak-to-peak voltage is relative to the applied vph DC level. AC limits are specified in Figure 7-5 (“III”).

Figure 7-6 Specified AC Limits**Overall limits:**

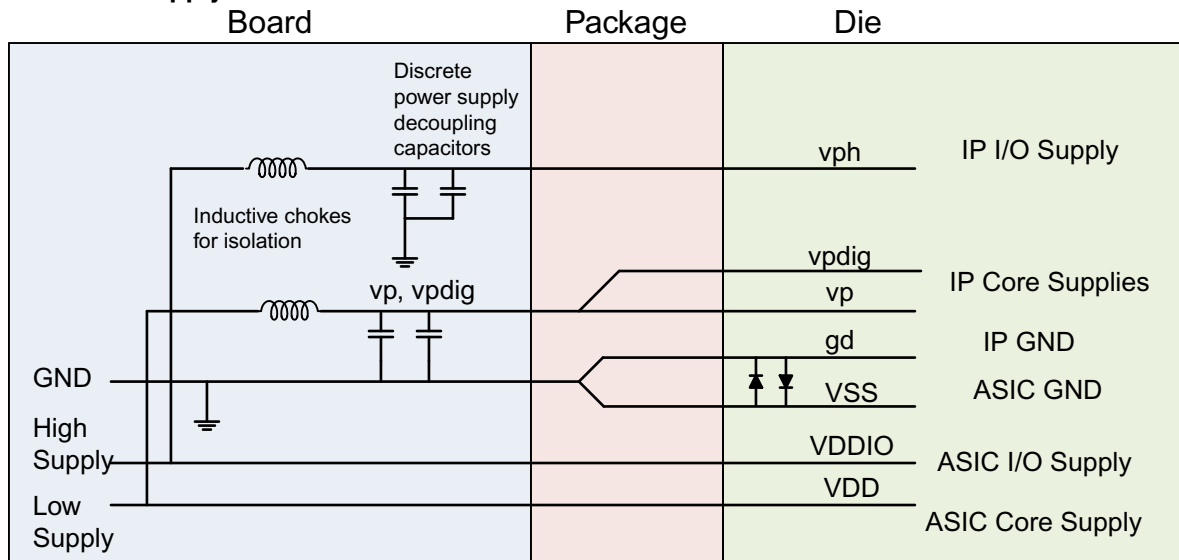
In addition to the previously defined DC/ Transition/ AC limits, there is also a limit to the minimum allowed instantaneous simulated vph voltage, that must be always above

- “Absolute min vph DC value” - 1.5%
- vph slew rate < “applied vph DC level” / 10us

7.3.2 Package and Board Guidelines—Power Supply Sharing

A diagram of the power supplies is shown in [Figure 7-7](#).

Figure 7-7 Power Supply Connections



To ensure good jitter performance, the PHY is designed to completely isolate power and ground supplies from the rest of the ASIC on-die. Per the power supply connections diagram, you need to isolate the PHY power supplies from other ASIC supplies by using inductive chokes (such as, ferrite bead).

To ensure good ESD performance, the PHY requires a common ground in the package with the rest of the ASIC.

The following supplies must be connected/shorted together within the package:

- PHY core supplies (vpdig, vp)
- PHY ground (gd) and ASIC ground

To reduce noise coupling from the ASIC, vsscore may be connected to PHY gd outside the package on the board.

When multiple PHYs are instantiated, their vp and vpdig may be connected together within the package according to the following requirements:

- If you are using multiple reference clocks with different sources, particularly if they are SSC and each independently spread, then recommendation is not to share the VPH supply in order to mitigate the risk of PLL pulling.
- Per-PHY supply parasitics and bypass capacitance are maintained.
- Power integrity simulation has been performed to confirm AC supply noise requirements are observed for all PHYs.

7.3.3 Recommended Maximum Off Silicon Die Power Supply Inductance

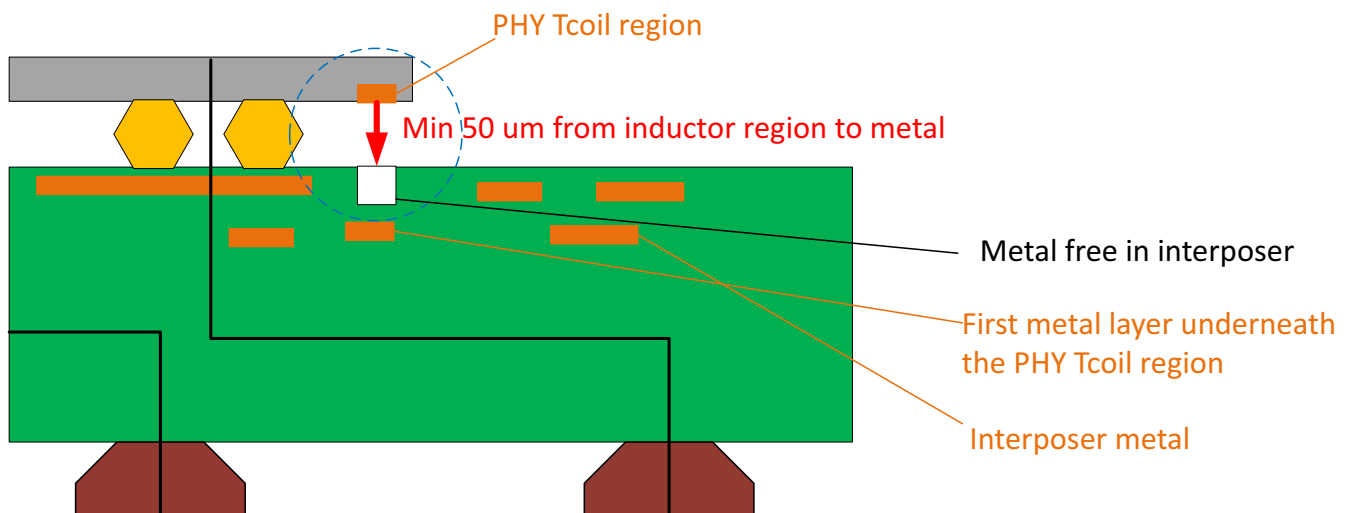
Recommended maximum supply inductance for each of vp/vpdig and vph is provided in [Table 7-2](#). For definition and interpretation of off die supply inductance, refer to SerDes Design Guide application note. A power integrity simulation must be performed to ensure supply voltage noise requirements/specifications are met with the implemented system power distribution network.

Table 7-2 Recommended Supply Inductance

Parameter	Values	Units	Comments
Recommended maximum total package and PCB inductance per PHY	400	pH	Measured from each chip connected (C4) cluster as a single port to the BGAs grouped as a single port. Recommended to add 100 nF PCB decoupling capacitors at each supply BGA to the closest return BGA. Synopsys releases a default testbench that must be considered just an example testbench. Customer must modify it to replace Synopsys generic package by customer's own package and PCB network, along with customer own LC filter network. Active mode simulation must be performed to confirm that customer package and board design will allow the PHY to meet the supply noise requirements. In addition of considering PHY internal (self) noise sources provided by Synopsys, customer must add in addition into the PI testbench any external noise (for example: switching supply noise and crosstalk) to ensure PHY compliance with respect to supply noise requirements

7.3.4 Package Guidelines—Substrate Routing

There are on-die inductors implemented in the PHY: Tcoil inductors in transmit and receive paths, and in MPLL. In the silicon interposer or package layout, keep a minimum of 50 μm clearance between on chip inductors and the metal plane underneath, as shown in [Figure 7-8](#).

Figure 7-8 Silicon Interposer Layout

Synopsys recommends that you avoid any routing or metal fill in the interposer area immediately beneath the on-chip inductors. Avoid any routing of signals, for example, TX or RX differential lines or reference clock lines, or any other toggling signals immediately beneath the MPLL on-chip inductors due to noise coupling concerns. Synopsys recommends to have a ground metal reference copper or an IO Supply reference copper populated under the MPLL inductor at a spacing of 50 μm or at a vertical spacing greater than 50 μm on package substrate or interposer. This helps to shield the inductor from noise coupling and at

the same time to ensure that the on-chip inductance is not affected by the noise. For Tcoil inductors in transmit and receive paths, ensure a minimum of 50 μm vertical spacing between the on-chip Tcoil and the metal shape on package or interposer to avoid the impact on the Tcoil inductance. The following figure shows an example of the location of the inductors within the lanes. For the coordinates of the keep-out areas illustrated in the PMA bump-map diagram, as shown in “[Flip-Chip Pinout](#)” on page 184, contact Synopsys.

Figure 7-9 On-die Inductors (not available for this preliminary release)

[Table 7-3](#) shows the coordinates of inductors.

Table 7-3 Inductors Coordinates (not available for this preliminary release)

Position	Coordinate
-	-

7.4 Reference Resistor

A reference resistor is used for calibration of the RX and TX termination. The reference resistor can be either external or internal (on-die) and is used when the PHY comes out of reset and when a manual tuning request is made. When a PHY is using the reference resistor (internal or external), it forces current across the resistor. When a PHY is not using the reference resistor, the current is disabled. [Table 7-4](#) provides the electrical requirements for the reference resistor.

For details on external reference resistor implementation for a single PMA, see [“Reference Resistor Implementation for a Single PMA”](#) on page 217.

For details on external reference resistor sharing between multiple PMAs, see [“Sharing a Reference Resistor Between Multiple PMAs”](#) on page 217.

For details on internal reference resistor ATE calibration, see [“Internal Reference Resistor ATE Calibration”](#) on page 218.



Note

For additional details on physical implementation and contention bus connectivity see [“Reference Resistor”](#) on page 194.

7.4.1 Package and Board Requirements—Reference Resistor

Reference resistor requirements are outlined in [Table 7-4](#).

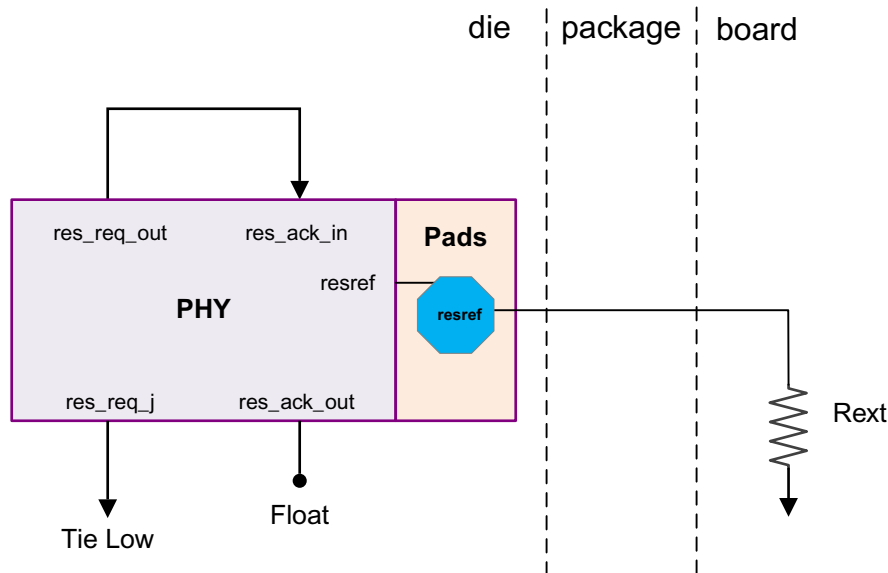
Table 7-4 Reference Resistor Requirements

Item	Value	Design Guidelines
External Reference Resistor Resistance	200 ohm precision resistor (+/-1%)	None
External Reference Resistor Maximum Current	2 mA	None
External Reference Resistor parasitic capacitance C_{load}	<20pF	It is important to meet the total capacitance at low frequency (in other words, the total capacitance of pad, package trace, and PCB).
Reference Resistor parasitic resistance through package and board R_{route}	1% of 200 ohm	This spec is for total resistance through the package and board to the board's external resistor. External Reference Resistor: - Take care to minimize the resistance of the connection through the package to the external reference resistor as this will affect the accuracy of the termination tuning. - The ground used on the board to terminate the external resistor should be as close as possible to the PMA ground voltage. - Noise adjacent signals (for example, clocks) should be spaced away from resref or should be shielded with ground, if possible. Internal Reference Resistor: - When doing ATE calibration on internal reference resistor this spec is applicable until the probe point (voltage or current mode). Refer to section “Internal Reference Resistor ATE Calibration” on page 218.

7.4.2 Reference Resistor Implementation for a Single PMA

Package and board connectivity for a single PMA implementation is shown in [Figure 7-10](#).

Figure 7-10 External Reference Resistor Connectivity for a Single PMA



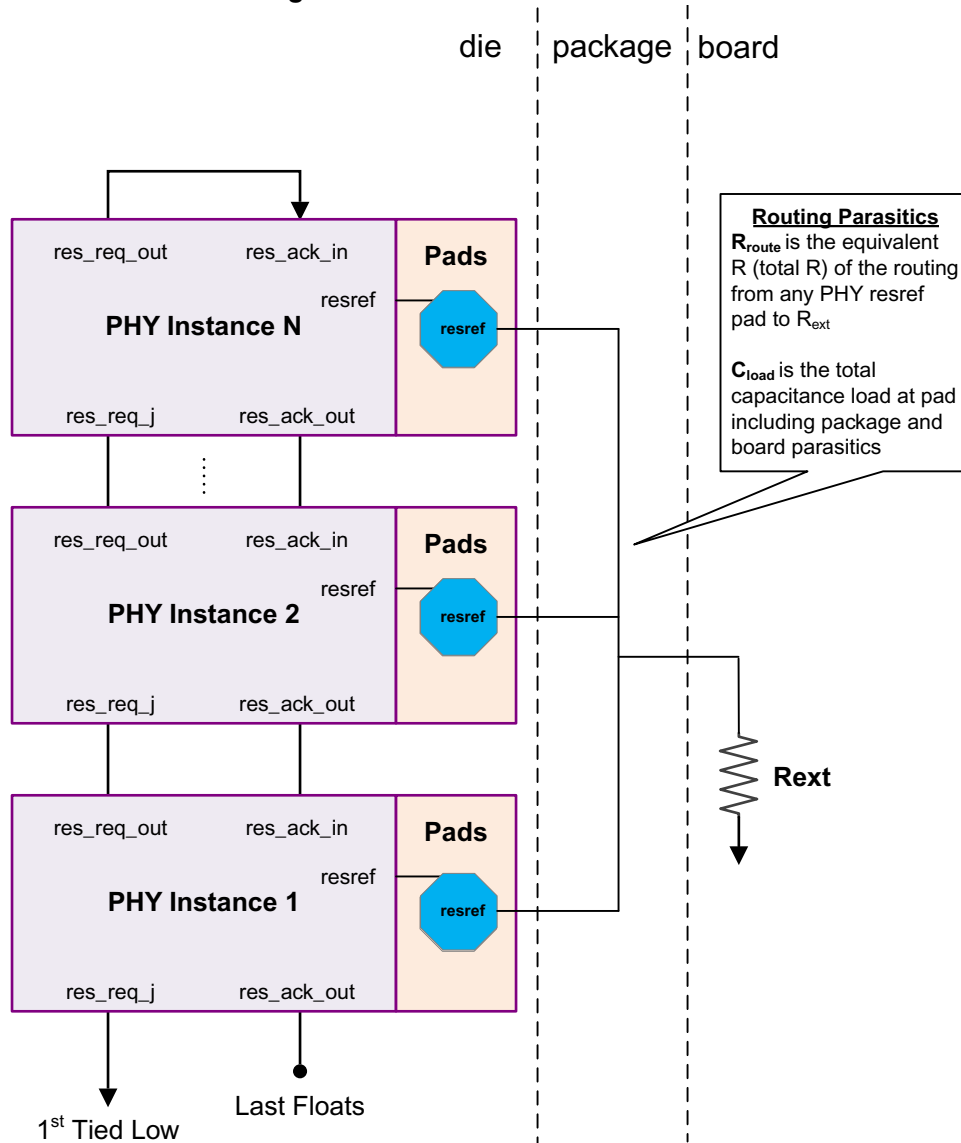
7.4.3 Sharing a Reference Resistor Between Multiple PMAs

The external reference resistor may be shared among a maximum of four instances of the *Synopsys PHY IP MIPI M-PHY Type 1 G5 for Intel3* PMA by connecting the `resref` signals together in the package substrate. Sharing an external reference resistor by tying the `resref` bus together in the package is shown in [Figure 7-11](#).



Note

- If multiple PHYs are sharing a single reference resistor and one or more of the PHYs are being put in a disabled state, see the specific requirements provided in section [“Configuration for a Disabled PHY”](#) on page 165)
- If multiple PHYs are sharing a single reference resistor, do not use the VP supply power-gating on any of the PMAs because it will disable the contention bus
- Reference resistors may be shared with other instantiations of the same PMA, but should not be shared with different PMAs
- Take care to meet the parasitic resistance and capacitance requirements noted in [“Package and Board Requirements—Reference Resistor”](#) on page 216
- Sharing internal reference resistor between multiple PMAs is not allowed.

Figure 7-11 External Resistor Sharing

7.4.4 Internal Reference Resistor ATE Calibration

Internal reference resistor calibration can be implemented in two different ways: current mode and voltage mode. On both methods it is required ATE access to `resref` bump:

- Current mode: inject a current ($1\text{ mA} \pm 1\%$) in `resref` bump, sweep calibration word (`res_ext_rcal[5:0]`) and measure the voltage until it reaches 0.2 V (refer to Figure 7-12). Store the calibration word that condition is met.
- Voltage mode: force a voltage ($0.2\text{ V} \pm 1\%$) in `resref` bump, sweep calibration word (`res_ext_rcal[5:0]`) and measure the current until it reaches 1 mA (refer to Figure 7-13). Store the calibration word that condition is met.

Calibration procedure can either be a linear or binary search. Procedures should be done at room temperature, typical supplies. Alternatively, calibrate for minimum and maximum temperature and average the result. Both modes require `res_ext_en='1'`.

The resulting calibration word that meets the condition must be stored for eFusing.

Figure 7-12 ATE Calibration of Internal Reference Resistor Using Current Mode

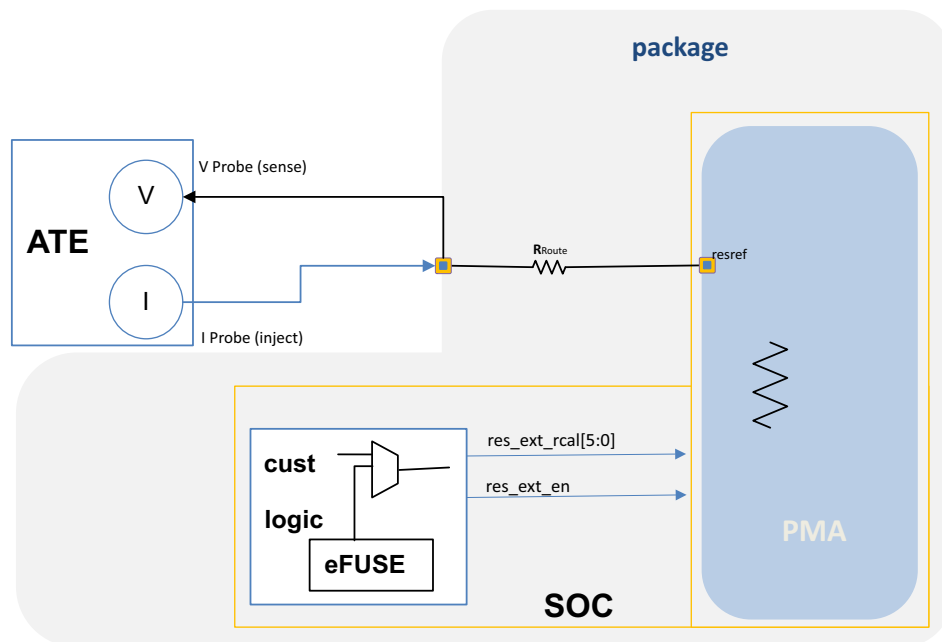
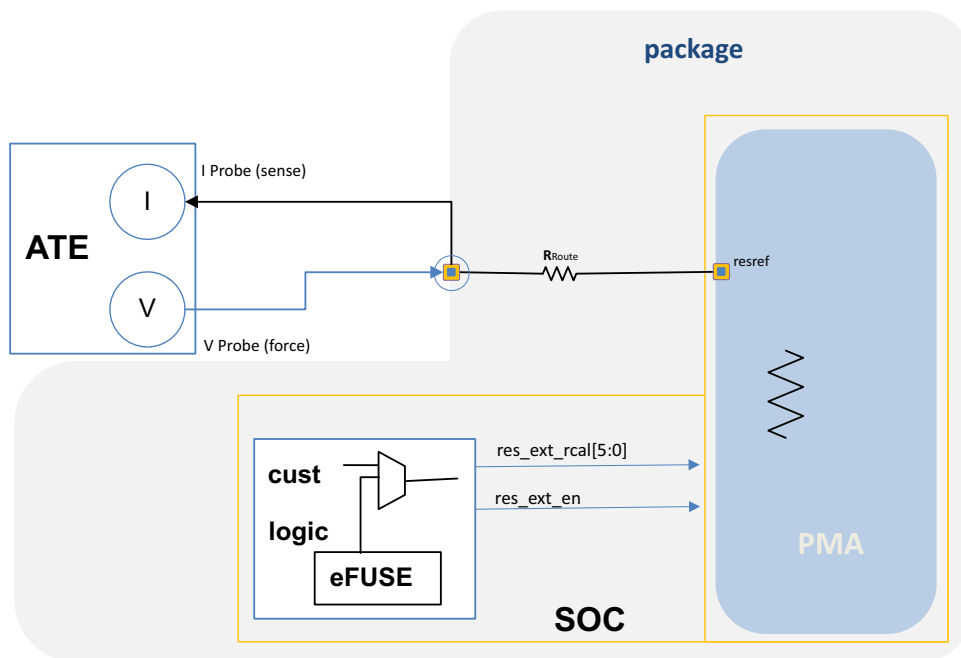


Figure 7-13 ATE Calibration of Internal Reference Resistor Using Voltage Mode



Note

- For R_{route} specifications and guidelines, refer to [Table 7-4](#) on page [216](#).
- Sense probing (voltage or current) needs to be done as close as possible to package pin.

7.5 Reference Clock

For details about reference clock board and package requirements, see “[Reference Clock Implementation](#)” on page [221](#).

7.6 Board Requirements—External Components

The PHY requires the following external components:

- Supply bypass capacitors selected to minimize supply noise
- Precision 200 ohm ($\pm 1\%$) resistor used for biasing and termination impedance calibration; the precision resistor requirement is shown in [Table 7-5](#).

Table 7-5 External Component Requirements

Component	Value
resref	200 ohm ($\pm 1\%$)

8

Reference Clock Implementation

The PHY is designed to handle a wide range of input clock frequencies to support a wide range of applications.

This chapter includes the following sections:

- [“Reference Clock Requirements”](#) on page [222](#)
- [“Reference Clock Implementation”](#) on page [226](#)

The information in this chapter is preliminary and subject to change.

8.1 Reference Clock Requirements

The PHY supports a differential reference clock source. The source may be driven through either external pads or internal pins. The reference clock must meet specific requirements for signal swing and jitter.



Note

You can use the Synopsys DesignWare PHY Jitter Estimator tool for detailed reference clock phase noise compliance analysis, which is available through the following link <https://www.synopsys.com/dw/jittercalc.php>. For details and access, consult Synopsys.

8.1.1 Reference Clock Specification

The reference clock random jitter must comply with the phase noise mask of [Table 8-1](#) the total RJ RMS jitter within the band defined by 50 kHz and the half clock frequency is listed in [Table 8-2](#).

Table 8-1 Reference Clock Phase Noise Mask Requirements

Offset Frequency (Hz)	Phase Noise (dBc/Hz)
5.00×10^4	-110
1.00×10^5	-120
3.00×10^5	-137
1.00×10^6	-137
3.00×10^6	-137
1.00×10^7	-137
5.00×10^7	-137

Table 8-2 Integrated Reference Clock Random Jitter

Reference Clock Frequency (MHz)	RJ (ps RMS Integrated Between 50kHz and Half Clock Frequency)
19.2	5.9
26.0	4.6
38.4	3.5
52.0	2.8
104.0	2.8

Table 8-3 Reference Clock Specification (DJ – 19.2/38.4 MHz)

Internal Refclk (MHz)	Gear	Fc (Hz)	A (HF Gain) (dB)	Total DJ after JTF (s)
19.2 / 38.4	G1B	6.33×10^5	-5.1	1.54×10^{-11}
	G2B	1.27×10^6	-10.9	7.71×10^{-12}
	G3B	2.53×10^6	-16.9	3.85×10^{-12}
	G4B	2.53×10^6	-16.9	1.93×10^{-12}
	G5B	5.06×10^6	-23.1	9.64×10^{-13}

Table 8-4 Reference Clock Specification (DJ – 26/52/104 MHz)

Internal Refclk (MHz)	Gear	Fc (Hz)	A (HF Gain) (dB)	Total DJ after JTF(s)
26/ 52/ 104	G1B	6.33×10^5	-3	1.54×10^{-11}
	G2B	1.27×10^6	-8.4	7.71×10^{-12}
	G3B	2.53×10^6	-14.2	3.85×10^{-12}
	G4B	2.53×10^6	-14.2	1.93×10^{-12}
	G5B	5.06×10^6	-20.4	9.64×10^{-13}

Weight DJ according to high-pass Jitter TF:

$$JTF(s) = \frac{s}{(s + 2\pi f_c)} \times 10^{A/20}$$

The sum of all spurs DJ_{pp}, after JTF(s) weighting shall not exceed Total DJ after JTF.

8.1.2 Reference Clock: Electrical Characteristics

Table 8-5 lists the reference clock electrical characteristics. For more information contact Synopsys.

Table 8-5 Reference Clock Electrical Characteristics

Symbol	Parameter	Min	Max	Units	Conditions
FREF_OFFSET	Reference clock frequency offset ^a	-300	300	ppm	
DCREF_CLK	Duty cycle	40	60	%	
	Maximum input voltage level		VP	V	Applies to ref_alt_clk_p only (CMOS level)
	Minimum input voltage level	0		V	
	Rise/Fall times (20-80%)		100	ps	
	Maximum input voltage level		1.2	V	Applies to ref_pad_clk_p / ref_pad_clk_m only
	Minimum input voltage level	-0.15		V	
	Absolute maximum voltage at pin including overshoot		1.32 ^b	V	
	Absolute minimum voltage at pin including overshoot	-0.3 ^b		V	
	Single-ended input swing	0.3		Vpp	
	Differential input swing	0.3	1.76	Vppdif	
SWREF_CLK	Differential ^c Input edge rate	0.6 ^d	4.0 ^d	V/ns	
REF_CLK_SKEW	Reference clock skew ($\pm$)		200	ps	

a. For supported reference clock frequencies see “[Support and Lane Configuration Settings](#)”.

b. Allowed overshoot time: 10% of period.

c. In case ref_pad_clk is used in single mode, input edge rate is required to be fulfilled on the single ended ref_pad_clk_p/m signal.

d. To be verified from -150 mV to +150 mV (centered on the differential zero crossing) on the differential waveform. During this 300 mV region, the signal must be monotonic through the both rise and fall times.



Note

Only one of the on-chip and off-chip clock inputs may be used at a time

- If an on-chip clock source is used, then you must tie the off-chip source to ground.
- If an off-chip clock source is used, then you must tie the on-chip source to ground.

8.1.3 Repeat Clock Specification

The PHY is able to repeat the input reference clocks through up to four (4) PHY macros. The requirements are provided in the following table.

Table 8-6 Repeat Clock Specification

Output Clocks		Min	Typ	Max	Units	Notes
ref_repeat_clk_p/m	Frequency	19.2		200	MHz	Mission mode
	Frequency		31.8		MHz	Burnin mode
	Duty-cycle	43		57	%	
	Amplitude	0		2*vp	Vdiff	
	Rise/Fall times	0.6		100	ps	
	Cload			50	fF	
	Additive Rj for 100MHz clock (for PAD path)			1.23	psrms	Integrated jitter: 10 kHz-10 GHz
	Skew between differential outputs			50	ps	@ 100 MHz clock
	Dj			10	Ps-p2p	Integrated up to 0.5 us; Ampnoise_vp=5% AmpNoise_vph=3%
	CMR	0		650	ps-p2p/V	Assumes a Single Tone (10 MHz ... 200 MHz) for common noise source; Rejection measured up to 0.5 us

8.2 Reference Clock Implementation

There are two types of reference clock implementations:

- External off-chip reference clocks using ref_pad_clk_p/ref_pad_clk_m
- On-chip reference clocks using ref_alt_clk_p

Both reference clock implementations support either a differential clock or a CMOS single-ended clock. The reference clock input is an unterminated, AC coupled input. If the reference clock is a single-ended CMOS clock, one of the clock inputs (such as ref_pad_clk_p) must be a full-swing CMOS clock, and the other input (such as ref_pad_clk_m) should be tied to either high or low. If the reference clock is differential, it may be CML or LVDS, or any differential specification that meets the requirements outlined in [Table 8-5](#) on page 224.

Tie to ground any unused reference clock inputs.



Note

External clock:

- Differential clock using ref_pad_clk_p and ref_pad_clk_m (CML input or LVDS input)
 - Single-ended using ref_pad_clk_p or ref_pad_clk_m; non-used input tied to GD
- On-chip clocks (CMOS only)
- Single-ended by ref_alt_clk_p

8.2.1 External Off-Chip Clock Sources

When routing reference clocks in from off-chip, a signal integrity (SI) analysis should be performed using a model of the clock input, package, board traces, and clock source. This analysis should confirm that the reference clock input specification is being met across the external on-chip reference clock input (ref_pad_clk_p/ref_pad_clk_m), as outlined in “[Reference Clock Requirements](#)” on page 222.

A model of the reference clock input is provided in [Figure 8-1](#) with its parameters in [Table 8-7](#).

Figure 8-1 Model of Reference Clock Pad Input Load

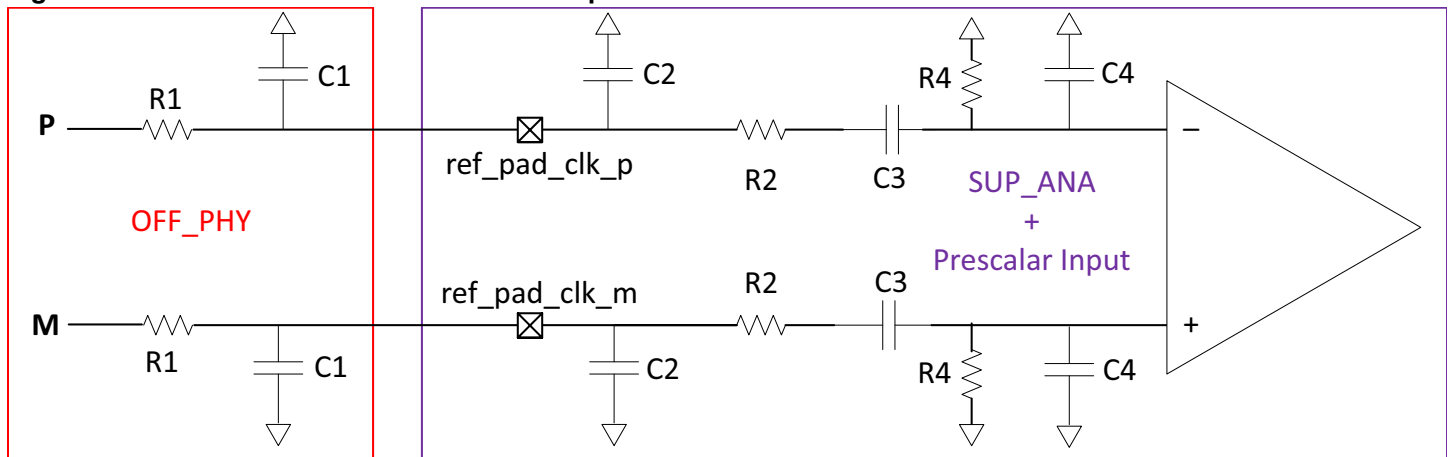


Table 8-7 Reference Clock External Pad Input Parasitics

Location	Parasitic Device	Value	Unit	Notes
OFF PHY	R1	<= 50	ohm	
OFF PHY	C1	<= 3	pF	
Within PHY: Bump to prescaler input	R2	<= 50	ohm	
Within PHY: Bump to prescaler input	C2	<= 1	pF	
Within prescaler: AC cap	C3	800	fF	
Within prescaler: DC Bias resistor	R4	400k	ohm	
Within prescaler: Input cap of prescaler input RX	C4	55	fF	

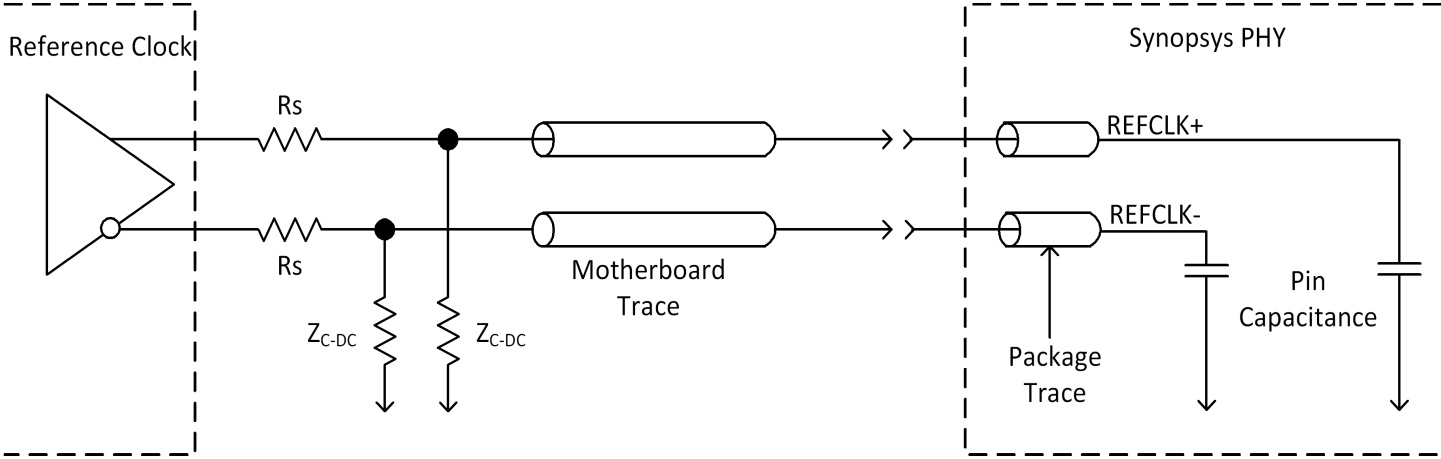
 **Note**

To improve common-mode rejection ratio (CMRR), differential traces should be as well matched as possible.

To enable connecting reference clocks to multiple PHYs at the same time, the reference clock inputs are not terminated. If termination is used, place it on your board as close to the chip as possible.

Also, make sure that the clock swing at the Synopsys PHY input is within reference clock specification. An example of this implementation is shown in [Figure 8-2](#).

Figure 8-2 Off-chip Clock Source Termination



8.2.2 On-Chip Clock Sources

Many applications require an on-chip reference clock. The challenge is getting the reference clock to the PHY without adding supply-noise-induced jitter to the clock.

When routing reference clocks on-chip, you should consider the parasitic loading of the clock distribution lines and the PHY clock input. This analysis should confirm that the reference clock input specification is being met across the on-chip reference clock input (ref_alt_clk_p) as outlined in [“Reference Clock Requirements”](#) on page 222. Make sure to buffer the clocks to be able to drive the following load:

ref_alt_clk_p pin capacitance = 100 fF

The following general guidelines should be followed when routing an on-chip reference clock:

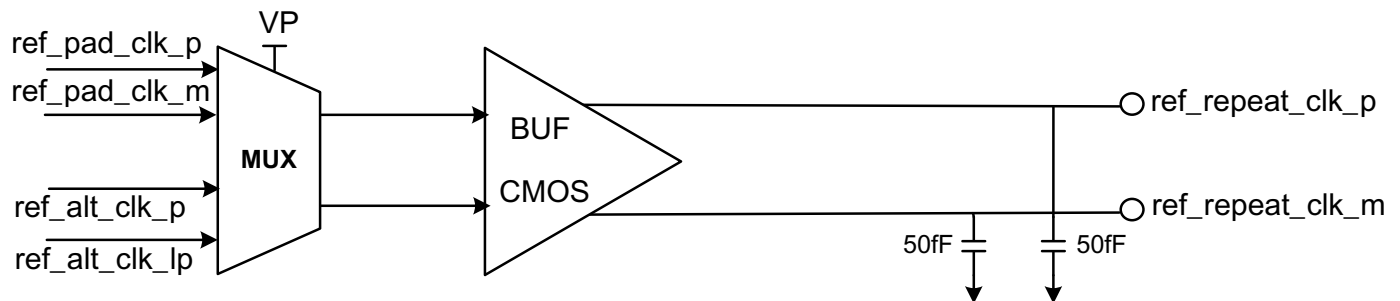
- Design refclk routes as differential routes
- Route on higher, thicker metal to maximize bandwidth (lower level metals tend to be too resistive)
- Use shielding on both sides of differential routes
- Minimize any source of noise that may couple in to the refclk
- Balance any supplies/signals that may couple in so that noise is common mode
- Avoid using an automatic PNR flow to route the reference clock
- Simulate the reference clock routing (consult with Synopsys on an appropriate model)
- Ensure that the reference clock at the far end of the route at the input to the next PMA still meet signal swing requirements
- Share your results with Synopsys for review

8.2.3 Reference Clock Repeater

There is a reference clock repeater output (ref_repeat_clk_{p,m}) that allows the PHY to buffer the reference clock and send it on to another PHY. The driver is a VP supply referenced CMOS output clock driver, as shown in [Figure 8-3](#). The repeater is designed to meet the specifications given in “[Repeat Clock Specification](#)” on page 225.

 **Note** Ref_clk repeat buffer and single-ended buffers are in an always-on power domain.

Figure 8-3 Reference Clock Repeater



 **Note** To improve common-mode rejection ratio (CMRR), differential traces should be as well matched as possible.

8.2.4 Reference Clock Distribution

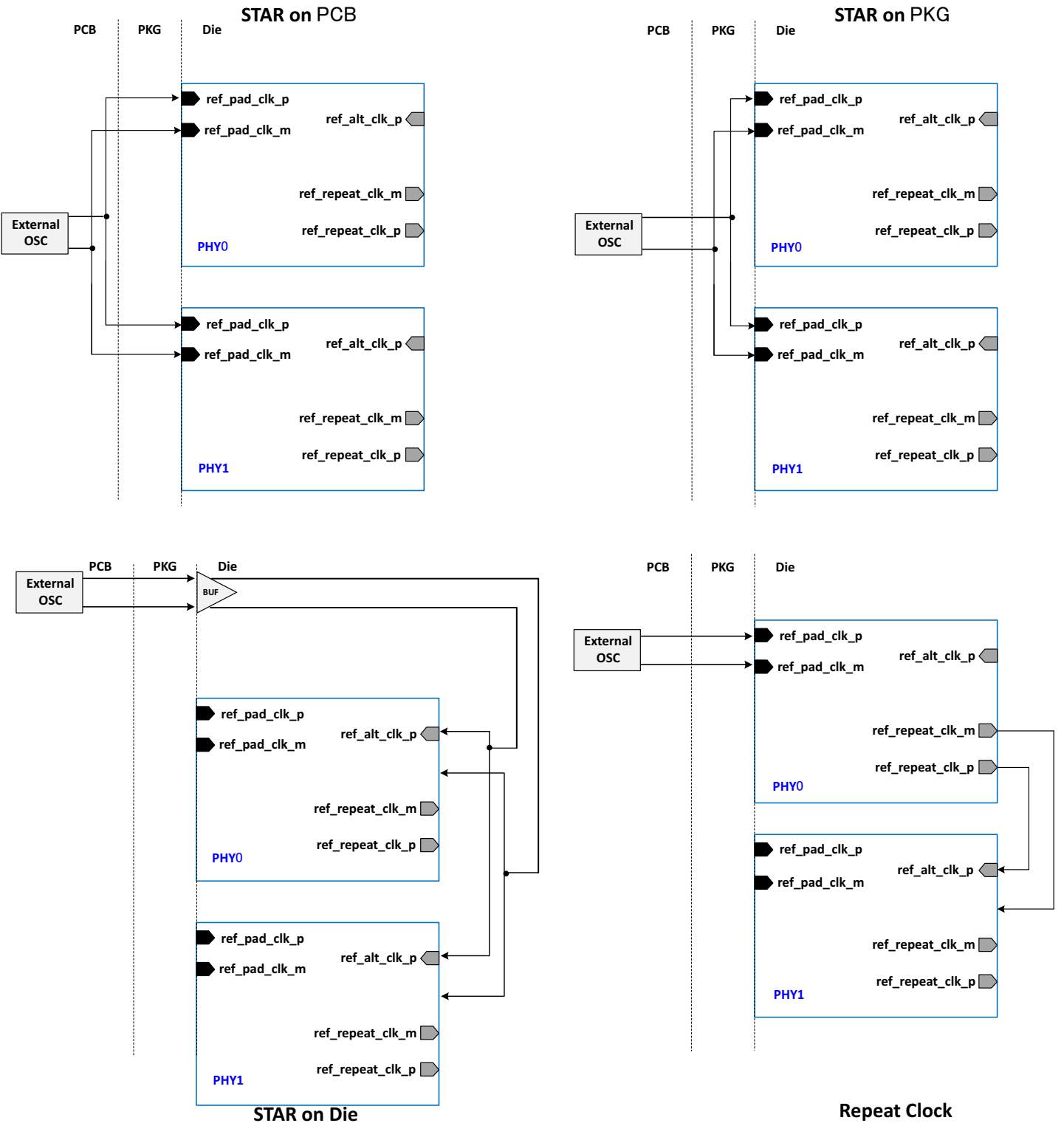
To minimize the amount of jitter on the reference clock, Synopsys recommends STAR configuration in the clock distribution. [Figure 8-4](#) on page 230 provides examples of the clock distribution on board, in the package and on the die. [Table 8-8](#) provides impact of each of the reference clock configurations.

Table 8-8 Clock Distribution Impact

#	Item	Clock Distribution Example				Remark
		STAR on PCB	STAR on Package	STAR on Die	Repeat Clock	
1	REFCLK input	PAD	PAD	Core	PAD/Core	
2	Jitter	-	-	Increase	Increase	Jitter increases if you use active devices
3	ATE Testbench	As is	As is	As is	Customize	Two PHYs setting becomes different if you use repeat clock. Then, you need to customize your ATE testbench.
4	REFCLK SI analysis	ref_pad_clk only	ref_pad_clk only	ref_alt_clk only	ref_pad_clk ref_alt_clk	
5	Additional buffer	-	-	Necessary	-	Additional REFCLK buffer is necessary for STAR on Die

Synopsys recommends to use STAR on PCB or PKG. It results in better reference clock quality (less jitter) and less implementation efforts. Depending on your design, you can use STAR on Die or repeat clock.

Figure 8-4 Reference Clock Distribution



9

Electrical Specifications

This chapter includes the following sections:

- [“Power Dissipation Overview”](#) on page 232
- [“Power Dissipation”](#) on page 233

9.1 Power Dissipation Overview

The PHY supports the power supply voltages in “[Power Dissipation](#)” on page 233.

The values are supported across the following temperature range:

-40 ~ 125°C

For more information, refer to section “[Operating Conditions](#)” on page 188.

The following sections provide simulated power consumption values for the PHY.

- The typical (Typ) values are for TT process corner, 25°C junction temperature, and typical voltage supplies for specified data rate.
- The worst-case (Wc) values are for FF process corner, 125°C junction temperature, and maximum voltage supplies for specified data rate.



Attention

- The power numbers in the following tables do not include raw PCS power consumption.
 - Target power dissipation values indicated in the following tables are based on simulations and are preliminary. Power dissipation is subject to change.
-

9.2 Power Dissipation

**Note**

Power dissipation values will be available in a future release.

A

Support and Lane Configuration Settings

For Support and Lane configuration values to configure the PHY for your application, refer to the excel file listed below (tab names SUPPORT_CONFIG_DATA_BOOK and LANE_CONFIG_DATA_BOOK respectively).

- <package>/doc/
 - dwc_mipi_mphy_type1_g5_22_in3_ns_config_reference_manual.xlsx

Contact Synopsys through your AE channel to request ROM/SRAM memory images for other supported reference clocks not included in PHY deliverables.

B

TX Equalization Values

This appendix contains TX equalization values per protocol for configuring the PHY for your application.

Table B-1 TX Equalization Values Per Protocol

Protocol	Preset	txX_eq_main [5:0]	txX_eq_pre [5:0]	txX_eq_post [5:0]	TX de-emphasis (dB)	TX pre-shoot (dB)	TX max swing (normalized)
MPHY	High_NoEQ	48	0	0	0.00	0.00	0.65
MPHY	High_EQ0	47	0	1	0.57	0.00	0.65
MPHY	High_EQ1	46	0	2	1.19	0.00	0.65
MPHY	High_EQ2	45	0	3	1.85	0.00	0.65
MPHY	High_EQ3	44	0	4	2.57	0.00	0.65
MPHY	High_EQ4	43	0	5	3.35	0.00	0.65
MPHY	High_EQ5	42	0	6	4.21	0.00	0.65
MPHY	High_EQ6	41	0	7	5.16	0.00	0.65
MPHY	High_EQ7	40	0	8	6.23	0.00	0.65
MPHY	High_EQ8	39	0	9	7.45	0.00	0.65
MPHY	High_EQ9	47	1	0	0.00	0.57	0.65
MPHY	High_EQ10	46	2	0	0.00	1.19	0.65
MPHY	High_EQ11	45	3	0	0.00	1.85	0.65
MPHY	High_EQ12	44	4	0	0.00	2.57	0.65
MPHY	High_EQ13	43	5	0	0.00	3.35	0.65
MPHY	Low_NoEQ	48	0	0	0.00	0.00	0.40
MPHY	Low_EQ0	47	0	1	0.95	0.00	0.40

Protocol	Preset	txX_eq_main [5:0]	txX_eq_pre [5:0]	txX_eq_post [5:0]	TX de-emphasis (dB)	TX pre-shoot (dB)	TX max swing (normalized)
MPHY	Low_EQ1	46	0	2	2.02	0.00	0.40
MPHY	Low_EQ2	45	0	3	3.24	0.00	0.40
MPHY	Low_EQ3	47	1	0	0.00	0.95	0.40
MPHY	Low_EQ4	46	2	0	0.00	2.02	0.40
MPHY	Low_EQ5	45	3	0	0.00	3.24	0.40

C

Configuration and Compile Macro ifdef Settings

This appendix contains the configuration and compile macro ifdef settings for your PHY.

Legend												
✓	Indicates that the macro must be set to achieve this Scaledown Mode.											
✓	Indicates that the macro is automatically set in the Scaledown Mode by a macro that must be set in this mode.											
Simulation Scaledown Mode			Scaledown0		ScaleDown1		Scaledown2		Scaledown3		Non Scale Down	
Analog Model			Behavioural		Netlisted		Netlisted		Netlisted		Netlisted	
TX/RX Power-up timing			Fast		Fast		Silicon		Silicon		Silicon	
TX/RX Interrupts			Disabled		Disabled		Enabled		Enabled		Enabled	
Adaptation			Fast		Fast		Fast		Silicon		Silicon	
MPLL Power-up timing			Fast		Fast		Fast		Fast		Silicon	
Calibrations			Skipped		Skipped		Skipped		Skipped		Enabled	
Superset	Macro	Description	PMA GTECH	PMA GATE	PMA GTECH	PMA GATE	PMA GTECH	PMA GATE	PMA GTECH	PMA GATE	GTECH	Gate
	IPNAME_SIM_MODE	Enables synchronizer metastability modeling and analog model functionality. Mandatory for running simulations	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	IPNAME_RANDOM	Randomizes behavioural modeling such as clock-tree delays. Mandatory for running simulations	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	IPNAME_RANDOM_SEED	Randomizes behavioural modeling such as clock-tree delays. Mandatory for running simulations	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	IPNAME_SIM_GATES	Required for PMA gate-level simulations		✓		✓		✓		✓		✓
	IPNAME_SIM_GTECH	Required for PMA GTECH simulations	✓		✓		✓		✓		✓	
	IPNAME_ANA_BEHAV	Use fast analog behavioural model	✓	✓								
	IPNAME_SHORT_RESET	Run in FAST Simulation Mode	✓	✓	✓	✓						
	IPNAME_DISABLE_RXTX_REQ_IRQ	Disable RX/TX request interrupts	✓	✓	✓	✓						
	IPNAME_FAST_RX_ADAPT	FAST RX Startup Adaptation	✓	✓	✓	✓	✓	✓				
	IPNAME_FAST_RX_STARTUP_CAL	FAST RX Startup Calibration	✓	✓	✓	✓	✓	✓	✓	✓		
	IPNAME_FAST_TX_STARTUP_CAL	FAST TX Startup Calibration	✓	✓	✓	✓	✓	✓	✓	✓		
SUP	IPNAME_FAST_SUP	Skip MPLL Calibration and Fast Rtune, bandgap, and prescaler powerup	✓	✓	✓	✓	✓	✓	✓	✓		
	IPNAME_FAST_BG_WAIT	Fast bandgap powerup	✓	✓	✓	✓	✓	✓	✓	✓		
	IPNAME_FAST_RX_PWRUP	Fast RX sigdet time, vreg_en, and clk_dcc_en	✓	✓	✓	✓						
	IPNAME_FAST_RX_VCO_WAIT	Fast RX cdr_startup, vco_cntr_pd, clk_en and vco_update	✓	✓	✓	✓						
RX	IPNAME_FAST_RX_VCO_CAL	Skip VCO Calibration	✓	✓	✓	✓						
	IPNAME_FAST_RX_CONT_ADAPT	FAST Continuous Adaptation	✓	✓	✓	✓						
	IPNAME_FAST_RX_CONT_CAL_ADAPT	FAST Continuous calibration/adaptation	✓	✓	✓	✓						
	IPNAME_FAST_TX_PWRUP	FAST TX clk_en, refgen_en, serial_en time	✓	✓	✓	✓						
TX	IPNAME_FAST_TX_VCM_HOLD	Fast common-mode charge-up time	✓	✓	✓	✓						
	IPNAME_FAST_TX_RXDET	Fast Receiver Detect time	✓	✓	✓	✓						
	IPNAME_FW_FAST_SIM	Reduces calibration and adaptation firmware counts.										
	IPNAME_FW_OVRD	Halt firmware										
	IPNAME_DISABLE_SRAM_REC	Disable SRAM adaptation parameter recording										

D

Cell Mapping for Hand-Placed Cells

For place and route, it is required that certain cells are hand placed. The MIPI M-PHY Type 1 G5 PHY uses the Synopsys Basic Core Module (BCM) library to describe the functionality. The cells provided in [Table D-1](#) must be converted to a technology equivalent standard cell using the verilog `define pragma.

To map the cells to the specific DesignWare Library part name, refer to <https://www.synopsys.com/dw/buildingblock.php>.

Table D-1 Cell Mapping for Hand-Placed Cells

Cell Define	Cell Instance Name	Is Used <yes/no>	Part Name	DesignWare Library Part Name	Description
IPNAME_SYNC_2_STAGE_SRC	U_SAMPLE_META_2	yes	*BCM_bcm21.v	DW_sync	2 stage data synchronizer asynchronous reset (active low)
IPNAME_SYNC_3_STAGE_SRC	U_SAMPLE_META_3	yes	*BCM_bcm21.v	DW_sync	3 stage data synchronizer asynchronous reset (active low)
IPNAME_SYNC_4_STAGE_SRC	U_SAMPLE_META_4	yes	*BCM_bcm21.v	DW_sync	4 stage data synchronizer asynchronous reset (active low)
IPNAME_CK_GT_LAT_SRC	I_hand_bcm_ck_gt_lat	yes	*BCM_bcm00_ck_gt_lat.v		Clock Gating Latch
IPNAME_CK_BUF_SRC	I_hand_bcm_ck_buf	yes	*BCM_bcm00_ck_buf.v		Pass-Through Buffer for Clock Trees
IPNAME_AND_SRC	I_hand_bcm_and	yes	*BCM_bcm00_and.v		2-Input AND Gate
IPNAME_OR_SRC	I_hand_bcm_or	yes	*BCM_bcm00_or.v		2-Input OR Gate

Cell Define	Cell Instance Name	Is Used <yes/no>	Part Name	DesignWare Library Part Name	Description
IPNAME_MX_SRC	I_hand_bcm_mx	yes	*BCM_bcm00_mx.v		2-Input Multiplexer Cell
IPNAME_CK_AND_SRC	I_hand_bcm_ck_and	yes	*BCM_bcm00_ck_and.v		AND Block for Clock Trees
IPNAME_CK_OR_SRC	I_hand_bcm_ck_or	yes	*BCM_bcm00_ck_or.v		2-Input Clock OR Gate
IPNAME_CK_MX_SRC	I_hand_bcm_ck_mx	yes	*BCM_bcm00_ck_mx.v		2-Input Clock Multiplexer
IPNAME_D_BUF_SRC	I_hand_bcm_d_buf	yes	*BCM_bcm00_buf.v		Non-inverting Buffer with Random DELAY Sim Model
IPNAME_CK_DFF_PRSTN_SRC	I_hand_bcm_ck_dff_prstn	yes	*BCM_bcm00_ck_dff_prstn.v		Positive-edge-triggered D Flip-Flop Dedicated for Clock Path with Active-Low Asynchronous Preset
IPNAME_CK_DFF_CLRN_SRC	I_hand_bcm_ck_dff_clrn	yes	*BCM_bcm00_ck_dff_clrn.v		Positive-edge-triggered D Flip-Flop Dedicated for Clock Path with Active-Low Asynchronous Clear
IPNAME_DFF_PRSTN_SRC	I_hand_bcm_dff_prstn	yes	*BCM_bcm00_dff_prstn.v		General Positive-edge-triggered D Flip-Flop with Active-Low Asynchronous Preset
IPNAME_DFF_CLRN_SRC	I_hand_bcm_dff_clrn	yes	*BCM_bcm00_dff_clrn.v		General Positive-edge-triggered D Flip-Flop with Active-Low Asynchronous Clear

**Note**

The list above is a superset of available hand-instantiated BCM cells. Not all of the cells may be used in this specific configuration.

The BCM cell technology mapping feature is enabled with the `ENABLE_Tech_CELL_MAPPING` define. When this define is enabled, the following two files are included in the flow.

- `phy/include/*_bcm_tech_map.v`

- This is a Verilog file that provides the standard cell mapping `define macros to instantiate the technology specific cells.
- The following is an example of a AND cell technology mapping:

```
`define IPNAME_AND_SRC \
HDBLVT03_AN2_2 I_hand_bcm_and ( \
  .A1(a), \
  .A2(b), \
  .X(z) \
);
```

where:

- HDBLVT03_AN2_2 is the cell name in the technology-specific library. This should be replaced with the specific standard cell to be used.
- A1/A2/X are the standard cell port names and a/b/z are the BCM pin names in the RTL to connect to the standard cell ports.
- phy/synth/constraints/*_bcm_pin_map.tcl
 - This file is a standard tcl file that provides the pin mapping variables that are used in the synthesis flow.
 - The following is an example of a AND cell technology mapping:

```
set and_src_inst_name      "I_hand_bcm_and"
set and_src_a_pin          "A1"
set and_src_b_pin          "A2"
set and_src_z_pin          "X"
```

where:

- and_src_a/b/z_pin are the tcl variables used in the synthesis flow and constraints.
- A1/A2/X are the standard cell port names that are mapped to the generic pins a/b/z respectively.

